

Programs Offered

Post Graduate Programmes (PG)

- Master of Business Administration
- Master of Computer Applications
- Master of Commerce (Financial Management / Financial Technology)
- Master of Arts (Journalism and Mass Communication)
- Master of Arts (Economics)
- Master of Arts (Public Policy and Governance)
- Master of Social Work
- Master of Arts (English)
- Master of Science (Information Technology) (ODL)
- Master of Science (Environmental Science) (ODL)

Diploma Programmes

- Post Graduate Diploma (Management)
- Post Graduate Diploma (Logistics)
- Post Graduate Diploma (Machine Learning and Artificial Intelligence)
- Post Graduate Diploma (Data Science)

Undergraduate Programmes (UG)

- Bachelor of Business Administration
- Bachelor of Computer Applications
- Bachelor of Commerce
- Bachelor of Arts (Journalism and Mass Communication)
- Bachelor of Arts (General / Political Science / Economics / English / Sociology)
- Bachelor of Social Work
- Bachelor of Science (Information Technology) (ODL)



Amity Helpline: (Toll free) 18001023434
For Student Support: +91-8826334455
Support Email id: studentsupport@amityonline.com | <https://amityonline.com>



Advanced Software Engineering Principles

AMITY



Advanced Software Engineering Principles

Advanced Software Engineering Principles

© Amity University Press

All Rights Reserved

No parts of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without the prior permission of the publisher.

SLM & Learning Resources Committee

Chairman : Prof. Abhinash Kumar

Members : Dr. Ranjit Varma
Dr. Maitree
Dr. Divya Bansal
Dr. Arun Som
Dr. Sunil Kumar
Dr. Reema Sharma
Dr. Winnie Sharma

Member Secretary : Ms. Rita Naskar

Contents

Page No.

Module - I: Life Cycle Models

01

- 1.1 Overview to Software Engineering
 - 1.1.1 Introduction to Software Engineering
 - 1.1.2 Introduction to Lifecycle Model
 - 1.1.3 Incremental Development
 - 1.1.4 Spiral Model
 - 1.1.5 Component Model
 - 1.1.6 Agile Software Development
 - 1.1.7 Waterfall Model
 - 1.1.8 Prototype Model
 - 1.1.9 Rapid Application Development (RAD) Model
 - 1.1.10 Selection of Appropriate SDLC Model

Module - II: Formal Methods

52

- 2.1 Basic Concepts
 - 2.1.1 Basic Concepts of Formal Specification
 - 2.1.2 Importance of Formal Methods
 - 2.1.3 Overview and Advantages of Formal Methods Model
 - 2.1.4 Use of Formal Methods in SDLC
- 2.2 Mathematical Preliminaries
 - 2.2.1 Basic Concepts of Mathematical Preliminaries
- 2.3 Mathematical Notations for Formal Specification
 - 2.3.1 Overview and Types of Mathematical Notations for Formal Specification
- 2.4 Formal Specification Languages
 - 2.4.1 Overview of Formal Specification Languages
 - 2.4.2 Types of Formal Specification Languages
 - 2.4.3 Difference between Informal and Formal Specification Language
- 2.5 Z-Notations
 - 2.5.1 Syntax, Type and Semantics of Z-Notations
 - 2.5.2 Benefits and Limitations of Z
 - 2.5.3 Specification to Convert Requirements Written in Natural Language to Z Formal
- 2.6 Ten commandments of Formal Methods
 - 2.6.1 Tips to Use Formal Methods

Module - III: Component-Based Software Engineering

98

- 3.1 Overview of Component-Based Software Engineering
 - 3.1.1 Introduction to Component-Based Software Engineering: Part 1
 - 3.1.2 Introduction to Component-Based Software Engineering: Part 2

- 3.1.3 Domain Engineering
- 3.1.4 Economics of Component-Based Software Engineering
- 3.2 Cleanroom Software Engineering
 - 3.2.1 Cleanroom Approach
 - 3.2.2 Functional Specification
 - 3.2.3 Cleanroom Testing
 - 3.2.4 Structure of Client/Server System

Module - IV: Client/Server Software Engineering

138

- 4.1 Overview to Client/Server Software Engineering
 - 4.1.1 Software Engineering for Client Server Systems
 - 4.1.2 Design for Client Server Systems and Testing Issues
 - 4.1.3 Peer to Peer Architecture
 - 4.1.4 Service Oriented Software Engineering
- 4.2 Web Engineering
 - 4.2.1 Service Engineering
 - 4.2.2 Software Development with Services
 - 4.2.3 Software Testing Issues
 - 4.2.4 Analysis Modelling Issues
 - 4.2.5 WebE Process
 - 4.2.6 Framework for WebE
 - 4.2.7 Formulating/Analysing Web-Based Systems
 - 4.2.8 Management Issues
- 4.3 Service Oriented Software Engineering
 - 4.3.1 Introduction to Service Oriented Software Engineering
 - 4.3.2 Services as Reusable Components
 - 4.3.3 Service Engineering
 - 4.3.4 Software Development with Services

Module - V: Re-engineering and CASE

202

- 5.1 Re-engineering
 - 5.1.1 Business Process Re-engineering and Software Re-engineering
 - 5.1.2 Introduction to Building Blocks of CASE
 - 5.1.3 Forward Re-engineering and Economics of Re-engineering
 - 5.1.4 Reverse Engineering and Restructuring Engineering
- 5.2 Computer-Aided Software Engineering
 - 5.2.1 Introduction and Building Blocks of CASE
 - 5.2.2 Taxonomy of Case Tools
 - 5.2.3 TCS Robot: Case Tools
 - 5.2.4 Integration Architecture and Case Repository

Module - I: Life Cycle Models

Notes

Learning Objectives

At the end of this module, you will be able to:

- Understand Software Engineering
- Analyze Lifecycle Model
- Know Spiral and Component Model
- Learn Agile Software Development
- Discuss Waterfall and Prototype Model

Introduction

Before a software product can be developed, user needs and constraints must be identified and made clear; the product must be made to be user- and implementer-friendly; the source code must be carefully implemented and tested; and supporting documentation must be created. Software maintenance tasks include reviewing and analysing change requests, redesigning and altering source code, thoroughly testing updated code, updating documentation to reflect changes and distributing updated work products to the right user.

The need for systematic approaches to software development and maintenance became apparent in the 1960s. Many software projects at the period suffered from cost overruns, schedule slippage, unreliability, inefficiency and lack of customer consent. It became evident that the demand for software was exceeding our capacity to produce and update it as computer systems became bigger and more sophisticated. Consequently, software engineering has developed into an important topic in technology.

The nature and complexity of software have seen significant changes in the past forty years. Applications from the 1970s returned alphanumeric results using a single processor and single line inputs. On the other hand, modern software are much more complex, rely on client-server technology and feature an intuitive user interface. They are compatible with a wide range of CPUs, operating systems and even hardware from different countries.

Software groups tackle development issues and backlogs in addition to doing their utmost to stay abreast of rapidly emerging new technologies. Improvements to the development process are even recommended by the Software Engineering Institute (SEI). It's an hourly requirement that cannot be avoided. But it often results in conflict between individuals who welcome change and others who adamantly stick to traditional working practices. Consequently, in order to prevent disputes and enhance software development and deliver high-quality software on schedule and under budget, it is imperative to embrace software engineering concepts, practices and methods.

Software is a collection of instructions that a user can use to gather inputs, manipulate them and then produce the desired output in terms of features and performance. It also comes with a collection of materials designed to help users comprehend the software system, such as the program handbook. In contrast, engineering focuses on creating goods through the application of precise, scientific ideas and techniques.

1.1 Overview to Software Engineering

Software is one of the most significant technologies in the world today and is essential to the development of computer-based systems and goods. Software has developed over

Notes

the past 50 years from a specialised tool for information analysis and problem solutions to a whole industry. However, we still struggle to provide high-caliber software on schedule and within budget.

Software covers a broad range of technologies and application domains through programs, data and descriptive information. Those that have to maintain legacy software still face unique difficulties. Simple collections of information content have given way to sophisticated systems that display complex functionality and multimedia content. These systems and applications are based on the web. These WebApps are software even though they have special features and needs.

Software engineering is the process, methods and tools that make it possible to create complex computer-based systems with speed and quality. All software initiatives can benefit from the five framework activities that make up the software process: planning, communication, modelling, construction and deployment. A set of fundamental principles guide the problem-solving process of software engineering practice. Even as our collective understanding of software and the tools needed to construct it advances, a wide range of software myths still mislead managers and practitioners. You'll start to see why these fallacies should always be disproved as you get more knowledge about software engineering.

1.1.1 Introduction to Software Engineering

Software systems are intricately designed intellectual works. Software development must guarantee that the system is delivered on schedule, stays within budget and satisfies the requirements of the intended application. In 1968, at a NATO meeting, the phrase "software engineering" was coined to advocate for the need of an engineering approach to software production in order to achieve these aims. Software engineering has advanced significantly and established itself as a field since that time.

"The goal of software engineering as a discipline is to significantly increase software productivity and quality while lowering software costs and time to market through research, education and practice of engineering processes, methods and techniques."

The process of turning a preliminary system concept into a software system that operates in the intended environment is known as software development. Software development activities include software specification, software design, implementation, testing, deployment and maintenance, just like many engineering projects. The user and customer's desires are determined by the software specification. These are stated as prerequisites or competencies that the software system needs to meet. In order to fulfil the software requirements, software design creates a software solution. Specifically, it establishes the software system's general software structure, also referred to as the software architecture.

The architecture shows the relationships, interfaces and interactions between the main parts of the system. High-level algorithms and user interfaces for the system's component parts are also defined by software design. The design is turned into computer programs, which are tested to make sure they function as the user and customer expects during implementation and testing. After installation, the software system is checked and adjusted to make sure it functions as intended in the target environment. The software system is continuously updated to fix bugs and improve functionality during the maintenance phase until it is replaced or abandoned.

Software engineering is an engineering profession that covers every facet of software creation, from system specification in its early phases to post-implementation maintenance. Two important words in this definition are as follows:

1. Engineering specialisation: Engineers are those who make things work. When applicable, they employ theories, techniques and instruments. They do, however, employ them sparingly and always look for answers to issues, even in the absence of relevant theories and techniques. Engineers search for answers within organisational and budgetary constraints because they are aware of these limitations.
2. Every facet of producing software: The technical procedures involved in software development are only one aspect of software engineering. In addition, it covers tasks like managing software projects and creating instruments, procedures and theories to aid in the creation of software.

Getting results of the necessary quality on time and within budget is the goal of engineering. Since engineers can't be perfectionists, this frequently requires accepting concessions. On the other hand, programmers can dedicate as much time as they like to the development of their own programs.

Since producing high-quality software is frequently best achieved by methodical and organised work practices, software engineers generally take this approach to their job. But engineering is all about choosing the best way for a given set of conditions, so in some cases, a less formal, more creative approach to development might work. For the creation of web-based systems, which calls for a combination of software and graphical design talents, less formal development is especially appropriate.

Activities related to software quality assurance (QA) are conducted concurrently with development activities. The purpose of quality assurance (QA) activities is to guarantee that development activities are executed accurately, that necessary artifacts—like software requirements document (SRD) and software design document (SDD)—are created and adhere to quality standards and that the software system will meet the requirements. Testing, requirements analysis, design evaluation, code review and inspection are the methods used to achieve these.

Activities related to software project management guarantee that the software system under development will be delivered within budget and on schedule. Project planning is a crucial component of project management. It happens just at the start of a project, right after the specifications for the software system are decided. Specifically, the expected amounts of time and effort needed to complete the three project activity tracks. A project schedule is created to provide direction. Project management is in charge of continuously monitoring the project's expenses and progress during the development and deployment phase, as well as carrying out the necessary adjustments to adjust the project to new circumstances.

Origins of the Term Software Engineering

In the 1960s, software engineering became its own independent field of engineering. "Software engineering" has been around since, at the very least, 1963–1964. It was first used by Margaret Hamilton, who worked on the Apollo space program, to differentiate software engineering from hardware and other engineering specialties. An hour on a computer cost hundreds of times as much as an hour on a programming at the time because hardware was so valuable.² In the name of efficiency, maintainability and clarity of the code were frequently compromised. Software became more and more important as computer applications became larger and more complex.

When the phrase "software engineering" was used as the name of a 1968 NATO conference, it was still more of a pipe dream than a practical field. The title was deliberately chosen by the organisers, particularly Fritz Bauer, to suggest that software manufacturing

Notes

should be grounded in the kinds of theoretical underpinnings and applied disciplines that are customary in the more established engineering departments.

Many things have changed since then. Margaret Hamilton received the Presidential Medal of Freedom on November 22, 2016, in recognition of her efforts developing software that helped prepare for the Apollo missions.

Why Software Engineering

First, every aspect of modern civilisation uses software. Software is essential to the operation and growth of enterprises. Software is essential to the operation of many machinery and gadgets, including cars, trucks, aeroplanes and medical equipment. Software is also essential to cloud computing, artificial intelligence (AI) and the Internet of Things (IoT). Software systems are growing exponentially in size, complexity and dispersion. These days, creating systems with millions of lines of code is not unusual. The F35 fighter, for instance, has 8 million lines of code; the Windows operating system from Microsoft has roughly 50 million lines of code; and Google Search, Gmail and Google Maps combined have 2 billion lines of code. Three decades ago, the software cost accounted for 5%–10% of the total system cost for many embedded systems, which are composed of both hardware and software. Today, that percentage is between 90% and 95%. Firmware, system on a chip (SoC) and/or application-specific integrated circuits (ASIC) are used in some embedded systems. These are integrated circuits, where the hardware and software are fused together. Since they are expensive to replace, the software's quality is essential. To develop systems, these demand a software engineering methodology.

Second, collaboration is aided by software engineering, which is necessary for the development of huge systems. It takes a lot of work to design, create, test and maintain large software systems. An average software developer can write between fifty and one hundred lines of source code a day. This covers the amount of time needed for analysis, design, implementation, testing and integration. For a system with 10,000 lines of code, a single software engineer would need to dedicate approximately 100–200 days, or 5–10 months, of effort. For a software engineer, 5,000–10,000 days, or 20–40 years, would be needed to complete a medium-sized system with 500,000 lines of source code. Any business cannot afford to wait this long. Thus, a team or teams of software engineers are required to design and implement real-world software systems. For instance, 20–40 software engineers are needed for a year to work on a medium-sized software system. Collaboration amongst two or more software engineers presents significant hurdles in terms of coordination, communication and conception when developing software systems.

The process of conceptualisation involves monitoring and categorising real-world occurrences in order to create a mental model that will aid in understanding the intended use of the system. Because software engineers may have various perspectives on the world based on variances in their education, cultural backgrounds, professional experiences, preconceptions and other variables, conceptualisation can be difficult for teams working together. This is explained in the tale of the blind men and the elephant. Comparable to the four blind guys attempting to see or comprehend an application are we software engineers. How can the team members create software that will accurately automate the application if they have the wrong impression of it? How can a team of people with disparate perspectives create and execute software components that complement one another? Software engineering helps developers create a shared knowledge of an application for which the software is designed by providing modelling languages like the Unified Modelling Language (UML), methods and procedures.

Software engineers must convey their analysis and design concepts to one another when working as a team. But the natural language is too colloquial and vague at times.

Once more, UML enhances developer communication. Lastly, how can software engineering teams coordinate and cooperate with each other while they work together? How do they assign the components to the teams and individual members, for instance and split up the work? How do they combine the elements created and put into practice by various teams and individuals within the team? Software engineering offers an answer. That is, these issues are resolved by software development methods and methodologies, software project management and quality assurance.

Software Engineering Ethics

Every part of our life is controlled and impacted by software, which permeates every area of our society. Software has the power to benefit or hurt our society and other people. Thus, when developing, deploying and testing software, software engineers need to take social and ethical duties into account. The “Software Engineering Code of Ethics and Professional Practice” was suggested by the ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices in this regard.

The goal of software engineers is to elevate the field of software analysis, specification, design, development, testing and maintenance to a useful and esteemed one. As part of their responsibility to the public's health, safety and welfare, software developers must abide by the following eight principles:

1. Public—Software developers are expected to behave in the public interest.
2. Client and employer—Software developers are expected to behave in a way that serves the public interest while acting in the best interests of their employers and clients.
3. Product—Software engineers are responsible for making sure that their creations and any associated changes adhere to the strictest industry standards.
4. Judgment—It is expected of software engineers to exercise professional judgement with independence and honesty.
5. Management—Managers and leaders in software engineering must support and adhere to an ethical management philosophy for software development and upkeep.
6. Profession—In line with the public interest, software engineers should enhance the integrity and credibility of their profession.
7. Colleagues—Software developers are expected to treat their peers fairly and to encourage them.
8. Self—Software engineers are expected to support an ethical attitude to the practice of their job and engage in lifelong learning about it.

These ethical guidelines should guide software engineers in both their daily and professional life. Software engineers, for instance, are required to maintain client or employer confidentiality. A software engineer's employer's or client's intellectual property must also be respected and safeguarded. A software engineer occasionally has to make a decision. For instance, a software engineer may be aware that, in rare situations, a component may behave abruptly, resulting in harm to property or even fatalities. He is also aware that his business needs to regain market share by releasing the product as soon as possible. If he discloses the issue, the release will need to be delayed significantly and he will be labelled as the “trouble maker.” If he doesn't report, a terrible tragedy could occur. In our sector, instances of this hypothetical situation have really happened time and time again. Those in management must also make moral decisions.

Notes

Software Engineering and Computer Science

What distinguishes computer science from software engineering? Both working professionals and students frequently ask this question. Computer science prioritises accuracy, performance, resource sharing, computational efficiency and optimisation. These are reasonably fast and accurately measurable. All of the time and money invested in computer science research during the past few decades (from 1950 to the present) has been directed towards enhancing these areas.

Software engineering prioritises software PQCT, in contrast to computer science. For instance, the aim of computer science is frequently to find the best answer. A good-enough solution would be used in software engineering to cut down on expenses and development or maintenance time. The goal of software engineering research and development is to greatly increase software PQCT. Unfortunately, it is difficult and time-consuming to quantify the influence of a software engineering process or technique. The influence needs to be evaluated over an extended period of time and with significant resources in order to be useful. For instance, it took experts over ten years to determine the detrimental effects of the unregulated goto statement. In other words, when the goto statement is used carelessly, the outcome is badly designed programs that are challenging to read, test and maintain.

Computer science is solely concerned with technical matters. Non-technical problems are dealt with by software engineering. For instance, the initial phases of the development process concentrate on determining the needs of the business and creating specifications and limitations. Domain expertise, experience with research and design, communication prowess and client interactions are prerequisites for these tasks. Project management expertise and knowledge are equally necessary for software engineering. Human variables like user preferences and system usage patterns must be taken into account when designing user interfaces. Political considerations must also be taken into account while developing software because the system may have an impact on a large number of individuals.

Understanding and appreciating software engineering processes, approaches and principles may be facilitated by being aware of the distinctions between software engineering and computer science. Take into consideration, for instance, the architecture of a software system that requires database access. Computer science may place an emphasis on effective data retrieval and storage and support program designs that allow direct database access. A program with such an architecture would be susceptible to modifications made to the database management system (DBMS) and database design. The program must be significantly altered if the database schema or DBMS are modified or replaced. This might be expensive and challenging. Software engineers would therefore not view this as a sensible design choice unless they really need efficient database access. In order to save maintenance time, money and effort, software engineers would rather have a design that minimises the effects of database changes.

Computer science and software engineering are closely connected fields, notwithstanding their differences. Similar to physics and electrical and electronics engineering or chemistry and chemical engineering, computer science and software engineering work together. In other words, software engineering is built on the theoretical and technological basis of computer science. Computer science is applied in software engineering. Software engineering does, however, have its own areas of study. These include, among other things, research on software processes and procedures, software validation, software verification and testing strategies.

The field of software engineering is vast. Programming languages, algorithms and data structures, database management systems, operating systems, artificial intelligence, and

computer networks are just a few of the computer science topics that a software engineer should be knowledgeable in. Software engineers working on embedded systems must possess a fundamental understanding of electronic circuits and hardware interface. Lastly, developing domain expertise and design experience is a gradual process for a software engineer to become a skilled software architect. Software engineering is an attractive field because of these problems as well as the capacity to develop and construct big, sophisticated systems to fulfil real-world objectives. Software engineers and researchers have a lot of options thanks to the constantly growing field of computer applications.

Software engineering is important because it plays a key part in creating software systems that are dependable, high-quality, and maintainable in order to address a variety of business, technological, and societal needs. The following are some salient features that underscore the importance of software engineering:

1. **Innovation Enabler:** Software engineering fosters innovation by offering an organised method for creating, developing, and implementing software solutions that tackle new possibilities and obstacles. It gives businesses the ability to develop novel goods, services, and business plans that promote social progress and economic expansion.
2. **Quality and Reliability:** To make sure that software systems fulfil requirements and function dependably in a range of operating settings, software engineering concepts and practices place a strong emphasis on quality assurance, testing, and verification. Good software increases productivity, user happiness, and faith in technology.
3. **Productivity and Efficiency:** By optimising workflows, automating tedious operations, and promoting teamwork, software engineering approaches, tools, and best practices raise productivity and efficiency in development. This makes it possible for businesses to provide software solutions more affordably and swiftly.
4. **Scalability and Adaptability:** The design and implementation of scalable and adaptable software structures that can take into account changing business requirements, technological improvements, and developing user needs are supported by software engineering principles. Software systems that are flexible and scalable can expand and change over time without sacrificing stability or performance.
5. **Risk Management:** Throughout the software development lifecycle, organisations can identify and manage potential risks and uncertainties with the aid of software engineering approaches like risk analysis and mitigation. The probability of budget overruns, quality problems, and project delays is reduced by proactive risk management.
6. **Global Collaboration:** Software engineering makes it possible for dispersed teams to collaborate successfully on software projects regardless of where they are physically located. Global team members may coordinate and share knowledge easily thanks to communication platforms, version control systems, and collaboration tools.
7. **Regulatory Compliance and Security:** Software engineering procedures take security and regulatory compliance into account to make sure software systems follow the law, morality, and industry norms. Adherence to regulatory frameworks like GDPR, HIPAA, and PCI DSS is crucial in safeguarding user privacy and minimising legal liabilities.
8. **Continuous Improvement:** Through procedures like code reviews, retrospectives, and post-implementation reviews, software engineering fosters a culture of continuous improvement. Organisations are able to improve software quality, performance, and user experience iteratively over time by drawing lessons from feedback and past experiences.
9. **Developer Empowerment:** Software engineering equips developers with the know-how, abilities, and resources required to take on challenging technical problems and make

Notes

significant contributions to software projects. A thriving and dynamic software engineering ecosystem is fostered via community participation, professional development, and ongoing learning.

Traditional software engineering and advanced software engineering represent different stages or approaches in the evolution of software development methodologies. Here's a comparison of the two:

Methodology:

- Traditional software engineering often refers to the Waterfall model or other sequential models where development progresses through fixed stages such as requirements gathering, design, implementation, testing, and maintenance, with little room for iteration.
- Advanced software engineering typically involves agile methodologies such as Scrum, Kanban, or Extreme Programming (XP). These methodologies emphasize iterative development, frequent collaboration with stakeholders, and adapting to change throughout the development process.

Flexibility and Adaptability:

- Traditional software engineering methodologies tend to be rigid and less adaptable to changes in requirements or technology.
- Advanced software engineering methodologies are designed to be flexible and adaptable, allowing teams to respond quickly to changes in requirements, technology, or market conditions.

Focus on Documentation:

- Traditional software engineering places a strong emphasis on comprehensive documentation at each stage of development.
- Advanced software engineering values working software over comprehensive documentation, although documentation is still important for communication and knowledge transfer within the team.

Team Structure and Collaboration:

- Traditional software engineering often involves separate teams for each stage of development (e.g., analysts, designers, developers, testers), with limited collaboration between team members.
- Advanced software engineering promotes cross-functional teams that include members with diverse skills (e.g., developers, testers, designers, product owners) who collaborate closely throughout the development process.

Customer Involvement:

- Traditional software engineering may have limited customer involvement, with requirements gathered upfront and little interaction during development.
- Advanced software engineering encourages continuous customer involvement through techniques such as frequent demos, user feedback sessions, and prioritization of features based on customer value.

Risk Management:

- Traditional software engineering tends to address risks upfront in the planning stages and relies on predictive methods to manage them.

- Advanced software engineering acknowledges that uncertainty and risks are inherent in software development and focuses on identifying and mitigating risks iteratively throughout the project.

Delivery Frequency:

- Traditional software engineering often results in longer development cycles, with software released in large, infrequent updates.
- Advanced software engineering enables more frequent and incremental delivery of working software, allowing for quicker feedback and faster response to changing requirements or market conditions.

1.1.2 Introduction to Lifecycle Model

The software process outlines the best way to oversee and plan a software development project while keeping constraints and limitations in mind. A software process is a set of operations connected by ordering constraints that, when carried out correctly and in accordance with the ordering constraints, should result in the desired output. The objective is to provide software of the highest calibre at a fair price. It is obvious that a method is unacceptable if it is unable to handle large software projects, scale up, or generate high-quality software.

Large software development firms typically have multiple processes going at once. Although many of these are unrelated to software engineering, they do affect software development. It is possible to categorise these process models as non-software engineering. This category includes training models, social process models and business process models. Though they fall beyond the purview of software engineering, these procedures have an effect on software development. A software process is the procedure that addresses the managerial and technical aspects of software development. It is obvious that developing software requires a wide range of tasks. It is preferable to consider the software process as a collection of component processes, each with a distinct type of activity, as different kinds of activities are typically carried out by different people. Even though they obviously cooperate to accomplish the overall software engineering goal, each of these component processes typically has a distinct objective in mind. A collection of principles, best practices and recommendations known as the Software Process Framework delineates high-level software engineering procedures. It makes no mention of the sequence or method by which these procedures are performed.

A software process outlines a software development methodology. On the other hand, a software project is a development endeavour that makes use of a software process. Software products are the end results of a software project. Every software development project starts with a set of specifications and is anticipated to produce software that satisfies those specifications by the end. A software process is an abstract sequence of steps that must be completed to translate user requirements into the final product. The software process can be thought of as an abstract type and every project is completed using it as an example of this type. Put another way, a process may involve multiple initiatives, each of which may result in a multitude of products.

The collection of actions and related outcomes that culminate in a software product is called a software process. These tasks are primarily completed by software engineers. All software processes have four basic process actions in common. These pursuits consist of:

- **Software Specification:** It is necessary to define the software's functionality as well as the limitations imposed on it.

Notes

- **Software Development:** It is necessary to build the program to meet the standard.
- **Software Validation:** To make sure the program accomplishes what the user desires, it needs to be verified.
- **Software Evolution:** Software must adapt to changing user requirements. These operations are organised differently and are explained in varying degrees of depth by different software processes. Both the schedule and the outcomes of the various activities differ.

To create the same kind of product, different companies could employ various procedures. Nonetheless, certain procedures are better suited for particular kinds of applications than others. The software product that is to be developed will most likely be of lower quality or less usefulness if an improper process is employed. These operations are organised differently and are explained in varying degrees of depth by different software processes. Both the schedule and the outcomes of the various activities differ. To create the same kind of product, different companies could employ various procedures. Nonetheless, certain procedures are better suited for particular kinds of applications than others. The software product that is to be developed will most likely be of lower quality or less usefulness if an improper process is employed.

A streamlined illustration of a software process given from a particular perspective is called a software process model. A software process model is an abstraction of the process it represents since models are by definition simplifications. Process models might incorporate tasks associated with software engineering personnel, software products and activities that are part of the software process.

Example: Here are a few instances of the several kinds of software process models that could be created:

A Workflow Model: This displays the order in which the process's inputs, outputs and dependencies are displayed. Human actions are represented by the activities in this model. **An Activity or Dataflow Model:** This depicts the procedure as a collection of tasks, each of which transforms data in some way. It demonstrates how an input, like a specification, gets converted into an output, like a design, during a process. Compared to the activities in a workflow model, these activities could be lower level. They could stand for human or computer-performed alterations. **An Action/Role Model:** This illustrates the responsibilities and tasks of the individuals working on the software process.

The general models or paradigms of software development vary widely and include:

- **The Waterfall Approach:** By using the aforementioned activities, this portrays them as distinct process phases, such as software design, implementation, testing and requirements definition. Each stage is "signed off" when it has been defined, at which point work moves on to the next.
- **Evolutionary Development:** The steps of specification, development and validation are interwoven in this method. From highly abstract specifications, a preliminary system is quickly created. After receiving feedback from the client, this is improved to create a system that meets their needs. After then, the system might be supplied. As an alternative, it might be reimplemented with a more methodical approach to create a system that is more reliable and manageable.
- **Formal Transformation:** This method is centred on creating a formal mathematical system definition and turning it into a program by applying mathematical techniques. Since these changes maintain "correctness," you can be certain that the created program complies with its specifications.

- **System Assembly from Reusable Components:** This methodology presupposes the existence of certain system components. Rather than creating these components from the ground up, the system development approach concentrates on their integration.

Characteristics of a Software Model

“What are the qualities that good software should have?” is the first question that every developer thinks of while designing any kind of software. Before delving into the technical aspects of any software, we would like to outline the fundamental expectations that users have. A software solution must, first and foremost, satisfy all end-user or client needs. Users The development and maintenance expenses of the program should also be kept to a minimum. The software development process must be completed in the allotted period.

These, then, were the obvious expectations for any project (recall that software development is a project in and of itself). Let's now examine the components of software quality. The Software Quality Triangle provides a clear explanation for this group of variables. Three qualities define quality application software:

- ❖ Operational Characteristics
- ❖ Transition Characteristic
- ❖ Revision Characteristics

Operational Characteristics of a Software

These elements pertain to the “exterior quality” of software and are dependent on functionality. Software has a number of operational characteristics, including:

- **Correctness:** All of the requirements specified by the client should be satisfied by the program that we are developing.
- **Usability/Learnability:** It should take less time or effort to become proficient with the software. Because of this, even those without any IT experience can easily utilise the software.
- **Integrity:** Software can have side effects, such as impairing the functionality of another application, just like medications can. However, good software shouldn't have any negative impacts.
- **Reliability:** There should be no flaws in the software product. In addition, it shouldn't malfunction while operation.
- **Efficiency:** This feature has to do with how the software makes use of the resources that are accessible. The program must utilise the storage capacity efficiently and carry out commands in accordance with the required temporal specifications.
- **Security:** These days, this component is becoming more and more important due to the rise in security risks. The hardware and data shouldn't be negatively impacted by the software. It is important to take the right precautions to protect data from outside dangers.
- **Safety:** The program shouldn't endanger lives or the environment.

Revision Characteristics of Software

These engineering-based variables, such as efficiency, documentation and structure, are related to the “interior quality” of the software. Any excellent software should have these built in. Software's various revision characteristics include:

- **Maintainability:** Any type of user should find it simple to maintain the software.

Notes

- **Flexibility:** It should be simple to make changes to the software.
- **Extensibility:** Increasing the functions it can do should be simple.
- **Scalability:** Upgrading it for additional work (or for more users) ought to be relatively simple.
- **Testability:** It ought to be simple to test the software.
- **Modularity:** It is said that all software is composed of independent parts and modules. The final software is then created by integrating these parts. Software has high modularity if it is broken up into independent, discrete components that can be tested and changed independently.

Transition Characteristics of the Software

- **Interoperability:** Software's ability to share data and use it transparently with other applications is known as interoperability.
- **Reusability:** Software is considered reusable if its code may be used for different purposes with minor alterations.
- **Portability:** Software is said to be portable if it can carry out the same tasks on multiple platforms and situations.

Each of these criteria has varying degrees of importance depending on the application.

Software Development Life Cycle (SDLC)

These are the models that support the development of the desired program. It is the comprehensive and diagrammatic visualisation of the software life cycle. It consists of all the tasks required to advance a software product through each stage of its life cycle. Stated differently, it organises the range of tasks carried out on a software product from inception to retest. Figure below illustrates the many stages of the SDLC.

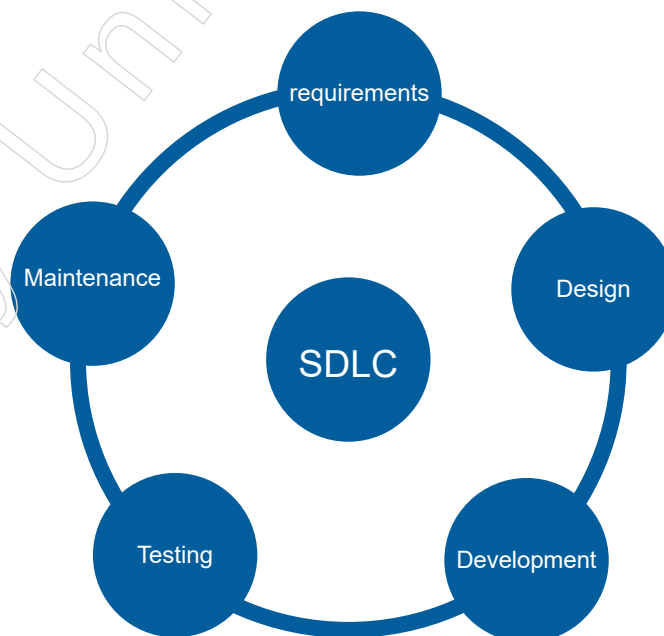


Figure: Illustrates various Software Development Life Cycle (SDLC) phases.

https://iaeme.com/MasterAdmin/Journal_uploads/IJARET/VOLUME_11_ISSUE_12/IJARET_11_12_019.pdf

Requirements: One of the most crucial stages in determining the client's need is requirement. There will be multiple review meetings to ensure that the criteria are consistent. Every review result ought to be recorded and monitored. They recommend

doing both official and informal interviews with the appropriate stakeholders of the applicant. This will make it easier for developers to understand exactly what is required of the application. Make sure to properly record these findings so that the group's retreat is cognizant of the necessity. As a result, it aids in lowering the flaws brought about by the criteria alone.

Design: The usage of case diagrams and thorough business-related design documentation is changing specifications.

Development: The development group is in charge of this phase, wherein the updated technical reviews and structural documents are inputs. Every piece of code needs to go through the team's inspection process, which includes going over the developed code and reviewing the unit's test cases before executing them.

Testing: The testing step is one of the SDLC's main validation stages. the emphasis on thoroughly testing the apps that were created using the requirements matrix.

Maintenance: To finalise and analyse the maintenance phase and organise the issues and findings under consideration, a technical analysis meeting ought to be conducted.

1.1.3 Incremental Development

The concept behind incremental development is to create a working prototype, share it with users, then iterate through multiple iterations until a workable solution is created. Activities for specification, development and validation are integrated rather than done in isolation and there is quick feedback between them all.

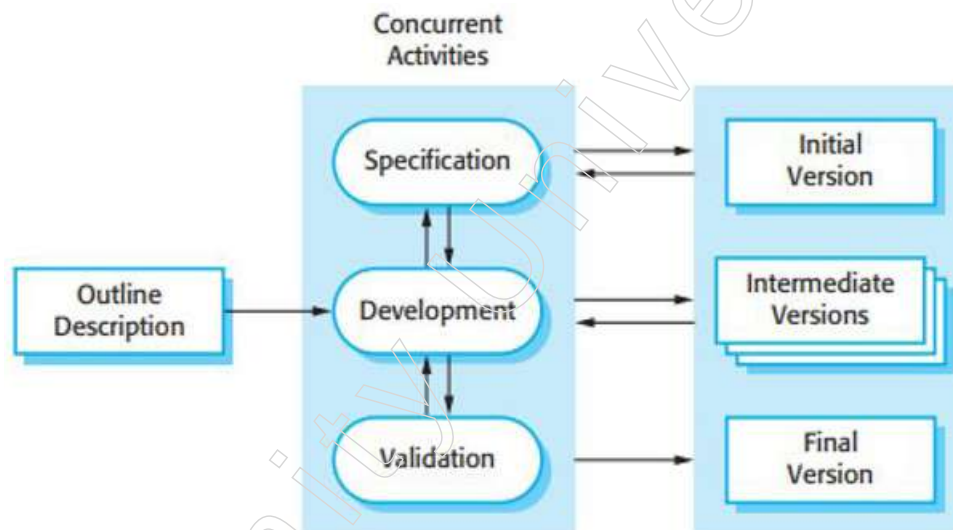


Figure: Incremental development

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

A key component of agile methodologies is incremental software development, which is superior to waterfall approaches for the majority of commercial, e-commerce and personal applications. The method we solve issues is reflected in incremental development. We rarely figure out the entire solution to an issue up front; instead, we approach a solution piecemeal and then go back when we see that we made a mistake. It is less expensive and simpler to make modifications to the program while it is being built when it is developed incrementally.

Part of the functionality required by the customer is incorporated into every system version or increment. Typically, the most crucial or urgently needed functionality is included

Notes

in the system's initial increments. This implies that the client can assess the system at a comparatively early development stage to determine whether it meets the needs. If not, then all that needs to be done is modify the current increment and maybe provide new functionality for future increments.

Comparing incremental development to the waterfall methodology reveals three key advantages:

1. It is less expensive to adapt to shifting customer needs. Comparatively speaking, substantially less analysis and documentation needs to be repeated than with the waterfall model.
2. Receiving input from customers regarding the development work completed is simpler. Consumers are able to provide feedback on software demos and observe the extent of implementation. It is challenging for customers to assess development from software design documentation.
3. Even in cases when all of the functionality has not been included, it is still possible to deliver and deploy valuable software to customers more quickly. Clients can utilise and benefit from the software more quickly than they might in a waterfall process.

Nowadays, the most popular method for developing application systems is incremental development in one form or another. This strategy can be agile, plan-driven, or, more frequently, a combination of these strategies. The system increments are predetermined in a plan-driven method; if an agile approach is used, the development of later increments is contingent upon progress and client goals, but the early increments are recognised.

The gradual method has two issues from a management standpoint:

1. It is impossible to see the process. To track their progress, managers require regular deliverables. Documents reflecting each iteration of the system are not cost-effective to prepare when systems are developed quickly.
2. With each additional increment, the structure of the system tends to deteriorate. Frequent change tends to destroy the software's structure unless time and resources are dedicated to refactoring to fix it. It gets harder and more expensive to incorporate new software updates.

When multiple teams work on separate parts of large, complex, long-term systems, the challenges associated with incremental development become especially severe. Big systems require a solid foundation or architecture and the roles of the many teams working on different components of the system must be distinctly outlined in relation to that architecture. Rather than being developed gradually, this needs to be thought out beforehand. It is possible to build a system piecemeal and get feedback from users without really delivering and implementing it in the user's environment. When software is deployed and delivered incrementally, it is incorporated into actual, functional processes. As testing out new software can interfere with regular company procedures, this isn't always feasible.

Initial software requirements are often quite well specified, but a strictly linear process is not possible due to the sheer size of the development effort. Furthermore, there can be a strong need to allow customers access to a small number of software features right away, then improve and expand on those features in upcoming software releases. In these situations, a process model built to generate the software incrementally can be selected.

The linear and parallel process flow components covered in earlier topics are combined in the incremental model. With reference to the figure below, as calendar time advances, the incremental model applies linear sequences in a staggered manner. Deliverable

“increments” of the software are generated by each linear sequence in a way that is comparable to the increments generated by an evolutionary process flow.

Word processing programs created with the incremental paradigm, for instance, might offer the following features in stages: basic file management, editing and document production in the first increment; more complex editing and document production in the second; advanced page layout in the third increment; and spelling and grammar checking in the fourth. It is important to remember that the prototyping paradigm can be included into any increment's process flow.

A core product is frequently the first increment in an incremental model. In other words, while many additional features—some known, some unknown—are still unmet, the fundamental needs are met. The consumer uses the core product (or has it thoroughly evaluated). A plan for the subsequent increment is created in response to use and/or assessment. The plan covers the development of new features and functionality as well as the modification of the core product to better suit the needs of the consumer. After each increment is delivered, this process is continued until the entire product is manufactured.

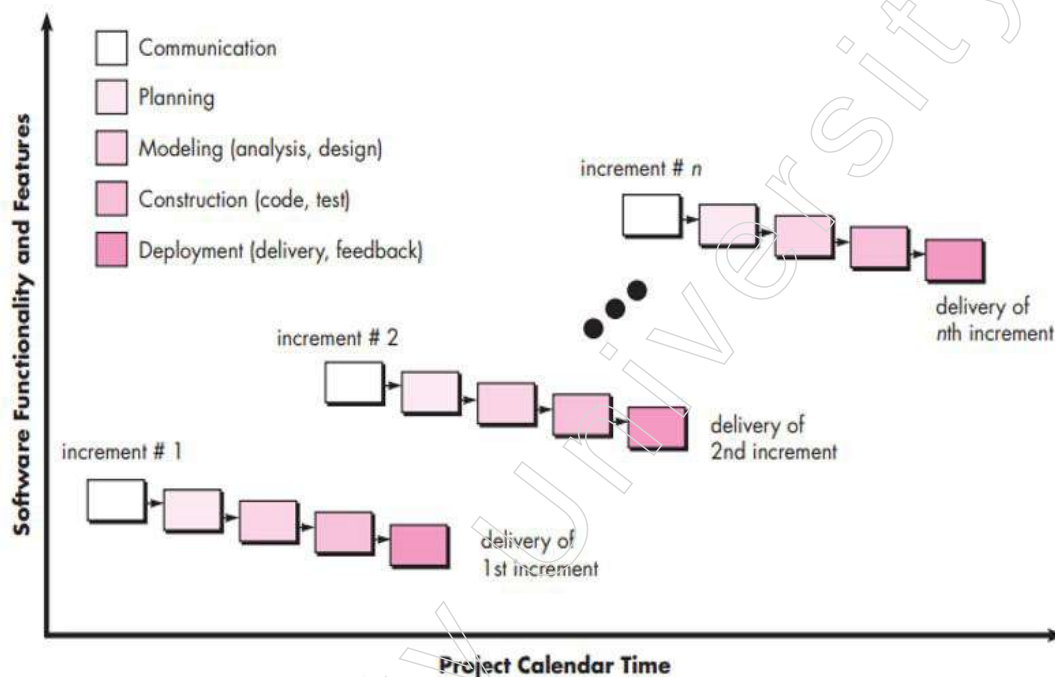


Figure: Incremental Process Models

https://www.mlsu.ac.in/econtents/16_EBOOK-7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf

A plan is created for the upcoming increment. The plan covers the development of new features and functionality as well as the modification of the core product to better suit the needs of the consumer. After each increment is delivered, this process is continued until the entire product is manufactured.

The delivery of a functioning product with each increment is the main goal of the incremental process model. Although early iterations are simplified copies of the finished product, they do include features that benefit the user and a platform for user assessment.

When staffing is not available for a full implementation by the project's set business deadline, incremental development is especially helpful. Less personnel is needed to implement early increments. If the main product is well received, more employees can be brought on board to carry out the following increment, if needed. Increments can also

Notes

be scheduled to control technical concerns. For instance, new hardware that is under development and whose delivery date is uncertain can be needed for a major system. Early increments may be able to be planned so as to avoid utilising this hardware, allowing for the prompt delivery of some functionality to end customers.

1.1.4 Spiral Model

Boehm proposed the spiral model, a paradigm for risk-driven software processes. This is depicted in the figure below. In this instance, the software process is depicted as a spiral as opposed to a list of tasks with some backtracking. Every spiral loop stands for a different stage of the software development process. As a result, the innermost loop may deal with system viability, the subsequent loop with requirements clarification, the following loop with system design and so forth. Change tolerance and change avoidance are combined in the spiral model. It makes the assumption that project risks are the cause of changes and incorporates explicit risk management techniques to lower these risks.

A risk-driven process model generator called the spiral development model is used to direct multi-stakeholder concurrent engineering of software-intensive systems. It stands out primarily for two reasons. One is a cyclical method that gradually increases the degree of definition and implementation of a system while lowering the degree of risk associated with it. The other is a series of anchor point benchmarks designed to guarantee stakeholder commitment to workable and agreeable system solutions.

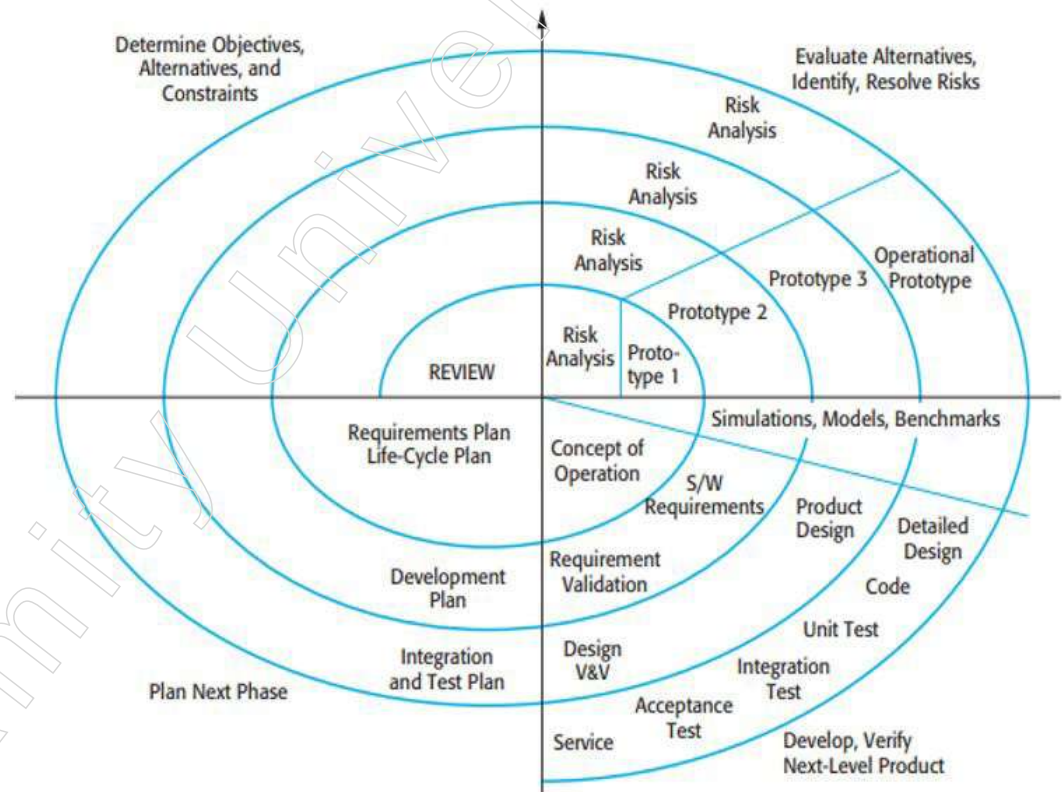


Figure: Boehm's spiral model of the software process

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

The spiral's loops are divided into four sectors:

1. Objective setting: There are set specific goals for that project phase. A thorough management strategy is created after identifying the process and product constraints.

Risks associated with the project are identified. Depending on these risks, different plans may be made for strategies.

2. Risk assessment and reduction: A thorough study is done for every project risk that has been identified. Measures are implemented to lower the risk. For example, a prototype system might be created if there's a chance the requirements aren't adequate.
3. Development and validation: A development model for the system is selected following the assessment of risks. Throwaway prototype, for instance, might be the ideal development strategy in cases where user interface hazards predominate. A development process based on formal transformations might be the best option if safety concerns are the primary concern and so on. The waterfall model might be the optimal development model to adopt if sub-system integration is the primary risk that has been identified.
4. Planning: After evaluating the project, a choice is made regarding whether to proceed with a second spiral loop. Plans are created for the project's subsequent phase in the event that it is agreed to proceed.

The spiral model's clear identification of risk sets it apart from other software process models. The spiral cycle starts with the elaboration of goals like functionality and performance. Then, some approaches to accomplishing these goals and resolving the obstacles in their path are listed. Sources of project risk are identified and each alternative is evaluated in relation to each goal. The following stage involves mitigating these risks through information-gathering exercises including simulation, prototyping and in-depth analysis.

After the risks have been evaluated, some development work is done and then the process moves on to planning the next step. Simply put, risk is the possibility of anything going wrong. One danger, for instance, is that the existing compilers may not generate object code that is sufficiently efficient, or they may be unreliable if a new programming language is to be used. Risk mitigation is a crucial component of project management since risks can result in suggested software changes as well as project issues like schedule and cost overruns.

Software is developed in a sequence of evolutionary releases using the spiral model. The release in the early stages could be a model or prototype. Later iterations result in ever-more-complete versions of the engineered system. A practical method for creating large-scale software and systems is the spiral model. Software changes as the process goes on, which helps the developer and the client recognise and respond to risks at every stage of development. The prototype approach can be applied at any point in the product's lifecycle thanks to the spiral model, which also leverages it as a risk reduction tool. It keeps the standard life cycle's methodical, step-by-step methodology while incorporating it into an iterative framework that more closely mimics the real world. When used correctly, the spiral model should lower risks before they become an issue by requiring a direct assessment of technical risks at every stage of the project.

However, the spiral model is not a cure-all, much like other paradigms. Customers may be hard to persuade that the evolutionary approach is controlled, especially in contract scenarios. It requires a high level of knowledge in risk assessment and depends on this expertise to succeed. Issues will surely arise if a significant risk is not identified and controlled.

Risk Handling in Spiral Model

Any unfavorable circumstance that could compromise the effective execution of a software project is considered a risk. The spiral model's ability to manage these unforeseen

Notes

risks once the project has begun is its most crucial component. It is easier to resolve such risks by creating a prototype.

1. By giving developers, the opportunity to create prototypes at every stage of the software development life cycle, the spiral model facilitates risk management.
2. Risk management is also supported by the prototyping model; however, risks have to be fully identified prior to the project's development activity commencing.
3. However, in practice, project risk could arise after development work begins; in such instance, the prototyping model cannot be applied.
4. Every stage of the Spiral Model involves dating and analyzing the product's attributes, as well as identifying and modelling the risks that exist at that particular moment.
5. As a result, this model has far greater flexibility than previous SDLC models.

Why Spiral Model is called Meta Model?

Because it incorporates every other SDLC model, the Spiral model is referred to as a Meta-Model. The Iterative Waterfall Model, for instance, is genuinely represented by a single loop spiral.

1. The spiral model applies the Classical Waterfall Model's step-by-step methodology.
2. As a risk-handling strategy, the spiral model builds a prototype at the beginning of each phase, adopting the Prototyping Model's methodology.
3. Additionally, it is possible to view the spiral model as a support for the evolutionary model, with each spiral iteration serving as a stage in the evolutionary process that builds the entire system.

Advantages of the Spiral Model

A few benefits of the Spiral Model are listed below.

1. Risk Handling: The Spiral Model is the ideal development model to use for projects with a lot of unknown risks that crop up during the development process since it analyses and manages risks at every stage of the process.
2. Good for large projects: For extensive and complicated undertakings, the Spiral Model is advised.
3. Flexibility in Requirements: This model allows for the accurate incorporation of change requests made at a later stage of the requirements.
4. Customer Satisfaction: Clients become accustomed to the system by using it before the full product is finished since they can observe the product's progress throughout the early stages of software development.
5. Iterative and Incremental Approach: Software development can be done incrementally and iteratively using the Spiral Model, which enables flexibility and adaptability in response to unforeseen circumstances or shifting requirements.
6. Emphasis on Risk Management: The Spiral Model emphasises risk management heavily, which lessens the effect that risk and uncertainty have on the software development process.
7. Improved Communication: Regular reviews and assessments are facilitated by the Spiral Model, which helps enhance communication between the development team and the client.
8. Improved Quality: Multiple iterations of the software development process are possible using the Spiral Model, which can lead to increased software quality and dependability.

Disadvantages of the Spiral Model

Some of the spiral model's primary drawbacks are listed below.

1. **Complex:** Compared to other SDLC models, the Spiral Model is far more sophisticated.
2. **Expensive:** Due to its high cost, the spiral model is not appropriate for minor projects.
3. **Too much dependability on Risk Analysis:** Risk analysis plays a major role in the project's successful completion. The development of a project employing this model will fail in the absence of very experienced specialists.
4. **Difficulty in time management:** Time estimation is especially challenging when the number of stages is unclear at the beginning of the project.
5. **Complexity:** Because the software development process is iterated several times, the Spiral Model can be complicated.
6. **Time-Consuming:** Because the Spiral Model necessitates numerous evaluations and reviews, it can be time-consuming.
7. **Resource Intensive:** Because the Spiral Model necessitates a large investment in planning, risk analysis and evaluations, it can be resource intensive.

The biggest problem with the cascade model is that it takes a long time to complete an item, which makes the product outdated. We have another way, called the Winding model or spiral model, to address this problem. Another name for the winding model is the cyclic model.

When to Use the Spiral Model?

1. In software engineering, a spiral model is used for large-scale projects.
2. When it's important to release something frequently, a spiral technique is used.
3. When developing a prototype makes sense
4. When weighing the costs and risks is essential
5. For projects that range in risk from moderate to high, the spiral strategy works well.
6. The spiral model of the SDLC is useful for complex and unclear requirements.
7. If changes are feasible at any time
8. when changing financial considerations make it impractical to commit to a long-term undertaking.

1.1.5 Component Model

A component model is a set of guidelines for the distribution, documentation and implementation of components. The purpose of these standards is to guarantee interoperability for component developers. They are also intended for companies who supply middleware to enable component operation and operate component execution infrastructures. Although other component models have been put out, the Webservices model, Microsoft's .NET model and Sun's Enterprise Java Beans (EJB) model are now the most significant models.

Weinreich and Sametinger describe the fundamental components of an ideal component model. These model components are summed together in the figure below. This graphic demonstrates how the components of a component model provide the component interfaces, the data required to use the component in a program and the deployment strategy for a component:

Notes

Notes

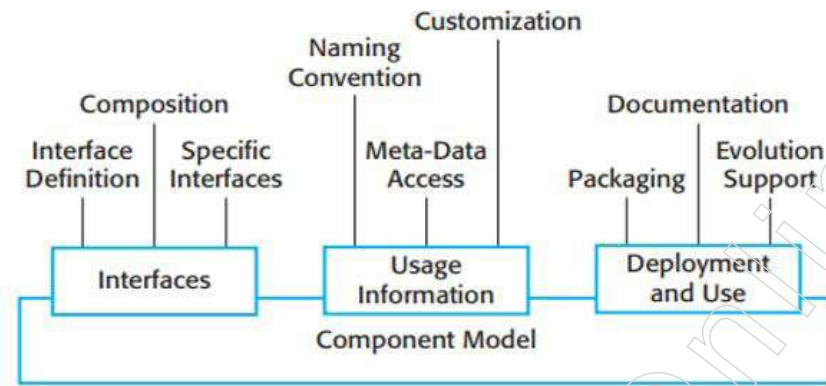


Figure: Basic elements of a component model

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

1. Points of contact Defining components involves defining their interfaces. The component model outlines the items that should be included in the interface specification, including operation names, arguments and exceptions, as well as the convention for defining interfaces. The language used to define the component interfaces should also be specified in the model. This is WSDL for web services. Java is the language used to define interfaces in EJB since it is specific to Java; in .NET, interfaces are written via the Common Intermediate Language (CIL). Certain component models demand that a component define certain interfaces. These are employed in the composition of the component with the infrastructure of the component model, which offers standardised services like transaction management and security.
2. Application Components must be assigned a distinct name or handle in order to be distributed and accessed remotely. For instance, in EJB, a hierarchical name is formed with the root based on an Internet domain name; this needs to be globally unique. Every service has its own Uniform Resource Identifier, or URI.

Data about the component itself, such as details about its interfaces and properties, is known as component meta-data. Because it enables users of the component to determine which services are needed and offered, the meta-data is crucial. Specific methods for accessing this component meta-data, like via a reflection interface in Java, are typically included in component model implementations.

Generic entities, components must be customised to fit into an application system before they can be deployed. One way to set up the Data collector component would be to specify how many sensors can be included in a sensor array. Consequently, the component model might outline how the binary components might be tailored for a certain deployment environment.

3. Deployment A specification for packaging components so they can be deployed as separate, executable entities is part of the component model. Components must be bundled with all supporting software that is not supplied by the component infrastructure or specified in a "requires" interface because they are autonomous entities. Information on a package's contents and binary organisation is included in deployment information.

It is inevitable that components will need to be modified or replaced when new requirements arise. Therefore, regulations controlling when and how component replacement is permitted may be included in the component model. Lastly, the documentation for the components that has to be created may be specified by the component model. This is used to locate the component and determine its suitability.

The component model specifies the services to be rendered by the middleware supporting the executing components for components that are implemented as program units as opposed to external services. Weinreich and Sametinger illustrate component models with an operating system example. Applications can use a set of generic services provided by an operating system. Comparable shared services for components are offered by an implementation of the component model. Some of the services that could be offered by implementing a component model are depicted in the figure below.

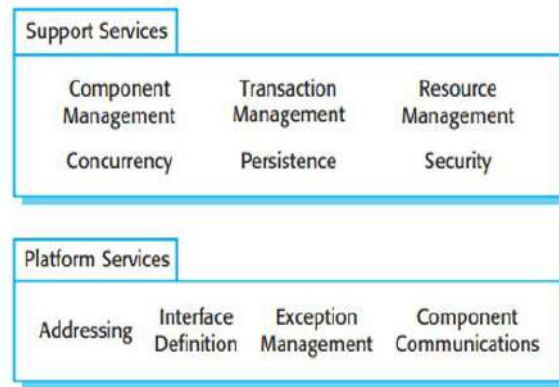


Figure: Middleware services defined in a component model

There are two types of services that a component model implementation offers:

1. Platform services, which let parts work together and communicate in a dispersed setting. All component-based systems need to have access to these essential services.
2. Common services that are probably needed by a wide range of components are known as support services. To guarantee that the user of component services is authorised, for instance, many components require authentication. It makes reasonable to give every component access to a uniform set of middleware services. By doing this, possible component incompatibilities can be prevented and component development expenses are decreased.

The middleware offers interfaces to various services and carries out the implementation of the component services. You can conceive of the components as being put in a “container” in order to utilise the services offered by an infrastructure built using the component model. A container is an application that carries out the support services together with a specification of the interfaces that an element needs to offer in order to be integrated into the container. When a component is included in a container, both the component and the container can access the component interfaces and support services. When in use, other components access the component interfaces through a container interface that calls code to access the embedded component’s interface instead of directly accessing the interfaces itself.

Containers are big and intricate and you may access all middleware services when you install a component inside of one. Simple components, however, might not require all of the features provided by the auxiliary middleware. As a result, the approach to common service offering used in web services is quite different. Program libraries have been developed to implement the standards that have been established for common online services, like security and transaction management. You only use the common services you require when constructing a service component.

Components

Within the community of CBSE, it is widely accepted that a component is a standalone

Notes

software item that can be combined with other components to form a software system. Beyond that, though, different definitions of a software component have been put forth by different persons. “A software element that conforms to a standard component model and can be independently deployed and composed without modification according to a composition standard” is how Council and Heineman describe a component.

criteria are basically the foundation of this concept, meaning that a software unit that complies with these criteria is considered a component. Szyperski, on the other hand, emphasises on the essential qualities of components rather than standards in his definition of a component:

“A software component is a composition unit that only has clear context dependencies and contractually established interfaces. A software component is subject to third-party composition and can be deployed separately.

Rather than referring to a service that the system uses, both of these definitions of a component rely their claims on the idea that a component is an element that is part of a system. They do, yet, also mesh well with the notion of a service as an element. A component does not have an externally observable state, according to Szyperski. This indicates that component copies are identical. Szyperski's concept does not apply to certain component models, such as the Enterprise Java Beans model, since they permit stateful components. While stateful components are more convenient and minimise system complexity in some systems, stateless components are undoubtedly easier to use.

The fundamental qualities of a component as they relate to CBSE are:

- **Standardised:** A component used in a CBSE process must adhere to a standard component model, which is known as component standardisation. Component interfaces, metadata, documentation, composition and deployment may all be specified by this approach.
- **Independent:** A component ought to be independent, meaning that it should be able to be assembled and used without requiring the use of other particular components. When a component requires services from outside sources, these should be specified clearly in a “requires” interface declaration.
- **Composable:** All external interactions must happen through publicly stated interfaces for a component to be considered composable. It must also provide other users access to details about it, including its characteristics and workings.
- **Deployable:** A component needs to be self-contained in order to be deployable. It needs to be able to function independently on a component platform that offers a component model implementation. This typically indicates that the component is binary and doesn't require compilation prior to deployment. A component user does not need to deploy a component if it is built as a service. Instead, the service provider deploys it.
- **Documented:** Complete documentation of components is necessary so that prospective users can determine whether or not the components fit their needs. All component interfaces should ideally have their syntax and semantics stated.

CBSE Processes

Software procedures known as CBSE processes enable component-based software engineering. They consider the potential for reuse as well as the various steps in the process that go into creating and utilising reusable components. An overview of the CBSE processes can be seen in the figure below. There are two categories of CBSE processes at the highest level:

1. Development for reuse: The goal of this procedure is to create services or components that may be utilised again in different applications. Usually, it entails extending the use of current components.
2. Development with reuse: This is the procedure for creating new apps with pre-existing parts and services.

CBSE Processes

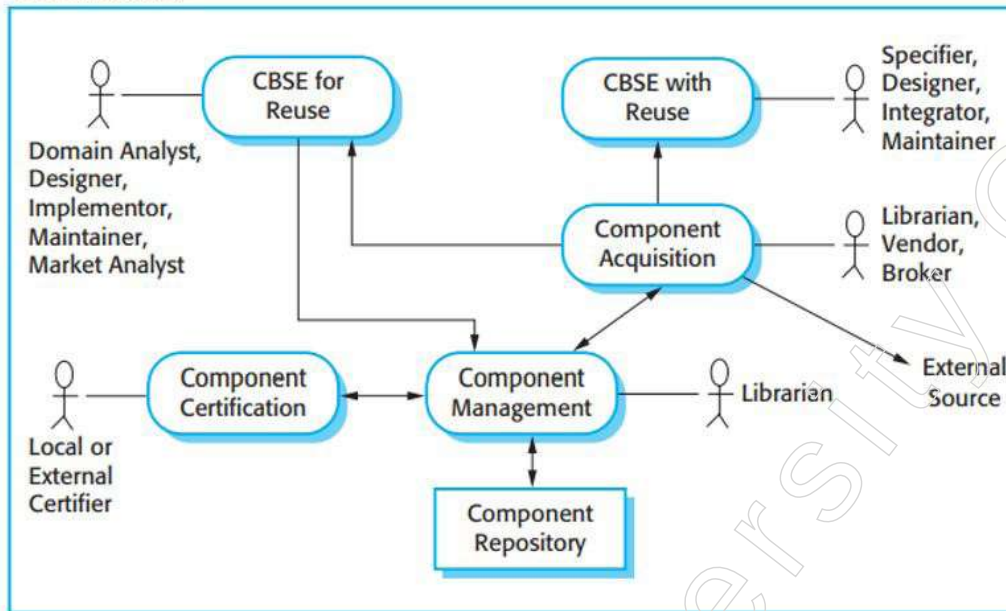


Figure: CBSE processes

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

These procedures involve various activities because they have distinct goals. The goal of the development for reuse process is to create one or more reusable parts. In order to generalise the components you will be working with, you have access to their source code and are aware of what they are. Since you don't know what components are accessible when developing using reuse, you must find them and build your system to utilise them as efficiently as possible. The source code for the component might not be available to you.

The above figure illustrates how component acquisition, component management and component certification are supported processes that go along with the fundamental CBSE procedures for reuse and with acquisition:

1. The process of obtaining components in order to develop them into reusable components is known as component acquisition. It could entail locating these components from an outside source or using locally produced services or components.
2. Managing a company's reusable components involves making sure they are appropriately categorised, stored and made available for reuse. This is known as component management.
3. The process of examining a component and attesting to its compliance with specifications is known as component certification.

Key characteristics of a component model include:

- **Encapsulation:** The implementation details of components are contained within them, with just a well defined interface available to the rest of the system. This lessens dependency and encourages the concealment of information.

Notes

- **Reusability:** The purpose of components is to enable their reuse in various scenarios. This enables developers to use pre-existing components in new projects, which encourages the development of modular and maintainable software.
- **Interoperability:** It should be possible for components to function together without any issues, irrespective of the platform, language, or environment in which they are implemented. Standards and well-defined interfaces are used to accomplish this.
- **Composability:** Larger systems can be constructed by composing or combining components. This makes it possible to put together and configure pre-existing components to create complicated applications.
- **Lifecycle Management:** The lifecycle of a component is well defined and includes its development, deployment and destruction. The correct initialisation, setup and cleanup of components are guaranteed via lifecycle management.
- **Versioning:** Versioning methods, which permit the coexistence of several versions of a component and guarantee backward and forward compatibility, are frequently included in component models.
- **Distribution:** Distributing components across various logical or physical boundaries is possible in networked systems, for example. This facilitates the creation of scalable and distributed applications.

There are numerous component models in use, some of which are as follows:

- **Component Object Model (COM):** The early 1990s saw the introduction of the Microsoft-developed COM binary interface standard for software components. It facilitates communication across processes and permits the development of components in several languages.
- **JavaBeans:** A Java component model that outlines reusable Java programming language components. Programming graphical user interfaces (GUIs) frequently makes use of JavaBeans.
- **Enterprise JavaBeans (EJB):** A Java component approach for developing distributed enterprise apps. Services like persistence, security and transaction management are offered by EJB components.
- **NET Framework:** The Common Language Runtime (CLR) is the foundation for the component model of the Microsoft.NET Framework. It facilitates the creation and implementation of language-neutral reusable components.

1.1.6 Agile Software Development

Agile software engineering is a development methodology that blends a philosophy with specific criteria. Small, highly motivated project teams, informal methodologies, minimum software engineering work items, early incremental software delivery, customer happiness and overall development simplicity are all encouraged by this philosophy. The development rules prioritise active and ongoing contact between developers and clients over analysis and design, however both tasks are still encouraged.

The term “agile” is now frequently used to characterise contemporary development processes. Agile is everyone. A team that is agile is quick to adapt and can react to changes in the environment. Software development is primarily about change. Modifications to the software being developed, adjustments made to the team members, modifications brought about by new technologies and modifications of any kind that might affect the project or product they are building. Since support for modifications is the essence of software, it needs to be incorporated into everything we do. An agile team understands that individuals

working in teams create software and that the success of the project depends heavily on these individuals' abilities to interact.

Many believed that meticulous project planning, formalised quality assurance, the use of analysis and design techniques supported by CASE tools and rigorous and controlled software development processes were the best ways to produce better software in the 1980s and early 1990s. The community of software engineers, who created huge, durable software systems like those for the government and aerospace industry, held this point of view.

Large teams of developers from various companies worked on this program. Teams worked on the program for extended periods of time and were frequently spread out geographically. The control systems of a contemporary aeroplane are an example of this kind of software; from original specification to deployment, it can take up to ten years. These plan-driven methods have a substantial system design, planning and documentation overhead. When several development teams must coordinate their efforts, the system must be considered vital and a wide range of individuals will be involved in the software's lifetime maintenance, this overhead becomes justifiable.

Nevertheless, the overhead associated with applying this rigid, plan-driven development method to small and medium-sized commercial systems is so great that it takes over the software development process. The development of the system takes more time than the creation and testing of programs. Rework is necessary when the requirements for the system change and the specification and design should, in theory, adapt to the program as well.

Several software developers proposed new "agile methods" in the 1990s as a result of their dissatisfaction with these antiquated approaches to software creation. As a result, the development team was able to concentrate on the software itself instead of the design and documentation. An incremental approach to software concept, development and delivery is a fundamental component of all agile techniques. They work best in application development, since system requirements frequently change quickly while the project is being developed. Their goal is to provide clients with functional software as soon as possible, allowing them to suggest modifications and additions to be incorporated into upcoming system releases. By avoiding labour with questionable long-term value and getting rid of paperwork that is likely never going to be used, they want to reduce the amount of bureaucracy associated with processes.

The agile manifesto, which was adopted by many of the top creators of these techniques, reflects the philosophy underlying agile methodologies. We are discovering better ways to develop software by doing it ourselves and by assisting others in doing so, according to this manifesto. As a result of this work, we now appreciate:

People and their interactions with procedures and equipment Functional software as opposed to thorough documentation Client cooperation as opposed to contract drafting Adapting to change as opposed to sticking to a plan in other words, even though the things on the right have worth, we place a higher value on the things on the left.

Extreme programming is arguably the most well-known agile methodology. Scrum, Crystal, Adaptive Software Development, (DSDM) and Feature Driven Development are further agile methodologies. Because of these techniques' effectiveness, there has been some integration with conventional system modeling-based development techniques, giving rise to the concepts of agile modelling and agile RUPs.

While incremental development and delivery is the foundation of all these agile methodologies, they suggest different ways to get there. But they have a lot in common

Notes

Notes

since they adhere to the same set of values, which are derived from the agile manifesto. The following illustrates these ideas.

For several kinds of system development, agile methodologies have proven to be particularly effective:

- Product development is the process by which a software business creates a small- to medium-sized product that will be sold.
- Custom system development within an organisation, where the customer has made a clear commitment to participate in the development process and where the software is not subject to numerous external laws and regulations.

Agile Principle

- Customer involvement: Clients ought to be actively involved in the entire development process. Their responsibilities include generating and ranking new system requirements and assessing system iterations.
- Incremental delivery: The customer specifies the needs to be incorporated in each increment of the software, which is created incrementally.
- People not process: Recognising and utilising the development team's skills is important. Without imposing rigid procedures, team members ought to be free to create their own methods of working together.
- Embrace change: Anticipate that the requirements for the system will evolve and build the system accordingly.
- Maintain simplicity: In both the software being built and the development process, simplicity should be prioritised. Make a concerted effort to remove as much complexity as possible from the system.

It can occasionally be challenging to put the guiding principles of agile methodologies into reality in practice:

- While it sounds good in theory, having a customer who can represent all system stakeholders and is willing and able to spend time with the development team is essential to the success of this approach. Customer service agents frequently face various demands and are unable to fully participate in the software development process.
- It's possible that certain team members won't get along with other members since their personalities aren't suited for the rigorous commitment that agile methods require.
- Setting change priorities may be very challenging, particularly in systems with a large number of stakeholders. Generally, every stakeholder assigns varying priorities to distinct modifications.
- Sustaining simplicity takes more effort. The team members might not have enough time due to delivery schedule pressure to implement the desired system simplifications.
- A lot of businesses, particularly big ones, have spent years transforming their cultures to ensure that protocols are established and adhered to. They find it challenging to switch to a working model where development teams define informal processes.

Utilising an outside organisation for system development is another non-technical issue, or more broadly, an issue with incremental development and delivery. Typically, the software requirements document is a component of the supplier-customer contract. Writing contracts for this kind of development may be challenging since agile approaches are inherently based on incremental specification. Therefore, rather than developing a

set of requirements, agile approaches must rely on contracts where the customer pays for the time needed to construct the system. Both the client and the developer gain from this, assuming everything goes according to plan. If issues do occur, though, there can be contentious disagreements about who is to blame and who should foot the bill for the additional time and materials needed to fix the issues.

Notes

Plan-driven and Agile Development

Design and implementation are viewed as the primary steps in the software development process by agile techniques. They integrate testing and requirements elicitation with other processes like design and execution. A plan-driven approach to software engineering, on the other hand, distinguishes distinct phases in the development process and assigns outputs to each one. The next process activity is planned using the outcomes from the previous stage. The differences between plan-driven and agile methods to system specification are depicted in the figure below.

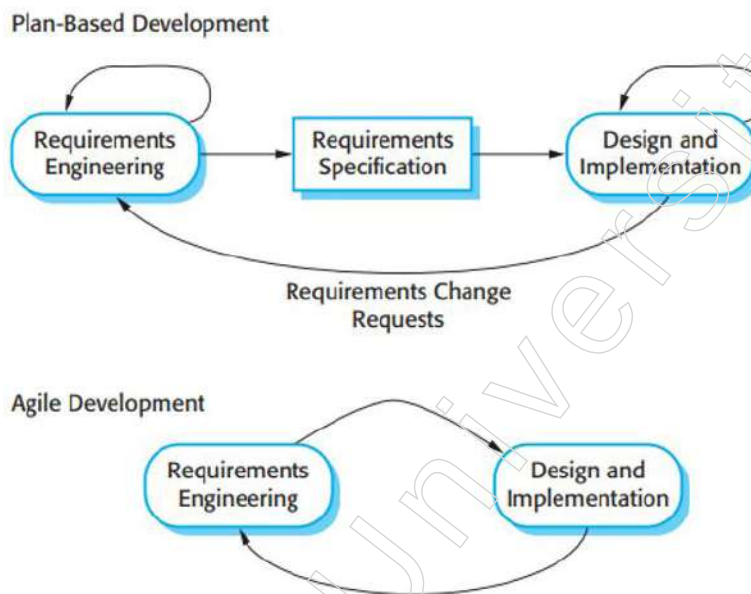


Figure: Plan-driven and agile specification

Iteration takes place within activities in a plan-driven method and formal documentation are utilised to communicate between process phases. For instance, as the needs develop, a requirements specification will be created in the end. The process of design and implementation then uses this as an input. Iteration happens across activities when using an agile methodology. As a result, rather than being developed independently, the requirements and the design are produced together.

Incremental development and delivery can be supported by a plan-driven software process. Assigning requirements and organising the design and development phase as a sequence of steps is entirely doable. The output of an agile process may include some design documentation but is not always code-focused. The agile development team may choose to implement a documentation “spike,” in which case the team produces system documentation as opposed to a new version of the system.

Human Factors

The importance of “people factors” is emphasised by agile software development proponents at great length. “Agile development focuses on the talents and skills of individuals, moulding the process to specific people and teams,” as stated by Cockburn and

Notes

Highsmith. This statement's main idea is that the team and people's requirements are met by the process, not the other way around.

Several essential characteristics must be present in both the members of an agile team and the team itself if the members of the software team are to drive the features of the process used to produce software:

Competence. "competence" in the context of agile development (and software engineering) refers to a combination of natural aptitude, specialised software-related abilities and general understanding of the method that the team has decided to use. All members of agile teams can and should be trained in process knowledge and skill.

Common focus. The aim of the agile team should be to provide a functional software increment to the client in the period given, even though team members may do diverse activities and contribute different expertise to the project. The team will also concentrate on ongoing modifications, both minor and major, to the process to better suit the needs of the group in order to accomplish this aim.

Collaboration. Regardless of process, software engineering is about evaluating, analysing and applying information that is shared with the software team; producing information that will aid in the understanding of the team's work by all parties involved; and developing information (computer software and pertinent databases) that offers the client business value. Team members must work together—with all other stakeholders as well as with one another—to complete these tasks.

Decision-making ability. Allowing a good software team, even an agile team, to take charge of its own future is essential. This suggests that autonomy—the ability to make decisions on technical and project-related matters—is granted to the team.

Fuzzy problem-solving ability. Software managers have to understand that the agile team will always be dealing with uncertainty and being thrown off guard by change. Sometimes the team has to acknowledge that the issue they are working on now might not be the one that needs to be resolved tomorrow. Nonetheless, the team may profit later in the project from the lessons learnt from any issue-solving exercise, even if it involves solving the incorrect problem.

Mutual trust and respect. The agile team needs to develop into a "jelled" team, as defined by DeMarco and Lister. When a team is cohesive, they demonstrate the mutual respect and trust that is required to become "so strongly knit that the whole is greater than the sum of the parts."

Self-management. Three things are implied by self-organisation in the context of agile development:

1. The agile team sets itself up for the tasks at hand,
2. Tailors the workflow to the specifics of its surroundings and
3. Plans its work in a way that maximises the delivery of software updates.

Although self-organisation has many technological advantages, its main advantages are enhanced teamwork and morale. The group functions as its own management, in a sense. When he writes, "The team commits to the work and decides how much work it believes it can complete within the iteration," Ken Schwaber addresses these problems. Nothing saps a team's motivation more than when someone else makes promises on its behalf. Nothing spurs a team on more than taking ownership of keeping the promises it made to itself.

Extreme Programming

Among the agile techniques, extreme programming (XP) is arguably the most popular and widely applied. Beck came up with the name since the methodology was established by taking accepted best practices—like iterative development—and pushing them to “extreme” lengths. For instance, with Windows XP, multiple new system versions might be created by various programmers, integrated and tested in a single day.

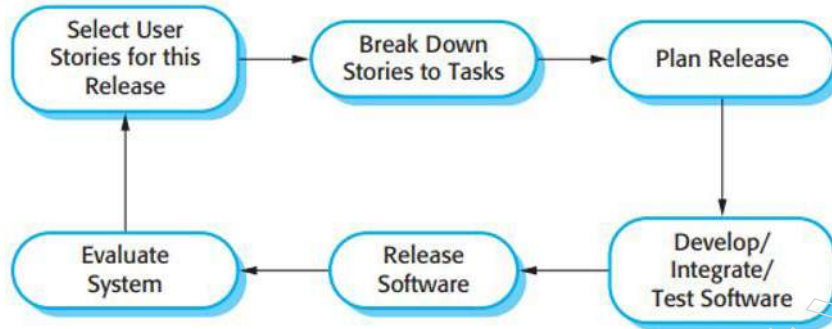


Figure: The extreme programming release cycle

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

Extreme programming expresses needs as user stories, or scenarios, which are then immediately implemented as a set of activities. Before writing the code, programmers create tests for every task in pairs. Every test has to run properly when new code is added to the system. The interval between system releases is brief. The XP procedure used to create an increment of the system under development is shown in the above figure.

There are several practices involved in extreme programming, including:

- **Incremental planning:** The Stories to be included in a release are chosen based on their relative priority and the time available, with requirements documented on Story Cards. These Stories are divided into development “Tasks” by the developers.
- **Small releases:** First, the most basic functional set that adds value to the business is built. The system is frequently released, with each release adding features one at a time.
- **Simple design:** Just enough design is done to satisfy the requirements as they stand right now.
- **Test-first development:** Before a new piece of functionality is implemented, tests are written for it using an automated unit test framework.
- **Refactoring:** It is expected of all developers to regularly refactor the code whenever feasible code enhancements are discovered. As a result, the code is clear and easy to maintain.
- **Pair programming:** Developers collaborate in pairs, reviewing each other’s work and provide encouragement to consistently perform well.
- **Collective ownership:** To prevent the emergence of isolated areas of expertise, the pairs of developers work on every aspect of the system and each developer is accountable for the entirety of the code. Everything is changeable by anyone.
- **Continuous integration:** A task is integrated into the entire system as soon as it is finished. Following any such integration, the system’s unit tests have to all pass.
- **Sustainable pace:** Excessive overtime is deemed unacceptable since it frequently lowers productivity in the medium term and the quality of the code.

Notes

Notes

- On-site customer: A full-time representative of the system's end-user, or the Customer, ought to be accessible to the XP team. The client is a part of the development team in an extreme programming process and they are in charge of providing system needs to the team for implementation.

Agile techniques' guiding principles are reflected in extreme programming:

1. The system is frequently and progressively released in tiny steps to facilitate incremental development. Simple client stories or scenarios serve as the foundation for requirements, which are then used to determine what functionality should be added to a system increment.
2. The ongoing participation of the client in the development team fosters customer involvement. In addition to participating in development, the client representative creates the system's acceptance tests.
3. Pair programming, group ownership of the system code and a sustainable development approach that does not need unduly long working hours are ways to help people, not processes.
4. Regular customer system releases, test-first development, code degeneration prevention refactoring and continuous integration of new features are all ways that change is welcomed.
5. Refactoring continuously to enhance the quality of the code and employing straightforward designs that do not needlessly predict future system modifications are two ways to maintain simplicity.

Agile Software Development represents a significant departure from traditional software engineering methodologies, but it can be implemented within both traditional and advanced software engineering contexts. Let's examine how Agile is approached in each:

Agile in Traditional Software Engineering:

- In traditional software engineering, Agile methods like Scrum, Kanban, or Extreme Programming (XP) may be adopted as an alternative to the Waterfall model.
- Agile practices such as iterative development, frequent collaboration, and adaptability to change are integrated into the existing development processes.
- Teams may face challenges in fully embracing Agile principles due to organizational hierarchies, rigid structures, and resistance to change.
- Agile adoption might be limited to certain projects or teams within the organization rather than being embraced organization-wide.

Agile in Advanced Software Engineering:

- In advanced software engineering environments, Agile methodologies are often the default approach to software development.
- Teams are more likely to fully embrace Agile principles such as self-organizing teams, continuous delivery, and responding to change over following a plan.
- Advanced software engineering practices are built upon Agile principles, with a focus on maximizing collaboration, delivering value iteratively, and adapting quickly to customer feedback.
- Agile practices are deeply ingrained in the organizational culture, with support from leadership and a commitment to continuous improvement.

In both traditional and advanced software engineering contexts, Agile methodologies offer benefits such as improved responsiveness to change, enhanced collaboration, faster

delivery of working software, and increased customer satisfaction. However, the degree to which Agile is adopted and integrated into the development process may vary depending on the organization's culture, structure, and level of maturity in software engineering practices.

Notes

Other Agile Process Models

Numerous outdated process descriptions and approaches, modelling techniques and notations, tools and technologies may be found throughout the history of software engineering. Each saw a brief period of fame before being surpassed by something fresh and (supposedly) superior. The agile movement is pursuing the same historical route with the advent of a diverse range of agile process models, all vying for recognition within the software development community.

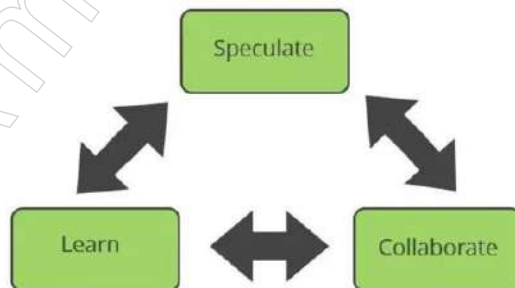
As mentioned in the previous topics, Extreme Programming (XP) is the most popular agile process model. However, a wide range of additional agile process models have been put out and are in use in the sector. Those that are most typical are:

- Adaptive Software Development (ASD)
- Scrum
- Dynamic Systems Development Method (DSDM)
- Crystal
- Feature Drive Development (FDD)
- Lean Software Development (LSD)
- Agile Modeling (AM)
- Agile Unified Process (AUP)

Adaptive Software Development (ASD)

Jim Highsmith introduced the concept of Adaptive Software Development (ASD) as a method for creating intricate software and systems. ASD is based on a philosophy that emphasises human cooperation and group self-organisation. Agile, adaptable development methods built on teamwork, according to Highsmith, are "just as much a source of order in our complex interactions as discipline and engineering." He describes a "life cycle" for ASD that includes three stages: learning, collaboration and conjecture.

The project is started and adaptive cycle planning is done during conjecture. Adaptive cycle planning defines the set of release cycles (software increments) that will be needed for the project using information from the project start, such as the customer's mission statement, project restrictions (such as delivery deadlines or user descriptions) and fundamental needs.



Regardless of how comprehensive and long-term the cycle plan is, it will inevitably alter. The plan is evaluated and modified in light of the data gathered at the end of the first cycle to better align the planned activities with the working environment of an ASD team. Collaboration is a tool that motivated individuals utilise to multiply their creativity and talent

Notes

beyond their absolute numbers. This methodology is a common thread throughout all agile techniques. However, working together is not always simple. It includes cooperation and communication, but it also places a strong emphasis on individualism since creative thinking requires individual inventiveness. Above all, everything comes down to trust. Collaborating individuals need to have faith in one another to: (1) provide constructive criticism without bias; (2) lend a helping hand without harbouring grudges; (3) exert equal or greater effort than they do; (4) possess the necessary skills to contribute to the task at hand; and (5) effectively convey issues or worries to one another.

When members of an ASD team start assembling the elements of an adaptive cycle, “learning” is prioritised just as much as moving the cycle closer to completion. As a matter of fact, Highsmith contends that education will enable software engineers to have a deeper understanding of the technology, process and project than they already possess. Three methods are used by ASD teams to learn: project postmortems, technical reviews and focus groups. Regardless of the process paradigm that is employed, the ASD philosophy provides value. Software project teams that apply ASD’s general emphasis on the dynamics of self-organising teams, interpersonal collaboration and individual and team learning are far more likely to succeed.

Scrum

Jeff Sutherland and his development team created the agile software development methodology known as Scrum in the early 1990s. The name of the process comes from an activity that takes place during a rugby match¹³. Schwaber and Beedle have been working to improve the Scrum techniques in recent years.

Scrum principles are used to development activities inside an integrated framework that includes requirements, analysis, design, evolution and delivery. They align with the agile manifesto. Tasks inside each framework activity take place in a pattern of work known as a sprint. The number of sprints needed for each framework activity will vary based on the complexity and size of the product. The work completed within a sprint is tailored to the current problem and is specified and frequently adjusted in real time by the Scrum team. The figure below shows how the Scrum method works overall.

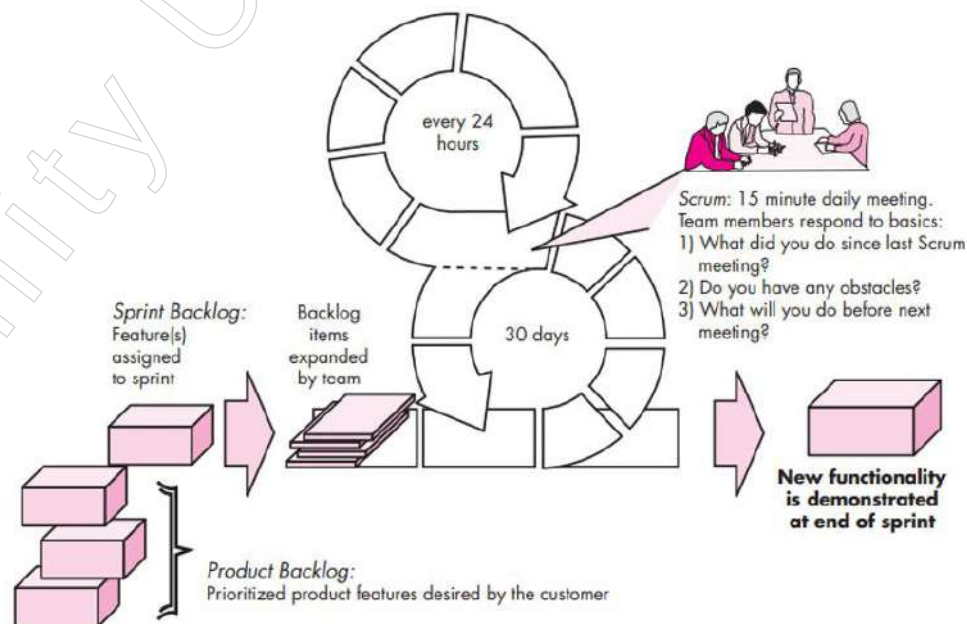


Figure: Scrum process flow

https://www.mlsu.ac.in/econtents/16_EBOOK-7th_ed_software_engineering_a_practitioners_approach_by_roger_s_pressman_.pdf

Scrum places a strong emphasis on applying a collection of software process patterns that have worked well for projects with constrained budgets, dynamic requirements and important business outcomes. A collection of development actions is defined by each of these process patterns:

- Backlog—an ordered list of the features or requirements for the project that provide value to the customer's business. The backlog is open to new items at any moment. The product manager evaluates the backlog and modifies the order of importance as needed.
- Sprints—comprise work units necessary to fulfil a backlog need that must be completed in a predetermined time-box¹⁴ (usually 30 days).
- Changes (for example, backlog tasks) are not added throughout the sprint. As a result, the sprint gives team members the chance to collaborate in a stable, temporary setting.
- Scrum meetings—are brief (usually lasting 15 minutes) meetings that the Scrum team holds every day. Every team member asks and responds to these three important questions:
 - ❖ Ever since the last team meeting, what have you done?
 - ❖ What challenges are you facing?
 - ❖ What are your goals for the upcoming team meeting?

The meeting is facilitated by a team leader known as a Scrum master who evaluates each participant's response. The team can identify any issues as soon as possible with the help of the Scrum meeting. Additionally, by fostering "knowledge socialisation," these daily sessions help the team structure to become self-organising.

Deliver the software increment to the client so they can see and test the implemented functionality. This is known as a demo. It's crucial to remember that the demo might only include features that can be delivered within the predetermined time frame, rather than all of the expected capability.

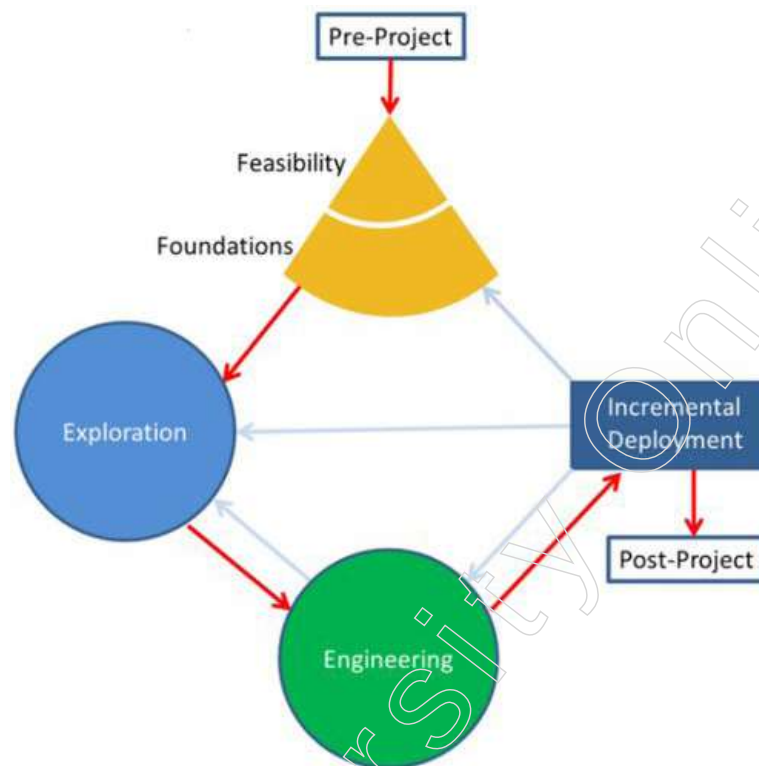
In their thorough analysis of these patterns, Beedle and his associates state that "Scrum assumes up-front the existence of chaos." The Scrum process patterns allow a software team to function effectively in a world where uncertainty cannot be completely eliminated.

Dynamic Systems Development Method (DSDM)

Using incremental prototyping in a supervised project environment, the Dynamic Systems Development Method (DSDM) is an agile software development methodology that "provides a framework for building and maintaining systems which meet tight time constraints." The Pareto principle, which states that 80% of an application may be delivered in 20% of the time needed to deliver the full (100%), can be adjusted to inform the DSDM concept.

Every iteration of the DSDM software process adheres to the 80 percent guideline. In other words, each increment just needs to take enough labour to make it easier to move on to the next. When further business requirements are discovered or adjustments have been requested and granted, the remaining details can be finished at a later time.

Notes



A global network of participating businesses known as the DSDM Consortium (www.dsdm.org) assumes the responsibility of being the “keeper” of the technique. The DSDM life cycle, an agile process model developed by the consortium, consists of three distinct iterative cycles that are preceded by two additional life cycle activities:

- Feasibility study—determines the fundamental business requirements and limitations related to the application that needs to be developed, after which it determines if the application is a good fit for the DSDM process.
- Business study—outlines the fundamental application architecture and specifies the maintainability criteria for the application. It also establishes the functional and information requirements necessary for the application to deliver business value.
- Functional model iteration—creates a series of iterative prototypes that show the consumer how the product works. (Note: The deliverable application is meant to evolve from all DSDM prototypes.) In this iteration cycle, user feedback is solicited while the prototype is being used in order to acquire more needs.
- Design and build iteration—reexamines the prototypes created during the functional model iteration to make that each one has been designed so end users can utilise it to generate operational business value. There are situations where iterations on the functional model and the design and build happen at the same time.
- Implementation—introduces the most recent software update—a “operationalised” prototype—into the working environment. It should be remembered that (1) the increment might not be finished completely or (2) adjustments might be needed when the increment is implemented. Either way, the functional model iteration activity is resumed as part of the ongoing DSDM development process.

When DSDM and XP are coupled, a combination method is created that combines the foundational techniques (XP) needed to build software increments with a robust process model (the DSDM life cycle). Additionally, a unified process model can be developed using the ASD principles of self-organising teams and cooperation.

Crystal

In order to achieve a software development approach that places a premium on “manoeuvrability” during what Cockburn describes as “a resource limited, cooperative game of invention and communication, with a primary goal of delivering useful, working software and a secondary goal of setting up for the next game,” he and Jim Highsmith created the Crystal family of agile methods¹⁵.

Cockburn and Highsmith have identified a collection of methods that each have roles, work products, process patterns and practices that are specific to them, but also essential aspects that are shared by all of them in order to attain manoeuvrability. The Crystal family comprises of a collection of exemplary agile processes that have demonstrated efficacy in many project scenarios. The idea is to give agile teams the freedom to choose the crystal family member best suited to their project and surroundings.

How does Crystal function?

Till now, we got to know that crystal is a family of various developmental approaches, and it is not a group of prescribed developmental tools and methods. In the beginning, the approach is set by considering the business requirements and the needs of the project. Various methodologies in the Crystal family also known as weights of the Crystal approach are represented by different colors of the spectrum.

Crystal family consists of many variants like Crystal Clear, Crystal Yellow, Crystal Red, Crystal Sapphire, Crystal Red, Crystal Orange Web, and Crystal Diamond.

- Crystal Clear- The team consists of only 1-6 members that is suitable for short-term projects where members work out in a single workspace.
- Crystal Yellow- It has a small team size of 7-20 members, where feedback is taken from Real Users. This variant involves automated testing which resolves bugs faster and reduces the use of too much documentation.
- Crystal Orange- It has a team size of 21-40 members, where the team is split according to their functional skills. Here the project generally lasts for 1-2 years and the release is required every 3 to 4 months.
- Crystal Orange Web- It has also a team size of 21-40 members where the projects that have a continually evolving code base that is being used by the public. It is also similar to Crystal Orange but here they do not deal with a single project but a series of initiatives that required programming.
- Crystal Red- The software development is led by 40-80 members where the teams can be formed and divided according to requirements.
- Crystal Maroon- It involves large-sized projects where the team size is 80-200 members and where methods are different and as per the requirement of the software.
- Crystal Diamond & Sapphire- This variant is used in large projects where there is a potential risk to human life.



Figure: Crystal Family (team members)

Notes

Notes

Feature Driven Development (FDD)

Peter Coad and his associates first proposed feature-driven development (FDD) as a workable process model for object-oriented software engineering. Coad's approach has been expanded upon and enhanced by Stephen Palmer and John Felsing, who have developed an agile, adaptable process that can be used for both smaller and bigger software projects.

Similar to other agile methodologies, feature-based decomposition (FDD) follows a philosophy that prioritises teamwork among members; (2) uses feature-based decomposition to manage project and problem complexity, followed by the integration of software increments; and (3) uses verbal, graphical and text-based methods to communicate technical details.

By promoting an incremental development approach, the use of design and code inspections, the use of software quality assurance audit, the gathering of metrics and the application of patterns (for analysis, design and construction), FDD places a strong emphasis on software quality assurance activities.



A feature is defined as “a client-valued function that can be implemented in two weeks or less” in the context of FDD. The following advantages result from the focus on feature definition:

- Users can better describe features, understand their relationships and check them for ambiguity, errors, or omissions since features are compact pieces of deliverable capability.
- Features can be arranged in a hierarchical grouping relevant to business.
- The FDD deliverable software increment is a feature, therefore every two weeks, the team builds functional features.
- Features are minimal, making it easy to efficiently inspect their design and code representations.
- Rather of being determined by an arbitrary software engineering job set, project planning, scheduling and tracking are guided by the feature hierarchy.

Lean Software Development (LSD)

The concepts of lean manufacturing have been applied to the field of software engineering through Lean Software Development (LSD). The LSD method is inspired by lean principles, which can be summed up as follows: remove waste, integrate quality, generate knowledge, postpone commitment, deliver quickly, respect people and optimise the entire.



It is possible to apply each of these ideas to the software process. In the context of an agile software project, for instance, “eliminate waste” can mean one of the following: (1) adding no unnecessary features or functions; (2) evaluating the impact on cost and schedule of any newly requested requirement; (3) eliminating any unnecessary process steps; (4) putting in place mechanisms to enhance team members’ information-finding abilities; (5) making sure testing catches as many errors as possible; (6) cutting down on the time needed to request and receive decisions that have an impact on the software or the process used to create it; and (7) streamlining the way information is communicated to all parties involved in the process.

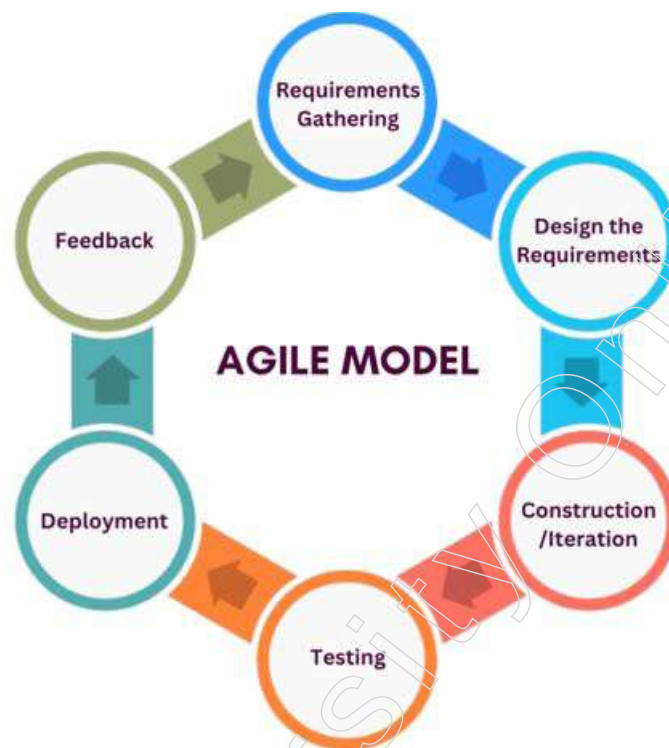
Agile Modeling (AM)

Software engineers are frequently required to develop massive, mission-critical systems. Such systems need to have their scope and complexity modelled in order to: (1) improve understanding of the tasks at hand among all stakeholders; (2) divide the problem among the necessary parties in an efficient manner; and (3) monitor system quality during the engineering and construction phases.

Many modelling techniques and notations for software engineering have been presented during the past 30 years for analysis and design (both architectural and component-level). Although these approaches have their advantages, they have shown to be hard to implement and maintain (across numerous projects). The “weight” of various modelling techniques is one aspect of the issue.

This refers to the amount of notation needed, the level of formalism advised, the size of the models for large projects and the challenge of keeping the model or models up to date when things change. However, enormous projects benefit much from analysis and design modelling, if only because they help to make the tasks more cognitively doable. Exists a flexible method for modelling software engineering that could serve as a substitute?

Notes



“Agile Modelling (AM) is a practice-based methodology for effective modelling and documentation of software-based systems,” according to Scott Ambler at “The Official Agile Modelling Site.” In a nutshell, Agile Modelling (AM) is a set of ideals, guidelines and software modelling techniques that may be easily and successfully used on a software development project. Agile models don’t have to be flawless; they might be merely marginally better than traditional models, which makes them more effective.

All of the values that align with the agile manifesto are embraced by agile modelling. Agile modelling acknowledges that an agile team needs to be brave enough to decide whether to reject a design and refactor it. The group must also possess the humility to acknowledge that business experts and other stakeholders should be respected and welcomed and that technologists do not have all the solutions.

While AM offers many “core” and “supplementary” modelling ideas, the following are the ones that set AM apart:

- Model with a purpose. When developing a model with AM, a developer should have a clear objective in mind (such as informing the client or assisting in a better understanding of a certain feature of the program). The kind of notation to be used and the degree of detail needed will become clearer once the model’s objective has been established.

- Use multiple models. Software can be described using a wide variety of models and notations. For the majority of undertakings, only a tiny portion is necessary. According to AM, each model should highlight a distinct facet of the system in order to provide the necessary information and only models that benefit the target audience should be employed.

- Travel light. As software engineering projects progress, save the models that will yield long-term benefits and discard the others. As modifications are made, every work product that is retained needs to be updated. This is an example of labour that causes the team to lag. According to Ambler, “every time you choose to stick with a model, you forfeit flexibility in exchange for the ease of having that information accessible to your team in an abstract way (potentially improving communication both within and between project stakeholders).”

Content is more important than representation. Information should be conveyed to the target audience through modelling. A model with excellent syntactic structure but little usable content is not as valuable as a model with imperfect notation but still offering the audience useful content.

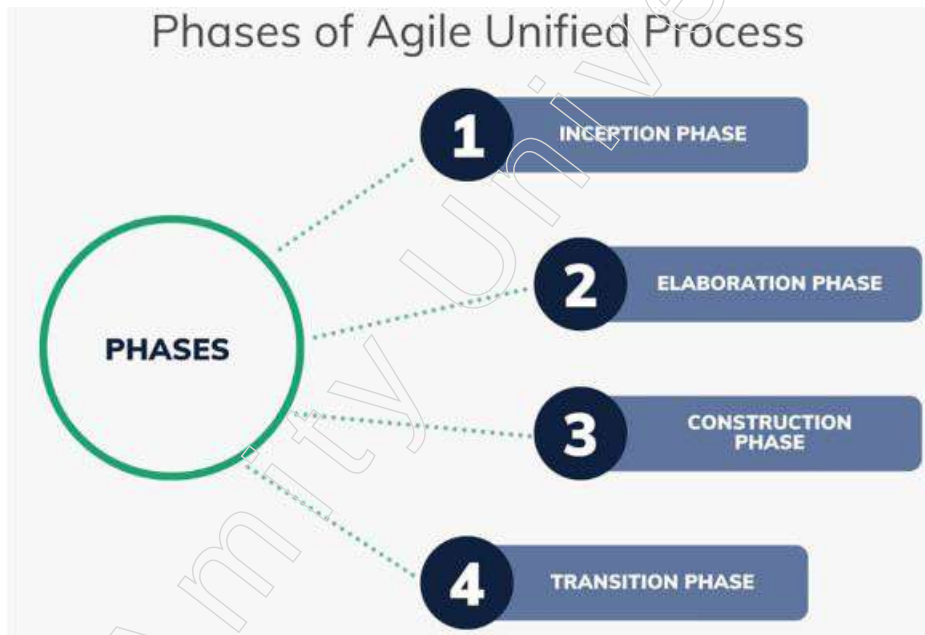
Know the models and the tools you use to create them. Recognise the advantages and disadvantages of every model as well as the instruments utilised in its creation.

Adapt locally. The modelling strategy ought to be modified to meet the requirements of the agile group.

The Unified Modelling Language (UML) has been widely accepted as the standard approach for representing analysis and design models within the software engineering community. The purpose of the Unified Process is to offer a framework for using UML. A condensed version of the UP including Scott Ambler's agile modelling methodology has been created.

Agile Unified Process (AUP)

When developing computer-based systems, the Agile Unified Process (AUP) uses the “serial in the large” and “iterative in the small” approaches. An AUP team can see the whole process flow for a software project by using the classic UP phased activities (inception, elaboration, construction and transition). This allows AUP to give a serial overlay, or a linear succession of software engineering activities. To attain agility and provide valuable software increments to end users as quickly as feasible, the team iterates within each activity. Every AUP iteration covers the following tasks:



- Modeling. The business and problem domains are represented in UML. These models must be “just barely good enough” to let the team go forward, though, in order to maintain agility.
- Implementation. Source code is translated from models.
- Testing. Similar to XP, the group creates and runs a number of tests to find bugs and make sure the source code complies with specifications.
- Deployment. Deployment in this context, like the generic process, is centred on delivering a software increment and gathering end-user feedback.

Notes

- Configuration and project management. Configuration management pertains to AUP and deals with risk management, change management and team control over any permanent work products. Project management organises team activities and keeps tabs on the team's development.
- Environment management. A process infrastructure comprising tools, standards and other team-available support technologies is coordinated by environment management.

It is crucial to remember that UML modelling can be used in conjunction with any of the agile process models, notwithstanding the AUP's historical and technical ties to UML.

1.1.7 Waterfall Model

The waterfall model, also known as the classic life cycle, proposes a methodical, sequential approach to software development that starts with requirements specified by the customer and moves through planning, modelling, building and deployment before ending with continued support for the finished product.

The more generic system engineering methods served as the basis for the first published model of the software development process. This model is shown in the figure below. This paradigm is referred to as the software life cycle or the "waterfall model" because of the way the phases flow into one another. One example of a plan-driven process is the waterfall model, which requires that all process activities be scheduled and planned out before any work is done on them.

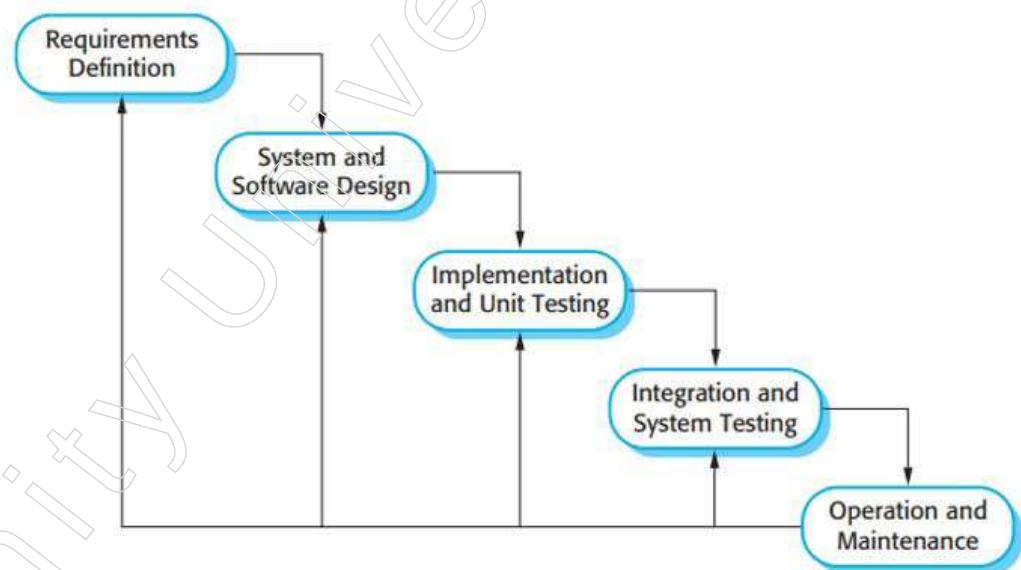


Figure: The waterfall model

<https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>

The core development activities are directly reflected in the main stages of the waterfall model:

1. Requirements analysis and definition: Users are consulted to determine the services, limitations and objectives of the system. After that, they receive a thorough definition that acts as a system specification.
2. System and software design: by creating a general system architecture, the systems design process assigns the needs to either hardware or software systems. Determining

and outlining the essential software system abstractions and their connections is part of software design.

3. Implementation and unit testing: The software design is realised as a collection of programs, or program units, at this point. Verifying that each unit satisfies its specification is the goal of unit testing.
4. Integration and system testing: To make sure the software criteria have been satisfied, the separate program parts or programs are combined and tested as a whole system. The customer receives the software system after testing.
5. Operation and maintenance: This is typically (though not always) the longest life cycle phase. Once implemented, the system is used in real life. In maintenance, mistakes that were missed in the early phases of the system's life cycle are fixed, system units are implemented better and services are improved when new requirements are identified.

Each phase should, in theory, provide one or more approved (or "signed off") documents. It is not advisable to begin the next step until the last one is complete. These phases actually overlap and exchange information with one another. Issues with requirements are found during the design process. Design flaws are discovered during coding and so forth. The software process involves feedback from one phase to the next and is not a straightforward linear model. It could subsequently be necessary to amend the documents created throughout each phase to reflect the adjustments made.

Iterations can be costly and require a lot of rework due to the expenses associated with creating and approving documents. Consequently, it is common practice to freeze certain aspects of the development process, such the specification and go on to other phases after a limited number of iterations. Issues are disregarded, programmed around, or put off for later settlement. The system might not function as the user desires as a result of this early freezing of requirements. Additionally, when implementation strategies are used to get around design flaws, it could result in systems with poor structure.

The software is utilised in the latter stage of the life cycle, known as operation and maintenance. It is found that the initial software specifications contained mistakes and omissions. Errors in the program and design appear and the requirement for more functionality is determined. Thus, for the system to continue to be useful, it must change. Repeating earlier steps of the procedure may be necessary to make these modifications (software maintenance).

The waterfall approach produces documentation at every stage and is compatible with different engineering process models. In order for managers to track advancement in relation to the growth strategy, this makes the process apparent. The rigid division of the project into discrete phases is its main issue. Early in the process, commitments must be made, which makes it challenging to adapt to shifting client needs.

The waterfall methodology should, in theory, only be applied in situations where the requirements are clear and unlikely to change significantly while the system is being developed. The waterfall model, however, mimics the kind of procedure applied to other technical undertakings. Software procedures based on the waterfall model are still widely employed since it is simpler to utilise a single management model for the entire project.

Formal system development, which involves creating a mathematical model of a system specification, is a significant variation of the waterfall paradigm. Then, this model is improved and turned into executable code by mathematical modifications that maintain its coherence. You can thus make a compelling case that a program developed in this manner is consistent with its specification, presuming that your mathematical transformations are

Notes

accurate. Systems with strict requirements for safety, dependability, or security are best developed using formal development methods, including those based on the B approach.

The process of creating a safety or security case is made easier by the formal approach. Customers and regulators will be able to see this as proof that the system satisfies safety and security criteria. Formal transformation-based processes are typically limited to the development of systems that are either security-critical or safety-critical. They call for specific knowledge. This strategy does not provide appreciable cost savings over alternative methods of system development for most systems.

The linear design of the traditional life cycle results in “blocking states,” where some project team members must wait for other team members to finish dependent tasks, according to an intriguing study of real projects. As a matter of fact, waiting times may really be longer than working productively! In a linear sequential process, the blocking states are more common at the start and finish. Work on software these days is fast-paced and constantly changing in terms of features, functionalities and information content. For this kind of job, the waterfall paradigm is frequently unsuitable. Nonetheless, in circumstances where criteria are set and work is to be completed in a linear fashion, it can be a helpful process model.

1.1.8 Prototype Model

It is a software development methodology that involves creating, testing and refining a prototype model until a workable version is produced. It is a demonstration of the real system or product in action. Users can assess and test developer proposals through prototyping before they are implemented. When compared to the real program, a prototype model typically shows less functional capabilities, poor dependability and inefficient performance.

It works well in situations where the client is unaware of all the project’s requirements. The process is iterative and involves both the client and the developer making mistakes.

Prototype model phase

- ❖ Requirements gathering and analysis
- ❖ Design
- ❖ Build prototype
- ❖ User evaluation
- ❖ Refining prototype
- ❖ Implementation and Maintenance

Requirements gathering and analysis: The system’s needs are specified. Users of the system are questioned during this step to find out what they expect from it. Utilise alternative methods to collect data.

Design: During this stage, the system’s basic design is constructed. It provides the user with a quick overview of the system.

Build prototype: During this stage, the first prototype is created, showcasing the fundamental specifications and supplying user interfaces. The information obtained during the design process is used to create the real prototype.

User evaluation: The customer and other key project stakeholders are shown the prototype during this phase for a preliminary assessment. The input is gathered and applied to the ongoing development of the product. This stage aids in determining the working model’s advantages and disadvantages.

Refining prototype: In this stage, discussions about issues like time and financial limits and the technical viability of the actual implementation take place based on input from the customers. The cycle continues until the customer's expectations are satisfied after modifications are accepted and included into the new Prototype model.

Implementation and Maintenance: The final system is developed using the final prototype, tested and then put into production. Regular maintenance and upgrades are performed on the system in response to changes in the real-time environment to minimise downtime and avoid major breakdowns.

Types of Prototyping Models

Rapid Throwaway prototypes: This strategy provides an effective way to test concepts and gather feedback from customers on each one. A produced prototype does not always have to be a part of the final, approved prototype when using this procedure. In order to avoid needless design flaws, customer feedback helps produce higher-quality final prototypes.

Evolutionary prototype: Up until it is ultimately approved, the prototype is gradually improved depending on input from the client. It facilitates effort and time savings. This is due to the fact that creating a prototype from the beginning for each step of the process can occasionally be quite unpleasant.

Incremental Prototype: Incremental prototyping involves breaking down the ultimate product into smaller prototypes that are then created separately. The several prototypes eventually combine to become a single product. The application development team and user feedback time can be shortened with the use of this technique.

Extreme Prototype: Web development is the main application for the extreme prototyping process. There are three successive phases to it.

- All of the pages from the basic prototype are available in HTML format.
- A prototype services layer can be used to simulate data processing.
- The services are put into practice and included in the finished prototype.

Advantages of prototype model

- ❖ The users who contribute to development. In the early phases of the software development process, errors might be found.
- ❖ Ideal in situations where needs are shifting.
- ❖ Diminish Upkeep expenses

Disadvantages of prototype model

- ❖ It is a laborious and slow process.
- ❖ Cost rises in relation to time.
- ❖ Because there is more customer participation, requirements are more dynamic and have an impact on the creation of the entire product.

1.1.9 Rapid Application Development (RAD) Model

The Rapid Application Development Model was first presented by IBM in the 1980s. The RAD model is a form of incremental process paradigm in which there is exceptionally short development cycle. When the requirements are completely understood and the component-based construction strategy is selected then the RAD model is applied. Various

Notes

phases in RAD are Requirements Gathering, Analysis and Planning, Design, Build or Construction and finally Deployment.

The important element of this paradigm is the utilisation of sophisticated development tools and methodologies. A software project can be implemented using this paradigm if the project can be broken down into distinct modules wherein each module can be assigned independently to separate teams. These components can finally be merged to produce the final product. Development of each module requires the numerous basic processes like in the waterfall model i.e. analysing, designing, coding and then testing, etc. as indicated in the image. Another noteworthy aspect of this model is a short time span i.e. the time window for delivery(time-box) is normally 60-90 days.

Multiple teams work on developing the software system utilising RAD paradigm parallelly.

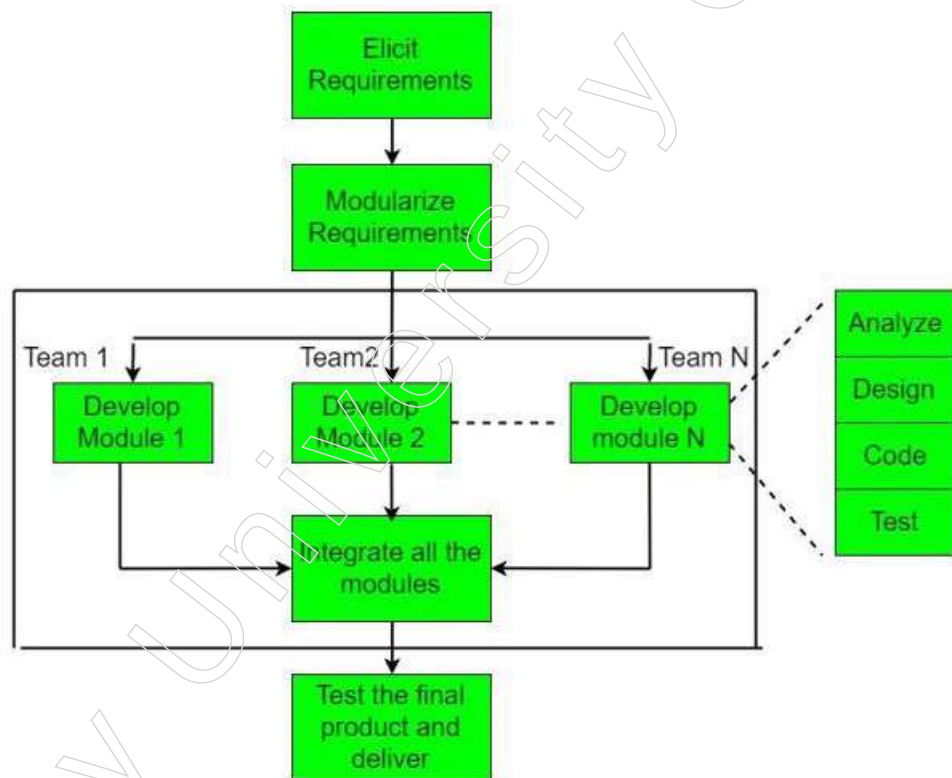


Figure: RAD model.

<https://www.geeksforgeeks.org/software-engineering-rapid-application-development-model-rad/>

Another essential component of the projects is the utilisation of strong development tools like XML, C++, Visual BASIC, Java and so on. There are four main stages to this model:

1. Requirements Planning – It entails applying a variety of requirements elicitation strategies, including user scenarios, task analysis, form analysis, brainstorming, FAST (Facilitated Application Development Technique), etc. It also includes the complete structured strategy outlining the necessary data, how to get it and how to process it to create a polished end model.
2. User Description – In this stage, developer tools are used to build the prototype based on user feedback. Stated differently, it entails a re-examination and validation of the data gathered during the initial phase. In this phase, the attributes of the dataset are also identified and clarified.

3. Construction – Prototype refinement and delivery occur during this phase. It involves really transforming processes and data models into the finished working result through the use of strong automated technologies. During this phase, all necessary improvements and modifications are also completed.
4. Cutover – It is necessary to thoroughly evaluate each interface created by different teams between their distinct components. Testing is made simpler by the use of highly automated tools and subparts. The user's acceptability testing comes next.

Quick prototype building, customer delivery and feedback gathering are all part of the process. The SRS document is created and the design is completed following the customer's validation.

When to use RAD Model?

When there are clear specifications from the client, when the user is involved at every stage of the project, when the project can be time-boxed, when functionality is supplied in small steps, when there are minimal technical risks and when the system can be modularised. We can apply the RAD Model in these situations. Sometimes it's imperative to create a system that can be split up into smaller components in a matter of two to three months. when the budget includes enough funds to cover the cost of both the designers' work and the automated tools used to create code.

Advantages	Disadvantages
Reusable parts are used in the project, which helps to shorten its cycle time.	Using strong and effective instruments calls for experts in their field.
Customer feedback is available in the early phases.	The project's failure may result from the lack of reusable parts.
Lower expenses because fewer developers are needed.	To complete the project on schedule, the team leader needs to collaborate closely with the developers and clients.
Utilising strong development tools produces goods of higher quality in much less time.	This concept is not applicable to systems that cannot be appropriately modularised.
Through the various stages, the project's development and progress can be evaluated.	Participation from customers is necessary at every stage of the process.
The small iteration time spans make it simpler to adapt to changing requirements.	It is not appropriate for small-scale projects since the expenses associated with employing automated tools
and procedures could outweigh the project's total budget in such circumstances.	
A smaller workforce can quickly increase productivity.	With RAD, not all applications may be used.

Applications:

- For a system with well-defined needs and a quick development period, this paradigm is appropriate.
- It is also appropriate for projects with modularizable requirements and development-ready reusable components.
- When creating a new system with the fewest possible modifications using already-existing system components, the model can also be applied.
- Only when the teams are made up of subject matter experts is this concept applicable.

Notes

This is due to the fact that having the necessary information and skills to employ effective strategies is essential.

- When the budget allows for the necessary usage of automated tools and procedures, the model should be selected.

Drawbacks of rapid Application Development:

- To work on the scalable projects, a big number of individuals or many teams are needed.
- Customers and developers that are really devoted are needed for this concept. A lack of dedication will lead to the failure of RAD projects.
- Projects that use the RAD approach demand a lot of resources.
- RAD initiatives fail if the right modularisation is not done. Performance issues with these projects are possible.
- It is challenging for projects employing the RAD approach to integrate new technology.

The Rapid Application Development (RAD) model is a traditional software engineering approach that prioritizes rapid prototyping and iterative development over comprehensive planning and upfront design. It aims to accelerate the development process by emphasizing user feedback and collaboration. Here's how the RAD model may be implemented in both traditional and advanced software engineering contexts:

RAD in Traditional Software Engineering:

- In traditional software engineering environments, the RAD model may be used as an alternative to the Waterfall model for certain projects.
- RAD involves rapid prototyping, where developers quickly build working prototypes of the software to gather feedback from users and stakeholders.
- Iterative development cycles are employed, with each cycle focusing on refining the prototype based on feedback received.
- While RAD emphasizes speed and flexibility, it may still involve some level of upfront planning and documentation, although not to the extent of traditional methodologies like Waterfall.
- RAD projects may face challenges in terms of managing scope creep and ensuring sufficient quality control, especially if there's a lack of clear requirements or governance processes in place.

RAD in Advanced Software Engineering:

- In advanced software engineering environments, RAD principles may be integrated into Agile methodologies such as Scrum or Extreme Programming (XP).
- RAD's emphasis on rapid prototyping and iterative development aligns well with Agile principles of delivering working software incrementally and responding quickly to change.
- Advanced software engineering teams practicing RAD within an Agile framework prioritize collaboration, customer feedback, and delivering value early and often.
- RAD projects in advanced software engineering environments benefit from the flexibility and adaptability of Agile methodologies, allowing teams to adjust course based on continuous feedback and changing requirements.
- Advanced software engineering practices such as continuous integration, automated testing, and DevOps complement RAD's focus on rapid development by ensuring that changes can be integrated and deployed smoothly.

In summary, while the RAD model originated as a traditional software engineering approach, its principles can be adapted and integrated into both traditional and advanced software engineering environments. Whether implemented within a Waterfall framework or as part of an Agile methodology, RAD aims to accelerate software development by prioritizing rapid prototyping, iterative development, and close collaboration with stakeholders.

1.1.10 Selection of Appropriate SDLC Model

Selection Process parameters are crucial to software development because they aid in selecting the most appropriate software life cycle model. The parameters listed below should be applied while choosing an SDLC. In the software development process, the selection of a software life cycle model is an important choice. Various models may be needed for different projects depending on variables including project size, complexity, volatility of needs and client engagement. There are a number of factors to take into account while selecting a software life cycle model.

Requirements Characteristics:

- Reliability of Requirements
- How often the requirements can change
- Types of requirements
- Number of requirements
- Can the requirements be defined at an early stage
- Requirements indicate the complexity of the system

Development Team:

- Team size
- Experience of developers on similar type of projects
- Level of understanding of user requirements by the developers
- Environment
- Domain knowledge of developers
- Experience on technologies to be used
- Availability of training

User Involvement in the Project

- Expertise of user in project
- Involvement of user in all phases of the project
- Experience of user in similar project in the past

Project Type and Associated Risk

- Stability of funds
- Tightness of project schedule
- Availability of resources
- Type of project
- Size of the project

Notes

- Expected duration for the completion of project
- Complexity of the project
- Level and the type of associated risk

A number of variables, including project needs, team expertise and project features, must be taken into account in order to choose the optimal Software Development Life Cycle (SDLC) model. Among the more well-liked models are Spiral, Agile and Waterfall.

- Because the Waterfall model is sequential and linear, it works best for projects with clear objectives and consistent needs. It isn't adaptable enough to modifications made throughout development, though.
- Agile is renowned for its flexible and iterative methodology, which fosters teamwork and adaptability to shifting demands. It works well for dynamic projects where ongoing feedback is essential.
- The Spiral paradigm allows for flexibility and risk control by combining elements of both Waterfall and iterative development. It helps with big, intricate undertakings.
- In the end, the decision is based on the details of the project. Agile is preferred since it is flexible; nevertheless, Waterfall or Spiral may be more suitable if needs are set in stone. The selection process will be guided by evaluating team competencies and project needs.
- Agile is very flexible, which sets it apart from all other SDLCs. Because change implementations in other SDLCs are predictive and necessitate meticulous planning, requirements and analysis, they might be difficult.
- Modern software development needs to support rapid modifications. In contrast to other predictive approaches, careful planning is not required for the Adaptive Agile paradigm. If necessary, adjustments can be done inside the same sprint.

A detailed grasp of the project's needs, limitations and objectives is necessary to make the optimum SDLC model selection, which is a strategic choice. Although every model has advantages and disadvantages, the most important thing is to match the selected model with the particulars of the project. Navigating the complexity of software development and guaranteeing the successful delivery of high-quality software solutions require flexibility, adaptability and good communication. The SDLC model that best suits the particular requirements and circumstances of the current project is ultimately the best one.

The selection of an appropriate Software Development Life Cycle (SDLC) model depends on various factors, including project requirements, team capabilities, organizational culture, and stakeholder expectations. Here's how the choice might differ between traditional software engineering and advanced software engineering contexts:

Traditional Software Engineering:

- In traditional software engineering environments, where there's a greater emphasis on predictability, thorough planning, and documentation, the Waterfall model might be more commonly used.
- The Waterfall model follows a sequential approach, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance. Each phase must be completed before moving on to the next.
- This model is suitable for projects with well-defined requirements and stable technologies, where changes are not expected to occur frequently.
- For projects with a clear and fixed scope, limited customer involvement, and a

preference for comprehensive documentation, the Waterfall model may be a suitable choice.

Advanced Software Engineering:

In advanced software engineering environments, where there's a focus on adaptability, collaboration, and delivering value iteratively, Agile methodologies such as Scrum, Kanban, or Extreme Programming (XP) are more commonly employed.

- Agile methodologies embrace iterative and incremental development, allowing for flexibility in responding to changing requirements, technologies, and market conditions.
- Agile frameworks promote close collaboration between cross-functional teams, frequent deliveries of working software, and continuous improvement based on feedback.
- For projects characterized by uncertainty, evolving requirements, and a need for rapid delivery, Agile methodologies are often preferred.
- Depending on the specific project requirements and team dynamics, organizations may choose to tailor Agile practices to suit their needs, combining elements from different methodologies or adopting hybrid approaches.

In summary, while the Waterfall model may be more suitable for traditional software engineering environments with well-defined requirements and stable technologies, Agile methodologies are typically preferred in advanced software engineering contexts, where there's a need for flexibility, collaboration, and rapid delivery of value. The selection of the appropriate SDLC model should consider the unique characteristics of the project, the organization, and the team involved.

Summary

- Software engineering is a discipline that deals with the systematic design, development, maintenance and evolution of software systems. It encompasses various principles, methods, tools and practices to create reliable, efficient and scalable software solutions.
- Lifecycle models in software engineering are methodologies that describe the stages a software product goes through from its inception to retirement. These models guide the development process, providing a structure for managing tasks, resources and milestones. Few common lifecycles models are: a) Waterfall model, b) Agile model, c) Spiral model, d) V-model, e) DevOps model, f) Incremental models, g) Hybrid models. Each model has its advantages and disadvantages and the choice of model often depends on project requirements, team dynamics, client needs and the nature of the software being developed. The key is to select a model that best fits the project's goals while allowing for adaptability to changes and ensuring quality throughout the software's lifecycle.
- Incremental development is a software development approach that involves building and delivering a system in smaller, manageable parts or increments. Instead of delivering the entire system at once, it's developed incrementally, with each increment adding to the functionality of the previous one. Incremental development allows for continuous refinement, adaptation to changing requirements and early value delivery, making it a popular choice for many software projects, especially in dynamic and evolving environments.
- Each model has its strengths and weaknesses and the choice depends on project requirements, team dynamics, client needs and the nature of the software being developed. Some models, like Agile, emphasise adaptability and collaboration, while

Notes

Notes

others, like the Waterfall model, prioritise structured planning and documentation. Combining elements from different models or adopting a hybrid approach is also common in software development to suit specific project needs.

- The Rapid Application Development (RAD) model is an iterative and accelerated software development approach that focuses on quickly producing high-quality software. It emphasises rapid prototyping and iterative development cycles to deliver functional software to customers in a shorter time frame. The RAD model is particularly beneficial for projects where rapid development and frequent changes are expected, such as in small to medium-sized applications, prototyping phases, or projects where quick market delivery is a priority. However, its success depends on strong communication, collaboration and active involvement of all stakeholders throughout the development process.
- Selecting the appropriate Software Development Life Cycle (SDLC) model depends on various factors including project requirements, team expertise, client needs, time constraints and the nature of the software being developed. Sometimes, a hybrid model combining elements from different methodologies might suit the project best. For instance, using Agile for development and incorporating some aspects of Waterfall for documentation and compliance can be effective. Evaluate the project's specific needs, risks, constraints and team dynamics before making a decision. The chosen SDLC model should align with the project's goals and constraints while ensuring efficient development, high-quality delivery and stakeholder satisfaction. Regularly assess and adapt the chosen model as the project progresses to ensure its continued suitability.

Glossary

- SDLC: Software Development Life Cycle
- RAD: Rapid Application Development
- SEI: Software Engineering Institute
- SRD: Software Requirements Document
- SDD: Software Design Document
- ASIC: Application-Specific Integrated Circuits
- SoC: System on a Chip
- UML: Unified Modelling Language
- COM: Component Object Model
- EJB: Enterprise JavaBeans
- CLR: Common Language Runtime
- XP: Extreme Programming

Check Your Understanding

1. What is the primary objective of software engineering?
 - a) Creating software without considering user needs
 - b) Designing and developing reliable and efficient software systems
 - c) Focusing solely on coding practices
 - d) Ignoring software testing and deployment
2. Which phase involves understanding and documenting what the software should do?

Notes

- a) Design phase
 - b) Implementation phase
 - c) Requirements engineering phase
 - d) Testing phase
3. In the Waterfall model, which phase comes after the testing phase?
- a) Maintenance phase
 - b) Implementation phase
 - c) Deployment phase
 - d) Requirements phase
4. What characterises the Agile software development lifecycle?
- a) Sequential and rigid phases
 - b) Iterative and adaptable approach
 - c) Emphasis on extensive documentation
 - d) Long development cycles
5. Which of the following is a key benefit of incremental development?
- a) Slow time to market
 - b) Flexibility to manage changes
 - c) Complete software delivery at once
 - d) Limited adaptability to user feedback

Exercise

- 1. Define software engineering.
- 2. Explain lifecycle model. Give examples.
- 3. What is incremental development?
- 4. Explain spiral model with examples.
- 5. What do you understand by component model?

Learning Activities

- 1. What measures would you take to minimise the risks associated with deployment, such as data migration and potential system downtime?
- 2. Explain the importance of end-user training and acceptance testing during the deployment phase.

Check Your Understanding- Answers

- 1. b)
- 2. c)
- 3. c)
- 4. b)
- 5. b)

Module -II: Formal Methods

Learning Objectives

At the end of this module, you will be able to:

- Define the basic concepts of formal specification
- Know the types of mathematical notations for formal specification
- Understand types of formal specification languages
- Analyse the difference between informal and formal specification language
- Discuss Syntax, Type and Semantics of Z-Notations

Introduction

Formal methods are mathematically based methods to describe the properties of computer systems. These formal methods offer frameworks that enable methodical, as opposed to haphazard, system specification, development and verification.

All specification methods aim to achieve the desired characteristics of a formal specification, which include completeness, consistency and absence of ambiguity. On the other hand, there is a far greater chance of obtaining these qualities with formal methods due to their mathematically based specification language. When a reader must interpret a graphical notation (like UML) or a natural language (like English) there is often uncertainty. This is eliminated when a specification language's formal syntax allows requirements or design to be understood in a single way. A precise description of needs is made possible by the descriptive powers of logic notation and set theory. Requirements in a specification should not contradict one another in order to be consistent. Mathematically demonstrating that early facts may be formally transferred (using inference rules) into later claims within the specification is the means of achieving consistency.

In software engineering, mathematically based methods to the design, development and testing of hardware and software systems are referred to as formal methods. These techniques make use of mathematical models to explain how a system behaves and to determine whether it is accurate. Making sure a system works as intended and satisfies its requirements is the aim.

The following are some essential features and elements of software engineering formal methods:

- **Formal Specification Languages:** These are languages, like Z, Alloy and Event-B, that let developers specify behaviour, design and system requirements in a clear and concise way.
- **Mathematical Logic:** Formal methods frequently use mathematical logic to represent and reason about system aspects. Examples of this logic include modal logic, temporal logic and predicate logic.
- **Model Checking:** To determine if a given property holds, this entails thoroughly examining every state that a system could possibly be in.
- **Theorem Proving:** This method shows that a system is proper in relation to its specifications by using formal mathematical proofs.
- **Abstract Interpretation:** This is estimating a program's behaviour to look for specific characteristics without thoroughly examining every state that could exist.

2.1 Basic Concepts

In computer science and software engineering, formal procedures are strategies we employ to guarantee the accuracy of our programs and minimise program faults. In order to describe and analyse system behaviour and increase system security and dependability, they rely on logic and mathematics.

There are several purposes for which we employ formal methods:

- **Correctness and reliability:** Formal methods reduce the likelihood of errors and inconsistencies in hardware and software, increasing the correctness, reliability and error-proneness of projects.
- **Early error detection:** Using formal methods helps us cut down on future time and financial costs by identifying flaws and inconsistencies early in the software development cycle.
- **Less unambiguity:** There are less misconceptions between us and the stakeholders when we use formal languages for system specifications since we can clearly grasp the behaviour and definition of our system.
- **Safety critical systems:** Systems like medical equipment and aerospace, among others, whose failure could have far-reaching effects, are referred to as safety-critical systems. We can uphold strict safety requirements for these systems with the aid of formal methodologies.
- **Verification and Validation:** To demonstrate and guarantee the accuracy of the hardware and software in our system, we can employ formal verification and validation techniques.
- **Security:** By testing and verifying the needs of our systems, formal assist us in ensuring the security of our gadgets.

Formal methods are mathematical techniques used in software engineering to specify, develop, and verify software systems. These methods are used to ensure the correctness and reliability of software systems by mathematically modeling their behavior and properties. Here's how formal methods might be applied in both traditional and advanced software engineering contexts:

Traditional Software Engineering:

- In traditional software engineering environments, formal methods may be employed in critical systems where safety and reliability are paramount, such as aerospace, defense, and healthcare.
- Formal methods are used to rigorously specify system requirements, design models, and behavior, often using formal specification languages such as Z, B, or VDM.
- Verification techniques such as formal proof, model checking, and theorem proving are used to ensure that the software meets its specifications and satisfies desired properties, such as safety, liveness, and security.
- However, the adoption of formal methods in traditional software engineering may be limited due to factors such as high cost, complexity, and the need for specialized expertise in formal verification techniques.

Advanced Software Engineering:

- In advanced software engineering environments, formal methods may be integrated into Agile methodologies or DevOps practices to ensure the correctness and reliability of software systems while maintaining a focus on flexibility and rapid delivery.

Notes

Notes

- Formal methods can be used iteratively throughout the development process, with developers specifying, verifying, and refining system requirements and designs in collaboration with stakeholders.
- Tools and techniques for formal specification and verification may be integrated into the development workflow, enabling continuous verification and validation of software artifacts.
- Advanced software engineering teams may also leverage automated testing, static analysis, and runtime verification techniques to complement formal methods and provide additional assurance of software correctness.
- By incorporating formal methods into Agile and DevOps practices, advanced software engineering environments can achieve a balance between agility and rigor, ensuring that software systems meet quality and reliability requirements while remaining responsive to changing customer needs and market conditions.

2.1.1 Basic Concepts of Formal Specification

Many researchers have promoted the use of formal methods in software development for over thirty years. Formal techniques are methods to software development that are mathematically oriented and include defining a formal model of the program. This model can then be formally analysed and it can serve as the foundation for a formal system specification. It is theoretically conceivable to eliminate software failures caused by programming errors by beginning with a formal model for the software and demonstrating that a created program is consistent with that model.

As a system specification, a formal system model is the foundation of all formal development procedures. The system's user needs, which are articulated in natural language, diagrams and tables, are translated into a mathematical language with rigorously specified semantics to build this model. The explicit definition of the system's intended functionality can be found in the formal specification. You can verify that a program behaves in accordance with the specification by hand or with the aid of tools.

Not only are formal specifications necessary for software design and implementation verification. They minimise the possibility of misunderstanding because they are the most exact method of system specification.

Moreover, creating a formal specification necessitates a thorough examination of the requirements, which is a useful method of identifying requirements issues. Errors in a natural language specification may be hidden by the language's imprecision. If the system is formally specified, this is not the case.

Formal specifications are typically created as a part of a software development process that is plan-based and involves fully specifying the system before it is developed. Before implementation starts, the system requirements and design are thoroughly outlined, thoroughly examined and verified. If a formal software specification is created, it often follows the specification of the system requirements but precedes the full system design. The formal specification and the detailed requirements specification are closely linked.

The phases of software definition and how they connect with software design in a plan-based development process are depicted in the figure below. You may want to restrict the application of this strategy to those parts that are essential to the system's functioning because it is costly to create formal specifications. These are recognised in the system's architectural design.

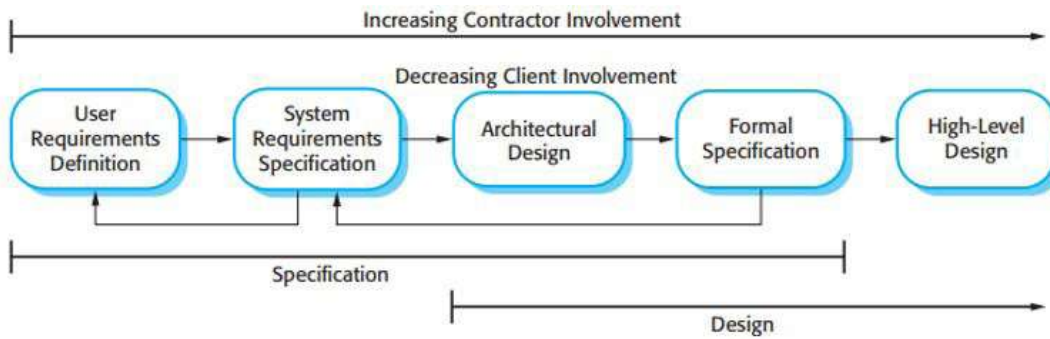


Figure: Formal specification in a plan-based software process

Image Source: Software-Engineering-9th-Edition-by-Ian-Sommerville

Automated assistance for formal specification analysis has been developed in the last few years. Model checkers are computer programs that accept as inputs a state-based formal specification (a system model) and a formally defined desirable attribute (e.g., “there are no unreachable states”). After conducting a thorough analysis of the specification, the model checking program either states that the model satisfies the system property or provides an example demonstrating that it does not. Static analysis is strongly related to the idea of model checking.

Creating a formal specification and applying it to a formal development process has the following benefits:

1. As you meticulously create a formal specification, you gain a comprehensive comprehension of the system requirements. Requirements error detection is a strong case for creating a formal specification, even if it is not used in a formal development process. Early requirement problem detection typically results in a substantially lower cost of correction than requirements problems discovered later in the development process.
2. The specification may be automatically analysed to find errors and incompleteness since it is written in a language with formally defined semantics.
3. A series of correctness-preserving transformations can be used to convert the formal specification into a program, such as the B approach. As a result, the final program is assured to fulfil its requirements.
4. Because you have confirmed the program's compliance with its specifications, the cost of program testing might be lowered.

Even for essential systems, formal methods have not had much of an impact on actual software development, despite these benefits. As a result, the community has relatively limited experience creating and utilising formal system specifications. The following are the justifications advanced against creating a formal system specification:

1. Since they are unable to comprehend a formal specification, problem owners and domain experts are unable to verify that it appropriately reflects their needs. Even software developers, who are knowledgeable about the formal specification, might not be familiar with the application domain, thus they too cannot be certain that the formal specification accurately reflects the needs of the system.
2. Estimating the potential cost savings from using a formal specification is more challenging than quantifying the costs of producing one. Managers are therefore reluctant to take the chance of using this strategy.
3. Formal specification languages are not commonly taught to software engineers. Because of this, they are hesitant to suggest using them in development procedures.

Notes

4. Scaling up existing formal specification methods to very big systems is a challenging task. Rather than describing entire systems, formal specification is mostly used to identify essential kernel software.
5. Agile development methodologies are incompatible with formal specifications.

2.1.2 Importance of Formal Methods

The requirements analysis and specification phase of the software development lifecycle is the most crucial. Poor requirements specification is the reason behind half of project failures, according to a Standish Chaos report. These are the first stages when formal methods work best. Writing a formal specification is more effective than drafting an informal one and then translating it. Early formal specification analysis is an effective way to find inconsistencies and incompleteness. In addition to the advantages mentioned above, there are a number of other advantages, which are covered as follows:

- **Measure of correctness:** Unlike the existing process quality measurements, the employment of formal method yields a measure of a system's correctness.
- **Early defect detection:** Early design artifacts can benefit from the application of formal method, which can help identify and remove design flaws sooner.
- **Guarantees of correctness:** Formal analysis tools, like model checkers, take into account every scenario that could occur during system operation. Any potential flaw or error will be discovered by a model checker. All possible interleaving and event orderings in a multithreaded system where concurrency is a problem can be explored through formal analysis. Testing will never be able to accomplish this degree of coverage.
- **Error Prone:** Writing a formal description compels the writer to consider several issues that they could put off until after coding. This lessens the number of mistakes that happen either during or after coding. Completeness is a feature of formal techniques, meaning they address every facet of the system.
- **Abstraction:** One can develop code for software or hardware products right away if their operation is straightforward, but most systems have far too much code, necessitating a thorough system description. On the other hand, a formal specification is a description that is exact, abstract and somewhat comprehensive. The abstraction makes it simple for a human reader to comprehend the software product's overall scope.
- **Rigorous Analysis:** We are able to conduct in-depth analysis because of the description's formality. Formal descriptions are typically written from several perspectives, allowing one to assess key characteristics like the proposal's accuracy or the degree to which high level requirements are satisfied.
- **Trustworthy:** The kind of evidence required in highly regulated industry like aviation is provided by formal processes. They give specific examples and explanations for the product's credibility.
- **Effective Test Cases:** We can methodically generate efficient test cases straight from the formal specification through a formal specification. It's an economical method of creating test cases.

Limitations of Formal

A significant part of the software development lifecycle is played by formal methods. These techniques do have certain drawbacks, though. The formal methods for software products are less effective because of these drawbacks. The following is a summary of several formal methods' drawbacks:

- **Correctness of Specifications:** In general, the user's stated requirements may not match the real requirements, which tend to change over time. It is impossible to ensure that a specification adheres to all of the user's informal needs while utilising formal methods. Nonetheless, a number of methods have been proposed in the literature to lessen the likelihood of inaccurate specifications; nonetheless, all inevitably begin informally. It is never certain that one has accurately acquired all user needs.
- **Correctness of Implementation:** Determining whether a given program satisfies the provided specifications is an extremely challenging task. For instance, it is not possible to automatically discover the loop invariants when using a verification checking approach like Hoare logic. Therefore, if an existing program was not created with the correctness proof in mind, it is frequently impossible to demonstrate its correctness. Proofs of correctness can only be achieved if programming and proof proceed at the same time.
- **Correctness of Proofs:** Proofs of correctness are crucial to formal methods. Proofs of correctness raise the likelihood that the program is right. Ensuring the accuracy of specifications and implementation is typically unfeasible. The primary issue with the evidence is how they were made. There is occasionally a chance that proof of accuracy will fall short. The following are potential causes of a failure in the proving of an implementation's correctness in relation to its specification:
 - ❖ The program is incorrect and needs to be modified.
 - ❖ The program is correct, but the correctness proof has not been found yet.
 - ❖ The program is correct, but there is no correctness proof.

2.1.3 Overview and Advantages of Formal Methods Model

The formal method model is focused with designing and implementing the program using a mathematical strategy. This model provides the framework for creating a sophisticated system and aiding in the creation of programs. Problems that are challenging to solve with other software process models can be solved through the use of formal methods in the development process. Formal specifications are created for this paradigm by the software developer. When the user starts using the system, there are fewer errors as a result of these 'minimisation of specification errors'.

Formal methods include formal specification, which specifies the desired properties of the system through mathematical expressions. Formal specification is articulated in terms of a language with formally defined syntax and semantics. This language consists of a set of relations that utilise rules to identify the objects for satisfying the specification, a semantic that uses objects to describe the system and a syntax that defines specific notation used for specification expression.

The formal method typically consists of two methods: model-based and property-based. The actions carried out on the system are described in the property-based specification. It also explains how these operations are related to one another. A property-based specification is composed of two elements: an equation that describes the semantics of the operations through a collection of equations called axioms and signatures that specify the syntax of the operations.

The model-based specification creates an abstract model of the system using logic, function theory and set theory as its tools. It also details the operations carried out on the abstract model. The resulting model is highly developed and idealised. A model-based specification includes definitions of the system's collection of states as well as legal

Notes

operations that are carried out on the system, along with an explanation of how these legal operations alter the system's current state.

A collection of mathematically based methodologies for the specification, design and verification of software systems is known as formal method in software engineering. These techniques guarantee the accuracy of a system's behaviour by describing it using exacting mathematical models. Formal method application has many benefits for the development process, but it also demands a certain amount of experience and money. We will explore the benefits of formal methods in depth in this thorough investigation.

1. Precision and Clarity:

Advantage: Formal methods use logic and mathematical languages to specify systems.

Explanation: System requirements are expressed more precisely and clearly when formal methods, which include mathematical notations and logic, are used. By reducing ambiguity, this precision makes sure that everyone involved understands the intended behaviour of the system.

2. Rigorous Verification:

Advantage: Rigorous methodologies for system verification are provided by formal methods.

Explanation: like as theorem proving and model checking allow for a thorough examination of system attributes. This thorough verification procedure raises trust in the software's correctness by guaranteeing that the system operates as intended and complies with its specifications.

3. Early Error Detection:

Advantage: In the early stages of the development lifecycle, formal methods can be used.

Explanation: Errors can be found and fixed prior to the implementation phase by using formal methods in the requirements analysis and design phases. As a result of early mistake detection, problems are found when they can be easily and affordably fixed, which lowers the overall cost and work involved in resolving them.

4. Improved System Reliability:

Advantage: Using formal methods helps create software systems that are more dependable.

Explanation: Formal techniques' methodical verification approach assists in locating and removing any mistake sources, enhancing the final product's dependability. Particularly for systems that are safety- and mission-critical, reliability is essential.

5. Safety and Security Assurance:

Advantage: In systems where security and safety are crucial, formal method are useful.

Explanation: By enabling the verification of safety and security attributes, these techniques lower the possibility of security flaws and catastrophic failures. Formal methods are essential for assuring the robustness of software systems in industries where safety and security are of utmost importance, such aerospace, healthcare and finance.

6. Concurrency and Distributed Systems:

Advantage: Concurrent and distributed systems can be effectively modelled and

reasoned about using formal methods.

Explanation: As concurrent and distributed architectures become more common, formal methods offer a methodical way to deal with the issues of coordination, synchronisation and communication in these systems. Creating reliable and scalable software solutions requires this.

7. Maintaining System Invariants:

Advantage: Invariants can be specified and verified using formal methods.

Explanation: Logical conditions known as invariants need to hold true for the entire time a system is operating. These invariants can be specified and verified by developers using formal methods, which guarantees that important system properties are preserved consistently. This enhances the accuracy and stability of the system.

8. Documentation and Knowledge Transfer:

Advantage: The development of formal models, which act as documentation for system behaviour, is facilitated by formal methods.

Explanation: Formal models serve as an accurate and thorough record of the behaviour of the system. These models are important resources for upkeep and future development since they help team members share expertise. They provide context for comprehending the nuances of the system and the reasoning behind its design.

9. Quality Assurance and Certification:

Advantage: In sectors where safety is crucial, formal methods play a significant role in supporting certification and quality assurance processes.

Explanation: Strict regulatory criteria and quality standards are followed by many industries. Software systems can be certified by using formal methods, which offer a way to prove and record adherence to certain criteria. This is especially important in industries where following standards is required by law or contract.

10. Enhanced Communication:

Advantage: Stakeholders can communicate in a shared language thanks to formal methods.

Explanation: The various stakeholders involved in the software development process can communicate with each other more effectively when a formalised mathematical language is used. Improved communication between developers, testers and system architects can lead to a common understanding of the needs and behaviour of the system.

11. Scalability:

Advantage: Scalability in both modelling and verification is possible with certain formal methods.

Explanation: It is essential that formal methods scale with the complexity of software systems. Scalable formal methods guarantee that the accuracy and rigor that define formal methods may be maintained while being successfully applied to real-world, industrial-scale projects.

12. Early Validation of Design:

Advantage: System design can be validated early on thanks to formal methods.

Explanation: A foundation for verifying the design versus requirements is provided by

Notes

formal specifications and models. Teams are able to improve the design before moving on to the implementation phase thanks to this early validation, which helps find design faults or inconsistencies.

13. Traceability:

Advantage: Traceability between specifications, design artifacts and implementation is made easier by formal methods.

Explanation: A transparent, traceable approach from requirements to design and implementation is made possible by formal models and specifications. Because every part of the system can be traced back to its original specifications, traceability makes it simpler to manage changes and comprehend their effects.

14. Formal Contractual Agreements:

Advantage: The creation of formal contractual agreements is supported by formal methods.

Explanation: Formal methods offer a way to describe and enforce agreements that regulate the development process in cases where contracts are in place. This is especially important in fields where software development is heavily influenced by contractual and legal requirements.

15. Formal Method Education and Skill Development:

Advantage: Software engineers' education and skill development are enhanced by the application of formal methods.

Explanation: Software engineers can improve their analytical and reasoning skills by learning and using formal methods. This training promotes a mindset that prioritises accuracy and comprehensive verification, which helps to maintain a culture of quality and accuracy in software development.

16. Tool Support:

Advantage: The use of formal methods is supported by a variety of settings and tools.

Explanation: Formal methods application is made easier by the availability of tools like theorem provers and model checkers. The formal modelling, analysis and verification processes are aided by these technologies, which makes the integration of formal methods into development teams' workflows more feasible.

17. Long-Term Maintenance:

Advantage: Software systems' long-term maintenance is facilitated by formal methods.

Explanation: The understanding of system behaviour during maintenance phases is facilitated by the use of formal models and specifications as documentation. With the constant evolution and changes that software systems receive, this is extremely beneficial.

18. Risk Mitigation:

Advantage: Using formal methods helps software developers reduce risk.

Explanation: Formal methods reduce the likelihood of important concerns such as security breaches and system breakdowns by methodically confirming the correctness of software. This is especially crucial for sectors of the economy where software malfunctions can have devastating repercussions.

19. Flexibility in Development Paradigms:

Advantage: Several development paradigms can use formal methods.

Explanation: The formal approach can be modified to fit many development paradigms, whether it is implemented through agile methodologies or conventional waterfall models. Teams can easily incorporate the formal method into their current development processes thanks to its flexibility.

20. Cross-Disciplinary Applications:

Advantage: Applications for formal methods extend beyond software engineering.

Explanation: The design and verification of hardware systems, protocols and even some business process elements can benefit from the use of formal method concepts, which are not just restricted to software. This cross-disciplinary applicability highlights the formal method's adaptability and generality.

To sum up, there are several benefits to using the formal method in software engineering, from supporting safety-critical applications to guaranteeing accuracy and precision. For sectors where software correctness is crucial, formal methods are an appealing option because to their benefits in terms of stability, security and long-term maintainability, even though they may require an initial investment and learning curve. Formal methods will probably become more crucial as technology develops for creating reliable and strong software systems.

An emerging model in Formal Methods is known as "Integrated Formal Methods" (IFM). This model represents a holistic approach to the application of formal methods throughout the software development lifecycle, integrating formal techniques with conventional software engineering practices. Here's an overview of the Integrated Formal Methods model:

Integrated Formal Methods (IFM):

Specification and Modeling: FM begins with the formal specification and modeling of system requirements, behavior, and architecture using mathematical notation or formal languages. Specifications may include functional requirements, safety properties, and performance constraints.

Design and Development: Formal methods are applied during the design and development phase to create formal models of the system architecture, components, and interactions. Models may include state machines, transition systems, temporal logic specifications, and other formal representations.

Verification and Validation: Formal verification techniques are used to rigorously analyze the correctness of the system design with respect to its formal specifications. This includes model checking, theorem proving, and static analysis to ensure that the system behaves as intended and meets its requirements under all possible scenarios.

Validation involves testing the system against its formal specifications to ensure that it behaves correctly in real-world conditions. This may include simulation-based testing, property-based testing, and runtime verification techniques.

Refinement and Iteration: The IFM model supports iterative refinement of system designs based on verification and validation results. If discrepancies or errors are found during verification or testing, the design may be refined, and the verification process repeated until the desired level of confidence is achieved.

Notes

Notes

Tool Support and Integration:

Various software tools and automated verification systems support the application of formal methods throughout the development lifecycle. These tools provide capabilities for specification, modeling, verification, and testing, helping developers apply formal methods effectively and efficiently.

Integration of formal methods into existing development processes and tool chains may require significant effort and investment in training, tool adoption, and process adaptation.

2.1.4 Use of Formal Methods in SDLC

The goal of formal methods is to provide rigour and organisation to every stage of the software development process. This keeps important issues from slipping our minds, gives us a common way to document different judgements and assumptions and establishes a foundation for uniformity across a wide range of related tasks. Formal methods aid in the understanding necessary to bring together the many stages of software development into a successful endeavour by offering clear and exact descriptive mechanisms.

Since individuals first started writing programs, the programming language used for software development has provided explicit syntax and semantics for the implementation phase. However, other sources must provide precision at every stage of software development except this one. "Formal" refers to a wide range of abstractions and formalisms meant to provide a similar degree of accuracy for various stages of software development. Even though some of these challenges are still being actively developed, practitioners can still profit from a number of approaches that have achieved a mature state.

For software engineering techniques, there is a noticeable trend towards the fusion of discrete mathematics and formal methods. When pursuing formal methods, it is not possible nor desirable to ignore these issues because many of them actually benefit software engineering. However, we will not adopt the stance that says using discrete mathematics in software engineering guarantees the use of appropriate formal methods. The primary goal of software engineering is to develop software systems of the highest calibre. We aim to incorporate "formal" into this endeavor in order to foster rigour and precision. A large portion of the content has been created (or adjusted) to fit the context of software development, even though our focus on the tasks that come before the actual programming itself does lead to machine independent abstractions frequently associated with mathematics.

- Formal languages are utilised in the requirements and testing phases of the SDLC.

Specification (Requirements Analysis Phase):

The process of defining a system's behaviour and intended characteristics is called specification. Formal specification languages are used to describe the internal structure, performance characteristics, functional behaviour, temporal behaviour and other attributes of a system.

While other formal methods like CSP, CCS, State charts, Temporal Logic, Lamport and I/O automata concentrate on expressing the behaviour of concurrent systems, Z, VDM and Larch are used to specify the behaviour of sequential systems. Rich state spaces are handled by RAISE, while concurrency-related complexity is handled by some languages like LOTOS.

Verification (Testing Phase):

The process of proving or disproving a system's correctness in relation to a formal

specification or property is known as verification. Model checking and theorem proving are two crucial methods for code verification.

- a. A finite state model of the system is constructed and its state space is mechanically examined during the model-checking process. NuSMV and SPIN are two well-known and comparable model checkers.
- b. Another method for confirming the accuracy of a program or verifying a specification is theorem proving. A theorem prover can demonstrate the desired attributes of a system model, which is expressed in a mathematical language. It is the logical proof mechanised. A mathematical notation is used to write the specification that a theorem prover is supposed to verify. One famous example of it is Z, which is pronounced “Zed.”

2.2 Mathematical Preliminaries

Mathematical preliminary works are the cornerstone of formal methods in software engineering, offering an organised framework for complicated system modelling, verification and design. The precise articulation of system behaviour and the construction of rigorous methods for assuring correctness are made possible by these fundamental ideas, which are founded in mathematical logic and set theory.

These mathematical foundations, which range from propositions and relations to functions, logic and automata theory, open the door to the creation of formal models that faithfully depict the complexities of software systems. These mathematical first steps, which emphasise precision and clarity, enable software engineers to critically analyse, verify and certify software systems—especially in security- and safety-critical areas.

These mathematical tools give stakeholders a common language, improving communication and understanding across the software development lifecycle. Examples of these mathematical tools are the expressive strength of predicate calculus, the modelling skill of automata and the abstraction powers of category theory. An understanding of these mathematical foundations is becoming more and more necessary for people who want to use formal methods to create software systems that are resilient, secure and dependable as software engineering advances.

2.2.1 Basic Concepts of Mathematical Preliminaries

Within the broad field of software engineering formal methods, the foundation of mathematical preliminary work provides a strong basis for modelling, describing and validating complicated systems. These mathematical ideas provide software engineers with an unsurpassed clarity in articulating needs and behaviours by acting as a precise and clear language. This paper delves into the fundamental ideas of mathematical preliminaries, revealing the complex web of functions, set theory, logic and other topics that support the creation of software systems that are secure and dependable.

The goal of rigour and precision in software engineering is at the core of formal methods. Mathematical preliminaries, which include a wide range of topics, offer the vocabulary and instruments required to describe, analyse and verify the behaviour of software systems. These ideas, which range from the fundamentals of mathematical logic to the complex nuances of set theory and beyond, influence the development of formal methods and help produce software that satisfies exacting standards for correctness and dependability.

Propositional Logic: The foundation of mathematical reasoning in formal methods is propositional logic. It handles with propositions, which are statements that can be true

Notes

or untrue. To represent the relationships between propositions, logical connectives like AND, OR and NOT are used. The basis for expressing conditions, limitations and logical ramifications inside a software system is provided by this logical language.

Predicate Logic: Propositional logic provides a strong foundation, but in more complicated situations, its shortcomings become evident. Predicates and quantifiers are introduced in predicate logic, which increases expressiveness. Predicates facilitate the declaration of attributes that involve variables, while quantifiers such as FOR ALL ($\forall$) and THERE EXISTS ($\exists$) allow the expression of properties that are universal and existential. For the purpose of defining complex interactions inside a system, predicate logic becomes indispensable.

Modal Logic: Modal logic arises from the necessity to encapsulate temporal and modal features inside a system. Certain modal operators, such as necessity ($\Box$) and possibility ($\Diamond$), enable the definition of conditions that either need to or might hold under certain circumstances. For the purpose of representing and reasoning about systems with dynamic states and possibilities, modal logic offers an essential dimension.

Temporal Logic: Software systems depend heavily on time and temporal logic gives us the means to analyse temporal aspects. The operators “eventually” ($\Diamond$) and “always” ($\Box$) allow attributes to be expressed that change over time. In order to model dynamic behaviours and guarantee that important conditions remain at particular times in time or consistently throughout a system’s execution, temporal logic becomes essential.

Sets and Elements: An essential idea in mathematical foundations, set theory offers a formal language to describe sets of discrete objects. Elements with the same property are referred to as sets and sets are used to express the domains and ranges of variables in a formal model. Sets are represented by curly braces.

Relationships and Relations: Although sets by themselves have great power, it’s the interactions between the pieces that reveal true expressiveness. Pairs of elements are connected by relations, which are subsets of the Cartesian product of sets. Modelling dependencies, interactions and relationships inside a software system requires an understanding of this fundamental idea.

Functions: Transformations and Operations: The idea of a function offers an organised method for expressing actions or transformations between sets. An element is mapped from one set (the domain) to another set (the codomain) by a function, represented as $f: A \rightarrow B$. Within a formal system, functions serve as the mathematical representation of the behaviours, transformations and operations of its constituent parts.

Propositional Calculus: Foundations of Logical Reasoning: Logical reasoning within formal methods is based on propositional calculus, a subset of formal logic. It handles propositions and logical operators, enabling the construction of intricate statements. Complex logical relationships can be expressed using logical connectives like AND, OR and NOT as building blocks.

Predicate Calculus: Expressing Quantified Relationships: By adding predicates and quantifiers, predicate calculus expands upon propositional calculus. Predicates involve variables and the extent of these variables is indicated by quantifiers like FOR ALL ($\forall$) and THERE EXISTS ($\exists$). A more sophisticated and expressive way to represent certain relationships and conditions inside a system is through the use of predicate calculus.

Finite-State Machines: Modeling System States: A formal framework for encapsulating the dynamic behaviour of systems is offered by automata theory. A basic idea in computer science, finite-state machines (FSMs) represent systems having a finite number of states,

transitions between states and related actions. FSMs play a crucial role in encapsulating distinct, state-dependent behaviours in software systems.

Model Checking: Ensuring System Properties: Model checking is a systematic method for confirming if a system meets a particular set of properties. Its foundation is found in automata theory. In order to verify that desired conditions, stated in temporal logic or other formal languages, hold, it entails investigating every potential state of a system. When used in complicated software systems, model checking proves to be an effective technique for guaranteeing accuracy and dependability.

Real and Complex Analysis: Ensuring Numerical Stability: Real and complex analysis becomes crucial when dealing with numerical computations and algorithms. The properties of real and complex numbers, continuity, limits and convergence are all covered in this area of mathematics. The accuracy and numerical stability of algorithms are guaranteed by real and complex analysis, which enhances the dependability of software systems that use numerical computations.

Model Theory: Understanding Knowledge of Formal Languages and Structures Within the field of mathematical logic, model theory explores the connections between formal languages and the structures they represent. It offers a theoretical framework for comprehending formal models' semantics. In order to guarantee that formal models faithfully represent the expected behaviours of software systems, model theory becomes essential.

Proof Theory: Constructing and Validating Proofs: A subfield of mathematical logic known as proof theory is concerned with the composition of mathematical proofs and the nature of mathematical reasoning. Proof theory becomes crucial in the setting of formal methods for creating and confirming system correctness proofs. It offers a methodical way to prove that a software system satisfies its requirements.

Lambda Calculus: Formalising Computation: A formal system in mathematical logic called lambda calculus offers a framework for expressing computation. Computations are represented by abstract variables and functions. Lambda calculus is especially useful in the context of formal methods for defining the semantics of functional programming languages and comprehending computation in software systems.

The foundational ideas of formal methods' mathematics foundations weave a complex and interwoven tapestry that characterises software engineering's precision language. All of these ideas—from the logical underpinnings of propositional and predicate calculus to the expressive potential of automata theory, functions and set theory—have a significant impact on how software engineers express, describe and think about complex systems. A thorough understanding of these mathematical foundations is essential for anybody looking to create software that is not only functional but also accurate, dependable and durable as software systems continue to increase in complexity and criticality. By navigating these mathematical ideas, we can move closer to a time when software systems will be built on the foundation of mathematical rigour, guaranteeing the reliability of the digital environment we live in.

2.3 Mathematical Notations for Formal Specification

Within software engineering, mathematical notations are a clear and accurate means of expressing system behaviours and needs in the context of formal specification. Software engineers are able to formally define complex specifications and assist rigorous analysis with the help of these mathematically grounded notations. These notations enable the explicit expression of system features, ranging from quantifiers like FOR ALL and THERE EXISTS in predicate logic to logical symbols like AND, OR and NOT in propositional

Notes

Notes

calculus. Functions are described using mathematical symbols like $f(x)$ to depict transformations, while set theory symbols like unions ($\cup$) and intersections ($\cap$) assist define domains and relationships between elements.

Operators like “eventually” ($\diamond$) and “always” ($\square$) are introduced by temporal logic to describe features of system behaviour that vary with time. The necessity ($\square$) and possibility ($\diamond$) symbols of modal logic enable the representation of modal attributes. Within automata theory, formal symbols are used in mathematical notations to express states, transitions and acceptance criteria.

A level of accuracy and clarity that spoken language may not be able to achieve is provided by the use of mathematical notations in formal specification, which helps to eliminate ambiguity and foster stakeholder consensus. When used effectively, these notations help create rigorous models that support software system validation and verification. Knowing these mathematical notations becomes crucial for reliable software development and communication as software developers use formal methods more and more.

2.3.1 Overview and Types of Mathematical Notations for Formal Specification

We go back to the block handler example below to show how mathematical notation can be used in the formal specification of a software component. To recap, user-created files are maintained by a crucial part of the operating system of a computer. The block handler keeps track of the blocks that are in use at all times in addition to maintaining a reservoir of unused blocks. Blocks are often added to a queue that is waiting to be added to the reservoir of unused blocks when they are released from a deleted file. The figure below shows a schematic representation of this.

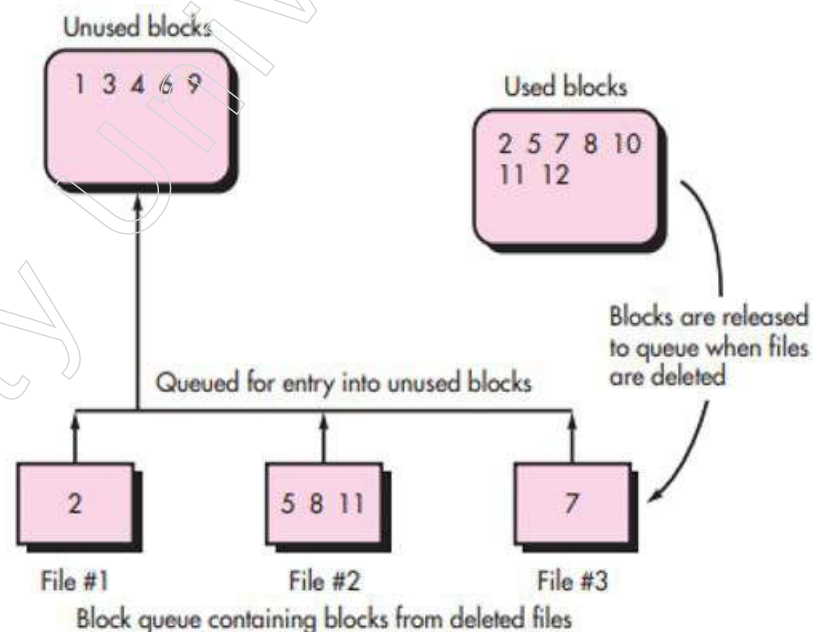


Figure: A block handler

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Example: A block handler. The subsystem responsible for maintaining user-generated files is one of the more crucial components of a basic operating system. The block handler is an element of the file subsystem. Blocks of storage that are kept on a file storage device make up files in the file store. Files will be added and removed while the machine

is operating, necessitating the purchase and release of storage blocks. The file subsystem will keep track of blocks that are currently in use and maintain a reservoir of unused (free) blocks to handle this. Blocks are often added to a queue that is waiting to be added to the reservoir of unused blocks when they are released from a deleted file. This is depicted in the figure above. The reservoir of idle blocks, the blocks that currently make up the files that the operating system is currently in charge of and the blocks that are awaiting addition to the reservoir are all depicted in this picture. The waiting blocks are kept in a queue, where each block in the queue is a collection of blocks from a file that has been erased.

The collection of available blocks, the collection of used blocks and the queue of returned blocks represent the state of this subsystem. When stated in plain English, the data invariant is

- No block will have the labels “used” and “unused” on it.
- Every set of blocks kept in the queue is a subset of the total number of blocks that are being used right now.
- There won't be any queue entries with the same block number.
- The entire collection of blocks that comprise files will be the combination of blocks that are utilised and unutilised.
- There won't be any duplicate block numbers in the collection of unused blocks.
- There won't be any duplicate block numbers in the used block collection.

Adding a collection of blocks to the end of the queue, removing a collection of used blocks from the front of the queue and adding them to the collection of unused blocks and checking to see if the block queue is empty are some of the activities related to this data.

The blocks to be added must be in the collection of used blocks in order for add() to function. The collection of blocks is now located at the end of the queue, which is the post condition. The queue needs to have at least one item in order for delete() to function. The addition of the blocks to the group of unused blocks is the post condition. There isn't a prerequisite for the check() function. This implies that, regardless of the value of the state, the operation is always specified. In the event that the queue is empty, the post condition returns true; otherwise, it returns false.

Each block number will be part of a set called BLOCKS. A collection of blocks that fall between 1 and MaxBlocks is called AllBlocks. Two sets and a sequence will be used to model the state. Both of the sets are free and in use. Each set has blocks: the free set has blocks that are accessible for use in new files and the used set has blocks that are currently used in files. Sets of blocks from deleted files that are prepared for release will be included in the sequence. One way to characterise the state is

used, free: P BLOCKS

BlockQueue: seqPBLOCKS

This is a lot like declaring variables in a program. It says that BlockQueue will be a sequence, with each element being a set of blocks and that used and free will be sets of blocks. One way to express the data invariant is as

$$\text{Used} \cap \text{free} = \emptyset \wedge$$

$$\text{Used} \cup \text{free} = \text{AllBlocks} \wedge$$

$$\forall i: \text{domBlockQueue} \bullet \text{BlockQueue}[i] \subseteq \text{used} \wedge$$

$$\forall i, j: \text{domBlockQueue} \bullet i \neq j \Rightarrow \text{BlockQueue}[i] \cap \text{BlockQueue}[j] = \emptyset$$

Notes

Four of the previously mentioned bulleted, natural language components match the mathematical components of the data invariant. The data invariant's first line declares that there won't be any common blocks in the utilised or free collections of blocks. The collection of free and used blocks will always equal the total collection of blocks in the system, according to the second statement. According to the third line, a subset of the used blocks will always make up the i th element in the block queue. The last line of the statement indicates that there won't be any shared blocks between any two distinct components in the block queue. Due to the fact that free and used are sets and won't include duplicates, the last two natural language components of the data invariant are realised.

An operation that removes an element from the top of the block queue must be defined first. The requirement is that there needs to be a minimum of one item in the queue:

$\#BlockQueue > 0,$

The removal of the queue head, placement of the head in the collection of free blocks and adjustment of the queue to reflect the removal are the post conditions.

$used' = used \setminus head\ BlockQueue \wedge$

$free' = free \cup head\ BlockQueue \wedge$

$BlockQueue' = tail\ BlockQueue$

A common practice in many formal methods is to prime a variable's value following an operation. Thus, the first part of the above equation says that the amount of new used blocks ($used'$) will be the same as the amount of old used blocks less the amount of blocks that have been eliminated. According to the second part, the head of the block queue will be added to the existing free blocks to create the new free blocks ($free$). The third component specifies that all components in the queue other than the first will make up the tail of the old block queue value, which is what the new block queue will be equal to. Ablocks, a group of blocks, are added to the block queue in a subsequent operation. The requirement is that Ablocks be a set of blocks that are in use at the moment:

$Ablocks \subseteq used$

The set of blocks must be added to the end of the block queue in order for the set of used and free blocks to stay the same.

$BlockQueue' = BlockQueue \sim \langle Ablocks \rangle \wedge$

$used' = used \wedge$

$free' = free$

Without a doubt, the block queue's mathematical specification is far more exacting than either a graphical representation or a narrative written in normal language. For certain application domains, the benefits of enhanced consistency and completeness outweigh the effort required to maintain the extra rigour.

A formal specification language is the name given to the representation utilised in formal methods. The language is formal in the sense that it may be used to express specifications in a straightforward and unambiguous way since it has a formal semantics. The task at hand can be clearly and succinctly specified using a formal specification language. Formal methods and specification language, with their strong mathematical foundation, offer a way to demonstrate that a specification is feasible, comprehensive, coherent and clear. Relatively simple mathematical constructs like sets, relations and functions can be used to model even the most complicated systems.

Typically, a formal specification language is made up of three main parts, or to put it mathematically, two sets: a set of relations and a set of syntax and semantics. The syntactic domain, often known as syntax, defines the precise notation used to convey specifications. Formal methods might differ significantly in their semantic area. The universe of objects that will be utilised to describe the system is defined in part by semantics. The rules that specify which objects correctly satisfy the specification are defined by a set of relations. The foundation of formal specification languages is mathematics. The majority of complex systems may be represented mathematically using elementary concepts like sets, relations and functions. A mathematical statement is clear and exact, making it possible to demonstrate that an implementation satisfies the mathematical specification and to offer strong justifications for one's answers.

A range of mathematical notations are used in the field of formal specification to precisely and concisely represent the behaviours, features and needs of a system. These mathematically based notations provide a vocabulary for expressing intricate relationships and circumstances inside software systems. Now let's examine some typical forms of mathematical notation that are employed in formal specification:

Predicate Logic Notation: A framework for expressing relationships and attributes involving variables is offered by predicate logic.

Example: "In predicate logic, one may express conditions such as 'For all elements x , $P(x)$ and $Q(x)$ are true.'"

Set Theory Notation: A mathematical foundation for characterising sets of elements and their interactions is provided by set theory.

Example: "Set theory notations, including union ($\cup$) and intersection ($\cap$), facilitate the concise representation of element relationships, such as ' $A = \{1, 2, 3\}$.'"

Temporal Logic Notation: Why and how qualities change throughout time are explained by temporal logic.

Example: "Temporal logic enables the articulation of dynamic behaviours, like 'The property P must always hold ($\forall t: \Box P(t)$).'"

Modal Logic Notation: Operators are introduced in modal logic to convey possibility and necessity.

Example: "In modal logic, statements such as 'It is possibly the case that Q ($\exists t: \Diamond Q(t)$).' capture the nuanced nature of system properties."

First-Order Logic Notation: Predicate logic is extended with quantifiers for existential and universal qualities in first-order logic.

Example: "First-order logic allows for statements like 'For every element x , if $P(x)$ holds, then $Q(x)$ must also hold ($\forall x: P(x) \rightarrow Q(x)$).'"

Higher-Order Logic Notation: Higher-order logic increases expressive capacity by allowing quantification over functions and predicates.

Example: "Expressing sophisticated relationships, higher-order logic allows statements like 'For any function f , there exists an x such that $f(x)$ equals zero ($\forall f \exists x: f(x) = 0$).'"

Z Notation: Z notation is a formal specification language based on first-order logic and set theory.

Example: "Z notation aids in formal specification with constructs like 'Given a set A , its cardinality $|A|$ represents the number of elements in A .'"

Notes

Linear Temporal Logic (LTL) Notation: A specific type of temporal logic called LTL is used to represent attributes over time sequences.

Example: "LTL allows succinct statements like 'Eventually, property P will hold ($\infty F P$).'"

Process Algebra Notation: A formalism for representing distributed and concurrent systems is offered by process algebra.

Example: "Process algebraic notations, such as ' $P \parallel Q$ ' denoting parallel composition, facilitate the representation of concurrent system behaviours."

State Machines Notation: State machines depict system states and transitions using graphical notations similar to state diagrams.

Example: "State machines express system dynamics visually, with transitions like 'On event E, transition from state A to state B.'"

Proof Theory Notation: Mathematical reasoning and proof structures are the subject of proof theory.

Example: "Proof theory notations assist in constructing and validating rigorous proofs, ensuring the correctness of specified properties."

Category Theory Notation: Mathematical links and abstract structures are provided by category theory.

Example: "Category theory notations, including morphisms and compositions, offer a high-level abstraction for understanding relationships between different structures."

Lambda Calculus Notation: A formal framework for representing computing is lambda calculus.

Example: "Lambda calculus notations, like ' $(\lambda x. x + 1)$ ', succinctly represent functional transformations within a system."

2.4 Formal Specification Languages

A formal specification language is the name given to the representation utilised in formal methods. The language is formal in the sense that it may be used to express specifications in a straightforward and unambiguous way since it has a formal semantics. The task at hand can be clearly and succinctly specified using a formal specification language. Formal methods and specification language, with their strong mathematical foundation, offer a way to demonstrate that a specification is feasible, comprehensive, coherent and clear. Relatively simple mathematical constructs like sets, relations and functions can be used to model even the most complicated systems.

Typically, a formal specification language is made up of three main parts, or to put it mathematically, two sets: a set of relations and a set of syntax and semantics. The syntactic domain, often known as syntax, defines the precise notation used to convey specifications. Formal methods might differ significantly in their semantic area. The universe of objects that will be utilised to describe the system is defined in part by semantics.

The rules that specify which objects correctly satisfy the specification are defined by a set of relations. The foundation of formal specification languages is mathematics. The majority of complex systems may be represented mathematically using elementary concepts like sets, relations and functions. A mathematical statement is clear and exact, making it possible to demonstrate that an implementation satisfies the mathematical specification and to offer strong justifications for one's answers.

2.4.1 Overview of Formal Specification Languages

Typically, a formal specification language consists of three main parts:

1. a collection of relations that specify the rules that indicate which objects correctly satisfy the specification,
2. semantics to help define a “universe of objects” that will be used to describe the system and
3. a syntax that defines the precise notation with which the specification is written

A formal specification language’s syntactic domain frequently stems from predicate calculus and standard set theory notation. A specification language’s semantic domain reveals how the language expresses system requirements.

Different semantic abstractions can be applied to explain the same system in various ways. There was representation of behaviour, function and information. For the same system, many modelling notations might be employed. Complementary perspectives on the system are provided by the semantics of each representation.

Assume that a formal specification language is being used to describe the sequence of events that lead to a specified state occurring in a system in order to demonstrate this approach when formal methods are applied. A different formal relation represents every function that takes place in a specific state. An indicator of the circumstances that will lead to the occurrence of particular functions can be found where these two relations overlap.

Various Formal Specification Language types include:

Model Based Languages:

A precise specification can be written in a variety of ways. Languages based on models are one method. It expresses the specification as a model of the system state. Sets, relations, sequences, functions and other well-known mathematical concepts are used in the construction of this state model. A system’s operations are defined by stating how they impact the system model’s state. The predicates provided in terms of pre and post conditions also characterise operations. For creating model-based languages, Zed (Z), B and Vienna Development Method (VDM) are the most often used notations.

Algebraic Specification Languages

Although algebraic manipulation is possible with process algebras, there exist languages that describe a system only in terms of its algebraic attributes. These algebraic specification languages characterise a system’s intended properties in terms of axioms that define its behaviour. The Common Algebraic Specification Language (CASL) and OBJ are two instances of algebraic specification languages. Mathematically speaking, an algebra (or algebraic system) is made up of two parts: (1) a collection of symbols, known as the algebra’s carrier set, that represent different kinds of values and (2) a set of operations on the carrier set.

Process oriented Languages

Process oriented formal specification language is used to define concurrent systems. These languages are based on a certain implicit concept of concurrency. Expressions and elementary expressions, which describe especially simple processes, respectively, serve as the denotation and building blocks for processes in these languages. For example, exchanging sequential processes.

Notes

Notes

Hybrid Languages

An analogue and digital component combination is used in the construction of many systems. Such systems require the use of a specification language that incorporates both discrete and continuous mathematics in order to be specified and verified. These hybrid languages, like CHARON, have drawn attention recently. A temperature controller is a basic illustration of a nonlinear hybrid system. A thermostat regulates a room's temperature by continuously measuring the ambient temperature and turning on and off the heater.

In software engineering, formal specification languages are crucial instruments that provide an organised and quantitative method for expressing, simulating and validating system behaviours and requirements. In this thorough investigation, we examine a variety of formal specification languages, clarifying their characteristics, uses and importance in the creation of dependable software systems. We take you on a thorough tour of the world of formal specification languages, starting with the fundamental ideas of Z Notation and ending with the useful uses of languages like Promela and ACL2.

Z Notation: A Mathematical Language for Rigorous Specifications

One of the mainstays of formal specification languages is Z Notation. Z Notation was developed in the late 1970s to give a clear and succinct way to express system requirements using mathematical constructs such as first-order logic and set theory. Because of its syntax, software developers may define sets, functions and schemas and accurately and mathematically describe complex system aspects. In safety-critical applications, where a strict specification is essential to guarantee the accuracy of crucial equipment, Z Notation is especially helpful.

Z Notation's expressive power comes from its capacity to depict intricate interactions and limitations inside a software system. The set of positive integers, for example, is indicated by a Z Notation statement like $\{x: \text{Integer} \mid x > 0\}$, demonstrating the language's capacity to concisely describe attributes. Z supports a step-by-step refining technique that makes it easier to gradually elaborate requirements and guarantees a methodical development process.

Vienna Development Method (VDM): Balancing Rigor and Practicality

The Vienna Development Method represents a dedication to mathematical rigour in software development and includes both VDM-SL (Specification Language) and VDM++. Since VDM places a strong emphasis on formal specification, design and verification, it is a useful tool in fields where accuracy is crucial. With its foundation in predicate logic and set theory, VDM-SL helps engineers precisely articulate system attributes. However, VDM++ expands on this expressiveness to incorporate object-oriented ideas in order to support contemporary software development paradigms.

The full software development lifecycle is supported by VDM, which is one of its strongest features. VDM offers a comprehensive framework from original specifications to design refinement and certification. Engineers may clearly specify types, functions and invariants by expressing properties in VDM using mathematical symbols and notation. This guarantees that the designated system follows its intended behaviour, promoting trust in the accuracy of the finished product.

B-Method: Stepwise Refinement for Systematic Development

The B-Method is well known for its methodical approach to software development through incremental refinement. It is based on mathematical notations and set theory. The B-Method was created with the intention of guaranteeing accuracy and dependability. It

enables engineers to gradually develop system specifications and verify their accuracy at every level of refinement.

The idea of refinement, which involves gradually turning an abstract specification into more executable and concrete forms, is fundamental to the B-Method. This iterative procedure offers a methodical way to attain correctness by guaranteeing that every refinement stage complies with the intended system attributes. The use of mathematical symbols and constructions in the B-Method's notation demonstrates its dedication to a formal and exacting development process.

Event-B: A Framework for Modeling and Refinement

A formal technique called Event-B, which has its roots in predicate logic and set theory, offers a foundation for system-level modelling and improvement. It places a strong emphasis on using events to record system behaviours and methodically converts abstract models into tangible applications. Because of Event-B's mathematical base, engineers are able to clearly and unambiguously depict system behaviour by expressing complicated relationships and attributes.

Complex features can be expressed because to the language's use of mathematical symbols like $\in$ (element of), $\cup$ (union) and $\Rightarrow$ (implies). With Event-B's modelling approach, an abstract model is built through a series of refinement phases, each of which adds additional information about the behaviour of the system. This methodical improvement procedure is in line with the language's objective of guaranteeing accuracy by using a development methodology that is grounded in mathematics.

Temporal Logic of Actions (TLA+): Capturing Dynamic System Behaviour

TLA+ expands the expressive capability of formal specification languages to capture dynamic system behaviours over time. It is based on temporal logic. TLA+, created by Leslie Lamport, emphasises the temporal evolution of system states and allows the modelling of concurrent and distributed systems. Temporal operators like $\diamond$ (eventually) and $\square$ (always) are incorporated into the language's notation, enabling engineers to express qualities that change over time.

Engineers can define system requirements precisely in TLA+ by defining temporal attributes and ensuring their accuracy. For example, the statement $\square(\text{Init} \Rightarrow \square \text{Next})$ states that the following state (Next) will always follow globally if the system satisfies the startup condition (Init). TLA+ is a useful tool in the development of dynamic systems since it encourages the production of clear and expressive specifications.

Alloy: Lightweight Modeling for Analysis

Modelling and analysis are the main areas of concentration for the lightweight formal specification language Alloy. Alloy, developed at MIT, describes system structures and restrictions using a relational logic approach. Because alloy specifications are clear and expressive, engineers can use the Alloy Analyzer tool to model complicated systems and conduct investigations.

System properties are compactly represented by the language's notation, which contains relational operators like $\subset$: (subset), $\Rightarrow$ (implies) and $\cap$ (intersection). Because alloy requirements are executable by nature, model checking can identify any problems early in the design phase. Alloy is a good choice for investigating the design space and confirming the consistency of complex system models because of its focus on analysis and simulation.

Notes

Communicating Sequential Processes (CSP): Modeling Concurrency with Mathematical Precision

A formal language called Communicating Sequential Processes (CSP) is used to model and analyse concurrent systems. Tony Hoare created CSP, which uses mathematical notation to describe how many processes interact and communicate with one another within a system. With operators like $\parallel$ (parallel composition), $\rightarrow$ (send) and $\leftarrow$ (receive) in its notation, CSP offers a succinct way to depict concurrent system behaviour.

In situations where the accuracy of concurrent processes is crucial, CSP is very useful. With mathematical accuracy, engineers can explain communication patterns, synchronisation and parallelism using this language. Software engineers can ensure that processes interact as intended without introducing catastrophic mistakes by modelling and verifying the behaviour of concurrent systems using CSP.

Lustre: Synchronous Dataflow for Reactive Systems

A synchronous dataflow language called Lustre is dedicated to describing and creating reactive systems. Designed for situations where safety is of utmost importance, Lustre utilises a declarative methodology to articulate data flow and the connections among various system components. When-else and $\rightarrow$ (arrow) are two examples of the structures used in Lustre's notation that give reactive behaviours a clear and straightforward visual representation.

With Lustre, engineers may model both the discrete and continuous components of reactive systems by using automata and equations to explain system behaviours. Because of its focus on synchronous dataflow, the language is predictable and appropriate for applications where event timing and coordination are crucial. Applications for Lustre can be found in fields like medical devices, automotive systems and avionics.

Promela (Process Meta Language): Modeling Concurrent Systems for Verification

Promela, which stands for Process Meta Language, is a modelling language used for concurrent system verification in conjunction with the Spin model checker. Promela, created by Gerard J. Holzmann, enables engineers to explain the behaviours and interactions of multiple processes running simultaneously. Keywords like proctype, active and atomic are included in the language's notation to facilitate the expression of concurrent system features.

Promela is very useful for model-checking concurrent systems to ensure they are correct. With Spin, engineers may methodically search the state space for possible faults, describe attributes using temporal logic and design system processes. The combination of Promela and Spin strengthens the verification process's rigour and guarantees the dependability of concurrent system architectures.

ACL2 (A Computational Logic for Applicative Common Lisp): Integrating Theorem Proving and Programming

Formal specification and verification are included in the computer language ACL2, which stands for A Computational Logic for Applicative Common Lisp. ACL2, which was created using Common Lisp, is a programming and reasoning language that facilitates the development of high-assurance systems. The notation of the language contains constructions like defun (define function) and defthm (define theorem), which allow one to express functional specifications along with the corresponding proofs.

The unique quality of ACL2 is that it provides a unified environment to assist the creation of algorithms and the associated correctness proofs. The language's theorem

prover provides a strong method for guaranteeing the dependability of crucial system components by mechanically confirming the accuracy of given attributes.

Navigating the Landscape of Formal Specification Languages

The difficult process of developing strong and dependable software systems is made easier by the huge array of formal specification languages, each of which adds special features and methods. Software developers have a wide range of tools at their disposal, depending on the particular requirements and features of their projects. These tools range from the fundamental concepts of Z Notation and VDM to the dynamic expressiveness of TLA+ and the lightweight modelling of Alloy.

A formalised mathematical foundation for expressing, modelling and testing system features is the unifying goal of these languages. Whether it is by temporal logic, lightweight modelling, or step-by-step refining, each language provides an organised method for system development that improves the accuracy and correctness of software systems.

The system's nature, the application's criticality and the development team's tastes are some of the variables that influence the choice of formal specification language. Formal specification languages play a critical role in ensuring that the software we develop is not just functional but also dependable, safe and able to meet the demanding requirements of contemporary applications, even as software engineering continues to advance.

2.4.2 Types of Formal Specification Languages

The main principle of a formal method is that formal methods employ a formal or mathematical vocabulary to provide a detailed specification of a system, which is beneficial. This syntax can be graphical, but it is typically textual. Additionally, a semantics is supplied, meaning that every description in the language has a specific meaning.

A system's architecture, structure and/or functional behaviour, as well as non-functional behaviour like timing and performance standards, can all be covered in one or more of these specifications.

There are several applications for a system's exact specification. First, it can be applied as a method for articulating a correct understanding of the system and identifying any flaws or incompleteness. In addition, the specification can be examined or its accuracy checked against relevant properties.

A specification can also be used to direct the development process, either directly through the generation of code or by revising the specification in the direction of code. Testing is, of course, a crucial part of the development process and it can also be supported by specifications. In fact, this paper's goal is to thoroughly examine this problem.

There are numerous formal specification approaches available; some are broad in nature, while others focus on elements pertinent to specific application areas (concurrent systems, for example). The majority are supported to differing degrees by tool support. In the following, we briefly review some of the most often used notations before talking about how the testing process might make use of them.

Model-Based Languages

A precise specification can be written in a variety of ways. One method is to create a model of the desired behaviour. Languages like Z, VDM and B do this by outlining the possible states of the system as well as the actions that can alter those states. Sets, sequences, relations and functions are commonly used to characterise the states of the system, whereas predicates expressed in terms of pre- and post conditions are used to

Notes

characterise operations. Such a specification can be structured in a variety of ways. For instance, Z does this by the use of a language of schemas, where each schema is made up of a declaration and a predicate that limits the schema.

Take the specification of a bounded stack, for instance. We use a schema to represent the possible states of the stack. The stack is intended to be used for storing elements that belong to a specific set, such as *Object*. This can be expressed as follows in Z:

[*Object*]

At this level of abstraction, the structure of the set's items is unimportant. This constructs a set called *Object*. Additionally, we provide a constant, *maxSize*, beyond which the size of the stack cannot increase in order to define the bounded stack.

$\text{maxSize} == 20000$

We can now use the following schema to describe the possible states of the stack:

Stack

$\text{items} : \text{seqObject}$

$\# \text{items} \leq \text{maxSize}$

By doing this, items are defined as a series of elements from the set *Object* whose length does not exceed *maxSize*. The system's initial configuration is provided by the *StackInit* startup protocol. The after state of a component is indicated by the priming (') of that variable. *Items'*, thus, refers to the condition of variable *items* after one.

StackInit

Stack'

$\text{Items}' = \langle \rangle$

If the stack is not empty, the subsequent operation, *Top*, returns the value at the top. The state of *Stack* is not altered by the *Top* operation.

Top

ΔStack

$x! : \text{Object}$

$\text{items} \neq \langle \rangle$

$x! = \text{head items}$

With clear functionality, we can further describe the standard *Pop* and *Push* operations on the stack as follows:

Pop

ΔStack

$x! : \text{Object}$

$\text{items} \neq \langle \rangle$

$\text{items}' = \text{tail items}$

$x! = \text{head items}$

Push

ΔStack

$x? : \text{Object}$

$\# \text{items} < \text{maxSize}$

$\text{items}' = \langle x? \rangle \hat{\ } \text{items}$

Z uses a number of conventions. For instance, names that finish in? stand for input and names that end in! stand for output (thus, $x?$ and $x!$ are inputs and outputs of type Object, respectively). For components declared in Stack, Δ Stack indicates a possible change in state, while Ξ Stack indicates no change in state in an operation model.

Finite State-Based Languages

Model-based languages with theoretically endless states, like Z, VDM and B, can describe systems that are arbitrary general. One disadvantage of this generality is that it reduces the automation potential of reasoning. However, finite state-based specification languages do not have this problem.

Finite state-based languages, as its name implies, define their state from a finite set of values. These values are frequently displayed graphically, with state transitions signifying state changes that resemble operations in a notation like Z. Finite state machines (FSMs), SDL, Statecharts and X-machines are a few examples of these languages.

When testing from such requirements, test approaches based on FSM have been taken into consideration. Protocol conformance testing has served as a major driving force behind the development of software testing from FSMs, as these tools are useful for defining a communications protocol's control structure. But more recently, FSM-based testing has been incorporated into a methodology known as model-based testing, in which testing is guided by the creation of a model. Keep in mind that model-based testing and model-based specification languages use the word "model-based" somewhat differently.

Formally speaking, an FSM is defined as

$$F = (S, X, Y, h, s_0, D_p)$$

Where

- S is a set of n states with s_0 as the initial state;
- X is a finite set of input symbols;
- Y is a finite set of output symbols;

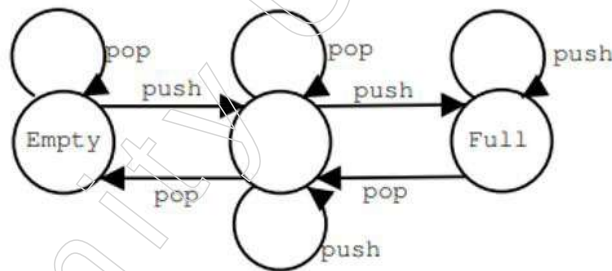


Figure: An FSM for a bounded stack

Image Source: https://bura.brunel.ac.uk/bitstream/2438/1871/1/landscapes_final.pdf

A bounded stack would have an FSM representation, as seen in the above figure. The conditions are represented by the following three states: the stack is empty, the stack is full and the stack is neither empty nor full. Take note that since top transitions do not change the state, they have not been included. Additionally, the FSM is finished in this diagram, so the actions of pushing on a full stack and popping on an empty stack are indicated.

The FSM as demonstrated is non-deterministic because the actions push and pop have the potential to either advance the stack to a different state or leave it in its current state when it is neither full nor empty. Pop, for instance, sends an element in the stack to the state Empty if it is the only one there; otherwise, pop leaves the state unchanged.

Notes

It is feasible to create a deterministic FSM that represents the stack if the stack's size is known. In this case, one state would exist for each $0 \leq i < n$ if the stack size was n . The state corresponding to i would indicate the circumstance under which the stack includes i elements.

There is additional internal data in many specification languages with finite state structures, like State charts, SDL and X-machines. Transitions are activities that have the ability to access, modify and have guards refer to this data. An extended finite state machine is one such specification (EFSM). An EFSM can be enlarged into an FSM if the data is made up of variables with finite types, albeit this could lead to a combinatorial explosion. Since such specifications depict the system using a finite set of states and an internal memory, we refer to them as finite state-based specifications even if the types are not finite.

While notations like SDL, Statecharts and X-machines provide the explicit depiction of concurrent activity, approaches like Z, VDM and B are largely focused on the description of sequential systems. These languages' specifications are frequently viewed as one or more potentially communicative EFSMs.

Process Algebra State-Based Languages

One can treat concurrency in a very beautiful algebraic way and process algebras characterise a system as a collection of interconnected concurrent processes. Systems that are composed of multiple concurrently communicating processes can also be described using finite-state languages like Statecharts and SDL. On the other hand, a rich theory of process algebras offers substitute conceptions of compliance expressed in terms of implementation relations. The implementation relations capture many features of the environment and traces, which are sequences of inputs and outputs, as well as various kinds of observations that can be made.

For instance, CSP defines a system as a group of interconnected, continuously operating processes that synchronise in response to events.

One possible CSP definition of a stack might be as follows:

$$\begin{aligned} \text{Stack}(\langle \rangle) &= \text{push}?x : X \rightarrow \text{Stack}(\langle x \rangle) \\ \text{Stack}(\langle y \rangle \hat{\ } s) &= \text{push}?x : X \rightarrow \text{Stack}(\langle x \rangle \hat{\ } \langle y \rangle \hat{\ } s) \\ &\quad \square \\ &\quad \text{pop}!y \rightarrow \text{Stack}(s) \end{aligned}$$

In this case, guarded equations describe a parameterised process. Push and pop channels can be associated with an input, $?x : X$, or an output, $!y$. These events have the ability to preface behaviour (denoted with $\rightarrow$), hence enabling regulated event sequencing. The operator $\square$ in this case represents choice and it's significant that there exist operators to depict concurrent and communicative action. Keep in mind that this stack example is comparable to one that can be expressed in a language with limited states. The distinctions between finite state-based languages and process algebras are illustrated in the example that follows.

When two processes are run simultaneously without the need for component synchronisation, this is known as interleaving. The process describes it.

$$P1 \parallel P2$$

wherein $P1$ and $P2$ processes run entirely apart from one another (without even synchronising on shared events). A variety of operators in CSP are also available to characterise parallel composition, which enables processes to selectively synchronise

on events. The interface parallel operator, for instance, interleaves all other events while synchronising on events within a specified set A. This permits P1 and P2 to develop independently, but events in A are only activated when they are activated in P1 and P2.

The behaviour of a specification stated in a process algebra can be described by a labelled transition system (LTS), hence testing from LTSs has been the main focus of work on testing from such specifications. It makes intuitive sense to think of an LTS as a representation of a system's behaviour, which is determined by the occurrence of events that alter the system's state. Informally, they are best described as graphs or trees, in which transitions are represented by edges labelled with events and states are represented by nodes. Consider, for instance an example is described in the figure below. Here, the input of a shilling is represented by the single initial action, shil. The system can now interact with its surroundings via the actions liq and choc, which stand for the output of a bar of chocolate and licorice, respectively, after receiving the input of a shilling. After doing this, the system is considered to deadlock because it is unable to take any more action.

The observable events that accompany labels indicate the range of possible actions that the system is capable of. An internal (quiet) action is indicated by a specific event τ . Internal processes are imperceptible.

Remember that the regular language $L(F)$ represents the semantics of an FSM F and that $L(F1) \subseteq L(F2)$ is a necessary condition for one FSM $F1$ to conform to another FSM $F2$. This is equivalent to looking at components of the set of traces that the SUT might generate.

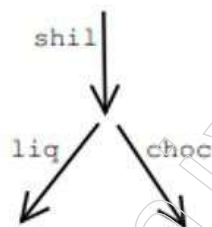


Figure: A simple LTS

https://bura.brunel.ac.uk/bitstream/2438/1871/1/landscapes_final.pdf

Hybrid Languages

An analogue and digital component combination is used in the construction of many systems. Such systems require the use of a specification language that incorporates both discrete and continuous mathematics in order to be specified and verified. These hybrid languages, like CHARON, have drawn attention recently.

A temperature controller is a basic illustration of a nonlinear hybrid system. A thermostat regulates a room's temperature by continuously measuring the ambient temperature and turning on and off the heater. The differential equations control the temperature. The temperature x drops when the heater is turned off, as per

$$x(t) = \theta e^{-Kt}$$

When t is the time, θ represents the initial temperature and K is a room-specific constant. The heater's operating temperature is controlled by

$$x(t) = \theta e^{-Kt} + h(1 - e^{-Kt})$$

where h is a constant that is influenced by the heater's power. The heater is assumed to be on and the temperature to be m degrees at first. Additionally, we would like the temperature to remain between m and M degrees. The system is described by a hybrid automaton in the figure below.

Notes

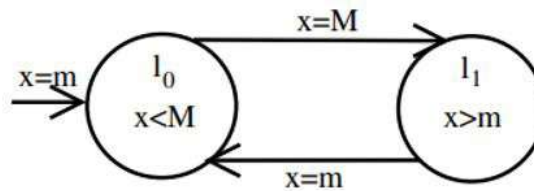


Figure: Hybrid Automaton for Temperature controller

The automaton, as observed, has two states, l_0 and l_1 , with an initial condition of $x = m$. Every state has a physical law that controls its rate of change and is accompanied by an invariant (these laws are not represented in the above Figure). For instance, in state l_1 , when the heater is off, the room temperature is always maintained at a higher level than the minimum temperature and $x(t) = \theta e^{-Kt}$ controls the rate of change in temperature. The heater is activated (state l_0) as soon as the ambient temperature falls to m degrees.

Algebraic Languages

Although algebraic manipulation is possible with process algebras, there exist languages that describe a system only in terms of its algebraic attributes. These algebraic specification languages use axioms to characterise desirable qualities of a system and then use those properties to describe the behaviour of the system. The Common Algebraic Specification Language (CASL) and OBJ are two instances of algebraic specification languages.

An algebra (or algebraic system) is defined mathematically as: (1) a set of symbols, known as the algebra's carrier set, that represent values of a certain kind; and (2) a set of operations performed on the carrier set. The following details must be included in order to define the rules that control how the operations behave:

- The syntax of the operations. This is accomplished by providing a signature for every operation that specifies the domain and range (or co-domain), which, in turn, correspond to the input parameters and the operation's output, respectively.
- The semantics of operations. Equations (or axioms) that implicitly specify the necessary qualities are used to do this. Typically, the axioms are expressed as equations, each of which could have a qualification.

A somewhat simplified bank system with an account type, for instance, might have the following operations in its algebraic specification:

- ❖ empty to create a new empty account;
- ❖ credit to credit money to an account;
- ❖ debit to withdraw money from an account;
- ❖ balance to enquire about the balance remaining in an account

For the sake of simplicity, let's assume that there is a specification of Nat , the natural number type, which is used to represent money. The full specification, complete with operation signatures and relevant equations, could look something like this:

```
spec Bank =
  Nat
then
  sort account
  ops
    empty: account
```

```

credit : Nat * account -> account;
debit  : Nat * account -> account;
balance : account -> Nat

vars
  n: Nat; acc: account
axioms
  balance( empty ) = 0;
  balance( credit(n,acc) ) = balance(acc) + n;
  balance(acc) ≥ n => balance( debit(n,acc) ) = balance(acc) - n;
  balance(acc) < n => balance( debit(n,acc) ) = 0 end

```

Notes

An empty account has a balance of zero, according to the first axiom. According to the second axiom, an account's balance before a transaction with n added is equal to its balance after n units of money are credited to it. Similar procedures are followed by the third axiom for handling debits, but it subtracts rather than adds and it only applies if the withdrawal does not result in a negative balance. The final axiom states that an effort to debit more money than is in the account results in a zero balance, meaning that overdrafts are not permitted.

Observers, constructors, transformers and constants are the several categories into which operations can occasionally be divided for practical purposes. An initial object of the considered abstract type is returned by a constant operation. A value of the type is modified (or transformed) in some way by constructor and transformer operations. Constructors and transformers are different in that the former, along with constant operations, form a minimal set of operations that can be used to generate any value of the type, or the carrier set. Values of a different type than the one being considered are returned by observer operations. In the given example, the operations `empty`, `credit` and `debit` are constructors and the operation `balance` is an observer. There are no transformer operations in the standard as it is written.

An essential justification for the algebraic method of specification is its ability to occasionally offer a means of assessing syntactically sound but otherwise arbitrary combinations of operations. For instance, the process of opening an account, adding \$100 to it and then asking what the account's balance is after all of that would be expressed as the following:

$E = \text{balance}(\text{credit}(100, \text{empty}))$

Using axiom 2 with $n=100$ and $\text{acc}=\text{empty}$, this can be written as:

$E = \text{balance}(\text{empty}) + 100$

Then, using axiom 1, this becomes:

$E = 0 + 100$

Term rewriting is the method previously described, in which the axioms are used as rewrite rules. A specification is considered canonical if all of its rewrite sequences converge to a single normal form in a finite number of steps. Since E does not include any free variables, in contrast to the axioms, it is an example of a ground term. When it comes to testing, term rewriting is especially helpful since it allows algebraic specifications to be practically executed using specified test scenarios. Generally speaking, though, theorem proving has more potential because it makes it possible to prove important properties of such requirements, this is sometimes necessary in testing.

There are numerous distinct algebraic specification notations, each having a unique concrete syntax, it should be emphasised. In fact, dialect can vary greatly even among

Notes

members of the same notation family. However, the concepts mentioned above—that is, that an operation's syntax is determined by its signature and its behaviour when combined with other operations is determined by equations—are universal.

2.4.3 Difference between Informal and Formal Specification Language

Formal and informal specification languages are two categories into which requirements specification languages can be divided. Natural language, such as English, is used to define needs in informal specification language. However, they frequently have a number of flaws, like contradictory, ambiguous, ambiguous and incomplete statements in a system definition.

Contradictions: sets of assertions that contradict one another. For instance, a system specification may specify in one section that the system must monitor every temperature in a chemical reactor, while a different section—possibly written by a different staff member—may specify that the system should only monitor temperatures that fall within a particular range. Contradictions that appear on the same page of a system specification are typically easy to spot. Contradictions, however, are frequently spaced out over many pages.

Ambiguities: statements with multiple possible interpretations. As an illustration, the following claim is unclear: The operator name and password, which is made up of six numbers, make up the operator identification. When an operator signs into the system, it ought to be shown on the security VDU and added to the login file. Does the term "it" in this excerpt refer to the identity of the operator or the password?

Vagueness: It frequently happens as a result of a system specification's large file size. It is quite impossible to continuously get a high degree of precision. It may result in claims like "the virtual interface should be based on simple overall concepts that are straightforward to understand and use and few in number" or "the interface to the system used by radar operators should be user-friendly." A cursory glance at these statements may miss the fundamental lack of information that is relevant.

Incompleteness: the system specification issues that arise most frequently. Take into account, for instance, the functional requirement: Depth sensors installed within the reservoir should be used by the system to maintain the reservoir's hourly level. It is recommended to save these numbers for the previous six months. This explains a system's primary data storage component. If one of the system's orders was: The AVERAGE command's purpose is to show the average water level for a certain sensor on a PC between two occasions. If no additional information was provided for this command, the details would be woefully lacking. For instance, what should happen if a system user enters a time that was more than six months prior to the current hour is not mentioned in the description of the command.

Conversely, though Formal specification languages use a formal notation to express system requirements and have a mathematical foundation, typically in the form of formal logic. All specification methods aim to achieve the desired characteristics of a formal specification, which include consistency, completeness and absence of ambiguity. On the other hand, the possibility of accomplishing these goals is significantly better when formal methods are used. The ambiguity that frequently arises when a reader must interpret a graphical notation or a natural language (such as English) is eliminated by the formal syntax of a specification language, which allows requirements or designs to be interpreted in only one way. Clear factual statements are made possible by the descriptive capabilities of logic notation and set theory (requirements). A specification must not contradict itself in two different places with the same facts in order to be considered consistent. The

mathematical proof that initial facts can be formally transferred (using inference rules) into later statements inside the specification ensures consistency. Even with the use of formal methods, achieving completeness can be challenging. When creating a specification, some features of a system might be left out on purpose to give designers more leeway in selecting an implementation strategy; lastly, it is impossible to take into account every possible operational scenario in a large, complex system. Things can just be inadvertently left out.

The complex process of developing software necessitates accuracy, lucidity and a methodical approach in order to convey the many aspects of a system. Specification languages, or tools that bridge the gap between human comprehension and machine execution, are at the vanguard of this articulation. Two main categories—formal and informal specification languages—emerge within this range. This talk explains how different domains differ from one another and explores their traits, uses and effects on the development lifecycle.

Informal Specification Language: The Language of Intuition

- Unofficial specification Languages differ from one other in that they are not based on mathematical formalism or strict structures, but rather in natural language.
- Their grammar is more human-readable and straightforward, which makes them more understandable to a wider range of users, including non-technical staff and stakeholders.
- Informal specifications frequently use narrative descriptions, diagrams and sentences that are similar to those in English.

Flexibility and Accessibility

- Informal specifications put flexibility and accessibility first, trying to convey system requirements at a high level without getting bogged down in minute technical specifics.
- They are frequently used at the beginning phases of a project to help clients, developers and other stakeholders communicate with one another.

Pros:

- ❖ Accessibility: Informal specifications promote understanding and cooperation since they are easily accessed by a large audience.
- ❖ Agility: They are perfect for dynamic and changing projects since they enable quick iteration and adaptability.

Cons:

- ❖ Ambiguity: The absence of a defined framework may cause confusion and different interpretations.
- ❖ Limited Rigor: For important systems or those with strict criteria, informal specifications could not have the level of precision required.

Examples

- ❖ Natural Language Descriptions: simple text descriptions in any natural language, such as English.
- ❖ Use Case Diagrams: diagrams at a high level that show system interactions without using formal restrictions.

Notes

Formal Specification Language: The Language of Precision

- Formal specification languages provide an accurate and clear description of system attributes since they are based on mathematical notation and logical constructions.
- Typically derived from formal methods such as set theory, mathematical logic and others, they are distinguished by a formal syntax.
- Prioritising expressiveness and precision allows for thorough analysis, validation and verification.

Precision and Rigor:

- Formal specifications provide priority to accuracy and consistency, facilitating mathematical inference about the characteristics of the system.
- In order to express needs, limitations and behaviours, they frequently entail the use of symbols, operators and mathematical notation.

Pros:

- ❖ Precision: Formal specifications reduce the possibility of misunderstanding by providing a precise and unambiguous representation.
- ❖ Verification: They support rigorous verification through formal methods like theorem proving and model checking.

Cons:

- ❖ Learning Curve: Formal languages may require a deeper understanding of mathematical principles and have a more difficult learning curve.
- ❖ Development Time: Formal specifications can take a lot of effort to create, particularly in the beginning of a project.

Examples

- ❖ Z Notation: a formal specification language that makes use of first-order logic and sets as mathematical structures.
- ❖ Temporal Logic: used to convey temporal features, guaranteeing that specific conditions are met at certain times.
- ❖ Event-B: a formal approach that emphasises mathematical accuracy for system-level modelling and development.

Table: Comparative Analysis

	Informal	Formal
Expressiveness	depends on natural language, which makes it eloquent when expressing abstract concepts but ambiguous at times.	uses mathematical notation to enable clear and precise communication, but it may limit the expressiveness of non-technical stakeholders.
Clarity and Ambiguity	Due to its reliance on natural language, it is ambiguity-prone and may cause misunderstandings.	minimises uncertainty and promotes a common understanding among technical professionals by placing a priority on clarity through mathematical rigour.
Verification and Validation	limited assistance for formal verification; testing and inspection are frequently done by hand.	increases trust in the accuracy of the system by supporting rigorous verification through formal methods like theorem proving and model checking.

Applicability	Ideal for first conversations, ideation and correspondence with non-technical stakeholders.	Ideal for situations requiring a high degree of assurance of correctness, safety-sensitive applications and critical systems.
Iterative Development	allows for quick adaptation and iteration, which gives dynamic projects flexibility.	Because of the need for precision, this could entail a more regulated and possibly delayed development process.

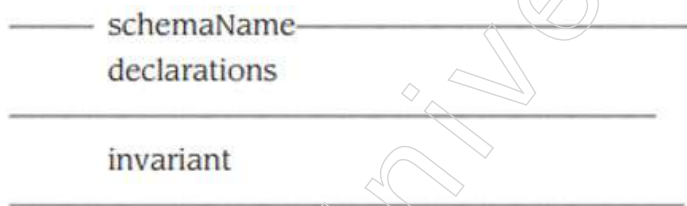
Notes

2.5 Z-Notations

The specification language Z, which is commonly used in the formal methods community, is pronounced correctly as “zed.” The Z language creates schemas, a way to organise the formal specification, by using typed sets, relations and functions in the framework of first-order predicate logic.

The Z specifications are arranged in the form of a collection of schemas, which are linguistic structures that introduce variables and define their relationships. The formal specification equivalent of a programming language component is basically called a schema. Just as components are used to organise a system, schemas are used to organise a formal specification.

The stored data that a system accesses and modifies is described by a schema. This is referred to as the “state” in Z. The schema also lists the relationships that exist inside the system and the operations that are used to change its state. A schema’s general structure looks like this:



Declarations denote the variables that make up the system state and the invariant sets limitations on how the state can change.

2.5.1 Syntax, Type and Semantics of Z-Notations

Developed at Oxford University’s programming research department, the Z language is a model-oriented, formal specification language that was first proposed in 1977 by Jean-Raymond Abrial, Steve Schuman and Bertrand Meyer. First order predicate logic and ZermeloFränkel axiomatic set theory serve as its foundations. A heavily typed mathematical specification language is the Z notation. Similar to how a compiler checks code in an executable programming language, it provides strong tool support for testing Z texts for syntax and type issues that is commercially accessible. It cannot be compiled into an operating program, executed, or interpreted.

It makes it possible to break down specifications into manageable chunks known as schemas. The primary characteristic that sets Z apart from other formal notations is its schema. Schemas can be used to define both the static and dynamic components of a system in Z. The data model, system status and system operations are all described in the Z standard. The Z specification is helpful for determining needs, implementing programs to meet requirements, testing implications and creating system instruction manuals.

Notes

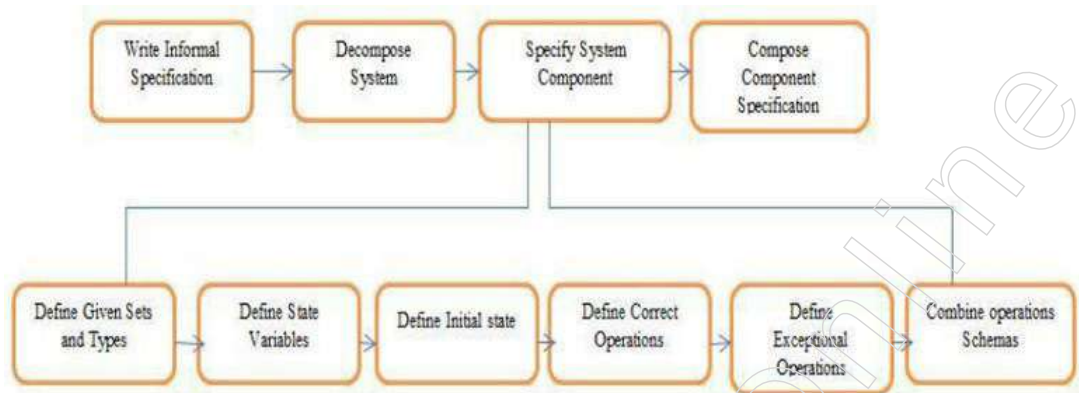


Figure: Z Process

<https://www.ijert.org/research/z-formal-specification-language-an-overview-IJERTV1IS6492.pdf>

Z links the abstract and concrete states mathematically, which aids in refinement towards an implementation. Many different types of companies employ Z for a wide range of purposes.

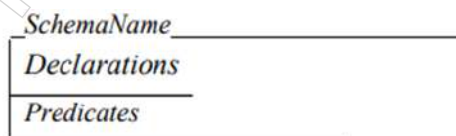
There are two languages used in the Z notation:

Mathematical Language: Objects and the relationships between them are described using propositional logic, predicate logic, sets, relations and functions in mathematical language.

Schema Language: Descriptions are organised and composed using the schema language, which gathers, encapsulates and names information for future usage.

Structure for Z Specification

Schemas are structures that resemble boxes and are used to introduce variables and outline how they relate to one another. Below is a schema. Predicates are defined below the middle line, while all declarations are made above it.



DECLARATION: The following will be found in the schema's declarations section:

- ❖ a set of declarations for variables; and
- ❖ references (referred to as schema inclusion) to other schemas.

PREDICATES: Variable values are restricted below the centre line. In a schema, the predicate portion consists of:

- ❖ a set of predicates with new lines or semicolons to separate them.
- a series of predicates divided by fresh semicolons or lines.

Z Conventions

- Any variable name, N, that has "e.g. N" after it indicates that it represents the state variable N's value following the operation. Z nomenclature describes N as having a dash on it.
- When a schema name is embellished with, the names declared in the specification are introduced along with the dashed values and the invariant that applies to these values.

- A variable name that has ! around it indicates that it is an output, such as a message.
- When ? is used to adorn a variable, it indicates that it is an input, such as amount?
- Dashed versions of the variables defined in the named schema are introduced if a schema name is preceded with the Greek character Xi (Ξ). The values of the associated dashed names for every variable name introduced in the schema are the same. That is, the operation has no effect on the values of the state variables.
- The Greek letter Delta (Δ) before a schema name indicates that the operation introducing the schema will modify the values of one or more state variables. Operational references to equivalent dashed names for any variable name introduced in the specified schema are possible.

2.5.2 Benefits and Limitations of Z

Benefits:

1. Precision and Clarity:

Benefit: Z notation offers a clear and concise method for expressing system specifications using mathematical constructs like schemas, sets and functions.

Explanation: Z notation's mathematical basis guarantees that specifications are unambiguous and leave little opportunity for interpretation. Developing a common understanding among stakeholders depends on this accuracy.

2. Rigorous Formalism:

Benefit: Because Z notation is based on strict formalism, it works well in critical systems where accuracy is crucial.

Explanation: System definition and verification can be done in a methodical and formal way thanks to the application of mathematical logic and set theory. This guarantees that a system's attributes may be logically reasoned about.

3. Stepwise Refinement:

Benefit: Stepwise refinement is supported by Z notation, enabling developers to progressively expand and improve system requirements.

Explanation: The methodical evolution of requirements from abstract to concrete levels is made possible by Z's step-by-step refining procedure. This makes the process of creating complicated systems more organised.

4. Tool Support:

Benefit: The usage of Z notation for specification, validation and verification is supported by some tools.

Explanation: Z/EVES theorem prover and other tools help with property verification and Z specification consistency checks. Z notation is more practically applicable in real-world tasks because to this tool support.

5. Expressiveness:

Benefit: Because Z notation is expressive, engineers may represent a variety of behaviours and attributes of systems.

Notes

Explanation: Z notation offers a complete way to express complicated systems since it can represent linkages, limitations and dynamic behaviours through mathematical constructs.

6. Integration with Development Process:

Benefit: Z notation can be used to support early specification through later stages of verification by integrating it into the software development lifecycle.

Explanation: Software reliability is increased when formal methods are employed consistently, which is made possible by the smooth integration of Z notation into development processes.

- A Z specification compels the software engineer to conduct a thorough domain analysis of the issue. (For instance, determine the state space and the initial and final conditions of each operation).
- All significant design choices must be made in accordance with a Z specification before the implementation is coded. You shouldn't start coding unless you are positive about what needs to be coded.
- A Z specification is a useful resource for creating test cases and performing conformance tests on finished systems.
- Formal investigation of a system's properties is possible with a Z specification.
- the adaptability to represent a specification that can result in code directly.
- Without higher-order characteristics, a wide class of structural models can be represented in Z and analysed well.
- It is simple to add independent conditions later.

Limitations

1. Learning Curve:

Limitation: Compared to informal notation, Z notation has a steeper learning curve, requiring practitioners to understand mathematical principles.

Explanation: Those who are not familiar with set theory and mathematical notation may find Z notation challenging due to its formal and mathematical nature. Effective adoption frequently requires instruction and training.

2. Complexity in Real-world Modeling:

Limitation: It can be challenging to express complicated relationships and some real-world scenarios in Z notation.

Explanation: Even though Z notation is expressive, it could be difficult to adequately represent some real-world situations using mathematical abstractions. In some situations, Z's practicality could be hampered by its complexity.

3. Verification Challenges:

Limitation: Z specification verification can require a lot of resources, particularly for big and sophisticated systems.

Explanation: Z allows for thorough verification, but as the system gets bigger and more complicated, the approach could get computationally expensive. This restriction can affect Z's scalability in some situations.

4. Limited Tool Ecosystem:

Limitation: Compared to some other formal methods, Z notation does not have as rich of a tool ecosystem.

Explanation: Although there are tools such as Z/EVES, the selection of auxiliary tools may be more constrained than in the case of widely used formal methods. The simplicity of integration into particular development environments may be impacted by this constraint.

5. Potential for Over-specification:

Limitation: A system's specifications run the danger of being overly thorough due to over specification of some features.

Explanation: Sometimes the need for accuracy can lead to extremely precise requirements that are difficult to manage and may even impede the development process's speed.

6. Limited Adoption in Some Industries:

Limitation: Z notation might not be used as frequently in some fields or sectors.

Explanation: Different businesses have different preferences when it comes to formal methods, therefore Z notation might not be the best option in every situation. This may restrict its suitability for use in various software development initiatives.

- Z does not offer any concurrent support.
- It offers no idea for timing-related issues.
- Z makes sequencing operations challenging.
- No determinism expressed explicitly (how to express unknown or undefinable parameters?!)
- To date, no single method has emerged as the most effective means of defining reasoning about behaviour in real time in Z.

Software systems can be precisely and formally specified and reasoned about with great strength when using Z notation. Its efficacy in critical systems arises from both its mathematical basis and its ability to facilitate incremental modification. When choosing whether to use Z notation, practitioners should take into account the learning curve, verification difficulties and the particular needs of their projects. Z notation, like any formal approach, needs to be used carefully to strike a balance between formality and practicality, matching its advantages with the limitations of the software development environment.

2.5.3 Specification to Convert Requirements Written in Natural Language to Z Formal

Below is a specification for converting natural language needs to the Z formal specification language approach. The specification shows a quick process for adding student information to the school database, including name, class, section and roll number.

Specification

[ROLLNO,NAME,CLASS,SECTION,ADDRESS]

Notes

StudentsDetails

$rollno : \mathbb{P} \text{ROLLNO}$
 $name : \mathbb{P} \text{NAME}$
 $class : \mathbb{P} \text{CLASS}$
 $section : \mathbb{P} \text{SECTION}$
 $address : \mathbb{P} \text{ADDRESS}$
 $stuname : \text{ROLLNO} \rightarrow \text{NAME}$
 $stuclass : \text{ROLLNO} \rightarrow \text{CLASS}$
 $stusection : \text{ROLLNO} \rightarrow \text{SECTION}$
 $stuadd : \text{ROLLNO} \rightarrow \text{ADDRESS}$

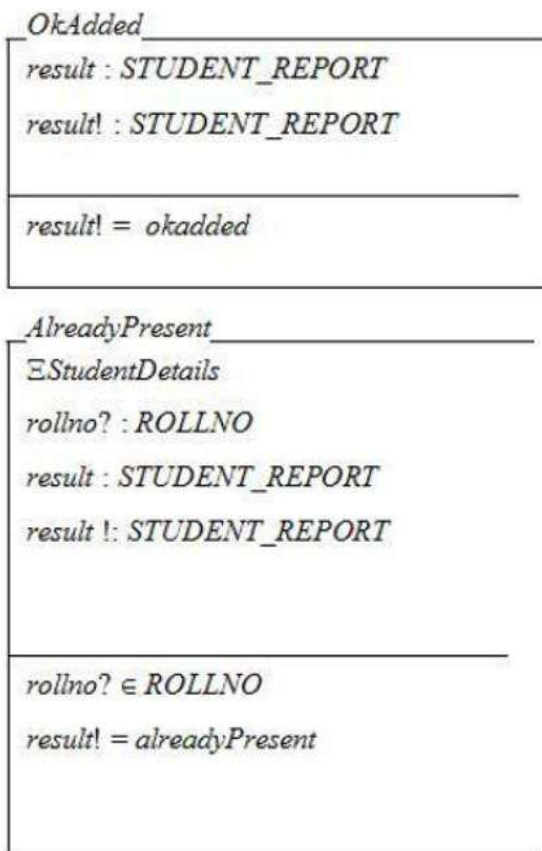
$rollno = \text{dom } stuname$
 $rollno = \text{dom } stuclass$
 $rollno = \text{dom } stusection$
 $rollno = \text{dom } stuadd$

AddStudentDetails $\Delta \text{StudentsDetails}$

$rollno? : \text{ROLLNO}$
 $name? : \text{NAME}$
 $class? : \text{CLASS}$
 $section? : \text{SECTION}$
 $address? : \text{ADDRESS}$

$rollno? \in rollno \wedge$
 $rollno' = rollno \cup \{rollno?\}$
 $stuname' = stuname \cup \{rollno? \mapsto name?\} \wedge$
 $stuclass' = stuclass \cup \{rollno? \mapsto class?\} \wedge$
 $stusection' = stusection \cup \{rollno? \mapsto section?\} \wedge$
 $stuadd' = stuadd \cup \{rollno? \mapsto address?\}$

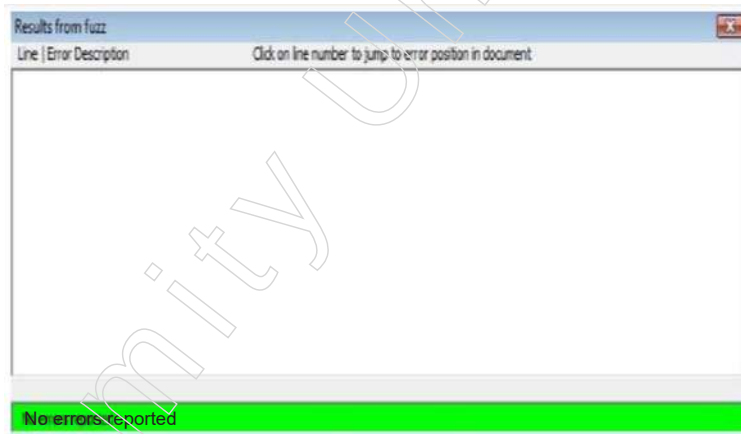
STUDENT_REPORT ::= okadded | alreadyPresent



Notes

Verification

In Z/Word tool, click the fuzz button to type check a document. A dialogue window with the results is shown.



2.6 Ten commandments of Formal Methods

Using formal methods in the actual world is not a decision that is made hastily. "The ten commandments of formal methods" are a set of guidelines developed by Bowan and Hinchley for individuals who are about to implement this crucial software engineering methodology.

1. You must select the proper notation. Software engineers should take into account the application type to be specified, language usage scope and vocabulary while selecting from the many formal specification languages available.

Notes

2. Formalise, but do not go overboard. Generally speaking, not every component of a large system needs to be studied using formal methods. First priority components are those whose failure cannot be allowed (due to business reasons), then safety-critical components.
3. You must project expenses. Formal methods are expensive to start up. High first-time costs are caused by hiring contract consultants, purchasing support tools and training people. When analysing the formal methods' return on investment, these expenses need to be taken into account.
4. You must always have an official methods guru available. When formal methods are applied for the first time, success depends on professional training and continuing advice.
5. You must not give up on your conventional methods of advancement. Formal methods can be integrated with traditional or object-oriented methods and this is often desirable. Everybody has advantages and disadvantages. When used correctly, a combo can yield very good outcomes.
6. You must adequately document. System requirements can be concisely, clearly and consistently documented using formal methods. It is advised, therefore, that a natural language commentary be included with the formal specification in order to reinforce the reader's comprehension of the system.
7. You must not lower your standards of excellence. Since "formal methods are not magical," additional SQA procedures must be implemented while systems are being created.
8. You must avoid becoming dogmatic. It is important for software engineers to understand that formal methods do not ensure correctness. Even in cases when formal methods are followed during development, it is possible—some would even say likely—that the final product will have minor errors, omissions and other features that fall short of expectations.
9. You must test, test and test some more. Formal methods do not relieve the software engineer of the responsibility to carry out meticulous, well-thought-out testing.
10. You must reuse. Reuse is the only practical strategy to save software costs and raise software quality over the long run. This reality remains unchanged by formal methods. In fact, when creating components for reuse libraries, formal methods might be the best course of action.

2.6.1 Tips to Use Formal Methods

It can be difficult and time-consuming to define and verify system requirements using formal methods and standards, but there are certain pointers and strategies that can be useful. Given the learning curve, complexity and effort involved, plan ahead and allot sufficient time and resources to adopt formal methods and standards. Rather than depending solely on a single technique or standard, think about combining formal methods and standards that best meet your objectives. Additionally, when creating, editing, verifying and documenting your system requirements, make use of tools and software that support formal methods and standards. Incorporating other stakeholders into the process and working together with them through formal methods and standards is the last step in gathering their feedback.

The process of developing software is intricate and multidimensional and it is crucial to guarantee the dependability, security and safety of software systems. Adherence to standards and formal methods are essential to accomplishing these objectives. This

thorough guide will cover everything from initial planning to final deployment, offering a wealth of advice on how to use formal methods and standards in software development.

Formal methods for defining, developing and validating software and hardware systems comprise a variety of approaches and instruments based on mathematical logic. System attributes can be precisely and unambiguously described with these methods, facilitating thorough study and verification. Z notation, B-Method, Event-B, Temporal Logic of Actions (TLA+) and model verification tools like SPIN and NuSMV are a few examples of formal methods.

In software development, standards act as best practices rules, specifications, or frameworks that guarantee software products are consistent, interoperable and of high quality. Respecting standards promotes compatibility between various platforms and technologies and increases the dependability of software systems. Software development standards include ISO/IEC 12207 for information security management, ISO 9001 for quality control and ISO/IEC 27001 for software life cycle processes.

Understand the Domain and System Requirements:

It is crucial to have a thorough grasp of the domain in which the program will function as well as the particular criteria it must meet before delving into formal methods and standards. The framework for later formal definition and verification procedures is established at this fundamental stage.

Perform Domain Analysis

To determine the domain's essential traits, difficulties and needs, do a thorough domain study. This will guide the development and verification phases that follow.

Compile all of the system requirements.

Involve stakeholders in order to compile thorough system requirements. Utilise methods such as surveys, user interviews and prototype evaluations to make sure you have a thorough grasp of the limitations and functioning of the system.

Select Appropriate Formal Methods:

Selecting the appropriate formal approach is essential to the verification process's success. Different kind of methods are more expressive in different ways and are more appropriate for particular kinds of systems or attributes.

Align Formal Approach with System Features

On the basis of your system's characteristics, evaluate several formal methods. Take into account elements like the development team's experience, the complexity of the system and time restrictions in real time.

Think About Precision Using Z Notation

Z Notation should be used if the system needs a thorough specification and accuracy is of the utmost importance. It is excellent at rigorously describing complex relationships and limitations mathematically.

Define Clear and Precise Specifications:

The correctness and clarity of specifications constitute the basis of formal verification. Specifications that lack clarity or ambiguity can cause misunderstandings and undermine the efficacy of formal methods.

Notes

Spend Time Developing Specifications

Provide enough time and resources to draft formal specifications that are precise and easy to understand. In the phases of validation and verification, this initial investment pays rewarded.

Utilise Equational Notations

To clearly explain specifications, use mathematical notations like temporal logic, predicate logic and set theory. The system attributes are represented in a clear and formal manner by these notations.

Utilise Model Checking Tools:

A useful method for methodically checking finite state models against predetermined properties is model checking. By using model checking tools, you may find any problems early in the development phase and automate the verification process.

Examine the Model Checking Resources

Examine and investigate various model checking tools associated with your preferred formal technique. Automated model testing is possible with tools like NuSMV for temporal logic specifications and SPIN for Promela.

Incorporate Model Checking into Pipelines for CI/CD

Your pipelines for continuous deployment and integration (CI/CD) should incorporate model checking. This guarantees that verification proceeds without hiccups as part of the development workflow, identifying problems early and minimising debugging labour.

Integrate Formal Methods into the Development Lifecycle:

The smooth integration of formal methods into the larger development lifecycle is essential for their effective deployment. This integration makes sure that formal verification is an essential component of the development process rather than a stand-alone activity.

Create Formal Workflows and Methods

Establish processes for formal methods that complement your development process. Give a detailed explanation of the application of formal methods at each level, ranging from requirements elicitation to code review.

Train Development Teams

Development teams should receive instruction and materials on the application of formal methods. Explain the procedure and highlight its advantages to promote adoption.

Document and Communicate:

Achieving success in software development requires both efficient documentation and effective communication, especially when utilising formal methods. All stakeholders will be able to comprehend the reasoning behind design choices and verification results when there is clear documentation.

Continue to Keep Thorough Records.

Record the formal specifications, the outcomes of the verification process and any modifications made to the original specifications. This documentation is an invaluable resource for development and upkeep in the future.

Apply Visualisation Methods

Make use of visualisation tools to convey verification results and intricate formal specifications. Using graphs, charts and diagrams can help make complex relationships easier to understand.

Automate Where Possible:

Automation decreases the possibility of human error while speeding up the verification process. Development teams are able to concentrate on more difficult areas of the product by automating mundane chores and verification stages.

Investigate Automated Testing Resources

Examine automated testing tools as an adjunct to model checking to enhance formal methods. Overall test coverage can be improved by using tools such as property-based testing frameworks.

Combine IDEs with Formal Verification

Examine how to integrate with formal methods-supporting Integrated Development Environments (IDEs). This makes it possible for developers to get rapid feedback while they're coding.

Ensure Traceability:

Every step of the development process, from requirements to implementation and verification, must be able to be tracked back to provide traceability. Understanding the effects of changes and adhering to standards depend on this.

Create Traceability Matrix

Make traceability matrices that connect each need to the formal specification and verification outcomes that match it. This offers a thorough understanding of the process of development and verification.

Utilise Tools for Traceability

Think about utilising technologies for traceability that automate the process of tracking connections between various artifacts. These tools lower the possibility of missing links and expedite the tracing procedure.

Consider Safety and Security Standards:

Adherence to safety and security requirements may be required, depending on the program. Standards offer a framework for making sure software systems fulfil specified requirements for security, quality and safety.

Summary

- Formal Specification refers to the rigorous, mathematical representation of a software system's requirements, design, or behaviour. It uses mathematical notations, logic and formal languages to describe system properties precisely.
- Formal Methods Model refers to the systematic application of mathematical techniques to describe, design and verify software systems. It encompasses various formal specification languages, formal verification methods and tools to ensure the correctness of software.
- Formal specification languages play a crucial role in software engineering by providing

Notes

Notes

a precise and unambiguous way to express system requirements and designs. These languages use mathematical notations, logic and formalisms to describe software artifacts, facilitating rigorous analysis and verification.

- Z-Notation is a formal specification language used in software engineering to precisely and unambiguously describe the requirements and designs of software systems. It is based on mathematical set theory and first-order predicate logic, providing a structured and rigorous approach to specification.
- Mathematical notations play a crucial role in formal specification, providing a precise and unambiguous way to express system requirements, designs and properties. Mathematical notations provide a foundation for expressing various aspects of formal specifications, including sets, logic, functions and temporal properties. Their use ensures clarity, precision and the ability to conduct formal analysis and verification of software systems.
- Z notation offers substantial benefits in terms of precision, formal verification and systematic development, it is not without its challenges. The success of Z in a development context depends on factors such as the team's familiarity with formal methods, the complexity of the system and the willingness to invest in the learning curve associated with Z notation.
- The "Ten Commandments of Formal Methods" represent a set of principles and guidelines that emphasise the best practices for applying formal methods in software engineering. These commandments aim to ensure the effective and successful application of formal methods to enhance the reliability and correctness of software systems.

Glossary

- SDLC: Software Development Life Cycle
- VDM: Vienna Development Method
- CASL: Common Algebraic Specification Language
- SL: Specification Language
- TLA: Temporal Logic of Actions
- CSP: Communicating Sequential Processes
- EFSM: Extended Finite State Machine
- UML: Unified Modelling Language

Check Your Understanding

1. What is the primary purpose of formal specification in software engineering?
 - a. Enhancing code readability
 - b. Improving system performance
 - c. Providing a precise and unambiguous description of system requirements and designs
 - d. Simplifying project management
2. Which mathematical notation is commonly used in formal specification for expressing relationships between sets?
 - a. Greek letters
 - b. Roman numerals
 - c. Set theory

- d. Binary code
- 3. What does the Universal Quantifier ($\forall$) represent in formal specification logic?
 - a. There exists
 - b. For some
 - c. For all
 - d. Not applicable
- 4. In Z notation, what does the Power Set (P) represent?
 - a. The set of natural numbers
 - b. The set of all subsets of a given set
 - c. The set of real numbers
 - d. The empty set
- 5. What is a key benefit of formal specification for software systems?
 - a. Increased project completion time
 - b. Reduced precision in requirements
 - c. Improved code maintainability
 - d. Rigorous verification of system properties against requirements

Exercise

- 1. Define the basic concepts of formal specification.
- 2. What is the importance of formal methods?
- 3. What are the advantages of formal methods model?
- 4. What is the use of formal methods in SDLC?
- 5. Define various types of mathematical notations for formal specification.

Learning Activities

- 1. Consider a simple system that manages a library of books. Each book has attributes such as title, author and publication year. The system should allow users to borrow and return books. Using Z-Notation, provide a formal specification for the basic functionalities of the book management system. Include relevant sets, schemas and operations.
- 2. Create a formal specification for the banking system using a suitable formal method. Define necessary sets, schema and operations to capture the key functionalities of the banking system.

Check Your Understanding- Answers

- 1. c) 2. c) 3. c) 4. b)
- 5. d)

Module - III: Component-Based Software Engineering

Learning Objectives

At the end of this module, you will be able to:

- Understand the component-based software engineering
- Define domain engineering
- Analyse economics of component-based software engineering
- Explain cleanroom approach
- Define cleanroom testing
- Understand structure of client/server system

Introduction

Component-Based Software Engineering (CBSE) is a paradigm shift in the rapidly changing field of software engineering that is redefining the way software systems are conceived, designed and built. Based on the core concepts of modularity, reusability and composability, this methodology seeks to improve the quality and maintainability of complex systems while streamlining the software development process. In this thorough investigation, we explore the complexities of CBSE, breaking down its fundamental ideas, analysing its benefits, tackling its drawbacks and looking at its practical uses.

Fundamentally, CBSE advocates for the assembly of independent and reusable components to construct software systems, marking a break from conventional monolithic techniques. This change is based on the realisation that creating software from scratch for each project is ineffective, prone to mistakes and hinders advancement in a field that requires flexibility and quick thinking. With the help of a modular approach, CBSE divides large, complex systems into smaller, independent parts, or components, each of which is in charge of a single function.

Key Principles of CBSE:

Interoperability: The importance of interoperability is one of the cornerstones of CBSE. A CBSE ecosystem's components must function in unison to enable the integration of various parts into a coherent whole. Achieving flexibility and preventing vendor lock-in depend on this interoperability.

Composability: CBSE promotes the building of systems through component composition. By following this idea, developers can create larger, more complicated systems by utilising pre-existing components rather than having to start from scratch. Flexibility and adaptability are fostered by composeability, which makes it possible to develop customised solutions from reusable building parts.

Encapsulation: A fundamental principle of CBSE is encapsulation, which mandates that every component enclose its internal operations. Encapsulation improves security, lowers complexity and fosters abstraction by revealing just the interfaces that are absolutely essential and concealing implementation details. This enables developers to work with components without having to comprehend all of their underlying workings.

Advantages of Component-Based Software Engineering

Reusability Benefits: The reuse of components across several projects is made easier by CBSE. This leverages pre-existing, proven solutions to save development expenses in

addition to saving time. Instead of creating new functionality, developers can concentrate on putting components together and integrating them.

Productivity Gains: Productivity rises when components are reused. By working with higher-level abstractions, developers can expand on the tried-and-true functionality contained within components. Faster development cycles and more effective resource usage are made possible by this.

Modularity's Impact: The modular technique used by CBSE divides a system into separate, modular components, streamlining the development process. The complexity of the system as a whole can be decreased by developing, testing and maintaining each component independently.

Maintainability Enhancement: Modular systems make maintenance easier to understand. Applying updates or fixes to certain parts of the system isolates them from the rest, which facilitates problem-solving and allows for the introduction of enhancements.

Flexibility Through Composability: Developers can alter and modify systems by reordering and merging pre-existing components because to the CBSE composability principle. This adaptability is especially useful in situations when requirements change on the fly.

Adaptability to Change: Systems developed using CBSE are more flexible and may adjust as business needs do. Without needing significant changes to the entire system, new components can be added and old ones updated or replaced.

Quality Assurance: Individual CBSE components can go through extensive testing and validation to guarantee high-quality implementations. Because components with proven functionality and dependability are integrated, this increases the total system reliability.

Reduced Error Propagation: It is less probable for errors in one component to affect the entire system. The impact of faults or difficulties is restricted to the particular component in which they arise due to the encapsulation of internal details and the usage of well-defined interfaces.

Efficient Development Cycles: Development cycles are sped up by component reuse. By utilising pre-existing components, developers can shorten the time needed to construct common features.

Cost-Efficiency: Cost reductions result from CBSE's emphasis on modularity and reusability. By reusing these components, organisations may spread the cost of generating high-quality components over a number of projects.

3.1 Overview of Component-Based Software Engineering

Reuse is a relatively new concept in the field of software engineering. Since the beginning of computing, programmers have reused concepts, abstractions and procedures; nevertheless, the early approach to reuse was ad hoc. Complex, high-caliber computer-based systems now need to be developed quickly. This contributes to a reuse strategy that is more structured.

The process of designing and building computer-based systems with reusable software "components" is known as component-based software engineering, or CBSE. CBSE "is changing the way large software systems are developed," according to Clements. The "buy, don't build" mentality promoted by Fred Brooks and others is embodied in [CBSE]. Similar to how the introduction of subroutines freed programmers from having to worry about specifics, [CBSE] moves the focus from writing code to creating software systems.

Notes

Integration is now the main focus instead of implementation. Its fundamental premise is that many large software systems have enough things in common for the development of reusable components to take advantage of and satisfy that commonality.

Real-World Applications and Case Studies

Enterprise Software Development: When developing enterprise software systems, where modularity, reusability and adaptability are crucial, CBSE is frequently used. Systems for enterprise resource planning (ERP) frequently use CBSE to smoothly combine a variety of functionality.

Embedded Systems: Within the field of embedded systems, CBSE is useful for creating software and firmware for a variety of devices, including automotive and Internet of Things systems. Updates are made easier and maintainability is improved by CBSE's modular architecture.

Web Application Development: By including reusable components for common operations like authentication modules, data processing and user interface elements, web application development benefits from the adoption of CBSE principles. CBSE concepts are frequently embodied in frameworks and libraries.

Aerospace and Defense: In the aerospace and defence sectors, CBSE is used to create intricate software systems that manage vital parts of spacecraft, aeroplanes and defence systems. In this field, CBSE's modularity and dependability are essential.

Component-Based Software Engineering provides a strategic approach to solving the difficulties of complexity, adaptability and efficiency as we negotiate the complicated terrain of contemporary software engineering. The CBSE-implemented concepts of modularity, reusability and composability improve the quality and dependability of software systems while simultaneously streamlining the development process. The advantages of CBSE in terms of productivity, maintainability and reusability are strong, even though issues with compatibility, granularity and changing technology still exist.

The practical uses of CBSE in a variety of fields highlight its adaptability and significance in a rapidly evolving technology environment. Enterprise solutions, embedded systems, aerospace projects—all benefit from the ongoing influence of CBSE on how we design, build and develop software systems. Software engineering could reach new heights of creativity and efficiency in the future if CBSE principles are applied wisely and are combined with continuing research and standardisation initiatives. Component-Based Software Engineering's modular and reusable building blocks serve as the journey's guide.

Challenges and Considerations in Component-Based Software Engineering

Versioning Challenges: It can be difficult to ensure compatibility between various component versions. Modifications to a component's behaviour or interface could cause problems in terms of compatibility with current systems.

Interoperability Concerns: It is possible for components from many sources or suppliers to not integrate smoothly because of variations in interface requirements or implementation. This necessitates giving significant thought to component integration and selection.

Granularity Balancing Act: Finding the right amount of detail for each component requires careful consideration. Overly fine-grained components can cause excessive coupling, whereas overly coarse-grained components can make them rigid.

Size Impact: Large components might have features that are superfluous, which would limit their reusability. On the other hand, an excessively small component count could lead to a combinatorial explosion of parts, which would be challenging to control.

Design Challenges: It can be difficult to design efficient interfaces and guarantee correct encapsulation. Component design needs to find a happy medium between offering enough capability and keeping things simple.

Dependency Management: Taking dependency management between components seriously is necessary. A stiff architecture may arise from having too many dependencies, whereas a cohesive architecture may be caused by having too few dependencies.

Discoverability Issues: Finding components that are appropriate for reuse can be difficult, especially in huge component repositories. Mechanisms for component discovery that work well are crucial.

Quality Assurance: It is essential to evaluate the dependability and quality of the components that are available. Insufficient assessment could result in the inclusion of parts that have unreported defects or insufficient record-keeping.

Technological Shifts: Technology is advancing so quickly that some parts can become outdated or incompatible. It is vital to consistently observe technology advancements to guarantee that constituents maintain their relevance.

Standardisation Challenges: The lack of established interfaces or protocols within specific domains may impede the capacity of components to communicate with one another. It will take standardisation efforts to overcome these obstacles.

3.1.1 Introduction to Component-Based Software Engineering: Part 1

At first glance, object-oriented or conventional software engineering and CBSE appear to be very similar. Using traditional requirements elicitation methodologies, a software team first defines requirements for the system to be constructed. After an architectural design is formed, the team looks at the requirements to see which subset can be directly composed of rather than constructed of, instead of diving right into further in-depth design work. In other words, for every system requirement, the team poses the following queries:

- ❖ Are commercial off-the-shelf (COTS) components available to implement the requirement?
- ❖ Are internally developed reusable components available to implement the requirement?
- ❖ Are the interfaces for available components compatible within the architecture of the system to be built?

The group makes an effort to change or eliminate any system requirements that are incompatible with proprietary or off-the-shelf components. Conventional or object-oriented software engineering methods are used to design the new components that need to be engineered to meet the requirement(s) if the requirement(s) cannot be amended or removed. However, a new set of software engineering tasks begin for those needs that can be satisfied by currently accessible components:

Component qualification. The components that will be needed are specified by the architecture and system requirements. The features of their interfaces are typically used to identify reusable components, whether they are in-house or off-the-shelf. In other words, the component interface is said to include “the services that are provided and the means by which consumers access these services.” However, the interface doesn’t give a clear image of how well the component will fit into the architecture and specifications. To determine whether each component fits, the software engineer must go through a discovery and analysis process.

Notes

Component adaptation. We pointed out that design patterns made up of components (functional units), relationships and coordination are represented by software architecture. The architecture, which specifies the forms of coordination and connectivity, essentially establishes the design guidelines for every component. The design guidelines of the architecture may not always align with the reusable components that are already in place. These elements must be modified to satisfy the requirements of the architecture, or they must be removed and swapped out for other, better elements.

Component composition. Once more, architectural design has a significant impact on how software parts are combined to create functional systems. The architecture determines the way the final product is put together by identifying mechanisms for coordination and connection (such as run-time characteristics of the design).

Component update. The imposition of a third party (i.e., the organisation that produced the reusable component may be outside the immediate control of the software engineering organisation) complicates updates when systems are implemented with COTS components.

The word “component” has been used several times in the first half of this section, although a clear definition of the phrase is elusive. Potentialities suggested by Brown and Wallnau include the following:

- **Component**—a nontrivial, practically independent and replaceable component of a system that, within the framework of a clearly specified design, performs a certain purpose.
- **Run-time software component**—a dynamically bindable collection of one or more programs that are maintained collectively and accessible via run-time discovery of documented interfaces.
- **Software component**—a compositional unit that only has explicit and contractually defined context dependencies.
- **Business component**—the use of software to implement a “autonomous” business idea or procedure.

Software components can also be classified according to how they are used in the CBSE process, in addition to these categories. Apart from the COTS components, the CBSE procedure produces:

- **Qualified components**—evaluated by software engineers to make sure that the system or product to be produced satisfies the requirements in terms of functionality, performance, reliability, usability and other quality factors.
- **Adapted components**—modified to change (also known as mask or wrap) undesirable or undesired traits.
- **Assembled components**—coordinated and efficiently controlled by the components when they are coupled with a suitable infrastructure and blended into an architectural style.
- **Updated components**—swapping off outdated software with updated versions when components become available.

Given the dynamic nature of CBSE, it is improbable that a cohesive definition would materialise anytime soon.

A standard evolutionary process model can be connected with a library of reusable “candidate components” through the usage of a “component-based development model.” However, the CBSE process needs to be described in a way that not only finds potential components but also verifies the interface of each one, modifies the components to

eliminate architectural inconsistencies, puts the components together in a chosen architectural style and updates the components when system requirements change.

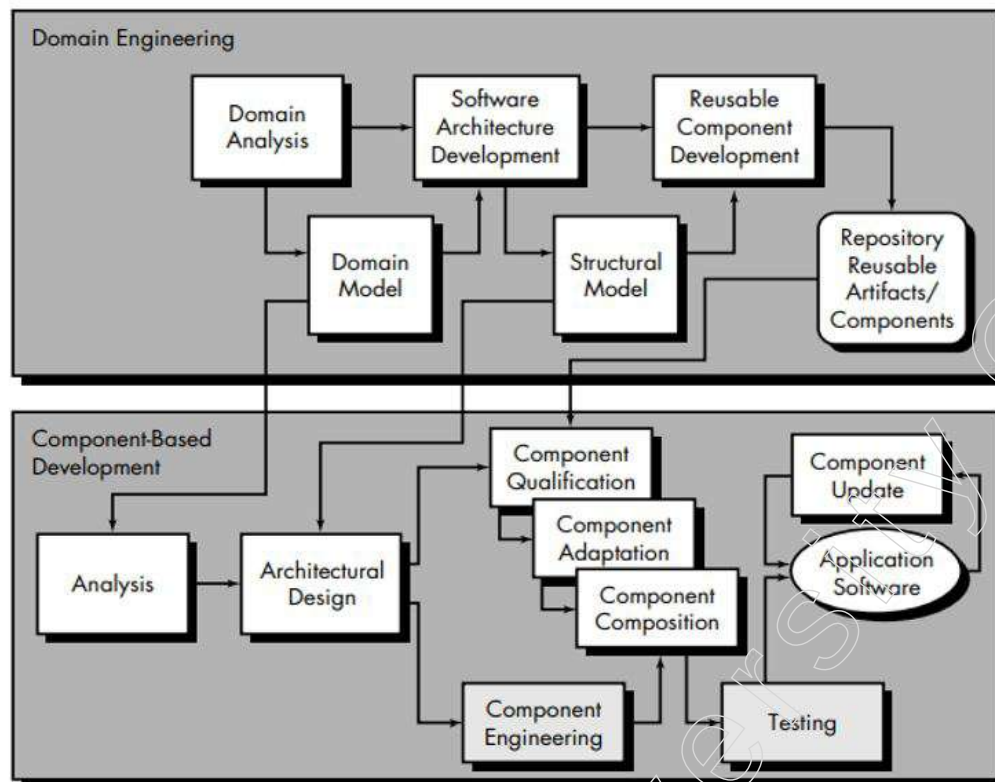


Figure: A process model that supports CBSE

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

The component-based software engineering process model places a strong emphasis on parallel tracks, where domain engineering takes place in tandem with component-based development. The work necessary to create a collection of software components that the software engineer can reuse is completed by domain engineering. After that, these components are moved over a "boundary" that divides component-based development from domain engineering.

A example process model that explicitly allows for CBSE is shown in the above figure. In the software engineering flow, domain engineering builds an application domain model that serves as a foundation for user demand analysis. Applications are designed using input from generic software architecture. Ultimately, reusable components are made available to software developers during component-based development after they have been acquired, chosen from pre-existing libraries, or created (as part of domain engineering).

Parallel to domain engineering is a CBSE practice called component-based development. The software team develops an architectural style that is appropriate for the analysis model developed for the application to be constructed using analysis and architectural design methods covered earlier in this book.

After the architecture has been defined, it needs to be filled with components that are either (1) built to satisfy specific needs or (2) available from reuse libraries. Therefore, there are two parallel channels in the task flow for component-based development. Reusable components need to be qualified and modified before they can be potentially integrated into the architecture. New components need to be engineered whenever they are needed. Following extensive testing, the resulting components are "composed" (integrated) into the architecture template.

Notes

Component Qualification, Adaptation and Composition

As we have already seen, the library of reusable components needed for component-based software engineering is provided by domain engineering. A portion of these reusable components are created internally, while others can be taken from already-existing programs or purchased from outside sources.

Regretfully, the mere fact that certain components are reusable does not ensure that integrating them into the architecture selected for a new application would be simple or successful. This is the reason that when a component is suggested for use, a series of component-based development activities are implemented.

Component Qualification

Component qualification verifies that a potential component will fulfil the necessary function, appropriately “fit” into the system’s architectural style and display the quality attributes (such as dependability, performance and usability) needed for the application.

The interface description offers helpful details regarding how a software component works and is used, but it falls short of providing all the information needed to assess if a suggested component can really be successfully reused in a new application. Among the several variables taken into account when qualifying a component are:

- Application programming interface (API).
- Development and integration tools required by the component.
- Run-time requirements, including resource usage (e.g., memory or storage), timing or speed and network protocol.
- Service requirements, including operating system interfaces and support from other components.
- Security features, including access controls and authentication protocol.
- Embedded design assumptions, including the use of specific numerical or non-numerical algorithms.
- Exception handling

Assessing each of these variables is not too difficult when reusable, internally generated components are suggested. The issues raised by the list can have their answers produced if effective software engineering principles were used in their creation. The internal workings of third-party or COTS components, however, are significantly harder to ascertain because the interface specification itself may be the only source of knowledge.

Component Adaptation

The ideal application architecture is one where domain engineering produces a library of easily integrated components. The phrase “easy integration” implies three things: (1) common activities, like data management, have been implemented for all components; (2) consistent methods of resource management have been implemented for all components in the library; and (3) consistent interfaces have been implemented into the architecture and with the external environment.

In actuality, a component may show conflict in one or more of the above mentioned areas even after it has been certified for use inside an application architecture. Component wrapping is a common adaptation strategy used to lessen the impact of these conflicts. White-box wrapping is utilised when a software team has complete access to the internal design and code of a component—which is frequently not the case when using commercial off-the-shelf components. Similar to its equivalent in software testing, white-box wrapping

looks at the component's internal processing details and modifies the code at the code level to eliminate any conflicts. When a component library offers a component extension language or API that makes it possible to eliminate or hide conflicts, this is known as gray-box wrapping. In order to eliminate or conceal conflicts, black-box wrapping necessitates the addition of pre- and post-processing at the component interface. The software team has to decide if it is worth the effort to properly wrap a component or if a bespoke component that avoids the conflicts that arise should be developed in its place.

Component Composition

Compiling qualified, adaptable and engineered components to fill the application's architecture is the task of component composition. An infrastructure must be built to connect the components and create a functional system in order to achieve this. The infrastructure, which is typically a library of specialised components, offers a framework for component coordination as well as particular services that let components work together to accomplish shared objectives.

One of the numerous methods for developing a functional infrastructure is the presence of four "architectural ingredients" in order to accomplish component composition:

Data exchange model. All reusable components should have defined mechanisms (such as drag and drop and cut and paste) that allow users and programs to interact and transfer data. The data exchange mechanisms facilitate the transfer of data not only between human and software, but also across components of the system (e.g., dragging a file to an output printer icon).

Automation. It is best to use a range of tools, scripts and macros to make it easier for reusable components to communicate with one another.

Structured storage. Instead of being accessed and organised as a collection of disparate files, heterogeneous data (such as text, voice/video, graphical and numerical data) contained in a "compound document" should be arranged and accessed as a single data structure. "A descriptive index of nesting structures is maintained by structured data, which applications can freely navigate to find, create, or edit individual data contents as required by the end user."

Underlying object model. The object model guarantees the interoperability of components created on multiple platforms and in different programming languages. That is, things need to be able to talk to each other over a network. The object model specifies a standard for component interchange in order to do this.

Standards for component software have been proposed by several large corporations and industry consortia due to the significant potential impact of reuse and CBSE on the software industry:

OMG/CORBA. A standard object request broker architecture (OMG/CORBA) has been released by the Object Management Group. Regardless of where they are located inside a system, reusable components (objects) can connect with other components thanks to the range of services offered by an object request broker (ORB). Provided that each component constructed in accordance with the OMG/CORBA standard has an interface definition language (IDL) interface, it is guaranteed that these components will integrate (unmodified) into the system. Objects in the client application make requests to the ORB server for one or more services using a client/server model. Requests can be made dynamically at run time or using an IDL. All the information required on the request and response forms for the service is contained in an interface repository.

Notes

Microsoft COM. Comprising two components, COM interfaces (implemented as COM objects) and a set of mechanisms for registering and passing messages between COM interfaces, is a specification that Microsoft developed for using components made by different vendors within a single Windows operating system application. From the application's perspective, "the focus is not on how [COM objects are] implemented, only on the fact that the object has an interface that it registers with the system and that it uses the component system to communicate with other COM objects."

Sun JavaBean Components.. The JavaBean component system was created with the Java programming language and is a platform-independent, portable CBSE framework. The Java applet4 is expanded by the JavaBean system to support the more complex software components needed for component-based development. The JavaBean component system consists of a collection of tools known as the Bean Development Kit (BDK), which enables developers to do the following tasks: (1) assess the functionality of pre-existing Beans (components); (2) alter their appearance and behaviour; (3) create mechanisms for communication and coordination; (4) create custom Beans for use in particular applications; and (5) test and assess Bean behaviour.

Which of these standards is going to rule the market? As of right now, there is no simple solution. Depending on the application categories and platforms selected, big software companies might decide to adopt all three standards, even though many developers have standardised on one of them.

Component Engineering

Utilising pre-existing software components is encouraged by the CBSE methodology. Sometimes, though, components need to be engineered. In other words, it is necessary to create new software components and combine them with already-existing COTS and internal components. These new components should be designed for reuse since they join the internal library of reusable components.

Reusing software components doesn't have any magical properties. Reusable software components are produced by combining object-oriented methods, testing, SQA and correctness verification methods with design principles like abstraction, hiding, functional independence, refinement and structured programming. We won't go over these subjects again in this part. Instead, we take into account the reuse-specific concerns that go hand in hand with sound software engineering techniques.

Analysis and Design for Reuse

It is possible to develop functional, behavioural and data models (expressed in a variety of different notations) to define the requirements of a given application. These models are then described using written specifications. The outcome is a detailed description of the needs.

The goal of analysing the analysis model should be to identify the parts that relate to already-existing, reusable components. Finding a way to extract data from the requirements model so that it may be used for "specification matching" is the issue. One method for object-oriented systems is described by Bellinzoni, Gugini and Pernici as follows:

At several levels of abstraction, components are developed and stored as specification, design and implementation classes; each class is an engineering description of a product from a prior application. The development knowledge, or specification knowledge, is kept in the form of reuse-suggestion classes that hold instructions on how to compose and customise reusable components after they have been retrieved based on their description.

In order to compare the requirements mentioned in the current specification with those listed for already-existing reusable components (classes), automated tools are utilised to search through a repository. To assist in locating possibly reusable components, characterisation functions and keywords are employed.

A reuse library, or repository, contains components that can be extracted by the designer and used in the design of new systems if specification matching produces components that meet the requirements of the present application. Software engineers must use conventional or OO design methods to develop design components if they cannot be located. Design for reuse (DFR) should be taken into consideration at this point, when the designer starts to construct a new component.

As we've already mentioned, DFR calls on the software engineer to use sound ideas and guidelines for software design. However, the application domain's features also need to be taken into account. Binder offers many important points that serve as the foundation for design for reuse:

Standard data. Standard global data structures (such as file structures or a whole database) should be identified and the application domain should be examined. Then, using these common data structures, all design components may be described.

Standard interface protocols. The design of external technical (nonhuman) interfaces, the human/machine interface and the nature of intramodular interfaces should all be considered at three different levels of interface protocol.

Program templates. For the architectural design of a new program, the structure model can function as a template.

The designer can operate within a framework once standard data, interfaces and program templates have been defined. New components that follow this architecture are more likely to be reused in the future.

Software engineering methods that have been covered elsewhere in this book are used in the building of reusable components, much like design. Conventional third-generation programming languages, fourth-generation languages and code generators, visual programming approaches, or more sophisticated tools can all be used to complete construction.

Advanced Component-Based Software Engineering

Advanced Component-Based Software Engineering (CBSE) refers to the application of sophisticated techniques, methodologies, and practices in the development of software systems based on the principles of componentization. CBSE emphasizes the construction of software systems by composing reusable and interchangeable software components. Here are some advanced aspects of CBSE:

Component Identification and Selection: Advanced software component identification, selection, and classification (CBSE) entails sophisticated methods for determining the functionality, quality qualities, and architecture compatibility of software components. Semantic analysis, repository mining, and automated component discovery may be examples of this.

Component Modeling and Specification: Formal techniques and languages are used in Advanced CBSE to model and specify software elements, interfaces, and interactions. Composition, verification, and validation are made easier by the explicit defining of component behaviour, contracts, and dependencies made possible by formal specifications.

Notes

Component Composition and Integration: In order to create complex systems, Advanced CBSE focuses on effective and adaptable ways to compose and integrate software components. Advanced composition frameworks, middleware, and integration patterns may be used in this to control dependencies, guarantee compatibility, and impose architectural limitations.

Component Adaptation and Customization: To modify and customise software components dynamically to changing requirements and runtime conditions, Advanced CBSE is supported. To allow for variable configuration and component specialisation, this may involve strategies like runtime variability management, aspect-oriented programming (AOP), and configuration management.

Component Testing and Verification: Advanced component-based systems engineering (CBSE) uses sophisticated testing and verification methods to guarantee the accuracy, dependability, and quality of software components. To find and stop errors at the component level, this could involve formal verification techniques, model checking, and automated testing frameworks.

Component Evolution and Maintenance: Over the course of the software lifecycle, Advanced CBSE handles issues with component evolution, versioning, and maintenance. The implementation of sophisticated dependency management, version control methodologies, and impact analysis approaches may be necessary to enable smooth upgrades, bug patches, and component improvements.

Component Reuse and Repository Management: In order to facilitate the organised reuse of software assets across projects and organisations, Advanced CBSE places a strong emphasis on the creation and administration of component repositories, libraries, and catalogues. To enable component reuse and sharing, this could involve sophisticated search and retrieval techniques, metadata management, and quality assurance procedures.

Component Deployment and Deployment Management: Advanced CBSE takes into account sophisticated methods for managing and distributing software elements in contexts that are heterogeneous and distributed. To guarantee consistent and dependable component deployment across various platforms and configurations, this may entail the use of orchestration, deployment automation, and containerisation technologies.

Quality Attributes and Non-Functional Requirements: The integration of non-functional requirements and quality criteria into the component-based development process is the focus of advanced CBSE. This could involve using sophisticated modelling tools, architectural designs, and analysis approaches to evaluate and maximise usability, performance, scalability, and security.

Tool Support and Infrastructure: To support component-based development activities, advanced CBSE depends on sophisticated tooling, infrastructure, and development environments. Runtime environments designed for component-based software engineering, build systems, collaborative tools, and integrated development environments (IDEs) are a few examples of this.

Component-Based Software Engineering (CBSE) is an approach to software development that emphasizes the assembly of software systems from reusable, self-contained software components. Here's how CBSE might be applied in both traditional and advanced software engineering contexts:

Traditional Software Engineering:

- In traditional software engineering environments, CBSE may be utilized to improve productivity, reduce development costs, and enhance software quality by leveraging existing components and libraries.

- Components are typically developed and maintained independently, with well-defined interfaces that enable them to be easily integrated into larger systems.
- The focus in traditional CBSE is often on component reuse, where developers search for existing components that meet their requirements rather than building everything from scratch.
- Integration and testing are crucial phases in traditional CBSE, as components must be thoroughly tested and verified to ensure compatibility and reliability when integrated into larger systems.
- Component repositories and registries may be established to manage and catalog reusable components, facilitating their discovery, selection, and reuse in different projects.

Advanced Software Engineering:

- In advanced software engineering environments, CBSE principles are often integrated into Agile and DevOps practices to promote rapid development, scalability, and maintainability of software systems.
- Advanced software engineering teams may adopt microservices architecture, a form of CBSE, where applications are built as a collection of loosely coupled services that communicate via well-defined APIs.
- The use of containerization technologies such as Docker and orchestration tools like Kubernetes facilitates the deployment and management of component-based systems in dynamic, cloud-native environments.
- Advanced software engineering teams prioritize continuous integration and delivery (CI/CD), automated testing, and monitoring to ensure the reliability and resilience of component-based systems in production.

Component-based development is complemented by practices such as service-oriented architecture (SOA), where services are treated as reusable components that encapsulate business functionality and can be orchestrated to create complex workflows.

3.1.2 Introduction to Component-Based Software Engineering: Part 2

Think about a sizable university library. You can use tens of thousands of books, magazines and other information sources. However, a system of classification needs to be created in order to access these resources. Librarians have developed a classification scheme that consists of author names, keywords, the Library of Congress classification code and other index entries to help users traverse this massive amount of information. All help the user locate the required resource with ease and speed.

Now imagine a sizable store of components. It has tens of thousands of reusable software components. How, though, can a software engineer locate the one she requires? Another question that comes up in trying to address this one is: How can we clearly and classifiably define software components? These are challenging questions for which there is currently no satisfactory solution. We examine current approaches in this part that will help software engineers in the future browse reuse libraries.

Describing Reusable Components

There are various methods to characterise a reusable software component, but Tracz's concept, content and context—or the 3C model—are the best three.

“A description of what the component does” is the definition of a software component. The semantics of the component—represented in the context of pre- and post conditions—

Notes

are specified and the interface to the component is fully described. The idea should make clear the component's purpose.

A component's substance explains how the idea is put into practice. The content is essentially information that is hidden from ordinary users and should only be known by individuals who plan to test or alter the component.

A reusable software component is situated inside its applicability domain by the context. That is, the context helps a software engineer identify the right component to satisfy application requirements by providing conceptual, operational and implementation features.

Concept, substance and context must be converted into a tangible specification scheme in order for it to be useful in a pragmatic situation. Classification strategies for reusable software components have been the subject of dozens of papers and articles (e.g. has an extensive bibliography). Three main categories apply to the methods that have been suggested: hypertext systems, artificial intelligence methods and library and information science methods. For component categorisation, the vast bulk of research to date points to the application of library science methods.

An indexing methods taxonomy for library science is shown in the figure below. The vocabulary or syntax that can be used to categorise an item (component) is restricted by controlled indexing vocabularies. There are no limitations on the type of description in uncontrolled indexing vocabulary. Three categories comprise most classification schemes for software components:

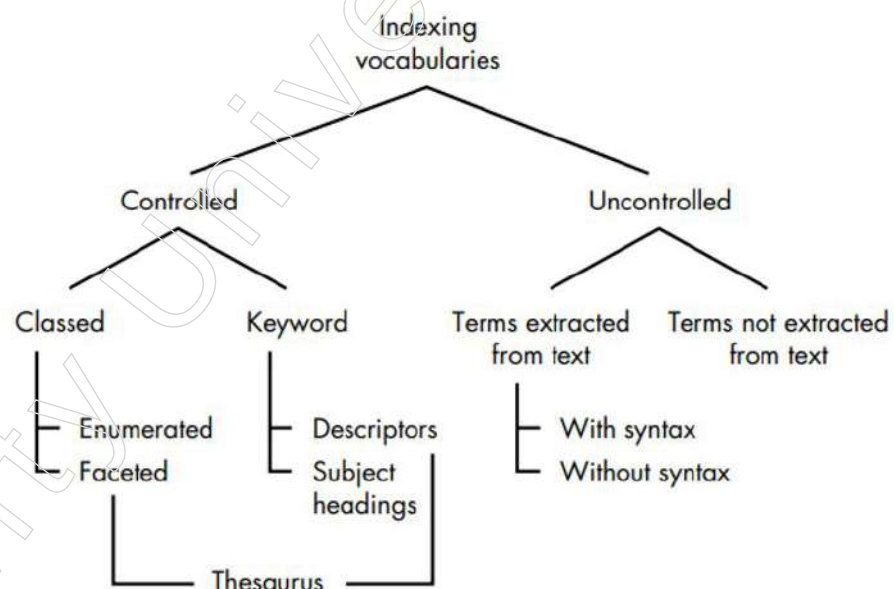


Figure: A taxonomy of indexing methods

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Enumerated classification. Software components are characterised by a hierarchical structure that defines classes and several tiers of subclasses. In the listed hierarchy, actual components are listed at the bottom of each path. For window operations, an enumerated hierarchy could look something like this:

- window operations
 - display
 - open

menu-based
 openWindow
 system-based
 sysWindow
 close
 via pointer
 ...
 resize
 via command
 setWindowSize, stdResize, shrinkWindow
 via drag
 pullWindow, stretchWindow
 up/down shuffle
 ...
 move
 ...
 close
 ...

An enumerated classification scheme has a hierarchical structure that facilitates understanding and application. Domain engineering must be done first, though, in order to have enough knowledge about the correct entries in the hierarchy before a hierarchy can be constructed.

Faceted classification. A domain region is examined and a number of fundamental descriptive characteristics are found. Then, these characteristics—known as facets—are linked to a component and prioritised. A facet can specify any feature, including the function performed by the component, the altered data and the context in which they are applied. The facet descriptor is the collection of facets that characterises a component. The aspect description is often restricted to a maximum of seven or eight facets.

Consider a system that uses the following facet descriptor as a basic example of how facets are used in component classification:

{function, object type, system type}

One or more values that are often descriptive keywords are assigned to each facet in the facet descriptor. For instance, if function is a component's facet, common values for this aspect could include

function = (copy, from) or (copy, replace, all)

A more thorough refinement of the primitive function copy is possible with the use of several facet values. For every component in a reuse library, a set of facets is given a keyword (value). A list of values is provided and the library is queried to find matches when a software engineer needs to query it for potential design components. A thesaurus function can be integrated using automated techniques. This allows the search to include technical synonyms for the keywords the software engineer selected in addition to the term

Notes

they specified. An intricate classification scheme provides the domain engineer with more versatility when defining intricate characteristics for individual components. The faceted classification technique is easier to expand and modify than the enumeration method since it is easy to add additional facet values.

Attribute-value classification. All components within a domain area have a defined set of properties. These qualities are subsequently given values in a manner similar to that of faceted classification. With the following exceptions, attribute value classification is actually comparable to faceted classification: (1) there is no upper limit on the number of attributes that can be used; (2) priority are not assigned to attributes; and (3) the thesaurus function is not employed.

Frakes and Pole state that there is no one “best” methodology and that “no method did more than moderately well in search effectiveness,” based on an empirical analysis of each of these classification techniques. It seems that more effort needs to be put into creating efficient classification algorithms for reuse libraries.

The Reuse Environment

Reusing software components needs to be enabled by an environment that includes the following components:

- A component database that holds both the classification data needed to retrieve software components and the components themselves.
- A library management system that grants database access.
- a method for retrieving software components from the library server that allows a client application to do so (such as an object request broker).
- tools from CBSE that facilitate reusing components in a fresh design or application.

Every one of these roles is included within or interacts with a reuse library.

One component of a broader CASE repository, the reuse library offers space for storing software components and a range of reusable artifacts, including as designs, requirements, code fragments, test cases and user manuals. The database and the tools required to query and retrieve components from the database are included in the library. Library queries are based on a component classification method.

Frequently, the context component of the 3C model, which was previously discussed, is used to characterise queries. A refined query is used to reduce the number of candidate components in a large list that is produced by an initial query. Once candidate components are located, concept and content information is collected to help the developer choose the right component.

3.1.3 Domain Engineering

Finding, creating, cataloguing and sharing a set of software components that may be used with current and upcoming software in a specific application domain is the aim of domain engineering. The ultimate objective is to create the means by which software engineers can reuse and exchange these components while working on both new and old systems.

Domain engineering has three primary tasks: disseminating, building and analysing. Reuse “will disappear, not by elimination, but by integration” into the routines of software engineering work, one could claim. Some predict that throughout the next ten years, domain engineering will overtake software engineering in importance as reuse becomes more of a focus.

The Domain Analysis Process

A comprehensive method for conducting domain analysis in the context of object-oriented software engineering. The process's steps were described as follows:

1. Define the domain to be investigated.
2. Categorize the items extracted from the domain.
3. Collect a representative sample of applications in the domain.
4. Analyze each application in the sample.
5. Develop an analysis model for the objects

It is crucial to remember that domain analysis may be used for both conventional and object-oriented development and it can be applied to any software engineering paradigm.

Prieto-Diaz offers an eight-step method for identifying and classifying reusable components, expanding on the second domain analysis stage:

1. Select specific functions or objects.
2. Abstract functions or objects.
3. Define a taxonomy.
4. Identify common features.
5. Identify specific relationships.
6. Abstract the relationships.
7. Derive a functional model.
8. Define a domain language.

The specification and subsequent creation of applications within the domain are made possible by a domain language.

While the previously mentioned phases offer a helpful framework for domain analysis, they don't offer any direction for selecting software components that could be repurposed. The following series of practical queries is recommended by Hutchinson and Hindley as a means of locating reusable software components:

- Example: Will future implementations require component functionality?
- As an illustration, how common is the function of the component inside the domain?
- For instance, is the function of the component duplicated inside the domain?
- For instance, is the part hardware dependent?
- For instance, does the hardware not change from implementation to implementation?
- Is it possible to transfer hardware specifications to a different component?
- For instance, is the design sufficiently optimised for the upcoming implementation?
- For instance, is it possible to parameterize a nonreusable component to make it reusable?
- For instance, can the component be reused with only small modifications in numerous implementations?
- Example: Is it possible to reuse something by changing it?
- For instance, is it possible for a nonreusable component to break down into reusable components?
- How valid is component decomposition for reuse, for instance?

Notes

Notes

Characterisation Functions

Determining if a supposedly reusable component is actually appropriate in a certain scenario might be challenging at times. Determining a collection of domain characteristics shared by all software inside a domain is important to arrive at this conclusion. A domain characteristic outlines a general feature shared by all products falling under the domain. Programming language, concurrency in processing, the significance of safety and reliability and many other things are examples of generic qualities.

A reusable component's domain characteristics can be expressed as $\{Dp\}$, where each item (Dpi) in the set denotes a distinct domain characteristic. An ordinal scale that indicates the significance of the attribute for component p is represented by the value given to Dpi . A standard scale could be

1. irrelevant to the question of appropriate reuse.
2. Only applicable in exceptional situations.
3. Relevant: Despite variations, the component can be altered to make it usable.
4. unquestionably important and reuse will be ineffective but still feasible if the new software lacks this feature.
5. reuse is not advised without this feature since it is obviously relevant and will be ineffective if the new software lacks it.

Table: Domain Characteristics Affecting Reuse

Product	Process	Personnel
Requirements stability	Process model	Motivation
Concurrent software	Process conformance	Education
Memory constraints	Project environment	Experience/training
Application size	Schedule constraints	• application domain
User interface complexity	Budget constraints	• process
Programming language(s)	Productivity	• platform
Safety/reliability		• language
Lifetime requirements		Development team
Product quality		productivity
Product reliability		

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Upon developing new software, w , in the application domain, a collection of domain attributes is determined for it. After that, a comparison between Dpi and Dwi is performed to see if the current component p may be successfully reused in application w .

The table above enumerates common domain attributes that may influence software reuse. To effectively reuse a component, consideration must be given to certain domain properties. The reusable components inside an application domain must be examined to ascertain their relevance even in cases where the program to be designed is evidently present within it. Occasionally (preferably, not too frequently), "reinventing the wheel" could still be the most economical option.

Structural Modeling and Structure Points

When conducting domain analysis, the analyst searches the apps that are housed within a domain for recurring patterns. Assuming that every application domain comprises

recurring patterns (of function, data and behaviour) with potential for reuse, structural modelling is a pattern-based approach to domain engineering.

The following is how Pollak and Rissman characterise structural models:

A limited set of structural parts with distinct patterns of interaction make up structural models. Multiple ensembles made up of these model pieces define the architectures of systems that use structural models. From these few constituents, simple patterns of interaction give rise to numerous architectural units.

A structural model can be used to describe any application area (for example, modern software in the aircraft avionics domain shares the same structural model despite significant differences in the systems' specifications). As a result, the structural model is an architectural style that belongs in the domain and may be applied to other applications.

A structure point is defined by McMaho as "a distinct construct within a structural model." Three unique features distinguish structure points:

1. An abstraction with a restricted number of instances is called a structural point. To put it in the vernacular of object-oriented programming, there should be little class hierarchy. Furthermore, the abstraction needs to be consistent across all applications inside the domain. If not, it is not reasonable to pay for the costs of confirming, recording and sharing the structural point.
2. It should be simple to understand the guidelines that control how the structural point is used. The interface to the structure point should also be somewhat straightforward.
3. Information hiding should be implemented by the structure point by isolating all of its internal complexity. As a result, the system's total perceived complexity is decreased.

Take the realm of alarm system software as an illustration of how structural points might serve as architectural patterns for a system. Systems as basic as SafeHome or as sophisticated as an industrial process alarm system may fall under this category. Nonetheless, a number of dependable structural patterns are seen in each instance:

- an interface that makes it possible for users to communicate with the system.
- a method for putting bounds on the parameters to be measured that the user can control.
- a system for managing sensors that can interface with every sensor under observation.
- a response system that responds to the information the sensor management system provides.
- a control system that gives the user authority over how the monitoring is conducted.

These structural elements are all included in a domain architecture. Generic structural points that cross several application areas can be defined as follows:

- Application front end—the GUI, which includes all panels, menus and tools for command and input editing.
- Database—the archive for every item pertinent to the application domain.
- Computational engine—the data-manipulating numerical and nonnumerical models.
- Reporting facility—the process that generates all types of output.
- Application editor—the system for adapting the application to certain users' requirements.

For software cost assessment, structure points have been proposed as a substitute for function points and lines of code.

Notes

3.1.4 Economics of Component-Based Software Engineering

Software engineering based on components is intuitively appealing. Theoretically, it should benefit a software company in terms of timeliness and quality. And savings on expenses ought to result from these. In the context of software engineering, we must first comprehend what can be reused and then what the true costs of reuse are. Consequently, a cost/benefit analysis for the reuse of components can be created.

Impact on Quality, Productivity and Cost

Aggressive software reuse can yield significant financial benefits, according to a large body of evidence from industry case studies. There is an improvement in overall cost, productivity in development and product quality.

Quality. A software component that is created for reuse should ideally be tested to ensure that it is error-free and flawless. In actuality, formal verification is not done on a regular basis and errors can and do happen. But with every reuse, flaws are discovered and fixed and as a result, the quality of a component increases. The component almost completely eliminates defects over time.

According to a Hewlett Packard study, the defect rate for newly produced software is 4.1 defects per KLOC, whereas the rate for reused code is 0.9 defects per KLOC. The defect rate for an application with 68 percent reused code was 2.0 defects per KLOC, which is 51 percent lower than the expected rate if the application had been constructed without reuse. A thirty-five percent increase in quality is reported by Henry and Faller. Reuse offers a nontrivial benefit in terms of quality and reliability for provided software, even though anecdotal accounts range across a very large band of quality improvement percentages.

Productivity. It takes less time to produce the plans, models, papers, code and data needed to create a deliverable system when reusable components are used throughout the software process. As a result, the consumer receives the same degree of functionality with less input work. As a result, productivity rises. It seems that 30 to 50 percent reuse can lead to productivity improvements in the 25–40 percent range, despite the fact that reports on percentage productivity enhancement are notoriously hard to understand.

Cost. In order to estimate the net cost savings for reuse, one must first predict the project's cost as if it were produced from scratch (Cs), from which the total cost of the program as delivered (Cd) and the costs associated with reuse (Cr) are subtracted.

The following expenses are related to reuse, Cr:

- Example: Domain analysis and modeling.
- Example: Domain architecture development.
- Example: Increased documentation to facilitate reuse.
- Example: Support and enhancement of reuse components.
- Example: Royalties and licenses for externally acquired components.
- Example: Creation or acquisition and operation of a reuse repository.
- Example: Training of personnel in design and construction for reuse

Whether reuse is a priority or not, many of the other charges mentioned below address challenges that are part of good software engineering practice. Domain analysis and running a reuse repository can come at a significant cost.

Cost Analysis Using Structure Points

An architectural pattern recurring in a specific application area is what we called a

structure point. By establishing domain architecture and then adding structure points to it, a software designer, also known as a system engineer, can create the architecture for a new product, system, or application. These structural components are either discrete reusable parts or sets of reusable parts.

Although structural points can be reused, there are nontrivial expenses associated with their qualification, adaption, integration and maintenance. Prior to reusing, the project manager needs to be aware of the expenses related to using structural points.

Cost information may be gathered for every structure point because they are all reusable components (and have a history in general). In a perfect world, every component in a reuse library has its qualification, adaption, integration and maintenance expenses maintained for every usage. After that, this data can be examined to estimate the expenses for the subsequent reuse.

Take the new application X, for instance, which needs to reuse three structure points (SP1, SP2 and SP3) and contain 60% new code. A range of other applications have been utilised by these reusable components and average costs for integration, qualification, adaption and maintenance are known.

The following needs to be ascertained in order to estimate the effort needed to provide X:

$$\text{Overall effort} = E_{\text{new}} + E_{\text{qual}} + E_{\text{adap}} + E_{\text{int}}$$

Where

E_{new} = effort required to engineer and construct new software components

E_{qual} = effort required to qualify SP1, SP2 and SP3.

E_{adapt} = effort required to adapt SP1, SP2 and SP3.

E_{int} = effort required to integrate SP1, SP2 and SP3.

By averaging the historical data gathered for the purpose of qualifying, adapting and integrating the reusable components in other applications, the effort needed to qualify, adapt and integrate SP1, SP2 and SP3 is ascertained.

Reuse Metrics

To quantify the advantages of reuse in computer-based systems, numerous software metrics have been created. A ratio can be used to represent the advantage of reuse in a system S.

$$R_b(S) = [C_{\text{noreuse}} - C_{\text{reuse}}] / C_{\text{noreuse}}$$

Where

C_{noreuse} = is the cost of developing S with no reuse.

C_{reuse} = is the cost of developing S with reuse.

Consequently, one can represent $R_b(S)$ as a non dimensional value in the range

$$0 \leq R_b(S) \leq 1$$

Devanbu and his colleagues propose that: (1) R_b will be impacted by the system's design; (2) it is crucial to include R_b in the evaluation of design alternatives because R_b is impacted by the design; and (3) the advantages of reuse are closely related to the cost-benefit of each individual reusable component.

Notes

Reuse leverage, a broad indicator of reuse in object-oriented systems, is described as

$$R_{lev} = \text{OBJ}_{reused} / \text{OBJ}_{built}$$

Where

OBJ_{reused} is the number of objects reused in a system.

OBJ_{built} is the number of objects built for a system.

3.2 Cleanroom Software Engineering

A strategy that can result in very high-quality software is the integrated use of statistical SQA, program verification (correctness proofs) and standard software engineering modelling (and possibly formal methods). The cleanroom software engineering methodology highlights the need of incorporating accuracy into software during the development process. In contrast to the traditional cycle of analysis, design, coding, testing and debugging, the cleanroom approach offers an alternative perspective.

Cleanroom software engineering aims to reduce reliance on expensive defect removal procedures by creating code increments correctly the first time and confirming their accuracy prior to testing. The statistical quality certification of code increments as they build up into a system is integrated into its process model.

The cleanroom method raises the bar for software engineering in a lot of ways. The cleanroom process places a strong emphasis on meticulous specification and design, as well as formal verification of every design aspect through the use of mathematically based correctness proofs. The cleanroom approach is an extension of the formal methods approach that places emphasis on statistical quality control measures, such as testing based on customers' expected software usage.

There are numerous short- and long-term risks associated with software failure in the actual world. The risks may pertain to financial loss, public infrastructure and commercial operations, or personal safety. A process paradigm called "cleanroom software engineering" eliminates flaws before they have the potential to cause major risks.

One method of software development used to create high-quality software is called "clean room software engineering." It is not the same as traditional software engineering, since traditional software engineering After every development stage is finished, there is a danger that the final phase, known as quality assurance (QA), would result in fewer, less dependable products that are prone to errors, defects and disgruntled clients, among other problems. However, because QA (Quality Assurance) is carried out at every stage of the software development process, clean room software engineering produces software that is both effective and high-quality when it is provided to the client.

Cleanroom software engineering is a quality approach to software development that adheres to a set of guidelines and best practices for requirements collecting, designing, coding, testing, managing and other processes. These practices not only lower development costs and boost productivity, but also improve product quality. From the start of system development until its conclusion, the focus is on reducing reliance on expensive processes and preventing errors in the process rather than fixing them after they arise.

3.2.1 Cleanroom Approach

The "cleanroom" concept in hardware fabrication methods is actually very straightforward: Establishing a fabrication process that prevents the introduction of product flaws is both time- and money-efficient. The cleanroom technique requires the discipline

necessary to remove defects in specification and design before producing in a “clean” way, as opposed to creating a product and then working to remove defects.

Mills, Dyer and Linger introduced the cleanroom concept to software engineering in the 1980s. This methodical approach to software development has not become widely used, despite early experiences showing great potential. Henderson offers three potential explanations:

- the conviction that using the cleanroom methodology in actual software development would be too radical, too theoretical and too mathematical.
- It recommends against unit testing by developers in favour of correctness verification and statistical quality control, which are radical ideas that differ greatly from the way most software is now built.
- the software development industry's level of maturity. Strict adherence to specified procedures throughout the whole life cycle is necessary when using cleanroom procedures. The industry has not been prepared to implement those strategies since the majority of it is still functioning at the Software Engineering Institute Capability Maturity Model's definition of the ad hoc level.

While there is some validity to each of these worries, the advantages of cleanroom software engineering surpass the costs involved in overcoming the underlying cultural opposition.

The Cleanroom Strategy

A customised incremental software model is used in the cleanroom methodology. Small, independent software engineering teams create a “pipeline of software increments.” Every certified increment is incorporated into the overall system. As a result, the system's functionality increases over time.

The figure below shows the order in which the cleanroom tasks are assigned at each increment. The pipeline of cleanroom increments begins once the software component of the system has been allocated functionality. The following assignments are completed:

Increment planning. An incremental strategy-based project plan is created. A cleanroom development schedule, the anticipated size of each increment and its functionality are created. To guarantee that authorised increments will be integrated on time, more caution must be used.

Requirements gathering. For every increment, a more thorough explanation of the customer-level requirements is created.

Box structure specification. The functional specification is described using a box-structuring-based specification method. Box structures “isolate and separate the creative definition of behaviour, data and procedures at each level of refinement,” in accordance with the principles of operational analysis.

Formal design. Cleanroom design is a logical and seamless extension of specification when using the box structure technique. Specifications (referred to as black boxes) are iteratively refined (within an increment) to become comparable to architectural and component-level designs (referred to as state boxes and clear boxes, respectively), despite the fact that it is feasible to distinguish clearly between the two activities.

Correctness verification. The cleanroom team rigorously verifies the validity of the design and subsequently the code through a series of steps. The highest level box structure (specification) is verified first, then design details and code. Using a series of “correctness

Notes

questions,” the first level of correctness checking is carried out. More formal (mathematical) methods of verification are employed if they fail to show that the specification is accurate.

Code generation, inspection and verification. The right programming language is translated from the box structural specifications, which are expressed in a specialised language. The semantic conformity of the code and box structures, as well as the syntactic accuracy of the code, are subsequently verified using standard walkthrough or inspection procedures. After that, the source code is checked for accuracy.

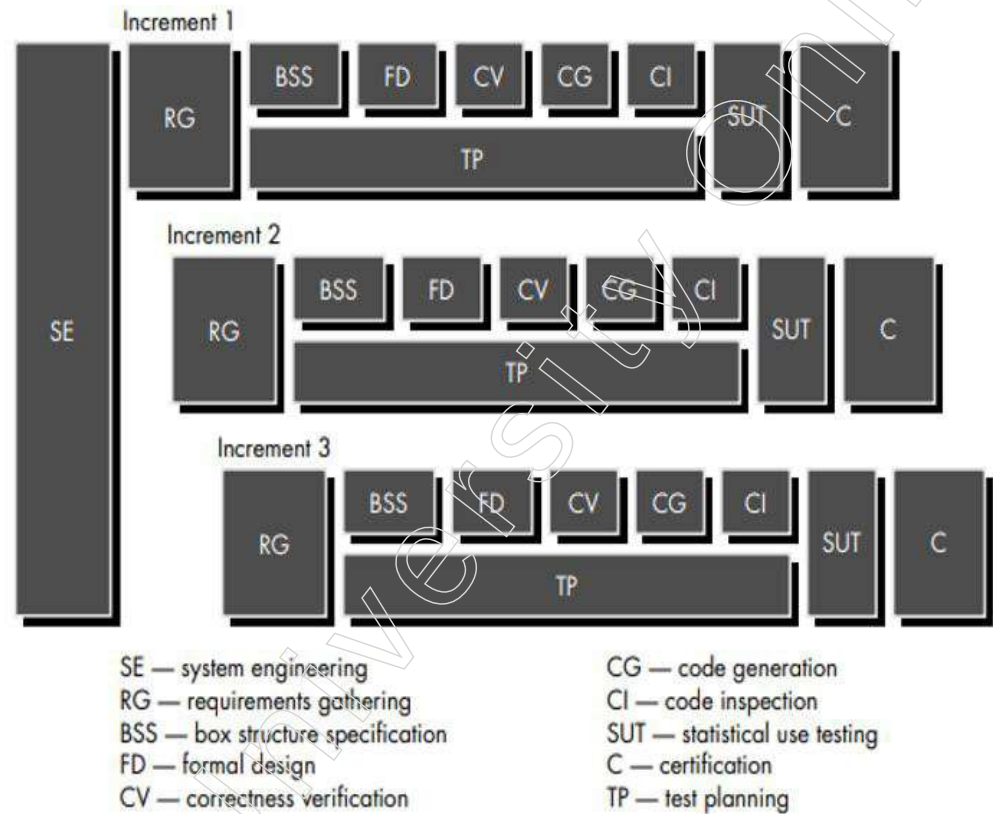


Figure: The cleanroom process model

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Statistical test planning. A set of test cases that exercise a “probability distribution” of usage are planned and constructed after the software’s anticipated usage is examined. This cleanroom operation is carried out concurrently with specification, verification and code production, as seen in the above figure.

Statistical use testing. Because it is impossible to test computer software thoroughly, there must always be a limited number of test cases designed. Statistical use approaches carry out a sequence of tests that are produced from a statistical sample of all possible program executions by all users from a specified group (the previously mentioned probability distribution).

Certification. The increment is validated as ready for integration once verification, inspection and usage testing are finished (and all mistakes are fixed).

The cleanroom process, like other software process models covered in this book, is largely dependent on the requirement to generate excellent analysis and design models. An additional method for a software engineer to express requirements and design is through box structure notation. The cleanroom approach differs primarily in that engineering models are subjected to formal verification.

With a clear plan for ongoing process improvement, Cleanroom is the first attempt at implementing statistical quality control in the software development process. In order to do this, a cleanroom unique life cycle was established, with an emphasis on software testing based on statistics for software reliability certification and software engineering based on mathematics for proper software designs.

Cleanroom software engineering is distinct from traditional, object-oriented software engineering because

1. It uses statistical quality control explicitly.
2. It uses a proof of correctness based on mathematics to validate the design specification.
3. To find high-impact errors, statistical use testing is largely relied upon.

It goes without saying that the most, if not all, of the fundamental ideas and principles of software engineering covered in this book are applicable to the cleanroom method. If high quality is to be produced, then sound analysis and design processes are necessary. However, cleanroom engineering differs from traditional software development methods in that it significantly reduces (or does away with) the quantity of testing carried out by the software developer and downplays (some would even say completely eliminates) the importance of unit testing and debugging.

In traditional software development, mistakes are considered inevitable. Since mistakes are accepted as inevitable, every program module needs to be unit tested in order to find errors and then debugged in order to fix them. Once the program is ultimately made available to the public, field testing reveals even more bugs and another round of testing and debugging starts. Rework related to these tasks is expensive and time-consuming. Even worse, error repair may have a degenerative effect, unintentionally introducing new errors.

Correctness verification and statistically based testing have taken the place of unit testing and debugging in cleanroom software engineering. These actions are what set the cleanroom method apart, along with the record keeping required for ongoing progress.

Dr. Harlan Mills of IBM's Federal Systems Division created the clean room technique, which was first published in 1981. However, it gained traction in 1987 when IBM and other organisations began implementing it.

Core Principles of Cleanroom

- **Formal Specification:** A major focus of Cleanroom is formal specification methods. To characterise the anticipated behaviour of the software, exact mathematical requirements are developed.
- **Statistical Testing:** Developing and implementing test cases based on statistical concepts is the hallmark of Cleanroom's statistical testing process. Without using conventional fault-based testing methods, this strategy seeks to find flaws.
- **Incremental Development:** Iterative and gradual development is encouraged by Cleanroom. The program is developed in phases and before each phase is incorporated into the main system, it is extensively tested.
- **Box Structure:** In Cleanroom, a visualisation technique called the Box Structure is employed to depict the information and control flow among various components. It facilitates comprehension and validation of the implementation and design.

Formal Methods in Cleanroom

- **Mathematical Specifications:** To define the specifications and design the software,

Notes

Cleanroom uses formal methods and mathematical notations. This formalism aids in the development process' accuracy and clarity.

- **Correctness Verification:** Formal methods offer a mathematical foundation for demonstrating that the software satisfies its requirements, which makes correctness verification easier. Cleanroom's dedication to creating high-assurance software depends on this phase.

Advantages of Cleanroom Software Engineering

1. High Reliability and Quality:

Defect Prevention: Defect avoidance is aided by Cleanroom's emphasis on statistical testing and formal methods. The methodology seeks to build highly dependable software by detecting and fixing flaws early in the development process.

Statistical Quality Control: A fundamental component of Cleanroom is statistical testing, which uses statistical quality control methods to gauge and regulate the software's quality. The end product's overall reliability is improved by this statistical method.

2. Early Detection of Defects:

Incremental Testing: Every module is put through a rigorous testing process before integration thanks to Cleanroom's incremental testing methodology. This makes it easier to identify flaws early on and fix them, keeping them from spreading to later phases of development.

Formal Specification: By using formal methods for specification, it is possible to validate requirements and designs early on, which reduces the possibility that errors may arise from misunderstandings.

3. Enhanced Maintainability:

Modular Development: The gradual and modular development methodology used by Cleanroom improves maintainability. Maintenance activities are made easier by the ability to make changes or upgrades to individual modules without affecting the system as a whole.

Formal Documentation: Cleanroom facilitates the creation of formal specifications and documentation that offer maintainers a thorough and lucid guide for comprehending and adjusting the software.

4. Predictable Development Process:

- ▣ **Statistical Project Planning:** Cleanroom uses statistical project planning approaches to calculate how much time and money will be needed for tasks related to development. By taking into account the uncertainty and unpredictability included in the development process, this statistical method improves predictability.

Progress Monitoring: Cleanroom's progressive design makes it possible to track development over time. When combined with statistical methods, this gives project managers the ability to efficiently monitor and control the development process.

5. Customer Satisfaction:

Reliable Software Delivery: Cleanroom's dedication to preventing defects and producing high-quality software helps them produce dependable and durable goods. Thus, by meeting or beyond expectations, this improves consumer happiness.

Transparent Development Process: The focus placed by Cleanroom on formal methods and documentation helps to ensure that the development process is transparent. Clients are able to comprehend the design, testing and specification procedures with clarity.

Challenges and Considerations in Cleanroom Software Engineering

1. Learning Curve and Skill Requirements:

Formal Methods Expertise: Because Cleanroom relies heavily on formal methods, development teams who are not familiar with mathematical notations and formal specification methodologies may find it challenging to understand.

Statistical Testing Knowledge: Expertise in statistical methods is necessary for the proper application of statistical tests. To properly implement the concepts of statistical testing, teams can require training.

2. Applicability to All Projects:

Suitability for Critical Systems: The strict methodology of cleanrooms is frequently most appropriate for vital systems where accuracy and dependability are crucial. For projects with less demanding specifications, Cleanroom's overhead can be too much.

Adaptability: Adapting the technique to projects that require numerous iterations or have rapidly changing requirements may provide issues.

3. Resource Intensiveness:

Time and Effort: The thorough testing procedures and iterative development methods used in Cleanroom might be resource-intensive. Compared to more agile alternatives, the methodology could take more time and effort.

Specialised Tools: Additional resources may be needed for formal methods and statistical testing in a cleanroom, such as specialised instruments.

4. Collaboration Challenges:

Communication and Collaboration: Team members may need to collaborate and communicate clearly in order to use the formal Cleanroom methods. The methodology's efficacy may be hampered by a lack of cooperation or by misinterpreting formal specifications.

Interdisciplinary Collaboration: It is essential for statisticians, mathematicians and software developers to work well together. Developing a common understanding may take more work when working across disciplines.

Real-World Applications and Case Studies

1. Aerospace and Defense:

High-assurance software is crucial for vital systems in the aerospace and defence industries, where cleanroom software engineering finds use. Cleanroom's reliability-focused approach is advantageous for projects involving control systems for defence or aeronautical applications.

2. Healthcare Software:

Software for the healthcare industry needs to be extremely accurate and reliable, especially when it comes to systems that handle patient data and medical records. Software for healthcare can have its integrity and security guaranteed by applying cleanroom principles.

3. Embedded Systems:

The development of embedded systems, such those found in industrial automation or automobile control units, can benefit from cleanroom conditions. Cleanroom's dependability and consistency are useful in these situations.

Notes

Notes

Cleanroom Software Engineering is a reliable and accurate guide in the constantly changing field of software engineering techniques. High-assurance software is developed as a result of its dedication to defect prevention, thorough testing and formal methods, especially in important domains where correctness is unavoidable.

With its high dependability, early defect detection and improved maintainability, Cleanroom is a viable method for projects when defect costs are significant. But given the difficulties—which include a learning curve, resource constraints and adaptability to changing project demands—careful examination of its applicability is required.

In practical uses, Cleanroom has shown its value in vital industries such as embedded systems, aircraft, defence and healthcare. Software engineering will continue to be greatly influenced by approaches like Cleanroom, which put correctness and stability first, as software systems become more and more essential to our daily lives. Working with Cleanroom requires focus, self-control and an unwavering quest for software quality.

3.2.2 Functional Specification

Cleanroom software engineering adheres to the concepts of operational analysis through the application of the box structure definition approach. A “box” contains the system in some detail, or at least a portion of it. Boxes are developed into a hierarchy where each box has referential transparency through a progressive refinement procedure. In other words, “each box specification’s information content is sufficient to define its refinement, independent of the implementation of any other box.” With crucial representation at the top and implementation-specific detail at the bottom, the analyst can now divide a system hierarchically. There are three kinds of boxes used:

Black box. The black box describes how a system or a subsystem behaves. A set of transition rules that translate a given stimulus into a reaction is applied by the system (or part) in response to particular stimuli (events).

State box. Similar to objects, the state box contains state information and services (operations). The state box’s inputs, or stimuli and outputs, or responses, are depicted in this specification view. The data contained in the state box that needs to be kept between the indicated transitions is also represented by the state box as the “stimulus history” of the black box.

Clear box. The transparent box defines the transition functions that the state box suggests. To put it simply, the state box’s procedural design is contained in a clear box.

The refinement method employing box structure specification is shown in the figure below. For a whole collection of stimuli, reactions are defined by a black box (BB1). A collection of black boxes, BB1.1 through BB1.n, that each handle a class of behaviour can be obtained by refining BB1. Until a cohesive class of behaviour (such as BB1.1.1) is found, refinement is carried out repeatedly. Next, a state box (SB1.1.1) for the black box (BB1.1.1) is defined. In this instance, all the information and resources needed to carry out the behaviour specified by BB1.1.1 are included in SB1.1.1. Eventually, procedural design details are defined and SB1.1.1 is further developed into clear boxes (CB1.1.1.n).

The verification of correctness takes place concurrently with each of these refining phases. State-box specifications are checked to make sure they all follow the parent black-box specification’s declared behaviour. In a similar manner, the parent state box is used to validate clear-box specifications. It should be mentioned that the box structure specification methodology can be substituted by specification techniques based on formal methods. The ability to formally verify each specification level is the only prerequisite.

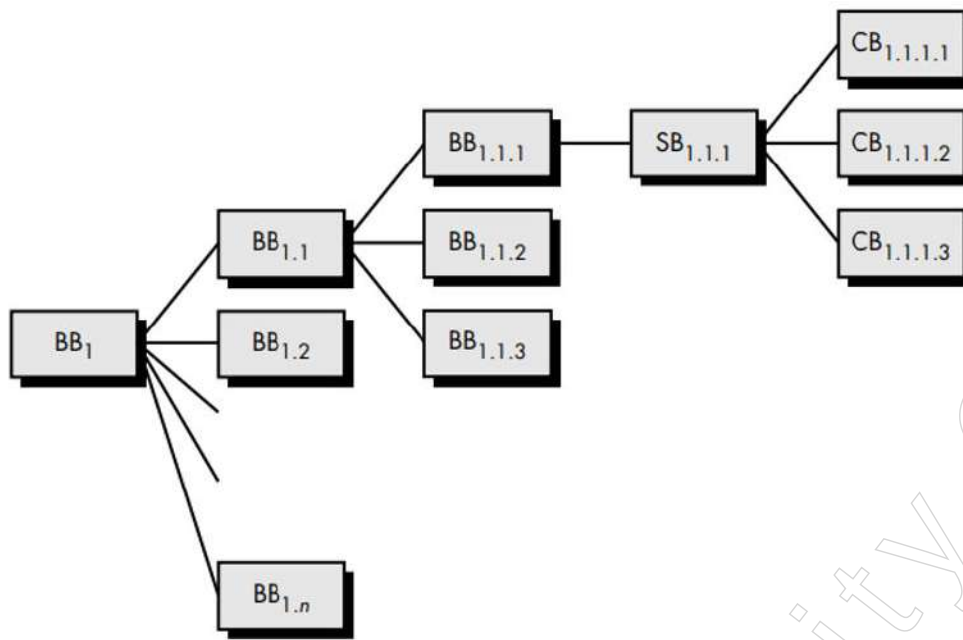


Figure: Box structure refinement

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Black-Box Specification

A black-box specification uses the syntax displayed in the figure below to express an abstraction, stimulus and response. A series, S^* , of inputs (stimuli), S , is subjected to the function f , which converts them into an output (response), R . A mathematical function f might be used for basic software components, although in most cases, f is described in natural language (or a formal specification language).

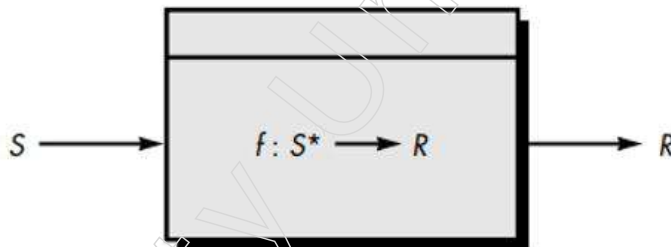


Figure: A black-box specification

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

A lot of the ideas that were presented for object-oriented systems can also be used with the black box. The black box encapsulates data abstractions and the procedures that work with those abstractions. The black box standard can show usage hierarchies, similar to class hierarchies, where lower-level boxes inherit the properties of those boxes higher in the tree structure.

State-Box Specification

"A simple generalisation of a state machine" is what the state box is. A state is a system's observable behaviour mode. A system changes from one state to another while processing takes place in response to events, or stimuli. Something might happen throughout the shift. The state box determines the action (reaction) that will happen as a result of the transition and the transition to the next state using a data abstraction.

Notes

With reference to the figure below, the state box has a black box. The input stimulus (S) for the black box originates from an external source and is accompanied by a set of internal system states. T. Mills gives the function, f , of the black box inside the state box a mathematical explanation:

$$g : S \times T \rightarrow R \times T$$

where t is a particular state and g is a subfunction associated with it. The state-subfunction pairs (t, g) taken as a whole define the black box function f .

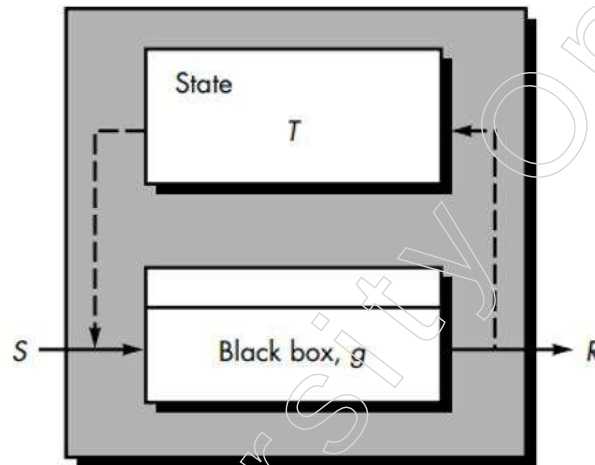


Figure: A state box specification

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Clear-Box Specification

The clear-box definition is quite similar to structured programming and procedural design. Essentially, the structured programming elements that implement g take the place of the state box subfunction g .

Take the clear box in the figure below as an illustration. A sequence construct with a conditional is used in place of the black box, g , in the above figure. Stepwise refinement can then be applied to these, leading to lower-level clear boxes.

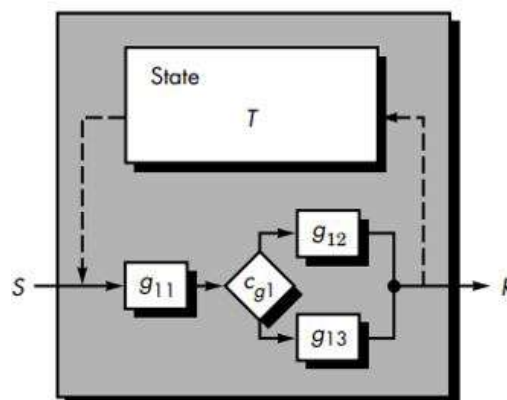


Figure: A clear-box specification

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

3.2.3 Cleanroom Testing

Cleanroom software engineering heavily utilises the structured programming paradigm in its design process. However, structured programming is used much more strictly in

this instance. A “stepwise expansion of mathematical functions into structures of logical connectives [e.g., if-then-else] and subfunctions, where the expansion [is] carried out until all identified subfunctions could be directly stated in the programming language used for implementation” is used to refine basic processing functions (described during earlier refinements of the specification).

Function can be efficiently refined through the use of structured programming, but what about data design? Several basic design principles are applied here. A collection of abstractions that are supported by subfunctions serve as the container for program data. The data design is derived from the ideas of data encapsulation, information concealing and data type.

Design Refinement and Verification

Every clear-box specification outlines the subfunction or method that must be designed in order to complete a state box transition. Stepwise refinement and structured programming constructs are utilised with the clear box, as seen in the figure below. A series of subfunctions, g and h , are derived from a program function, f . These are further developed into conditional expressions (do-while and if-then-else). Additional refinement serves as an example of ongoing logical refinement.

The cleanroom team2 verifies formal correctness at every stage of refinement. In order to achieve this, the structured programming techniques are coupled with a collection of general correctness requirements. The correctness requirement for each and every input to a function f that is expanded into the sequence g and h is

Does g do f when followed by h ?

The correctness condition for all input to a function p , when refined into a conditional of the form, if c then q , else r , is

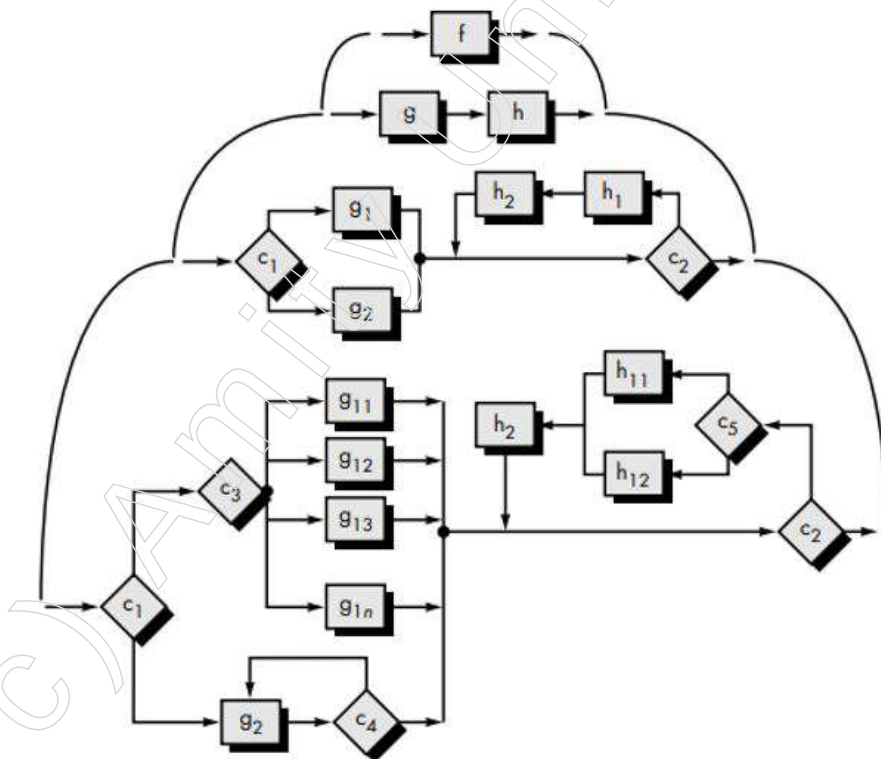


Figure: Stepwise refinement

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Notes

Whenever condition c is true, does q do p ; and whenever c is false, does r do p ?

The accuracy requirements for each input to function m , when refined as a loop, are

Is termination guaranteed?

Whenever c is true, does n followed by m do m ; and whenever c is false, does skipping the loop still do m ?

These correctness conditions apply each time a clear box is enhanced to the next level of detail.

It is significant to remember that the quantity of correctness tests required is limited by the use of structured programming features. For sequences, one condition is checked; for if-then-else, two conditions are examined; and for loops, three conditions are confirmed.

We employ a straightforward example that was previously presented by Linger, Mills and Witt to demonstrate correctness verification for a procedural design. The goal is to create and test a little program that calculates the integer portion, y , of a given integer's square root, x . The flowchart in the figure below serves as a representation of the procedural design.

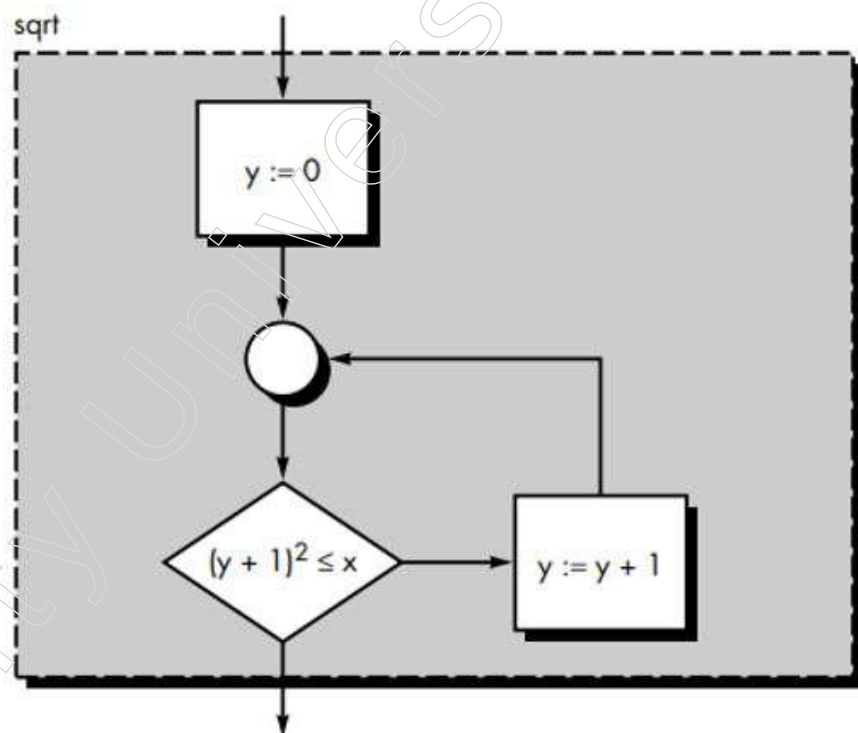


Figure: Computing the integer part of a square root

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

We must specify entry and exit conditions, as shown in the above Figure, in order to confirm the accuracy of this design. It is stated in the entry condition that x must equal or exceed 0. For there to be an exit, x must stay constant and take on a value that falls inside the range shown in the figure. It is vital to demonstrate that the conditions init, loop, cont, yes and exit indicated in the accompanying figure are true in every situation in order to demonstrate the correctness of the design. These are known as subproofs at times.

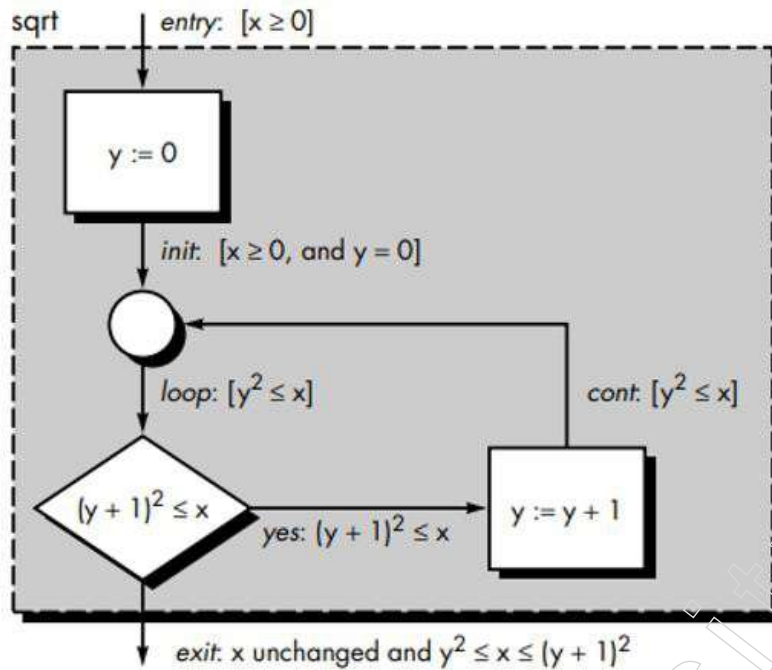


Figure: Proving the design correct

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

1. It is required by the condition init that $[x \geq 0 \text{ and } y = 0]$. The entrance condition is taken to be correct based on the needs of the problem. As a result, $x \geq 0$, the first component of the init condition, is met. With reference to the flowchart, $y = 0$ is set in the statement that comes right before the init condition. As a result, the init condition's second portion is likewise met. Thus, that is accurate.
2. There are two possible methods to encounter the loop condition: (1) through control flow that goes through the condition cont, or (2) straight from init (in this instance, the loop condition is satisfied directly). Loop is true independent of the flow channel that leads to it since the condition cont and the loop condition are the same.
3. Only after y has increased by one does the cont condition come into play. Furthermore, only in the event that the yes condition is also true can the control flow path leading to cont be called. Therefore, it follows that $y^2 \leq x$ if $(y + 1)^2 \leq x$. The requirement for cont is met.
4. The conditional logic displayed tests the yes condition. Therefore, when control flow follows the indicated path, the yes condition has to be true.
5. First, it is required by the exit condition that x stay unaltered. Looking closely at the design, we can see that x never appears to the left of an assignment operator. No function calls that make use of x exist. Thus, it hasn't altered. It follows that $(y + 1)^2 \leq x$ as the conditional test $(y + 1)^2 \leq x$ must fail to reach the exit condition. Furthermore, the loop condition $(y^2 \leq x)$ must still be true. Consequently, the exit condition can be satisfied by combining $(y + 1)^2 > x$ and $y^2 \leq x$.

We also need to make sure the loop ends. Analysing the loop condition reveals that the loop must eventually end because $x \geq 0$ and y is incremented.

The five steps mentioned above serve as evidence that the algorithm shown in the above figure is correctly designed. We can now be positive that the design will calculate the square root's integer portion.

Notes

Notes

Advantages of Design Verification

There are many clear benefits to rigorously verifying the validity of every clear-box design refinement. Linger puts these into the following terms:

- Verification becomes a limited procedure as a result. Control structures are arranged in a clear box in a nested, sequential manner that naturally defines a hierarchy that displays the correctness requirements that need to be confirmed. We can replace intended functions in the hierarchy of subproofs with corresponding control structure improvements thanks to an axiom of substitution. For instance, demonstrating that the combination of the operations g1 and g2 with the intended function f2 has the same impact on data as f1 is necessary for the subproof for the intended function f1 in the figure below. Keep in mind that in the proof, f2 stands in for every detail of its refining. The proof argument is localised to the current control structure by this replacement. It actually allows the software engineer to perform the proofs in any sequence.

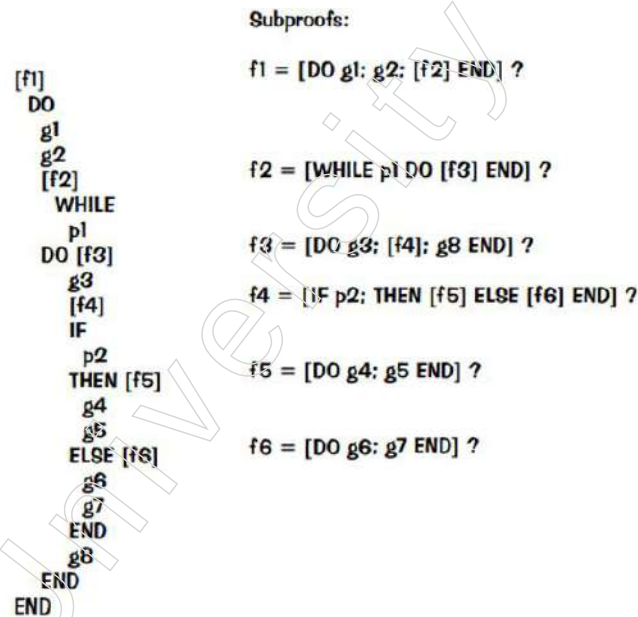


Figure: A design with subproofs

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

- It is impossible to overemphasise the positive effect that reducing verification to a finite process has on quality. All programs, except the simplest, can be validated in a finite number of steps, even though they display an almost endless number of execution pathways.
- It lets cleanroom teams verify every line of design and code. On the basis of the correctness theorem, teams can conduct group analysis and discussion to verify the system and when further assurance is needed for a mission- or life-critical system, they can generate written proofs.
- It results in a near zero defect level. Each control structure's correctness condition is checked one at a time during a team review. Each condition must be agreed upon by the entire team, hence an error can only occur if every team member confirms a condition wrongly. Software with few or no faults before initial execution is produced when unanimous consent is required based on individual verification.
- It scales up. Sequence, alternation and iteration structures make up the top-level, clear-box methods found in all software systems, regardless of size. A big subsystem with thousands of lines of code is usually invoked by each of these and each of those

subsystems has its own top-level intended functions and procedures. Thus, these high-level control structures' accuracy requirements are checked in the same manner as low-level structures'. High-level verification could need more time, but it doesn't require additional theory.

- It produces better code than unit testing. Unit testing examines the results of running a subset of test paths from a large pool of possible paths. The cleanroom method can check every potential impact on all data by relying on function theory for verification. This is because, although a program may have multiple execution routes, it only has one function. Additionally, verification is more effective than unit testing. It only takes a few minutes to verify most verification conditions, but it takes a long time to create, run and verify unit tests.

It is crucial to remember that source code itself must eventually be subjected to design verification. It is frequently referred to as correctness verification in this context.

3.2.4 Structure of Client/Server System

Distributed and cooperative computer architectures are facilitated by contributions from hardware, software, databases and network technologies. A distributed and cooperative computer architecture looks like the illustration below in its most basic form. Corporate data is kept in a root system, occasionally a mainframe. Servers, which are usually powerful workstations or PCs with multiple roles, are connected to the root system. The root system maintains company data, which the servers request and update. They also play a crucial position in user-level PC networking via a local area network (LAN) and manage departmental systems locally.

The computer that is located above another computer in a c/s structure is referred to as the server and the computer or computers that are located below it is referred to as the client. Services are requested by the client and supplied by the server. However, a variety of implementations are possible within the framework of the architecture shown in the figure below:

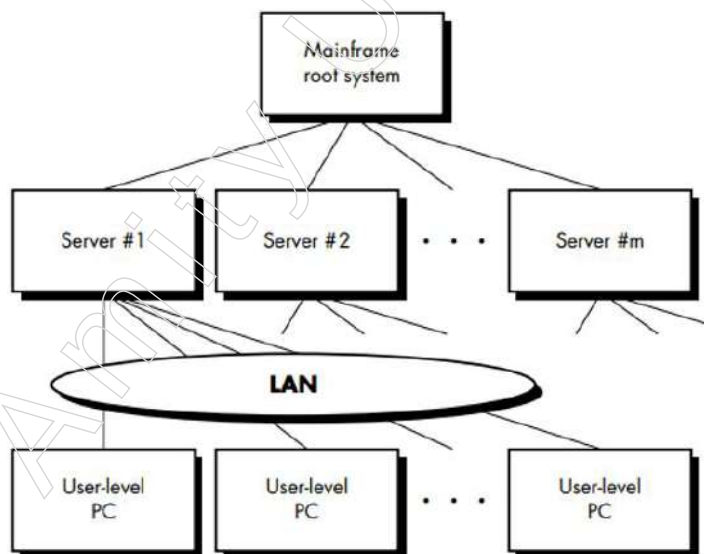


Figure: Distributed, cooperative computer architectures in a corporate setting
Image Source: Software Engineering A Practitioner's approach by roger s. pressman

File servers. The client asks to see particular documents in a file. These records are sent over the network from the server to the client.

Notes

Database servers. The server receives requests written in structured query language (SQL) from the client. These are sent via the network as messages. Only the client receives the results when the server processes the SQL request and locates the needed data.

Transaction servers. The server site's remote processes are triggered by a request sent by the client. A series of SQL statements make up the remote procedures. When a request triggers the remote procedure and the outcome is sent back to the client, a transaction has taken place.

Groupware servers. A groupware architecture is present when the server offers a collection of apps that permit communication between clients (and the users of them) through text, photos, videos, bulletin boards and other representations.

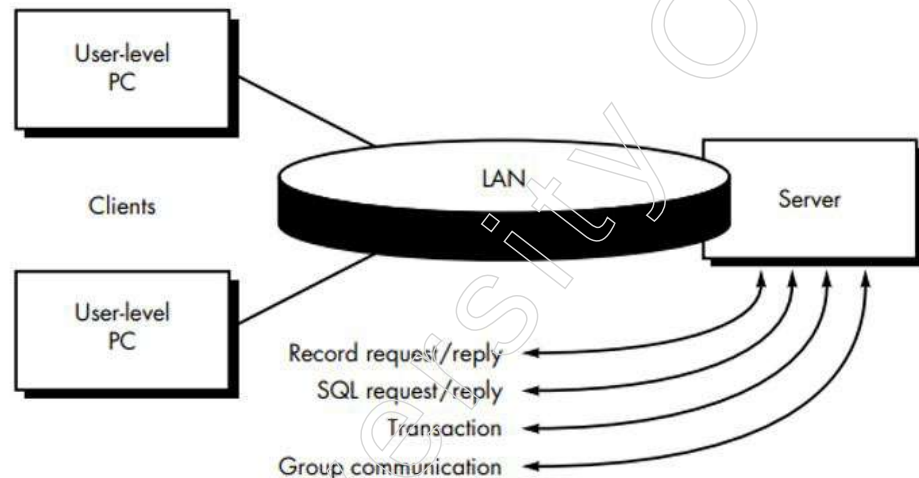


Figure: Client/server options

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Software Components for c/s Systems

Software that is suitable for a c/s architecture contains multiple discrete subsystems that can be assigned to the client, the server, or spread between both machines, as opposed to seeing software as a monolithic application to be implemented on one machine:

User interaction/presentation subsystem. All of the features that are often connected to a graphical user interface are implemented by this subsystem.

Application subsystem. Within the parameters of the domain in which the application functions, this subsystem carries out the requirements specified by the application. For instance, depending on numerical input, calculations, database data and other factors, a business program may generate several printed reports. Email and bulletin board communication capabilities could be enabled by a groupware program. In either situation, the application software may be divided into client-side and server-side components.

Database management subsystem. The data management and manipulation that an application requires is handled by this subsystem. Data administration and manipulation can range from simple tasks like moving a record to more complicated ones like handling intricate SQL transactions.

Every c/s system also has another software building piece, known as middleware, in addition to these subsystems. Middleware is a term for software components that are present on both the client and the server. It includes components of network operating systems and specialised application software that supports features that help with client/server connections, such as communication management, object request broker standards,

groupware technologies and database-specific applications. Middleware has been described as “the nervous system of a client/server system” by Orfali, Harkey and Edwards.

The Distribution of Software Components

between the client and the server after the fundamental specifications for a client/server application have been established. A fat server architecture is produced when the server is assigned the majority of the functionality related to each of the three subsystems. On the other hand, a fat client architecture is produced when the client implements the majority of the database, application and user interaction/presentation components.

Implementing file server and database server architectures frequently results in fat clients. In this instance, all application and GUI software is client-side, while the server supports data administration. When groupware and transaction systems are put into place, fat servers are frequently created. In order to reply to client communications and transactions, the server offers the application support needed. The client software prioritises communication management and the GUI.

You can use fat clients and fat servers to show how client/server software systems are allocated generally. Nonetheless, a more detailed method of allocating software components delineates five distinct configurations:

Distributed presentation. The application logic and database logic in this basic client/server model stay on the server, which is usually a mainframe. The logic for preparing screen data using programs like CICS is also housed on the server. Character-based screen data that is transferred from the server is transformed into a graphical user interface (GUI) presentation on a PC using specialised PC-based software.

Remote presentation. As an extension of the distributed presentation strategy, the client uses data sent by the server to construct the user presentation, while the primary database and application logic are kept on the server.

Distributed logic. All user presentation tasks and data entering procedures, including field-level validation, server query formulation and server update information and requests, are delegated to the client. Database management, client version control, server file updates and enterprise-wide application procedures are all delegated to the server.

Remote data management. By formatting data that has been retrieved from somewhere else (like a corporate level source), server applications generate a new data source. The new data that has been prepared by the server is exploited by applications that are assigned to the client. Systems for making decisions fall under this group.

Distributed databases. The information that makes up the database is dispersed among several clients and servers. As a result, in addition to application and GUI components, the client must provide data management software components.

Thin-client technology has also received a lot of attention in recent years. A so-called “network computer” that outsources all application processing to a fat server is known as a thin client. When compared to desktop computers, thin clients, or network computers, offer significantly reduced cost per unit with little to no noticeable performance loss.

Guidelines for Distributing Application Subsystems

Although there are no hard and fast rules on how application subsystems should be distributed between the client and server, general best practices include the following:

The presentation/interaction subsystem is generally placed on the client. This strategy is cost-effective since PC-based, Windows-based environments are readily available and the processing power needed for a graphical user interface is minimal.

Notes

If the database is to be shared by multiple users connected by the LAN, it is typically located on the server. Along with the actual database, the database management system and database access capabilities are housed on the server.

Static data that are used for reference should be allocated to the client. This reduces needless network traffic and server loading by putting the data closest to the consumers who need it.

Based on the distribution that optimises the server and client configurations as well as the network that links them, the application subsystem balance is divided between the client and server. For instance, implementing a mutually exclusive relationship usually entails searching the database to see if any records satisfy the specifications for a certain search pattern. An alternative search pattern is employed if no match is discovered. Network traffic is reduced if the program in charge of this search pattern is entirely contained on the server.

The parameters for the primary and secondary search patterns would be included in the first network transmission sent by the client to the server. If a secondary search is necessary, it would be decided by the server's application logic. The record that was discovered as a consequence of the primary or secondary search would be included in the response message sent to the client.

A message for the first record retrieval, a response over the network if the record is not found, a second message with the parameters for the second search and a final response with the retrieved record would be the alternate approach, which places the logic to determine whether a second search is necessary on the client. Network traffic would be reduced by 33 percent if the logic to assess the results of the first search and start the second search, if needed, was placed on the server. This is assuming that the second search is needed fifty percent of the time.

The mix of applications running on the system should be taken into consideration when making the ultimate decision on subsystem distribution, in addition to the specific application. For instance, certain apps in an installation can need a lot of work on the GUI and not much on the central database. This would result in the usage of a basic server and powerful workstations on the client side. Other apps might choose the fat client method with this configuration in place, negating the need to improve the server's capabilities.

Placement of volatile application logic on the server has become more common as client/server architecture has grown in popularity. When modifications are made to the application logic, this makes it easier to deliver software upgrades.

Linking c/s Software Subsystems

The client/server architecture's numerous subsystems are connected via a variety of methods. These procedures are transparent to the end user at the client site and are integrated into the operating system and network architecture. The following connection mechanism types are most prevalent:

- pipes. Pipes, which are widely used in UNIX-based systems, allow messages to be sent across machines that are running different operating systems.
- calls for remote procedures. These allow a process running on one computer to call upon the execution of a different processor module.
- SQL communication between a client and a server. This is how SQL requests and related data are passed from one component—usually on the client—to another—usually on the server, at the DBMS. Only applications using relational database management systems (RDBMS) can use this method.

Middleware and Object Request Broker Architectures

The components (objects) that implement the c/s software subsystems covered in the previous sections must be able to communicate with one another both within and across a single machine (either a client or server). An object on a client can send a message to a method contained by an object on a server through the use of middleware called an object request broker. To put it simply, the ORB intercepts the message and manages all the coordination and communication needed to locate the object to which the message was addressed, call its method, send the necessary data to the object and then return the resultant data to the original object that generated the message.

There are three popular standards that leverage the object request broker concept: JavaBeans, COM and CORBA.

A CORBA architecture's fundamental structure is depicted in the figure below. When implementing CORBA in a client/server system, an interface description language—a declarative language that enables a software engineer to define objects, attributes, methods and the messages needed to invoke them—is used to define objects and object classes on both the client and the server. Client and server IDL stubs are constructed to support a client-resident object's request for a server-resident method. Requests for objects throughout the c/s system are handled by the stubs, which act as a gateway.

A system for storing the object description must be built because requests for objects over the network happen at run time. This will ensure that relevant details about the item and its location are accessible when needed. The interface repository makes this possible.

CORBA uses dynamic invocation to: (1) retrieve relevant information about the desired method from the interface repository; (2) create a data structure with parameters to be passed to the object; (3) create a request for the object; and (4) invoke the request when a client application needs to call a method that is contained within an object somewhere else in the system. The request is then forwarded to the ORB core, a request management component of the network operating system that is specific to an implementation and it is subsequently fulfilled.

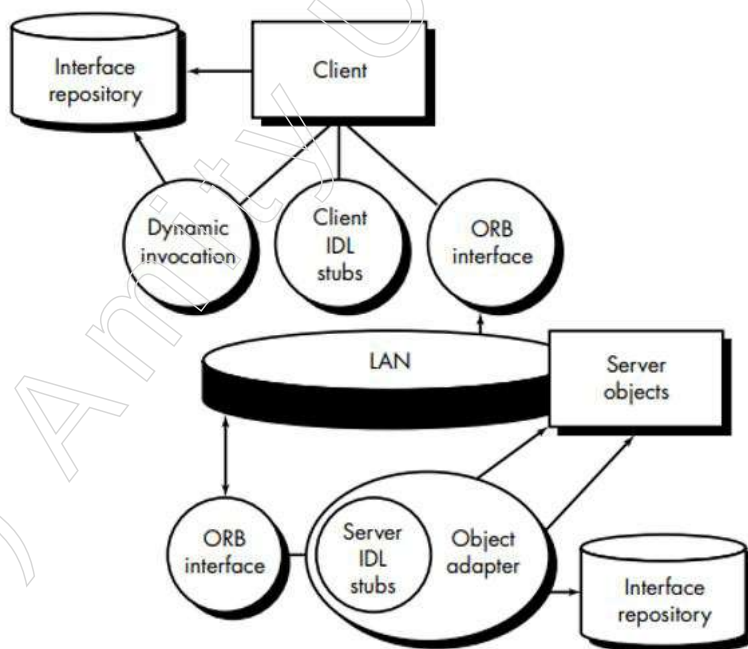


Figure: The basic CORBA architecture

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Notes

The server processes the request after it has passed through the core. Object adapters are used at the server site to handle incoming requests from clients, save class and object information in a server-resident interface repository and carry out various other object management tasks. The real object implementation located at the server location is accessed via IDL stubs that resemble those defined at the client computer.

A modern C/S system develops its software using object-oriented programming. Software developers can design an environment where items can be reused across a broad network environment by utilising the CORBA architecture

Summary

- Component-Based Software Engineering (CBSE) is an approach to software development that emphasises the construction of software systems by assembling pre-built, reusable components. These components encapsulate specific functionalities and can be combined to create complex applications. CBSE aims to enhance productivity, maintainability and reusability in software development.
- Domain Engineering is a systematic approach in software engineering that focuses on creating reusable assets and capturing commonalities within a specific application domain. The goal of domain engineering is to streamline the development process by identifying and designing components, patterns and models that can be shared across multiple projects within the same domain.
- The Cleanroom Approach is a software development methodology that focuses on producing high-quality software with minimal defects. It emphasises a rigorous and disciplined process to achieve reliability, correctness and efficiency in software development. The Cleanroom Approach was developed as a response to the need for high-assurance software, especially in critical systems such as aerospace, healthcare and defense.
- Cleanroom Testing is an integral part of the Cleanroom software development methodology. It is a systematic and disciplined approach to software testing that focuses on verifying the correctness and reliability of software components. Cleanroom Testing complements the overall Cleanroom Approach, which aims to produce high-quality software with minimal defects.
- A Client/Server System is a distributed computing architecture that divides tasks or processes between clients and servers, allowing for efficient and scalable interaction between users and resources. The structure of a client/server system is characterised by the separation of roles and responsibilities between client devices and server systems.

Glossary

- CBSE: Component-Based Software Engineering
- COTS: Commercial Off-The-Shelf
- API: Application programming interface
- ORB: Object Request Broker
- IDL: Interface Definition Language
- BDk: Bean Development Kit
- DFR: Design For Reuse
- QA: Quality Assurance
- LAN: Local Area Network
- GUI: Graphical User Interface

Check Your Understanding

1. What is a key emphasis in Component-Based Software Engineering (CBSE)?
 - a. Sequential development
 - b. Parallel development
 - c. Reusable components
 - d. Proprietary components
2. What is the primary goal of CBSE in terms of software development?
 - a. Maximising defects
 - b. Minimising reusability
 - c. Enhancing productivity
 - d. Ignoring software modularity
3. What is a typical characteristic of a software component in CBSE?
 - a. Monolithic and tightly coupled
 - b. Reusable and independent
 - c. Limited functionality
 - d. Embedded within a single project
4. Which phase in CBSE involves identifying and designing reusable assets within a specific domain?
 - a. Implementation
 - b. Testing
 - c. Domain Engineering
 - d. Deployment
5. What does CBSE stand for?
 - a. Concurrent-Based Software Engineering
 - b. Component-Based Software Evolution
 - c. Component-Based Software Engineering
 - d. Common-Based Software Enhancement

Exercise

1. Explain component-based software engineering.
2. What do you mean by domain engineering?
3. What is the economics of component-based software engineering?
4. Explain cleanroom approach.
5. Define functional specification.

Learning Activities

1. Imagine you are tasked with developing a web-based e-commerce application using Component-Based Software Engineering. Describe the process of component identification for this application. Identify at least three potential reusable components and explain why they are suitable for reuse.
2. As part of a CBSE project, you are responsible for setting up a component repository to store and manage reusable components. Describe the key steps involved in building and maintaining this component repository. Discuss the criteria for selecting components to be included, the version control mechanisms and how you would ensure the quality and reliability of the components in the repository.

Check Your Understanding- Answers

1. c)
2. c)
3. b)
4. c)
5. c)

Notes

Module - IV: Client/Server Software Engineering

Learning Objectives

At the end of this module, you will be able:

- Understand software engineering for client server systems
- Analyse the design for client server systems and testing issues
- Define peer to peer architecture
- Analyse software testing issues
- Define WebE process and framework for it
- Know Service Oriented Software Engineering

Introduction

Building scalable, modular and distributed systems still starts with client/server software engineering. Technological developments like as server less computing, containerisation and microservices are changing the game and presenting new problems and opportunities for architects and developers. In the ever-evolving world of software engineering, client/server-based systems require a careful design that takes scalability, modularity and security into account.

The client, which handles user interface and user interactions and the server, which handles request processing, data management and business logic enforcement, are the two main components of a system according to the client/server architectural core model in software engineering. The creation of scalable, distributed and modular systems has been greatly aided by this design. Let's examine the main facets of software engineering for clients and servers.

1. Components of Client/Server Architecture:

- a. Client: The application's user-facing component, the client, is in charge of informing users and gathering their input. Depending on the system, it may be a mobile app, a web browser, or a desktop application.
- b. Server: The application logic is hosted on the server, which also handles client requests, maintains data and guards the system's integrity. It can manage several client connections at once and be either a physical server or a cloud-based service.

2. Types of Client/Server Models:

- a. Two-Tier (Client/Server): The client and server have direct communication in the two-tier architecture. This architecture can result in reduced scalability and higher client complexity, although it is appropriate for simpler applications.
- b. Three-Tier (Client/Application Server/Database Server): An application server is introduced in the three-tier concept, dividing the data storage and business logic. This improves maintainability and scalability and makes deployment options more flexible.
- c. N-Tier (Multiple Application Servers): The burden is distributed among several application servers that manage distinct capabilities in an N-tier architecture. Although this paradigm is very flexible and scalable, it needs to be carefully designed and managed.

3. Communication Protocols:

- a. HTTP/HTTPS: Frequently employed for client/server web-based interactions. offers a stateless communication mechanism appropriate for a variety of uses.
 - b. TCP/IP: Offers dependable, connection oriented communication. Ideal in situations where delivery order and data integrity are crucial.
 - c. RESTful APIs: Web services are designed using the Representational State Transfer (REST) architectural style. is extensively used due to its scalability and simplicity and it communicates using normal HTTP methods.
4. Benefits of Client/Server Software Engineering:
- a. Scalability: Distributed processing made possible by client/server design enables systems to manage a high volume of users and data.
 - b. Modularity: Modularity and simpler maintenance are encouraged by the separation of client and server components. Modifications to one element do not always affect the other.
 - c. Centralised Data Management: Centralised data management on the server preserves data integrity. Enforcing data consistency, backup and security is simpler.
 - d. Enhanced Security: Because sensitive business logic and data are stored on the server, there is less chance that clients may be exposed. It is possible to build secure communication protocols between the client and server.
5. Challenges and Considerations:
- a. Network Dependency: Systems that use clients and servers rely on network connection. Performance can be impacted by problems such as latency and network failures.
 - b. Complexity in Deployment: It can be difficult to deploy and maintain server and client components, particularly in large-scale applications.
 - c. Compatibility Issues: It might be difficult to ensure compatibility across many operating systems, browsers and devices.
 - d. Scalability Planning: Careful planning is necessary when designing for scalability in order to divide the workload among servers efficiently.
6. Modern Trends in Client/Server Software Engineering:
- a. Microservices Architecture: dividing programs into manageable, separately deployable services. improves maintainability, scalability and eases continuous deployment.
 - b. Serverless Computing: abstracts server administration so that programmers can concentrate only on creating code. automatically adjusts for demand.
 - c. Containerisation: packages apps and their dependencies using containers. increases scalability and consistency across many contexts.

4.1 Overview to Client/Server Software Engineering

Client/Server Software Engineering is a fundamental architectural paradigm that controls how two key components—the client, which handles user interfaces and interactions and the server, which handles application logic, data management and business processes—interact with one another. Tasks are divided effectively by this paradigm, allowing distributed, modular and scalable systems. It includes a range of architectures, each serving a particular purpose, such as two-tier, three-tier and N-tier models. Communication protocols that enable smooth interaction between clients and

Notes

servers include TCP/IP, HTTP/HTTPS and RESTful APIs. This method encourages improved security, centralised data management, scalability and modularity. In the current software development landscape, client/server software engineering is still essential despite obstacles like network dependency and deployment complexity. It has evolved alongside trends like microservices, server less computing and containerisation to fulfill the needs of modern applications.

4.1.1 Software Engineering for Client Server Systems

Several models of the software process were presented. The two most popular approaches are (1) an evolutionary paradigm that uses event-based and/or object-oriented software engineering and (2) component-based software engineering that draws on a library of commercial off-the-shelf (COTS) and in-house software components, though any of them could be modified for use during the development of software for c/s systems.

The traditional software engineering processes of analysis, design, construction and testing are used in the development of client/server systems as they progress from a collection of general business requirements to a set of verified software components that have been installed on client and server computers.

The methodical design, development, testing and upkeep of software applications that adhere to the client-server architecture are all part of software engineering for client-server systems. This design encourages effective communication and resource sharing by dividing duties or processes between the client (user interface) and the server (application logic and data storage). There are several steps and factors to take into account in the software engineering process for client-server systems.

Key Aspects of Software Engineering for Client-Server Systems:

1. **Requirements Analysis:** Determine and comprehend the client-server system's unique requirements. Determining user interactions, data storage requirements and performance expectations are all part of this. Ascertain the functions of the user interfaces, application logic and database interactions that are part of the client and server components.
2. **Architecture Design:** Describe the general client-server system architecture, including the components' means of interaction and communication. To guarantee a smooth integration, select suitable communication protocols and data exchange formats between the client and server. Develop the application logic on the server side and the user interface components on the client side.
3. **Technology Selection:** Select frameworks and technologies that facilitate the client-server architecture. This entails choosing database management systems, communication protocols (such HTTP and WebSocket) and computer languages. Take performance and scalability into account to manage changing client and server loads.
4. **Development:** Create a user experience that is responsive and easy to use by implementing the user interface on the client side. Create the data storage systems, business rules and application logic for the server-side components. Assure appropriate data validation, error handling and communication between the client and server components.
5. **Testing:** Make sure that the client and server components are thoroughly tested. Unit, integration and system testing are all included in this. Check that the client and server are interacting and exchanging data without any problems. To assess the system's performance under high loads, do stress testing.
6. **Security Considerations:** Put security measures in place to safeguard data while it's being transmitted and stored. This entails the use of secure storage techniques and

encryption for communication. Put authorisation and authentication procedures in place to limit access to private information and features.

7. **Scalability and Performance Optimisation:** By building the system to accommodate an expanding number of users and data, you can plan for scalability. This could entail distributed databases and load balancing. To guarantee a responsive and effective system, optimise the performance of the client and server components as well.
8. **Deployment:** Install the client-server system in the intended setting, taking security protocols, network setups and server configurations into account. To make upgrades and maintenance easier, use release management and version control procedures.
9. **Monitoring and Maintenance:** Install monitoring software to keep an eye on the client-server system's health and performance. In order to keep the system current and safe, do routine maintenance and respond quickly to any difficulties.
10. **Documentation:** Provide thorough user manuals, system architecture diagrams and code documentation for the client-server system. To make future development or system integration easier, document APIs and interfaces.

Challenges in Client-Server Software Engineering:

- **Network Latency:** Network connection is necessary for client-server systems and latency can impact how responsive an application is.
- **Data Consistency:** It might be difficult to guarantee data consistency between the client and server, particularly in distributed applications.
- **Scalability:** Careful planning is needed to balance the load and scale the client and server components to handle growing user loads.
- **Security Risks:** It is essential to safeguard against vulnerabilities like injection attacks and secure communication channels.
- **Compatibility Issues:** Complexity is increased by having to ensure interoperability across various client devices, browsers and server environments.

Client-server software engineering necessitates a thorough and planned approach to design, development and maintenance. Software developers may develop dependable and scalable client-server applications that satisfy changing user and business objectives by taking architectural concerns into account, choosing relevant technologies, guaranteeing security and performance-enhancing.

4.1.2 Design for Client Server Systems and Testing Issues

When developing software for a particular computer architecture, the design methodology needs to take the particular construction environment into account. Essentially, the hardware architecture should be accommodated into the design.

When designing software for client/server architecture, the design methodology needs to be “adapted” to address the following problems:

- In the design process, data and architectural design are paramount. The data design becomes even more important than in traditional applications in order to utilise the capabilities of an object-oriented database management system (OODBMS) or relational database management system (RDBMS) efficiently.
- Behavioural modelling should be done when the event-driven paradigm is selected and the design model should incorporate the control-oriented elements that the behavioural model suggests.

Notes

- A c/s system's user interaction/presentation component carries out all of the operations often connected to a graphical user interface. Interface design is thus given more weight.
- Often, an object-oriented perspective on design is selected. An event that is started at the GUI and linked to an event handling function in the client-based software provides an object structure as opposed to the sequential structure found in procedural languages.

Architectural Design for Client/Server Systems

A client/server system's architectural design is frequently described as having a communicative processes style. The following is how Bass, Clements and Kazman characterise this architecture:

Reaching the scalability quality is the aim. The purpose of a server is to provide data to one or more clients, who are usually spread out over a network. The server receives a call from the client and it responds to the request either synchronously or asynchronously. When a server operates synchronously, control is returned to the client simultaneously with data. The client (which has its own thread of control) receives only data from the server if it operates asynchronously.

An object request broker architecture is utilised to implement this synchronous or asynchronous communication because contemporary C/S systems are component-based.

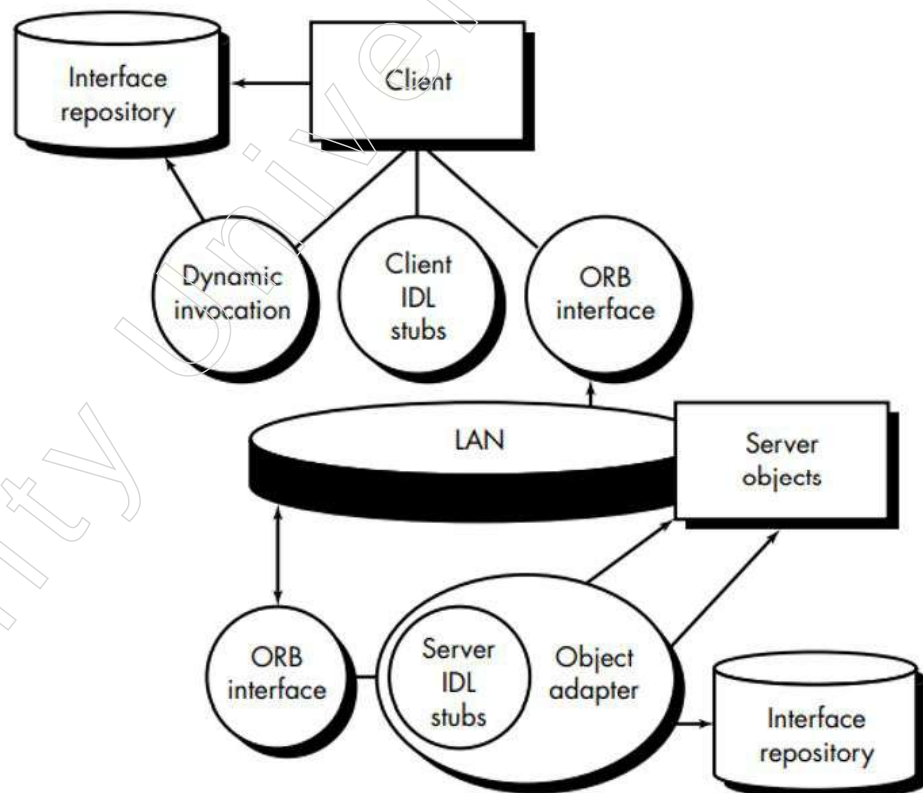


Figure: The basic CORBA architecture

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Interface details are specified at the architectural level using the CORBA interface description language. Application software components can use ORB services (components) without being aware of their internal workings thanks to the use of IDL. The coordination of communication between client and server components falls under the

purview of the ORB as well. The designer specifies an object adaptor, also known as a wrapper, that offers the following services in order to achieve this.

- Component (object) implementations are registered.
- All component (object) references are interpreted and reconciled.
- Component (object) references are mapped to the corresponding component implementation.
- Objects are activated and deactivated.
- Methods (operations) are invoked when messages are transmitted.
- Security features are implemented.

The ORB architecture needs to be built with component interoperability in mind in order to support both in-house components that might have been constructed using a different technology and COTS components provided by various vendors. CORBA employs a bridge notion to achieve this.

Let us assume that ORB protocol X was used to create a client and ORB protocol Y was used to implement a server. Both protocols are CORBA compliant, however they need to talk to a “bridge” in order to translate between internal protocols due to variations in internal implementation. For seamless communication between the client and server, the bridge interprets messages.

Conventional Design Approaches for Application Software

The data flow diagram can be used in client/server systems to determine the subject data areas (data stores), high-level functions and system scope. It can also be used to allow for the deconstruction of the high-level functions. However, instead of going all the way to the level of an atomic process as in the conventional DFD technique, decomposition ends at the level of a basic business process.

A basic business process (EBP) in the context of c/s is a series of tasks carried out at a client site by a single user continuously. Either all of the chores are completed, or none at all.

Additionally, the role of the entity relationship diagram is expanded. The DFD's subject data regions (data stores) are still broken down utilising it to provide a high-level picture of the database that will be implemented with an RDBMS. Its new function is to give high-level business object definitions a framework.

The structure chart is now used as an assembly diagram to display the components involved in the solution for a basic business process, rather than as a tool for functional decomposition. The components that comprise interface objects, application objects and database objects determine the processing methods for the data.

Database Design

The structure of business objects utilised in the client/server system is defined and then specified through database design. Business objects can be defined using conventional analytical modelling notation, such the ERD. However, in order to capture the additional information that cannot be properly described using a graphic notation like an ERD, a database repository needs be constructed.

Information visible to buyers and users of the system—not to its implementers—is referred to in this repository as a business object. A design repository can be used to store this data when it is implemented using a relational database. The client/server database is gathered with the following design data.

Notes

- The new system's Entities are identified in the ERD.
- The entities listed in the ERD are implemented using files.
- File-to-field relationships determine which fields are contained in which files, hence establishing the file layout.
- The data dictionary, or fields in the design, are defined by fields.
- Relationships between files allow for the creation of logical views and queries by joining similar files.
- The kind of file-to-file or file-to-field relationships that are employed for validation are identified by relationship validation.
- By using field type, field attributes from field superclasses—such as date, text, number, value and price—can be inherited.
- The properties of the data in a field are specified by the data type.
- The file's location can be determined using the file type.
- Key, foreign key, attribute, virtual field, derived field and similar functions are examples of field functions.
- The values permitted for status type fields are indicated by authorised values.
- The rules for updating, computing derived fields and other tasks are known as business rules.

The increasing prevalence of c/s architectures has pushed the trend towards distributed data management. The data management component is located on both the client and the server in c/s systems that use this technique. Data dispersion is a crucial consideration in database architecture. In other words, how is data spread throughout a network's nodes and between the client and server?

A relational database system uses a structured query language to make scattered data easily accessible. The fact that SQL is "nonnavigational" is advantageous in a c/s architecture. The kind of data in an RDBMS is stated using SQL; navigational information is not needed. This implies, of course, that the RDBMS needs to be intelligent enough to keep track of every piece of data's position and be able to determine the optimal route to it. A request for data in a less advanced database system has to specify what needs to be accessed and where. The maintenance of navigational data by application software makes data management for C/S systems significantly more complex.

It should be mentioned that the designer has access to additional data dissemination and administration methods.

- Manual extract. The relevant data can be manually copied by the user from a server to a client. When a user need static data and can maintain control over the extract, this method can be helpful.
- Snapshot. By defining a "snapshot" of the data to be delivered from a server to a client at predetermined intervals, this technique automates the manual extract. When providing comparatively static data that needs to be updated only occasionally, this method can be helpful.
- Replication. When it's necessary to keep numerous copies of the data at various locations (such as different servers or clients and servers), this method can be applied. Because data consistency, updates, security and processing need to be coordinated at several places, the level of complexity increases in this situation.

- Fragmentation. The system database is divided up among several machines in this method. While theoretically fascinating, fragmentation is very hard to apply and is not commonly encountered.

Notes

An Overview of a Design Approach

Porter offers a series of instructions for creating a basic business process that blends aspects of object-oriented and conventional design. It is presumed that before beginning the design of basic business processes, a requirements model that specifies business objects has been created and improved. The design is then derived using the subsequent steps:

1. Determine which files are produced, changed, referenced, or destroyed for each basic business operation.
2. To define components or objects, start with the files found in step 1.
3. Get the business rules and other information about the business objects that have been established for the relevant file for each component.
4. Sort the rules according to relevance to the process and break them down to the level of a method.
5. Define any further components required to put the methods into practice as needed.

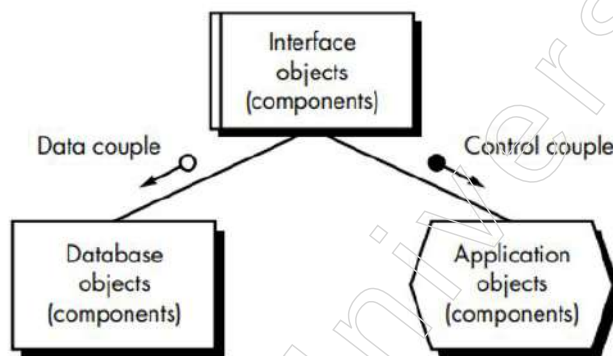


Figure: Structure chart notation for c/s components

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Porter proposes a notation for structure charts to depict the component structure of a basic business process. To ensure that the chart adheres to the object-oriented nature of C/S program, an alternative symbology is employed. There are five distinct symbols that may be seen in the figure:

Interface object. Known also as the user interaction/presentation component, this kind of component is usually constructed over one file and associated files that have been connected via a query. The methods for formatting the GUI interface and client-resident application logic are included. Additionally, it has embedded SQL, which describes database operations carried out on the main file that the interface is based around. In case the application logic that is usually linked with an interface object is executed on a server, usually by utilising middleware technologies, the server-side application logic needs to be recognised as a distinct application object.

Database object. This kind of component is used to indicate database operations, including selecting a file different than the main file upon which an interface object is formed, or creating records. Note that you may need to use a second SQL statement to get a file in a different order if the primary file that an interface object is created over is treated differently. For instance, the structure chart should designate the second file processing method as a distinct database item.

Notes

Application object. This component is called by a remote procedure call or a database trigger and it can be used by an interface object or a database object. It can also be utilised to locate business logic that was sent to the server for execution and is typically connected to interface processing.

Data couple. A message is delivered between two independent objects when one of them summons the other. This occurrence is indicated by the data couple symbol.

Control couple. A control couple symbol is used when one object calls another independent object and no data is sent between the two items.

Process Design Iteration

Database, application and interface objects are all represented by the same design repository that is used to represent business objects. The entities listed below are recognised:

- Procedures specify how a business regulation should be put into practice.
- The analytical model identifies elementary business processes, which are defined by elementary processes.
- The components of a basic business process can be identified using the process/component connection.
- The elements depicted on the structural chart are described by components.
- The components that are important for carrying out a specific business rule are identified by the business rule/component connection.

The designer will have access to a helpful design tool that offers reporting to help with both the development and ongoing management of a c/s system if a repository is established utilising an RDBMS.

Testing Issues

Software testers have particular challenges because of the dispersed nature of client/server systems. Binder recommends concentrating on the following areas:

- Client GUI considerations.
- Target environment and platform diversity considerations.
- Distributed database considerations (including replicated data).
- Distributed processing considerations (including replicated processes).
- Nonrobust target environment.
- Nonlinear performance relationships

It is necessary to create the c/s testing strategy and tactics in a way that makes it possible to address each of these problems.

Overall c/s Testing Strategy

Generally speaking, there are three stages to testing client/server software: (1) testing individual client apps in a “disconnected” mode, without taking into account server or underlying network operation; (2) testing client software and related server apps together, without explicitly testing network operations; and (3) testing the entire c/s architecture, including network performance and operation.

At each of these levels of detail, a wide variety of tests are carried out, although the following testing methodologies are frequently used for c/s applications:

Application function tests. To find operational flaws, the application is tested separately from other applications.

Server tests. The server's coordination and data management features are put to the test. Data throughput and total reaction time of the server are also taken into account.

Database tests. The server's data storage is checked for accuracy and integrity. To make sure that data is correctly saved, updated and retrieved, transactions posted by client applications are reviewed. Testing is also done on archiving.

Transaction tests. To make sure that every class of transaction is handled in accordance with the specifications, a number of tests are developed. Tests concentrate on processing accuracy as well as performance concerns (such as transaction volume and processing durations).

Network communication tests. These tests confirm that message passing, transactions and other relevant network traffic all happen without mistake and that communication between the network's nodes is carried out as intended. These testing may include network security tests as well.

Musa suggests creating operational profiles based on client/server usage situations in order to carry out these testing methods. An operational profile shows how various user types communicate with the c/s system. In other words, the profiles offer a "pattern of usage" that can be used in the planning and execution of tests. For instance, what proportion of transactions will be inquiries for a specific user type? orders? updates?

In order to create the operational profile, a series of user scenarios resembling the use-cases covered previously in this book must be derived. What, where, who and why are all covered in each scenario. That is, the identity of the user, the nature of the transaction, the location of the system interaction within the actual C/S architecture and the reason behind it. Requirements elicitation techniques or informal conversations with end users might be used to develop scenarios. But the outcome ought to be the same. For each scenario, it should be indicated which system functions are necessary to support a certain user, in what order they are needed, what kind of response and timing are anticipated and how frequently each function is used. After that, the operational profile is created by combining this data (for all users).

Small-scale testing is where testing starts. In other words, just one client application is examined. Testing is done gradually to ensure client, server and network integration. Lastly, a test of the system as a whole operating unit is conducted. Module/subsystem/system integration and testing are viewed as top down, bottom up, or a combination of the two in traditional testing. While there may be some top-down or bottom-up components to module integration in C/S development, parallel development and integration of modules at all design levels are more common in C/S projects. Thus, in certain cases, a non-incremental or "big bang" strategy is the most effective way to carry out integration testing in C/S projects.

System testing is impacted by the fact that the system is not being developed to employ prespecified hardware and software. We need to focus much more on compatibility and configuration testing because c/s systems are networked cross-platform systems. The system must be tested in every known hardware and software environment in which it will be used, according to configuration testing doctrine. A functionally consistent interface across hardware and software systems is ensured via compatibility testing. Depending on the implementation environment, a Windows-style interface, for instance, may seem different, but regardless of the client interface standard, the same fundamental user behaviours should provide the same outcomes.

Notes

A validation scenario in testing refers to a specific test case or scenario designed to validate that a software system or component meets its intended requirements and functions correctly within its operational context. Validation scenarios are typically derived from the system's functional and non-functional requirements and are used to verify that the system behaves as expected from the end user's perspective. Here's how a validation scenario is typically structured:

Components of a Validation Scenario:

- **Scenario Description:** A brief description or title that identifies the specific functionality or feature being tested.
- **Preconditions:** Any necessary conditions or prerequisites that must be met before the scenario can be executed. This may include setup steps, configuration settings, or data preloading.
- **Inputs:** The input data or stimuli required to trigger the functionality being tested. This may include user inputs, API requests, database queries, or system events.
- **Expected Behavior:** The expected outcome or result of executing the scenario. This defines the criteria for determining whether the system behaves correctly. Expected behavior may include system responses, output values, state changes, or error conditions.
- **Validation Criteria:** Specific criteria or conditions used to validate the correctness of the system's behavior. This may include comparing actual outcomes against expected outcomes, checking data consistency, or verifying compliance with regulatory requirements.
- **Test Steps:** The sequence of steps or actions to be performed to execute the scenario. This includes interacting with the system, executing specific functions or operations, and observing system responses.
- **Assertions:** Explicit statements or conditions that must be true during the execution of the scenario. Assertions are used to validate the system's behavior and detect deviations from expected outcomes.
- **Postconditions:** Any conditions or states that should exist after the scenario has been executed. This may include cleanup steps, data reset, or system state restoration.

C/S Testing Tactics

Because the duplicated data and processes can be grouped into classes of objects that share the same set of features, object-oriented testing approaches make sense even if the c/s system has not been designed using object technology. Test cases for a class of objects (or its equivalent in a traditionally created system) should be universally applicable to all instances of the class after they have been derived.

When taking into account the graphical user interface of contemporary C/S systems, the OO point of view is especially helpful. Because the GUI must function across multiple platforms, it deviates from traditional interfaces and is intrinsically object-oriented. Additionally, because the GUI generates, edits and works with a wide variety of graphical items, testing needs to investigate a lot of different logic paths. The objects might appear wherever on the desktop, be present or absent and exist for a prolonged period of time, which further complicates testing.

This means that in order to accommodate the complexity of the GUI environment, the typical capture/playback approach for testing character-based interfaces must be adjusted. Structured capture/playback is a functional variant of the capture/playback paradigm that was developed for GUI testing.

Conventional capture/playback saves and compares inputs and output images of subsequent tests to the keystrokes recorded during input and the screen images produced during output. The foundation of structured capture and replay is an internal, logical perspective on exterior activity. Interactions between the application program and the GUI are captured as internal events, which can be stored as “scripts” in the vendor’s proprietary language, one of the C variations, or Microsoft Visual Basic.

4.1.3 Peer to Peer Architecture

Peer-to-peer, or p2p, systems are decentralised systems in which any node on the network can do computations and where clients and servers are not distinguished—at least not in theory. The system as a whole is built for peer-to-peer applications to use the processing and storage capacity of a potentially massive computer network. Every node needs to run a copy of the program, which includes the standards and protocols needed to facilitate communication between the nodes.

Peer-to-peer technology have primarily been utilised for personal systems as of this writing. For instance, users can share files on their PCs using file-sharing programs based on the Gnutella and Kazaa protocols and users can communicate directly with one another without the need of an intermediary server using instant messaging programs like ICQ and Jabber. In order to look for signs of extraterrestrial life, the long-running SETI@home project processes data from radio telescopes on home computers. Freenet, on the other hand, is a decentralised database that was created to make it simpler to publish information anonymously and to make it more difficult for authorities to suppress this information.

Nonetheless, there are signs that corporations are using this technology more frequently to maximise the potential of their PC networks. P2P systems have been implemented for computationally demanding applications by both Boeing and Intel. This appears to be the most useful technology for cooperative applications that facilitate distributed working.

There are two ways to examine the architecture of peer-to-peer applications. Whereas the application architecture is the general arrangement of components in each application type, the logical network architecture is the system's distribution architecture. We concentrate on the two main categories of logical network designs that might be applied in this topic: semi-centralised and decentralised systems.

Peer-to-peer systems allow each node in the network to be aware of every other node, establish a connection with it and exchange data with it. Since this is obviously not feasible in practice, nodes are arranged into “localities,” with certain nodes serving as links to other node localities. This decentralised peer-to-peer architecture is shown in the figure below.

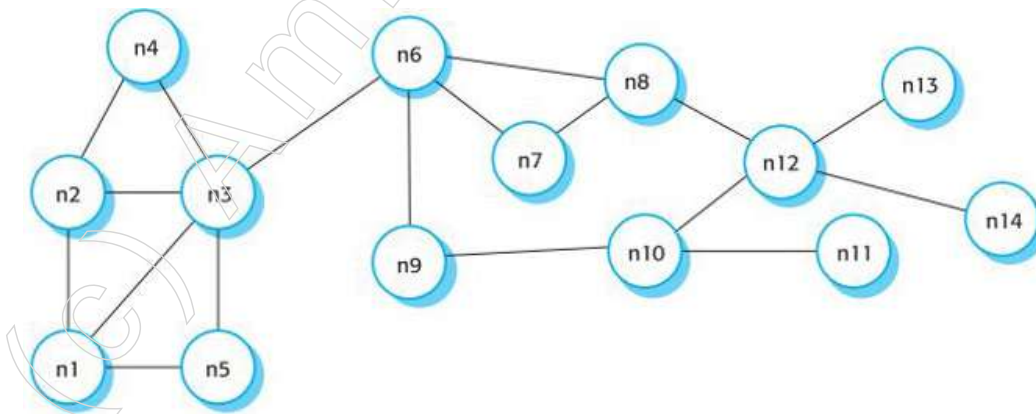


Figure: Decentralised p2p architecture

Notes

The nodes in a decentralised architecture serve as communications switches that can transfer control signals and data between nodes in addition to being functional components of the network. Let's take an example where the decentralised document management system shown in the above figure is used. A group of researchers uses this system to exchange documents and each member of the group keeps track of documents on their own system. On the other hand, upon retrieving a document, the node doing so also makes it accessible to other nodes. A search command is delivered to nodes in that "locality" by someone who needs a document. After verifying if they have the document, these nodes send it back to the requester. When a document is eventually found, the node can route it back to the original requestor. If they don't have it, they forward the search to other nodes. Consequently, if node n1 searches for a document stored at node n10, the search is directed to node n10 via nodes n3, n6 and n9.

The decentralised architecture has several benefits, including high redundancy, fault tolerance and network node disconnect tolerance. Nonetheless, the system clearly has overheads because multiple nodes may conduct the same search and duplicated peer communications have a large overhead. A semi-centralised architecture is an alternate peer-to-peer architectural model that deviates from a pure peer-to-peer architecture. In this architecture, one or more nodes function as servers within the network to enable node connections. Figure below shows this model.

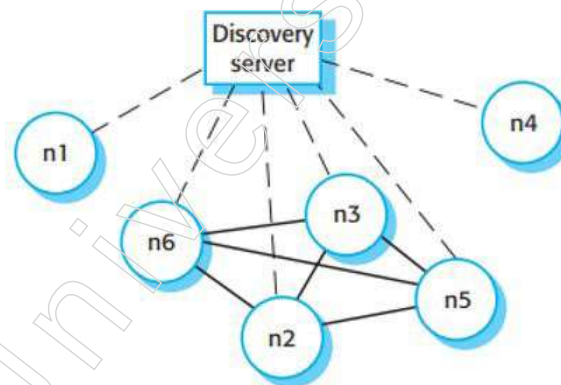


Figure: A semicentralised p2p architecture

It is common for some nodes to stand out in a computational peer-to-peer system where a processor-intensive computation is split among numerous nodes. These nodes are responsible for assigning work to other nodes and compiling and verifying the computation's output.

Peer-to-peer systems have clear overheads, but overall, they are a far more effective method of inter-organisational computing than service-based methods. Due to unresolved difficulties with security and trust, employing peer-to-peer (P2P) techniques for inter-organisational computing still presents challenges. Accordingly, peer-to-peer (P2P) systems are more likely to be employed in situations where there are established working connections between companies or for non-critical information systems.

4.1.4 Service Oriented Software Engineering

Organisational information exchange was revolutionised in the 1990s with the introduction of the Web. Information on distant servers maintained by organisations other than their own could be accessed by client PCs. However, direct access to the data by other programs proved impractical and access was limited to a web browser. This meant that it was unable to create opportunistic connections between servers, where a program may query many catalogues from various providers, for example.

Web services were created as a workaround to this issue, enabling programs to access and update online resources. Organisations can define and publish a programmatic web service interface to allow their information to be accessed by other programs via a web service. The data that is available and its methods of access and use are specified by this interface.

In a broader sense, a web service is a standard representation of a computational or information resource that other applications can utilise. These could be computing resources like a specialised CPU, storage resources, or information resources like a parts catalogue. An archive service, for instance, might be put into place to securely and permanently retain organisational data that must be kept up to date for many years as required by law.

One example of a broader concept of a service is a web service, which Lovelock et al. Defined as: A performance or act that is provided to another person by another. Ownership of any production variables is typically not the outcome of the performance, which is essentially ethereal even though the process may be connected to a tangible product.

Software components naturally evolve into services and the component model is really a set of web service standards. Thus, a loosely linked, reusable software component that encapsulates discrete functionality and can be accessed programmatically and distributed can be classified as a web service. A web service is one that can be accessible through XML-based protocols and the regular Internet.

According to the definition of component-based software engineering, a crucial difference between a service and a software component is that services have to be independent and loosely connected. That is, regardless of the execution environment, they should always function in the same manner. External components that might exhibit varying functional and non-functional behaviours should not be relied upon. The “requires” interface, which in CBSE specifies the other system components that must exist, is therefore absent from web services. All that a web service interface is, is a “provides” interface that describes the parameters and capabilities of the service.

A method for creating distributed systems that operate on geographically dispersed computers and include stand-alone services as system components is called service-oriented systems. Services are agnostic of implementation language and platform. Local services and external services from various suppliers can be combined to create software systems that seamlessly communicate with one another.

“Software as a service” and “service-oriented systems” are two distinct concepts. Offering software capability to customers remotely via the internet as opposed to using applications installed on their computers is known as software as a service. Systems that are implemented with reusable service components and that are accessed by other programs as opposed to users directly are known as service-oriented systems. A system that is service-oriented can be used to implement software that is provided as a service. But, in order to provide software as a user service, it is not necessary to implement it in this manner.

There are several significant advantages of approaching software engineering from a service-oriented perspective:

1. Any service provider, whether within or external to an organisation, can offer services. By integrating services from several suppliers, organisations can develop applications, provided that these services meet specific requirements. A manufacturing company, for instance, can establish a direct connection with the services offered by its suppliers.
2. To enable any authorised user to use the service, the service provider publishes information about the service. Before a service may be included in an application

Notes

- program, there is no requirement for the service user and the service provider to haggle about what the service does.
- Applications have the option to postpone service binding until after they are executed or deployed. Thus, in theory, an application that made use of a stock price service, for example, could switch service providers on the fly while the system ran. This implies that programs are able to respond quickly to modifications in their execution environment and modify how they operate accordingly.
 - It is feasible to build new services opportunistically. A service provider might be able to identify new services that can be produced by creatively connecting preexisting services.
 - Customers might choose to pay for services based on how they use them rather than how they are provided. Therefore, the application writer can use an external service that will only be paid for when needed, rather than purchasing a costly component that is rarely used.
 - It is possible to reduce the size of applications, which is crucial for mobile devices with constrained memory and computing power. It is possible to delegate some exception handling and computationally demanding processing to other services.

Loosely linked architectures characterise service-oriented systems, where service bindings are subject to change while the system operates. As a result, multiple versions of the service that are identical might run at different times. While certain systems will only be constructed with online services, others will combine web services with locally created components. Examine the following example to show how applications that utilise a variety of services and components could be set up:

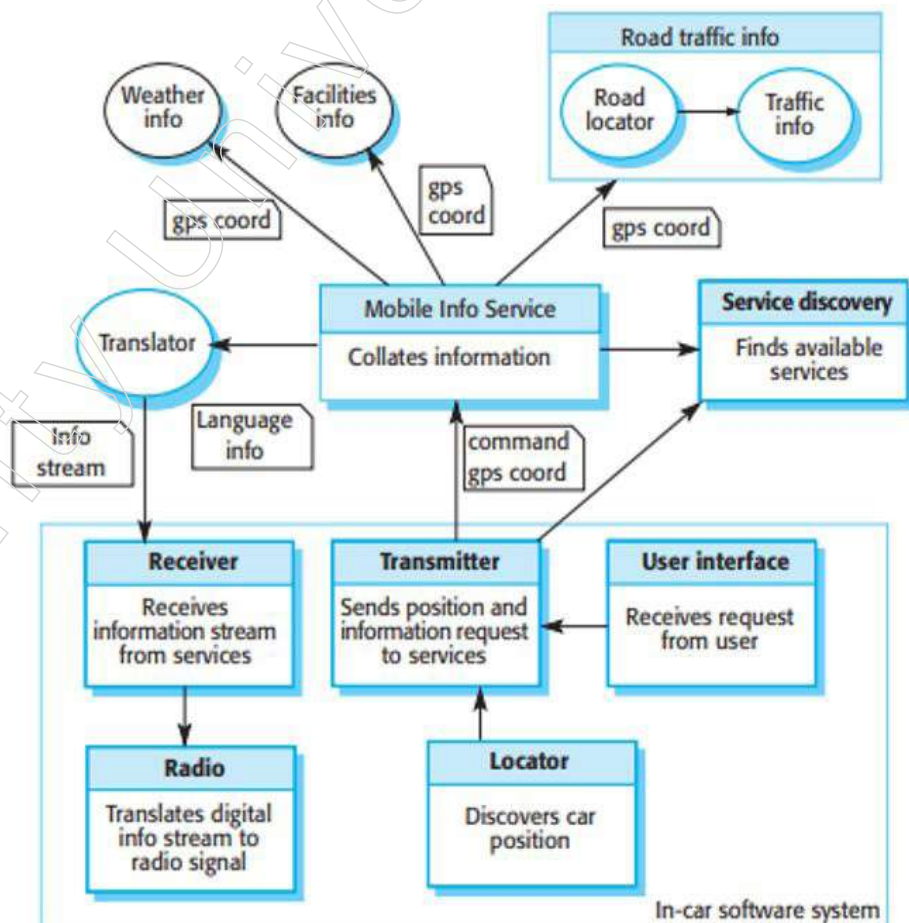


Figure: A service based, in-car information system

Drivers can get information on the weather, traffic, local events and other topics via an in-car information system. This is connected to the vehicle radio in order to transmit data as a signal on a designated radio channel. The vehicle has a GPS receiver installed to determine its location. The system then uses that position to access various information services. After then, information might be given in the language the driver has chosen.

An example of how such a system might be organised is shown in the figure above. Five components are included in the in-car software. These manage communications between the automobile radio, the GPS receiver that provides the vehicle's position and the driver. All contact with outside services is handled via the Transmitter and Receiver modules.

The vehicle is connected to an outside mobile information service that provides information on local facilities, traffic, weather and other topics by aggregating data from several different services. These services are provided by various providers in various locations and the in-car system uses an external discovery service to locate the services offered nearby. In order to connect to the proper weather, traffic and facility services, the mobile information service also makes use of the discovery service. After then, the combined data is transmitted to the vehicle via a service that converts it into the driver's selected language.

This example highlights one of the main benefits of the service-oriented methodology. You are not required to select the service provider or the particular services to be accessed at the time the system is designed or installed. The in-car software searches for the most helpful local information service while the vehicle is moving by utilising the service discovery service. It can travel across borders and provide local information to those who do not speak the language by using a translation service.

In my opinion, the evolution of the service-oriented approach to software engineering is just as significant as that of the object-oriented method. Cloud and mobile systems depend on service-oriented systems. In their SOA book, Newcomer and Lomow provide an overview of the promise of service-oriented approaches, which is currently being realised:

"The service-oriented enterprise promises to significantly improve corporate agility, speed time-to-market for new products and services, reduce IT costs and improve operational efficiency. It is driven by the convergence of key technologies and the universal adoption of Web services."

Developing apps based on services enables collaboration and utilisation of one another's business operations by businesses and other organisations. As a result, systems involving a lot of information sharing between businesses, like supply chain systems where one company gets goods from another, are easily automatable. Using a normal programming language or a specialised workflow language, services from different providers can be linked to create service-based applications.

The inability of the software industry to come to an agreement on component standards had a significant impact on the early work on service provision and implementation. As a result, it was standards-driven, with participation from all of the major industrial corporations in standards development. This gave rise to the idea of service-oriented architectures and a wide range of standards known as WS* standards. These were put up as architectures for standards-based service communication in service-based systems. The suggested standards, however, had a large execution cost and were complicated. Numerous businesses have adopted an alternate architectural strategy based on so-called RESTful services in response to this issue. Although a RESTful method is less complicated than a service-oriented design, it is not as appropriate for services that provide intricate features.

Notes

4.2 Web Engineering

The public has become more interested in computers thanks to the Internet and the World Wide Web. We use the Internet for almost everything: buying stock and mutual funds, downloading music, watching films, booking hotel rooms, selling personal belongings, booking flights, meeting people, banking, taking college courses, grocery shopping and more. The Web and the Internet that supports it are arguably the most significant advancements in computer history. The information era has been beckoned to us all by these computing tools and billions more will follow. In the early years of the twenty-first century, they have become indispensable to day-to-day existence.

A wide range of end users can access a sophisticated array of functionality and material using web-based systems and applications, or WebApps. Web engineering is the process that goes into making good Web applications. Although it is not a perfect replica of software engineering, web engineering emphasises many of the same technical and managerial tasks and shares many of the essential ideas and principles of software engineering. Although these activities are carried out in slightly different ways, they are all guided by the same overarching concept, which calls for a methodical approach to the creation of computer-based systems.

The creation and use of solid scientific, technical and management principles as well as methodical, disciplined approaches to the successful development, implementation and upkeep of superior Web-based systems and applications are the focus of Web Engineering (WebE).

4.2.1 Service Engineering

The practice of creating services for reuse in applications that are service-oriented is known as service engineering. It is similar to component engineering in many ways. It is the responsibility of service engineers to guarantee that the service is a reusable abstraction that can be applied to many systems. Along with designing and creating generally helpful features related to that abstraction, they also have to make sure the service is dependable and stable. In order for prospective consumers to find and comprehend the service, they must document it.

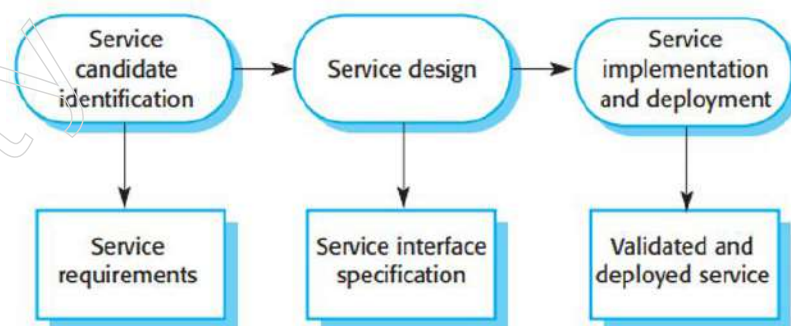


Figure: The service engineering process

Image Source: Software Engineering by roger s. pressman TENTH Edition

The service engineering process consists of three logical stages, as depicted in the above figure:

1. Service candidate identification, where you identify possible services that might be implemented and define the service requirements.
2. Service design, where you design the logical service interface and its implementation interfaces (SOAP-based and/or RESTful).

3. Service implementation and deployment, where you implement and test the service and make it available for use.

An existing component that has been put into practice and utilised in an application might serve as the foundation for the creation of a reusable component. The same is true for services; a component that needs to be turned into a service or an existing service will frequently serve as the beginning point for this procedure. In this case, the design approach entails generalising the current component to eliminate features unique to this application. Implementation entails modifying the component by incorporating the necessary generalisations and introducing service interfaces.

Service Candidate Identification

The fundamental tenet of service-oriented computing is that business operations ought to be supported by services. Since every organisation has a diverse variety of procedures, it is possible to deploy a wide range of services. Thus, identifying potential service candidates requires comprehending and evaluating the business procedures of the company in order to determine which reusable services could be used to assist these procedures:

According to Erl, there are three main categories of services:

1. Utility services. Certain general functionality that can be utilised by many business processes is implemented by these services. A currency conversion service, which may be used to calculate the conversion of one currency (such as dollars) to another (such as euros), is an example of a utility service.
2. Business services. These services are connected to a certain line of business. The process of enrolling students in classes is an illustration of a commercial function in a university.
3. Coordination or process services. These services facilitate a broader business process that typically includes a variety of participants and actions. An ordering service that enables orders to be placed with suppliers, goods to be received and payments to be made is an example of a coordination service in a business.

Additionally, Erl proposes that services may be seen as either entity- or task-oriented. While entity-oriented services are linked to a system resource, task-oriented services are connected to an activity. A business entity, such a job application form, serves as the resource. Business or utility services might be task- or entity-oriented. Services for coordination are invariably task-oriented.

Finding logically consistent, autonomous and reusable services should be your aim when searching for potential candidates. In this regard, Erl's classification is useful since it offers a way to identify reusable services by viewing business entities as resources and business activities. It can be challenging to discover potential service candidates, though, as you need to consider the potential uses for the services. To determine if a potential candidate is likely to be a beneficial service, you must first brainstorm potential prospects and then ask them a series of questions. To find services that might be reused, you could ask the following questions:

- Is a single logical resource utilised in several business processes connected to an entity-oriented service? Which actions on that entity are typically taken that need to be supported? Are these compatible with PUT, CREATE, POST and DELETE RESTful service operations?
- Is the task for a task-oriented service one that is completed by several employees inside the company? When a single support service is offered, standardisation

Notes

inevitably results. Will they be able to accept this? Does this need to be rewritten as an entity-oriented service, or can it still be included in the RESTful model?

- Does the service operate independently? That is, how much does it depend on other services being available?
- Is state maintenance required for the service? If state information is needed, it needs to be either supplied as a parameter to the service or kept up to date in a database. Because the service and the necessary database are dependent upon one another, using a database reduces the reusability of services. Because there is no need for database binding, services that get the state as input are typically easier to reuse.
- Could there be outside clients using this service? For instance, both internal and external users may have access to an entity-oriented service linked to a catalogue.
- Are there likely to be differences in the non-functional requirements of different service users? If they do, it might be wise to deploy multiple versions of a service.

You can choose and improve the abstractions that can be used as services with the aid of the answers to these questions. Still, there's no set formula for selecting the best services. To determine which services are best for you, you must apply your business acumen and experience.

A list of recognised services together with the requirements that go along with them is the result of the service selection process. What the service is expected to do should be specified in the functional service requirements. The security, performance and availability requirements for the service should be specified in the non-functional requirements.

Take into consideration the following example to better understand the process of identifying and implementing service candidates:

A computer equipment sales organisation has set aside special rates for approved configurations for a few major clients. The company wants to create a catalogue service that lets clients choose the equipment they require, making automated ordering easier. Orders are not placed using a catalogue interface directly, in contrast to a consumer catalogue. Rather, products are ordered using each company's web-based procurement system, which uses the catalogue as a web service. This is because, when an order is placed, big businesses typically have their own budgeting and order approval processes that need to be followed.

One instance of an entity-oriented service is the catalogue service, which uses the catalogue as its underlying resource. The following are the prerequisites for a functional catalogue service:

1. For every user company, a customised version of the catalogue will be provided. This will comprise the equipment pricing that have been agreed upon with the customer company as well as the authorised configurations and equipment that employees of that company may order.
2. A customer staff must be able to download a copy of the catalogue for offline viewing.
3. Users of the catalogue will be able to compare the features and costs of up to six catalogue items.
4. Users will have the ability to browse and search the catalogue.
5. Users of the catalogue will be able to find the estimated date of delivery for a specified quantity of catalogue products.
6. Through "virtual orders," which they can place using the catalogue, users will be able to reserve the necessary items for a period of 48 hours. A genuine order submitted by

a procurement system needs to validate virtual orders. After the virtual order, the actual order needs to arrive within 48 hours.

The catalogue also has a number of nonfunctional requirements in addition to these functional needs:

1. Only personnel from approved organisations will be able to use the catalogue service.
2. Only staff members of each customer will have access to the prices and configurations that are given and they will remain private.
3. The catalogue will be accessible without any interruptions between 0700 GMT and 1100 GMT.
4. Peak load for the catalogue service should not exceed 100 queries per second.

There isn't any non-functional requirement concerning the catalogue service's response time. This is contingent upon the magnitude of the catalogue and the anticipated count of concurrent users. There is currently no need to indicate the needed performance because this is not a time-critical service.

Service Interface Design

Designing the service interfaces is the next step in the service engineering process once you have selected potential services. This entails specifying the parameters and operations connected to the service. You are responsible for designing the input and output messages if SOAP-based services are used. When using RESTful services, you need to consider the resources needed and how the service operations should be implemented using the standard procedures.

Abstract interface design serves as the foundation for service interface design, as it allows you to define the entities, operations, inputs, outputs and exceptions related to the various operations that make up the service. Next, you should consider implementing this abstract interface as RESTful or SOAP-based services.

Should you opt for a SOAP-based method, you will need to create the XML message structures that the service sends and receives. A WSDL interface definition is built around the operations and messages. You must plan how the service operations map onto the RESTful activities if you decide to use a RESTful approach.

In order to specify the names and parameters of the operations, the abstract interface design begins with the service requirements. You should now specify any potential exceptions that could occur from invoking a service operation. The catalogue activities that carry out the requirements are depicted in the figure below. These don't need to be detailed in great depth; that comes later in the design process. The next step is to provide more information about the inputs and outputs of the service once you have created an informal description of what it should perform.

Operation	Description
MakeCatalog	Creates a version of the catalog tailored for a specific customer. Includes an optional parameter to create a downloadable PDF version of the catalog.
Lookup	Displays all of the data associated with a specified catalog item.
Search	Takes a logical expression and searches the catalog according to that expression. It displays a list of all items that match the search expression.
Compare	Provides a comparison of up to six characteristics (e.g., price, dimensions, processor speed, etc.) of up to four catalog items.

Notes

CheckDelivery	Returns the predicted delivery date for an item if ordered that day.
MakeVirtualOrder	Reserves the number of items to be ordered by a customer and provides item information for the customer's own procurement system.

Figure: Catalog operations

Image Source: Software Engineering by roger s. pressman TENTH Edition

It is especially crucial to define exceptions and provide service users with information about them. The way that their services are used is unknown to service engineers. Assuming that service customers have a thorough understanding of the service specification is typically a bad idea. You should create exceptions that alert the service client to improper inputs since input messages can be mistyped. In the building of reusable components, it is often recommended to delegate all exception handling to the component's user. It is not the place of service developers to dictate how exceptions should be handled.

Sometimes all that's needed is a written explanation of the operations, together with their inputs and outputs. The decision to implement the service in detail is left up to the user. However, there are situations when you need a more thorough design. In these cases, a thorough interface description can be provided using readable description formats like JSON or graphical notations like the UML.

An example of a useful service that shows that it's not always easy to decide between a SOAP-based and RESTful approach to service implementation is the catalogue service. The catalogue can be viewed as a resource because it is an entity-based service, indicating that a RESTful approach would be more appropriate. However, you must keep some state during an interaction session with the catalogue because operations on it are not straightforward GET operations. This implies applying a SOAP-based methodology. These kinds of conundrums are typical in the field of service engineering and the choice of which strategy to take is typically influenced by local conditions (such as the availability of experts). You must choose which resources will be used to represent the catalogue and how the basic GET, POST and PUT operations will be performed on these resources in order to construct a set of RESTful services. A few of these design choices are simple:

1. A resource that resembles a catalogue exclusive to a corporation ought to exist. This needs to be made using a POST operation and have the form's URL, /.
2. A unique URL of the format <base catalog>/<companyname>/<item identifier> should be assigned to each catalogue item.
3. To obtain items, utilise the GET procedure. The URL of an item in a catalogue is used as the GET parameter when implementing lookup. Utilising GET and the firm catalogue as the URL and the search string as a query parameter, search is implemented. A list of URLs for the items that match the search is returned by this GET operation.

The operations for Compare, CheckDelivery and MakeVirtualOrder are more intricate, though:

1. A series of GET actions can be used to collect the individual items for the Compare operation, then a POST activity can be used to generate the comparison table and then a final GET operation can be used to return the results to the user.
2. A virtual order is represented by an additional resource needed for the CheckDelivery and MakeVirtualOrder procedures. This resource is created with the necessary number of objects using a POST operation. The delivery date is computed and the order form is automatically filled in using the company ID. A GET operation can then be used to obtain the resource.

It's important to consider carefully how exceptions are translated into the common http response codes, such as 404, which indicates that a URL cannot be obtained. We won't get into this problem here, but it makes the service interface design even more complicated.

The realisation procedure is less complicated in this instance for SOAP-based services since the logical interface design may be immediately converted into WSDL. The majority of programming environments (like the ECLIPSE environment) that facilitate service-oriented development are equipped with tools that can convert a logical interface definition into the matching WSDL representation.

4.2.2 Software Development with Services

The foundation of service-based software development is the notion that services can be combined and configured to build new, composite services. These might be used as components in another service composition, or they could be combined with a browser-based user interface to form a web application. The services included in the composition could come from an outside source, come from a company that built business services specifically for the application, or both.

These days, a lot of businesses are transforming their enterprise apps into service-oriented systems, in which a service rather than a component serves as the fundamental application building block. This creates the opportunity for increased internal corporate reuse. The creation of inter organisational applications amongst reliable suppliers who will utilise one another's services will be the following phase. In order for SOAs to achieve their long-term goals, a "services market" where services are purchased from outside vendors must emerge.

Separate business processes can be combined using service composition to create an integrated process with more capability. Let's say an airline wants to provide passengers a whole trip package. In addition to scheduling their flights, travelers can reserve hotels in the locations of their choice, hire a car or hail a taxi from the airport, peruse a travel guide and make plans to see nearby sites. The airline combines its own booking service with those provided by taxi and auto rental businesses, hotel booking agencies and local attraction owners to produce this application. A single service that combines the offerings of several providers is the end product.

This procedure can be understood as a series of distinct steps, as illustrated in the figure below. From one stage to the next, information is transferred; for instance, the time the aeroplane is expected to arrive is communicated to the vehicle rental business. A workflow is a series of actions arranged in a timely manner, each of which completes a portion of the task at hand. A workflow is a representation of a business process; that is, it outlines the processes necessary to accomplish a specific objective that is significant to the organisation. The airline's vacation booking service is the business process in this instance.

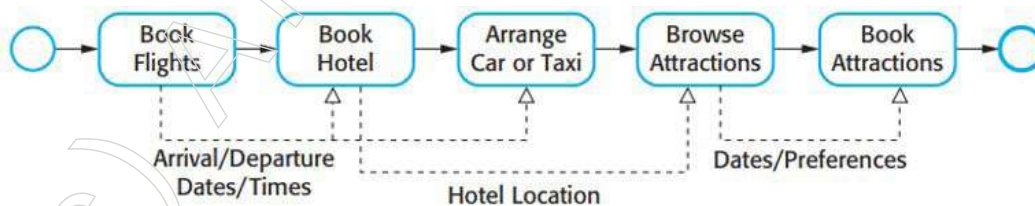


Figure: Vacation package workflow

Image Source: Software Engineering by roger s. pressman 9th Edition

The concept of workflow is basic and scheduling a vacation in the example above appears like an easy task. Service composition is far more complicated in reality than this

Notes

straightforward model suggests. For instance, you need to build in procedures to deal with service outages and account for the potential for them. You also need to consider the extraordinary requests that users of the program may make. Let's take an example where a passenger needed a wheelchair to be rented and transported to the airport due to their disability. This would necessitate adding more steps to the workflow and implementing and creating new services.

Because the regular operation of one service typically results in an incompatibility with the normal operation of another service, you will need to be able to handle scenarios when the workflow needs to be modified. Let's take an example where a flight is scheduled to depart on June 1 and return on June 7. The process then moves on to the stage of booking hotels. Nevertheless, there are no hotel rooms available because the resort is hosting a significant convention through June 2. This lack of availability is reported by the hotel booking agency. This is not a failure; being unavailable is a frequent occurrence. As a result, you must "undo" the flight reservation and advise the user that the flight is not available. After then, he or she must choose whether to adjust the resort or the dates. Workflow jargon refers to this as a "compensation action." Actions that have previously been executed but need to be modified due to subsequent workflow activities are undone using compensation actions.

In essence, software design with reuse is the process of creating new services through the reuse of preexisting services. Reusing design always necessitates compromising needs. The "ideal" system requirements must be adjusted to take into account the services that are really offered, whose prices are reasonable and whose level of service is acceptable.

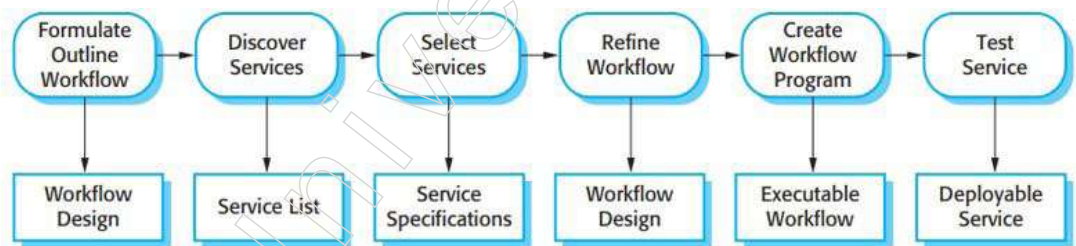


Figure: Service construction by composition

Image Source: Software Engineering by roger s. pressman 9th Edition

We've highlighted six crucial steps in the process of building services via composition in the above figure:

1. Create a workflow outline. During the initial phase of service design, you create a "ideal" service design based on the criteria for the composite service. At this point, you should design something pretty abstract, intending to add features later on when you have more information about the services that are available.
2. Find out about services In order to find out what services are available, who offers them and how they are provided, you search service registries or catalogues at this step of the process.
3. Choose potential services. You next choose potential services that can carry out process tasks from the list of potential service candidates that you have found. Of course, one of your choosing criteria will be how well the services work. They might also cover the price of the services and their level of quality (availability, responsiveness, etc.). Depending on the specifics of pricing and service quality, you may select a variety of functionally identical services, some of which may be tied to a workflow activity.
4. Streamline the process You then fine-tune the procedure based on details about the services you have chosen. This entails expanding on the abstract description and

possibly changing the order of workflow tasks. The steps of service discovery and selection can then be repeated. You proceed to the next step of the process once a stable set of services has been selected and the final workflow architecture has been defined.

5. Make a workflow application. In this phase, the service interface is defined and the abstract workflow design is converted into an executable program. For service implementation, you can use a workflow language like WS-BPEL or a traditional programming language like Java or C#; the service interface specification needs to be written in WSDL. The development of web-based user interfaces to enable browser access to the new service may also be part of this phase.
6. Check the finished product or program. When external services are involved, testing the full composite service is a more complicated procedure than testing individual components.

We address the design and testing of workflows in the remaining sections of this topic. Service discovery doesn't seem to be a big deal in real life. The majority of service reuse still occurs within companies, where internal registries and casual conversations amongst software engineers can be used to find services. To find services that are open to the public, use standard search engines.

Workflow Design and Implementation

Analysis of current or proposed business processes is necessary for workflow design in order to comprehend the various operations and the information exchanged between them.

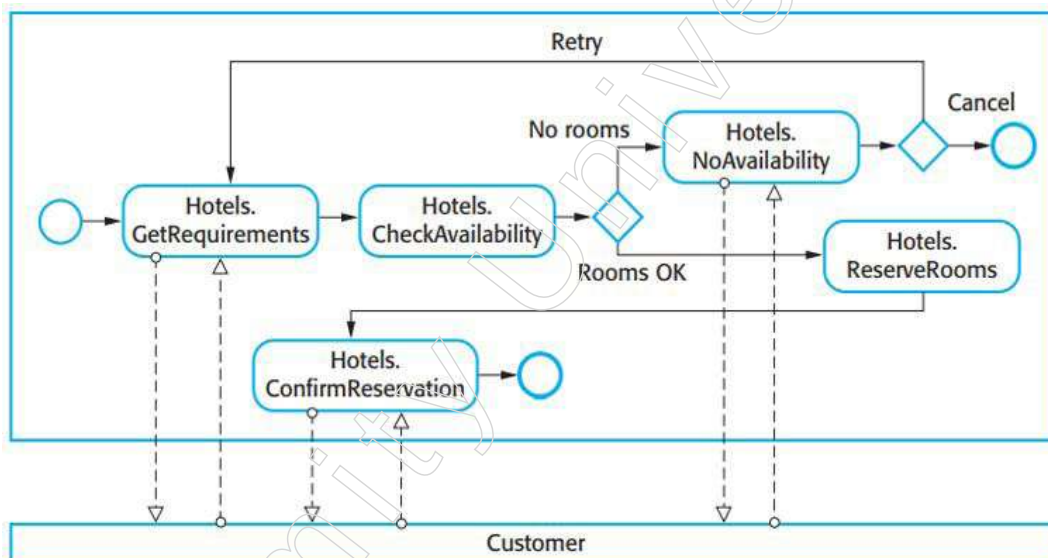


Figure: A fragment of a hotel booking workflow

Image Source: Software Engineering by roger s. pressman 9th Edition

Next, you create a workflow design notation where the new business process is defined. This lays down the steps needed to carry out the process as well as the data that is transferred between the various steps. There could not be a "normal" style of working or a defined process; instead, the processes that are in place could be informal and reliant on the abilities and skills of the individuals involved. In these situations, you must create a workflow that meets the same objectives by applying your understanding of the existing process.

Workflows, or business process modelling notation, are graphical representations of business process models that are typically made with UML activity diagrams or BPMN.

Notes

These provide comparable characteristics. It is likely that in the future, activity diagrams from UML and BPMN will be combined and this combined language will serve as the foundation for a standard for workflow modelling.

The pictorial language known as BPMN is rather simple to comprehend. To translate the language to XML-based, lower-level descriptions in WS-BPEL, mappings have been established. As a result, BPMN complies with the web service standards stack.

A basic BPMN model of a portion of the vacation package scenario mentioned above is shown in the above figure. The model assumes the presence of a Hotels service with related actions named GetRequirements, CheckAvailability, ReserveRooms, NoAvailability, ConfirmReservation and CancelReservation. It also depicts a simplified workflow for reserving a hotel. Obtaining the customer's requirements, determining whether rooms are available and then making a reservation for the necessary dates are the steps in the process.

Some of the fundamental BPMN ideas that are utilised to build process models are introduced in this model:

1. A rectangular shape with rounded corners is used to symbolise activities. An automated service or a human can carry out an activity.
2. Circles are used to symbolise events. Anything that occurs throughout a business process is called an event. A beginning event is represented by a simple circle, while an ending event is represented by a darker circle. A middle event is represented by a double circle (not visible). Events can be clock events, enabling the periodic execution or time-out of workflows.
3. A gateway is symbolised by a diamond. A gateway is a decision-making point in the process. For instance, a decision is made in the above Figure based on the availability of rooms.
4. The order of the activities is indicated by a solid arrow, while the message flow between the activities is shown by a dashed arrow. These communications are sent between the customer and the hotel booking service, as seen in the above image.

The majority of workflows may be summarised by these essential components. But there are a tonne of other features in BPMN. These provide details to a business process description so that it can be converted into an executable service automatically. As a result, web services can be produced straight from a business process model using the service compositions outlined in BPMN.

The procedure used by one organisation—a booking service provider—is depicted in the above figure. But a service-oriented approach's main advantage is that it facilitates inter organisational computing. This indicates that services from various businesses are involved in a computation. Creating unique workflows with interactions for each of the participating organisations is how BPMN represents this.

We'll take a different example from high-performance computing to demonstrate this. It has been suggested that a service-oriented approach be used to enable the sharing of resources like high-performance computers.

Assume for the purposes of this example that a research lab is providing a vector processing computer (a device that can do parallel computations on arrays of values) as a service (VectorProcService). Setup Computation is a different service that allows access to this. The figure below illustrates these services and how they interact.

Notes

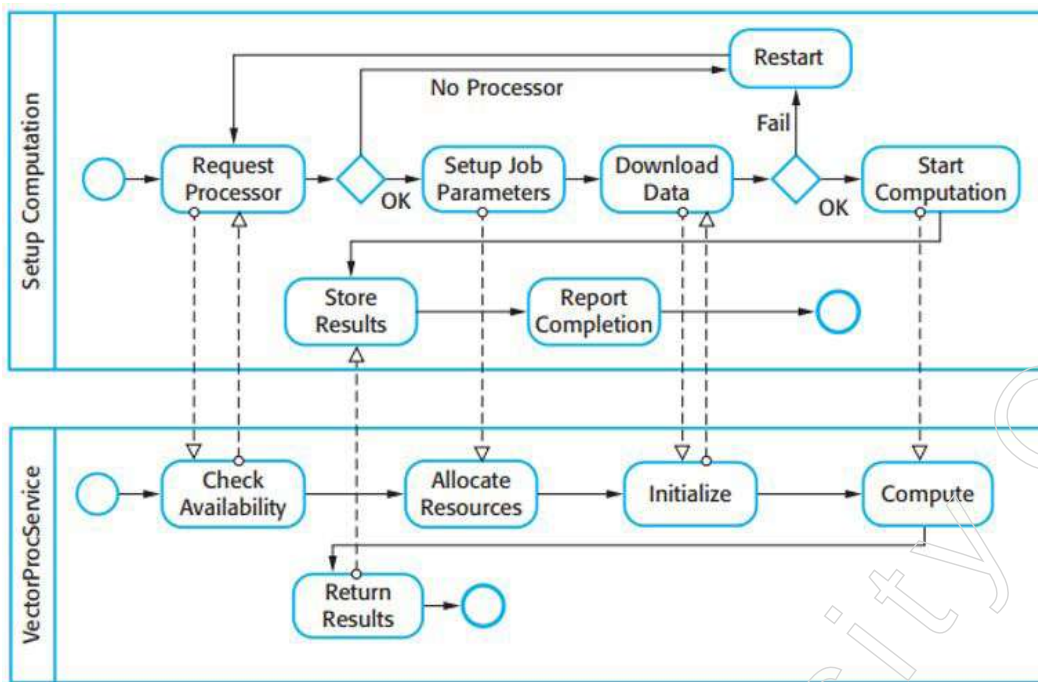


Figure: Interacting workflows

Image Source: Software Engineering by roger s. pressman 9th Edition

The workflow for the Setup Computation service in this example determines the necessary computation and downloads data to the processing service after requesting access to a vector processor, if one is available. The output is saved locally on the computer after the computation is finished. In the VectorProcService workflow, a processor's availability is checked, resources are allocated for the computation, the system is initialised, the computation is performed and the results are returned to the client service.

Each organisation's workflow is represented in a different pool in BPMN. The process is represented visually by enclosing each participant's workflow in a rectangle and writing their name vertically on the left border. Each pool's stated workflows are coordinated through message exchanges; sequence flow between activities inside different pools is prohibited. When many departments within an organisation are involved in a workflow, pools might be divided into designated "lanes" to illustrate this. Every lane displays the operations within that division of the company.

The final design needs to be turned into an executable program once it's accessible. This could entail the following two actions:

1. putting into practice the services that are not reusable. These services can be written in any language because they are implementation-language independent. The development environments for Java and C# both support the composition of web services.
2. creating a workflow model executable version. This usually entails translating the model, either manually or automatically, into WS-BPEL. While there are numerous tools for automating the BPMN-WS-BPEL process, there are situations in which converting a workflow model into legible WS-BPEL code is challenging.

A number of web service standards have been established to directly assist the implementation of web service compositions. One such standard is WS-BPEL (Business Process Execution Language), an XML-based "programming language" that controls interactions between services. Other standards that support this include WS-CDL

Notes

(Choreography Description Language), which defines the message exchanges between participants and WSCoordination, which specifies how services are coordinated.

Service Testing

In order to show that a system satisfies both functional and non-functional criteria and to find errors that may have been introduced during the development process, testing is crucial to all system development procedures. An examination of the software source code is a prerequisite for several testing methods, including coverage testing and program inspections. Nevertheless, the source code for the service implementation is unavailable when services are provided by an outside source. Thus, tried-and-true source code-based methodologies cannot be used to service-based system testing.

Testers may encounter other challenges when evaluating services and service compositions in addition to comprehension issues with the service's implementation:

1. Instead of the service user, the service provider is in charge of external services. Any prior application testing is nullified if the service provider decides to modify or withdraw these services at any time. Different versions of software components are maintained to address these issues. To cope with service versions, however, no standards have yet been established.
2. The long-term goal of service-oriented architectures (SOAs) is to dynamically bind services to service-oriented applications. This implies that an application might not always run on the same service every time. As a result, while an application linked to a certain service may pass tests, there is no guarantee that the same service will be used when the system is actually executed.
3. A service's non-functional behaviour depends on more factors than just how the application being tested uses it. When undergoing testing, a service might function properly since it isn't experiencing a high load. Because of the requests made by other service users, the observed service behaviour may change in practice.
4. Service testing could be quite costly due to the services' payment model. There are various alternative payment methods: certain services can be offered for free, while others might require a subscription or be paid for on an as-needed basis. If a service is free, the provider won't want testing programs to load it; if a subscription is needed, a service user could be hesitant to sign up for a subscription before trying the service. Similarly, service customers can perceive the expense of testing to be prohibitive if usage is contingent on payment for each use.
5. We've talked about the idea of compensation actions that are brought about when an exception happens and prior agreements (like a plane ticket) need to be cancelled. Testing such actions presents a challenge because they can be dependent on other services failing. It may be rather challenging to guarantee that these services genuinely malfunction within the testing phase.

When outside services are used, these issues become much more severe. When services are used within the same organisation or when cooperating enterprises have faith in the services provided by their partners, they are less serious. In these situations, paying for services is probably not going to be an issue and source code might be accessible to help with the testing process. It is still a major research concern to find solutions to these testing issues and develop standards, resources and methods for testing service-oriented applications.

4.2.3 Software Testing Issues

Software testers have particular challenges because of the dispersed nature of client/server systems. Binder recommends concentrating on the following areas:

- Client GUI considerations.
- Target environment and platform diversity considerations.
- Distributed database considerations (including replicated data).
- Distributed processing considerations (including replicated processes).
- Nonrobust target environment.
- Nonlinear performance relationships.

It is necessary to create the c/s testing strategy and tactics in a way that makes it possible to address each of these problems.

Overall c/s Testing Strategy

Generally speaking, there are three stages to testing client/server software: (1) testing individual client apps in a “disconnected” mode, without taking into account server or underlying network operation; (2) testing client software and related server apps together, without explicitly testing network operations; and (3) testing the entire c/s architecture, including network performance and operation.

At each of these levels of detail, a wide variety of tests are carried out, although the following testing methodologies are frequently used for c/s applications:

Application function tests. To put it simply, the application is tested independently in an effort to find any operational flaws.

Server tests. The server's coordination and data management features are put to the test. Data throughput and total reaction time of the server are also taken into account.

Database tests. The server's data storage is checked for accuracy and integrity. To make sure that data is correctly saved, updated and retrieved, transactions posted by client applications are reviewed. Testing is also done on archiving.

Transaction tests. To make sure that every class of transaction is handled in accordance with the specifications, a number of tests are developed. Tests concentrate on processing accuracy as well as performance concerns (such as transaction volume and processing durations).

Network communication tests. These tests confirm that message passing, transactions and other relevant network traffic all happen without mistake and that communication between the network's nodes is carried out as intended. These testing may include network security tests as well.

Musa suggests creating operational profiles based on client/server usage situations in order to carry out these testing methods. An operational profile shows how various user types communicate with the c/s system. In other words, the profiles offer a “pattern of usage” that can be used in the planning and execution of tests. For instance, what proportion of transactions will be inquiries for a specific user type? orders? updates?

It is vital to construct a collection of user scenarios that are comparable to the use-cases covered earlier in this book in order to develop the operational profile. What, where, who and why are all covered in each scenario. That is, the identity of the user, the nature of the transaction, the location of the system interaction within the actual C/S architecture and the reason behind it. Requirements elicitation techniques or informal conversations with end

Notes

Notes

users might be used to develop scenarios. But the outcome ought to be the same. For each scenario, it should be indicated which system functions are necessary to support a certain user, in what order they are needed, what kind of response and timing are anticipated and how frequently each function is used. After that, the operational profile is created by combining this data (for all users).

Small-scale testing is where testing starts. In other words, just one client application is examined. Testing is done gradually to ensure client, server and network integration. Lastly, a test of the system as a whole operating unit is conducted.

Module/subsystem/system integration and testing are viewed as top down, bottom up, or a combination of the two in traditional testing. While there may be some top-down or bottom-up components to module integration in C/S development, parallel development and integration of modules at all design levels are more common in C/S projects. Thus, in certain cases, a non incremental or “big bang” strategy is the most effective way to carry out integration testing in C/S projects.

System testing is impacted by the fact that the system is not being developed to employ prespecified hardware and software. We need to focus much more on compatibility and configuration testing because c/s systems are networked cross-platform systems.

The system must be tested in every known hardware and software environment in which it will be used, according to configuration testing doctrine. A functionally consistent interface across hardware and software systems is ensured via compatibility testing. Depending on the implementation environment, a Windows-style interface, for instance, may seem different, but regardless of the client interface standard, the same fundamental user behaviours should provide the same outcomes.

c/s Testing Tactics

Object-oriented testing procedures make sense even in cases where object technology has not been used to create the c/s system, because replicated data and processes can be grouped into classes of objects that have similar attributes. Test cases for a class of objects (or its equivalent in a traditionally created system) should be universally applicable to all instances of the class after they have been derived.

When taking into account the graphical user interface of contemporary C/S systems, the OO point of view is especially helpful. Because the GUI must function across multiple platforms, it deviates from traditional interfaces and is intrinsically object-oriented. Additionally, because the GUI generates, edits and works with a wide variety of graphical items, testing needs to investigate a lot of different logic paths. The objects might appear wherever on the desktop, be present or absent and exist for a prolonged period of time, which further complicates testing.

This means that in order to accommodate the complexity of the GUI environment, the typical capture/playback approach for testing character-based interfaces must be adjusted. Structured capture/playback is a functional variant of the capture/playback paradigm that was developed for GUI testing.

Conventional capture/playback saves and compares inputs and output images of subsequent tests to the keystrokes recorded during input and the screen images produced during output. The foundation of structured capture and replay is an internal, logical perspective on exterior activity. Interactions between the application program and the GUI are captured as internal events, which can be stored as “scripts” in the vendor’s proprietary language, one of the C variations, or Microsoft Visual Basic. Traditional needs for path testing and data validation are not met by GUI-exercising tools.

4.2.4 Analysis Modelling Issues

The analysis modelling methods used for more traditional computer architectures and the requirements modelling activity for c/s systems are very similar. It should be emphasised, however, that the qualification activities related to CBSE also apply because many contemporary c/s systems employ reusable components.

Analysis modelling avoids specifying implementation details, so problems related to dividing up software components between client and server are only taken into consideration while designing a system. Early analysis and design iterations may be used to make implementation decisions on the overall c/s approach, such as fat client vs. fat server, because an evolutionary approach to software engineering is used for c/s systems.

In software engineering, analysis modelling is an essential stage where system requirements are examined, comprehended and structurally represented. Requirements are analysed and documented using a variety of modelling methodologies, although analysis modelling is not without its difficulties, just like any other stage of software development. The following are some typical problems with analysis modelling:

Ambiguous Requirements:

Issue: Misunderstandings between the development team and stakeholders may result from ambiguities in the requirements. Uncertain criteria could lead to a system that doesn't fulfil users' real demands.

Solution: Reduced uncertainty can be achieved through cooperation with stakeholders, clear communication routes and thorough documentation. Uncertain requirements can be found and clarified via regular evaluations and feedback sessions.

Incomplete Requirements:

Issue: Gaps in knowledge caused by incomplete requirements lead to a system devoid of essential functionalities. Development process delays and rework may result from this problem.

Solution: To capture all pertinent needs, a thorough analysis and stakeholder interaction are necessary. Requirements are continuously refined and validated to make sure nothing important is missed.

Inconsistent Requirements:

Issue: When criteria are stated in a contradictory way or when they clash with one another, inconsistencies occur. Building a coherent system requires resolving discrepancies.

Solution: Inconsistencies can be found and fixed early in the analytical phase by regularly reviewing and validating requirements with stakeholders. Evolving requirements can be addressed by a well-defined change management procedure without creating discrepancies.

Scope Creep:

Issue: When new features or modifications are added after the analysis phase has started, this is known as scope creep. Delays, higher expenses and a loss of attention to the essential requirements may result from this.

Solution: It is necessary to establish a change control procedure and precisely define the project's scope. Every modification that is suggested should undergo a rigorous impact study and a formal approval process.

Notes

Notes

Difficulty in Prioritisation:

Issue: Setting a requirement priority can be difficult since different stakeholders may have different priorities. Ineffective prioritisation may cause less important features to be developed at the expense of more important ones.

Solution: It is essential to work together with stakeholders to comprehend their priorities and how each requirement will affect the firm. Prioritisation can be aided by employing strategies like MoSCoW (Must-haves, Should-haves, Could-haves and Won't-haves).

Lack of User Involvement:

Issue: Requirements that do not meet end users' needs may arise from their insufficient involvement in the analysis process. Rework and unhappiness may result from this later on in the development phase.

Solution: Engage stakeholders and end users in a proactive manner during the analytical stage. Hold interviews, seminars and prototype reviews to make sure you're getting ongoing validation and feedback.

Overemphasis on Documentation:

Issue: An overemphasis on documentation may cause a gap between the analytical model and the project's real requirements. It could lead to an inflexible strategy that impedes flexibility.

Solution: Give communication more importance than paperwork. Although documentation is necessary, comprehending and meeting the underlying criteria should take precedence. Agile approaches frequently promote functional software over extensive documentation.

Difficulty in Modeling Complex Systems:

Issue: Certain systems may be difficult to adequately represent, particularly those that are big and complicated. The analysis model may become unclear or oversimplified as a result of the complexity.

Solution: Focus on modelling one aspect at a time and break down complicated systems into manageable components. Prototypes, diagrams and visualisation tools can be used to improve comprehension and facilitate successful stakeholder communication.

Inadequate Tool Support:

Issue: Lack of appropriate instruments for analysis modelling can impair process accuracy and efficiency. Manual methods can result in mistakes and inefficiency.

Solution: Invest in modelling tools that are appropriate and complement the selected analysis methods. These solutions offer a centralised repository for requirements, improve collaboration and automate some operations.

Resistance to Change:

Issue: Changes suggested during the analysis phase may encounter resistance from stakeholders or team members, resulting in a deficiency of understanding and alignment.

Solution: Overcoming opposition can be aided by efficient change management techniques, communication tactics and education about the advantages of changes. Including stakeholders at the outset of the process might also help them feel more invested in the suggested modifications.

It takes a combination of efficient communication, stakeholder involvement and the application of suitable techniques and tools to address these analysis modelling issues. Frequent feedback loops, validation sessions and an emphasis on teamwork all help make the software engineering analysis modelling phase more successful.

4.2.5 WebE Process

The WebE process is greatly impacted by the features of Web-based applications and systems. WebApp releases are generated in a rapid fire sequence through an incremental, iterative process model driven by continuous evolution and immediacy. Applications in this domain are network-intensive, which implies a diversified user population (which places unique demands on requirements elicitation and modelling) and a highly specialised application architecture (which places demands on design).

Parallel development efforts will probably be scheduled inside the WebE process and involve a team of both technical and non-technical professionals (e.g., copywriters, graphic designers), as WebApps are typically content driven with an emphasis on aesthetics.

The following steps are part of the development process of a web application, which frequently employs the incremental process model:

1. Client communication: Describe how users interact with the system using the use case approach to gather demand information about the issue. This information may then be utilised to support plan, analysis and design modelling follow-up.
2. Plan: Create a project incremental plan. To modify the plan in accordance with the needs
3. Web analysis: Examine the requirements and create a demand analysis model that explains the requirements of the system. UML use case figures are frequently used to represent functions in analysis models, UML dynamic models are used to represent behaviour and UML class diagrams are used to represent the system's static structure.
4. Web design: Create a system's interface, navigation, architecture and other relevant models.
5. Building: To create and test the online application, use web development tools.
6. Deployment: Set it up so that it works well in the terminal client environment.

Web Engineering (WebE) is a methodical and structured approach to creating web-based applications that recognises the particular difficulties brought about by the ever-changing online landscape. We dive into the details of the WebE process in this in-depth analysis, looking at its stages, techniques and tools used to manage the complexity of contemporary web development.

The Stages of the WebE Process

1. Requirements Engineering: User Stories and Personas: It's critical to comprehend user requirements. WebE uses methods like user stories and personas to gather user requirements and expectations, which serves as a basis for further development phases.
Both functional and non-functional requirements are discovered and recorded. Functional requirements are features and capabilities; non-functional requirements include security, scalability and performance.
2. System Design: Information Architecture: Information architecture defines the way that data is arranged and structured within a web application. Sitemaps, navigation models and content hierarchies are examples of this.

Notes

Prototypes and wireframes: Early in the design process, high-fidelity prototypes and low-fidelity wireframes are produced to visualise the user interface and get feedback.

3. **Implementation: Front-End Development:** Through front-end development, the user interface and experience are realised. Web interfaces that are interactive and responsive are made possible by technologies like JavaScript, HTML and CSS.

Back-End Development: The back-end is designed to manage data processing, business logic and front-end interactions. It is frequently powered by databases and server-side technologies.

4. **Testing: Functional Testing:** The web application is put through extensive functional testing to make sure it satisfies the requirements. Unit, integration and system testing are all included in this.

Usability Testing: Real users' feedback is gathered during usability testing in order to evaluate the web application's usability, accessibility and general level of user satisfaction.

5. **Deployment and Maintenance: Deployment Strategies:** To expedite the release process and reduce downtime, WebE takes into account a variety of deployment methodologies, such as continuous integration and continuous deployment (CI/CD).

Monitoring and Maintenance: After deployment, ongoing monitoring aids in finding and resolving problems. To maintain the longevity of the online application, regular maintenance entails updates, security patches and optimisations.

Methodologies in Web Engineering

1. **Agile Development: Iterative Development:** Agile techniques like Scrum and Kanban are frequently used in WebE. They make it easier for teams to engage in iterative development, which enables them to adapt to changing needs and produce small, gradual improvements.

Collaboration and Communication: Agile places a strong emphasis on teamwork and communication among cross-functional members, which promotes a flexible and adaptable development environment.

2. **DevOps Practices: Continuous Integration/Continuous Deployment (CI/CD):** These DevOps techniques automate the deployment and integration procedures. This guarantees the dependability of releases and quickens development cycles.
Infrastructure as Code (IaC): By adopting IaC principles, infrastructure may be managed and provisioned programmatically, improving scalability and consistency.

Tools Utilised in the WebE Process

1. **Version Control Systems: Git:** Version management with Git facilitates collaborative development and allows the development team to follow changes in real time.
2. **Project Management Tools: Jira, Trello, Asana:** These technologies support agile approaches by facilitating team collaboration, work tracking and project planning.
3. **Design and Prototyping Tools: Figma, Sketch, Adobe XD:** By generating visual representations of the web application, design and prototype tools facilitate the design and feedback processes.
4. **Development Frameworks: React, Angular, Vue.js:** These frameworks are frequently used in front-end development to create flexible and interactive user interfaces.

Django, Ruby on Rails, Express.js: Frameworks for back-end development speed up server-side development while putting an emphasis on maintainability and efficiency.

5. **Testing Tools:** Selenium, Jest, Cypress: Functional and end-to-end testing are supported by testing tools, which guarantee the dependability and calibre of the web application.
6. **Deployment and Containerisation:** Docker, Kubernetes: By isolating apps and their dependencies, containerisation solutions improve scalability and simplify deployment.

Challenges and Considerations

1. **Browser Compatibility:** Because different rendering engines conform to different standards, it is still difficult to guarantee uniform performance and appearance across web browsers.
2. **Security Concerns:** Web applications are vulnerable to a number of security risks, such as data breaches and cross-site scripting (XSS and CSRF). Ensuring development requires the implementation of strong security measures at every stage.
3. **Performance Optimisation:** The optimisation of performance is a task that WebE must take on, taking into account variables like responsiveness, resource usage and page load times, especially when dealing with changing network conditions.

Real-World Applications

1. **E-Commerce Platforms:** WebE principles are used by major e-commerce platforms to provide smooth and intuitive online purchasing experiences. Secure transactions, user interactions and a variety of product catalogues must all be handled by these platforms.
2. **Social Media Networks:** WebE is used by social media networks to handle user interactions, dynamic content and real-time changes. These platforms' responsiveness and scalability are essential to their success.
3. **Content Management Systems (CMS):** WebE is used by CMS platforms to let users create, manage and publish digital material. WebE's modular design is useful in the ever-changing world of content generation.

A thorough and flexible method for negotiating the challenges of online development is web engineering. WebE makes it easier to create dependable, user-focused and scalable online applications by promoting interdisciplinary collaboration, agile approaches and a set of specialised tools. The WebE process is always changing in the ever-changing web environment in order to take advantage of new technology and solve new difficulties. Since online development is still essential to many different businesses, teams who want to create creative, dependable and user-friendly web solutions should use the guidelines provided by WebE's principles and practices.

4.2.6 Framework for WebE

As WebApps transform from content-directed, static information sources to user-directed, dynamic application environments, it becomes increasingly crucial to implement sound engineering and management concepts. In order to do this, a WebE framework comprising an efficient process model filled with engineering tasks and framework activities must be developed. The figure below suggests a WebE process paradigm.

The first step in the WebE process is called formulation and it defines the scope of the first increment as well as the aims and objectives of the WebApp. Planning determines a finely granulated development plan for the initial WebApp increment, with a more coarsely granulated schedule for following increments. It also calculates the total project cost and assesses the risks related to the development effort. Analysis determines the WebApp's technical specifications and the content pieces that will be included. Additionally specified are the aesthetic requirements for graphic design.

Notes

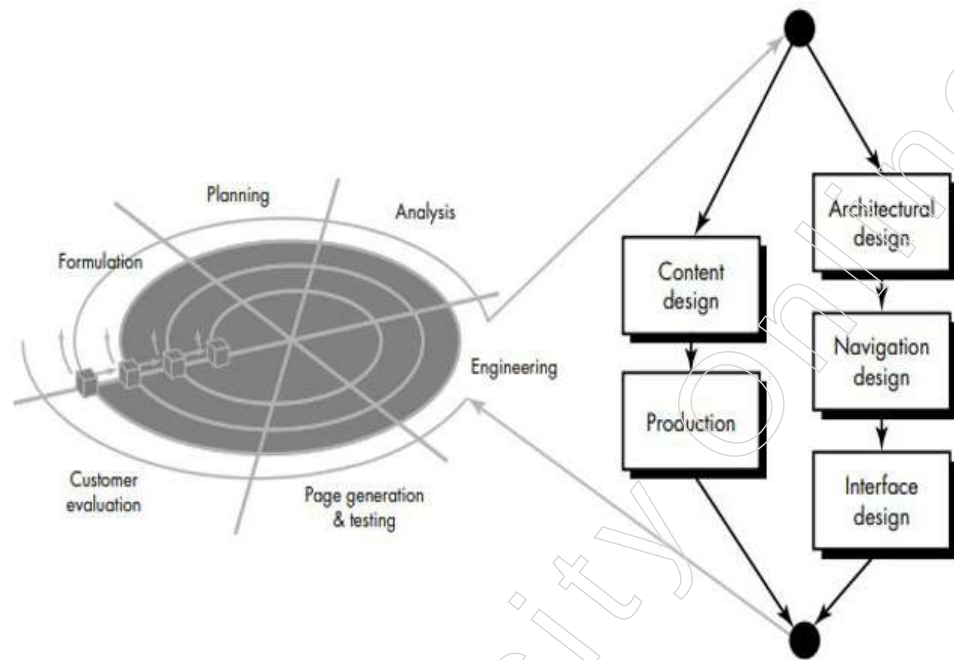


Figure: The WebE process model

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Two concurrent jobs are included in the engineering activity, as seen on the right side of the figure below. Nontechnical team members of WebE handle responsibilities like content production and design. The goal of these jobs is to create, produce and/or obtain all of the audio, video, text and graphic content that will be incorporated into the WebApp. A series of technical design tasks are carried out concurrently.

One construction task that heavily relies on automated tools for WebApp creation is page generation. The architectural, navigational and interface designs are combined with the content described in the engineering activity to create executable Web pages in HTML, XML and other process-oriented languages (like Java). This activity also involves integrating with component middleware, such as CORBA, DCOM, or JavaBeans. Testing helps make sure the WebApp will function properly in various situations (such as with different browsers), tests the WebApp's navigation and looks for faults in applets, scripts and forms.

throughout the customer evaluation phase, every increment generated throughout the WebE process is examined. This is the time when requests for modifications are made (scope extensions occur). Through the incremental process flow, these modifications are incorporated into the following path.

4.2.7 Formulating/Analysing Web-Based Systems

The process of creating and analysing Web-based systems and applications involves a series of Web engineering tasks that culminate in the creation of an analysis model or requirements specification for the system. The process starts with determining the overarching objectives of a WebApp. Through formulation, the developer and the client can agree on a set of priorities for the WebApp's development. It also establishes the parameters of the development endeavour and offers a way to gauge an effective result. The technical process of analysis determines the behavioural, functional and data requirements for the WebApp.

Formulation

Powell offers a list of inquiries that need to be addressed before moving on to the formulation stage:

- What is the main motivation for the WebApp?
- Why is the WebApp needed?
- Who will use the WebApp?

Each of these straightforward questions should have a brief response provided. Assume, for instance, that a company that makes home security systems has chosen to launch an online store to sell its goods to customers directly. A sentence outlining the WebApp's purpose could be.

Customers can design and buy all the components needed to install a home or business security system at SafeHomeInc.com.

It is significant to highlight that this statement lacks specificity. Bounding the site's overarching intent is the goal.

Following deliberation with several stakeholders within SafeHome Inc., the following response is provided to the second question:

We will be able to sell directly to customers thanks to SafeHomeInc.com, which will reduce our expenses as retailers and increase our profit margins. Additionally, it will enable us to expand into areas where we do not currently have sales outlets and grow sales by a predicted 25% over current yearly sales.

The demographic for the WebApp is finally defined by the company: "Homeowners and small business owners are projected users of SafeHomeInc.com."

These responses allude to certain objectives for the SafeHomeInc.com website. Generally speaking, two types of objectives are distinguished:

- Informational objectives. Declare your aim to give the user a certain piece of content or information.
- Relevant objectives. Show that you can complete a task inside the WebApp.

One educational objective inside the SafeHomeInc.com WebApp's content could be

Users will be able to access comprehensive product specs on the website, which will include technical details, installation guidelines and cost details.

An application goal might be stated by looking at the responses to the previous questions:

- After asking the customer about the type of facility (home, business, or retail space) that needs to be secured, SafeHomeInc.com will provide personalised recommendations for the appropriate product and setup.
- A user profile is created after all applicable and informative goals have been determined. "Relevant features related to potential users including their background, knowledge, preferences and even more" are captured in the user profile. A user profile for SafeHomeInc.com would list the traits of a normal security system buyer (this data would be provided by the marketing department of SafeHome Inc.).
- Following the development of goals and user profiles, the formulation activity concentrates on a WebApp's statement of scope. The goals that have already been established are frequently incorporated into the scope declaration. However, it's also helpful to let people know how integrated the WebApp should be expected to be. That

Notes

Notes

is to say, integrating an existing database application with a Web-based front end is often necessary for current information systems. At this point, connectivity concerns are taken into account.

Analysis

The ideas and guidelines covered in the context of software requirements analysis are applicable to the analysis activity of web engineering without modification. A comprehensive analysis model for the WebApp is created by expanding on the scope that was established during the formulation activity. During WebE, four distinct kinds of analysis are carried out:

- Content analysis. The entire range of material that the WebApp will offer is specified. Text, pictures, music and video data are all considered forms of content. Every data object to be used in the WebApp can be identified and described using data modelling.
- Interaction analysis. A detailed description is provided of how the user interacts with the WebApp. Use-cases that give thorough explanations of this interaction can be created.
- Functional analysis. As part of interaction analysis, usage scenarios (also known as use-cases) are developed that specify the operations to be performed on WebApp material and also suggest further processing capabilities. Every function and operation has a detailed description.
- Configuration analysis. A thorough description is given of the WebApp's infrastructure and surroundings. The WebApp may be located on an extranet, intranet, or the Internet. At this point, it's also important to identify the WebApp's architecture, which includes the component infrastructure and the extent to which a database will be used to produce content.

For large, sophisticated WebApps, a thorough requirements definition is advised, but these documents are uncommon. One could argue that any document could become outdated before it is completed due to the ongoing expansion of WebApp standards. While this might be the case in its most extreme form, it is still important to develop an analysis model that can operate as a basis for the subsequent design work. The data gathered in the four previous analysis activities should, at the very least, be evaluated, adjusted as needed and then put into a document that WebApp designers can use.

4.2.8 Management Issues

It makes sense to wonder, "Do we really need to spend time managing a WebApp effort?" given the instantaneous nature of WebApps. Isn't it best to let a WebApp develop organically, requiring little to no deliberate management? Many web developers might choose minimal or no administration, but that doesn't mean they're correct!

The field of web engineering is highly technical. There are many participants, many of whom work simultaneously. Any team of professionals faces a difficulty when it comes to completing the technical and nontechnical activities required to develop a high-quality WebApp on schedule and within budget. To prevent uncertainty, annoyance and failure, risks need to be taken into account, a schedule has to be made and monitored and controls need to be specified. These are the main tasks that fall under the umbrella of project management.

The WebE Team

Building a good Web application requires a wide range of abilities. "There are so many different aspects to [Web] application software that there is a (re)emergence of the renaissance person, one who is comfortable operating in several disciplines," Tilley and

Huang write in response to this problem. Although the writers are entirely true, there aren't many "renaissance" individuals around. Additionally, considering the demands of large-scale WebApp development projects, a WebE team may be a better fit for the wide skill set needed.

But the characters and their parts are frequently very different. Component-based software engineering, networking, architectural and navigational design, Internet standards/languages, human interface design, graphic design, content layout and WebApp testing are just a few of the numerous talents that need to be shared among the members of the WebE team. The WebE team members should be divided into the following roles:

Content developer and providers. One WebE team member's function must be centred on content creation or collecting because WebApps are content-driven by nature. Keeping in mind that content encompasses a wide range of data objects, content suppliers and developers may have a variety of backgrounds (other than software). For instance, media producers may offer audio and video, graphic designers may offer layout design and aesthetically pleasing material, copywriters may supply text-based content and marketing or sales personnel may offer product details and graphical pictures. Research personnel might also need to locate and prepare outside material for the WebApp for use as references or placement.

Web publisher. The diverse content generated by content developers and providers must be organised for inclusion within the WebApp. In addition, someone must act as liaison between technical staff that engineers the WebApp and nontechnical content developers and providers. This role is filled by the Web publisher, who must understand both content and WebApp technology including HTML (or its next generation extensions, such as XML), database functionality, scripts and general Web-site navigation.

Web engineer. Throughout the course of creating a WebApp, a Web engineer is involved in many different tasks, such as gathering requirements, modelling analysis, creating interfaces and architectures, implementing the WebApp and testing it. Along with having a working knowledge of hardware/software platforms, network security, multi-media ideas, client/server architectures, HTML/XML and database technologies, the Web engineer should also have a firm grasp of component technologies.

Support specialist. This position is given to the person or people in charge of maintaining WebApp support. WebApps are constantly changing, thus the support specialist is in charge of making adjustments, modifications and improvements to the website. This includes updating content, putting new policies and forms into place and adjusting the way the site is accessed.

Administrator. Frequently referred to as the Web master, this individual is in charge of the day-to-day management of the WebApp, which includes

- Development and implementation of policies for the operation of the WebApp.
- Establishment of support and feedback procedures.
- Implementation of security procedures and access rights.
- Measurement and analysis of Web-site traffic.
- Coordination of change control procedures.
- Coordination with support specialists.

Project Management

We took into account every action that falls under the umbrella term of project management. A thorough consideration was given to process and project metrics, project

Notes

planning (including estimation), risk analysis and management, scheduling and tracking, SQA and SCM. However, the WebE approach to project management is very different in reality.

Initially, a significant portion of WebApps are contracted out to companies who (allegedly) specialise in creating Web-based systems and applications. In these situations, a company (the client) requests a fixed-price estimate for the creation of a WebApp from two or more vendors, compares the prices and chooses one of the suppliers to complete the work. However, what does the hiring company search for? How does one assess a WebApp vendor's competence? How can one determine the reasonableness of a price quote? To what extent should an organisation (or its outsourcing contractor) prepare for, schedule and assess risks before beginning a significant WebApp development project?

Secondly, there is limited historical data available for estimation as WebApp development is still a relatively young application sector. Almost no WebE metrics have been published in the literature as of yet. To be honest, not much has been said about what those measures might be. As a result, estimating is entirely qualitative and is predicated on prior experience with like projects. However, nearly all WebApps aspire to be inventive, providing users with something fresh and unique. As a result, experience estimating is subject to significant mistake even though it is useful. Thus, how are trustworthy approximations generated? To what extent may specified timelines be guaranteed to be met?

Third, having a precise grasp of the project's scope is essential to scheduling, risk analysis and estimation. However, the scope of the "continuous evolution" WebApp will change over time. In light of the fact that project needs are likely to vary significantly as it moves forward, how can the contracting company and the outsourced vendor maintain cost and schedule control? Given the special characteristics of Web-based systems and applications, how can scope creep be controlled and more importantly, should it be controlled at all?

Currently, there is a lack of clarity on the questions raised by the aforementioned variances in WebApp project management history. Some rules, nonetheless, are important to take into account.

Initiating a project. Even if outsourcing is the best course of action for developing WebApps, an organisation needs to complete a number of activities before looking for an outside vendor to complete the work:

1. The WebApp's target audience is determined; potential internal users are enumerated; the general objectives of the WebApp are established and examined; the content and services the WebApp will provide are detailed; rival websites are mentioned; and both qualitative and quantitative "measures" of a successful WebApp are established. A product specification ought to have documentation on this information.
2. The WebApp should have a preliminary design created internally. While a skilled Web developer will undoubtedly produce a full design, outsourcing vendors can save time and money by identifying the overall style and feel of the WebApp (which can always be changed in the early stages of the project). The kind and amount of content that the WebApp will display, as well as the kinds of interaction processing (such order input and forms) that will be done, should all be indicated in the design. The product specification ought to provide this information.
3. It is necessary to create a preliminary project timeline that includes milestone dates in addition to the final delivery date. Deliverable versions of the WebApp should have milestones associated with them as they develop.

4. It is important to determine the extent of the contractor's supervision and communication with the vendor. As development moves forward, this should involve defining quality review points, designating a vendor liaison and outlining the liaison's power and responsibilities and outlining the vendor's obligations for inter organisational communication.

A request for quotes that is sent to potential vendors should include all of the data gathered during these stages.

Selection of candidate outsourcing vendors. Thousands of "Web design" firms have sprung up in recent years to assist enterprises with e-commerce and Web presence. While many have mastered the WebE procedure, many more are merely hackers. The contractor needs to conduct due diligence in order to choose potential Web developers: Determine the name of the chief Web engineer(s) of the vendor for successful past projects (and, later, make sure that this person is contractually obligated to be involved in your project); (3) carefully examine samples of the vendor's work that are similar in look and feel (and business area) to the WebApp that is to be contracted. These steps will help you assess the Web vendor's professionalism, ability to meet schedule and cost commitments and ability to communicate effectively. Meeting in person can reveal a lot about the "fit" between vendor and contractor even before a request for quotes is made.

Assessing the validity of price quotes and the reliability of estimates. Estimation is inherently problematic because the scope of WebApps is notoriously flexible and there is comparatively little historical data available. Because of this, some suppliers will include sizeable safety margins in the project pricing proposal. This makes sense and is appropriate. "Have we gotten the best bang for our buck?" is not the pertinent issue. Instead, the inquiries ought to be

- Does the WebApp's estimated cost offer a direct or indirect return on investment that makes the project worthwhile?
- Does the quotation provider possess the necessary level of professionalism and experience?

The price quote is reasonable if "Yes" responses to these inquiries are received.

The degree of project management you can expect or perform. The degree of formality involved in project management tasks, which are carried out by both the vendor and the contractor, is strongly correlated with the WebApp's size, cost and complexity. A thorough project schedule that outlines work assignments, quality assurance checkpoints, engineering work products, customer review points and significant milestones should be created for large, complicated projects. Together, the vendor and contractor should evaluate the risks and create plans for reducing, tracking and managing the risks that are considered significant. It is important to have written definitions for quality assurance and change control procedures. It is important to set up procedures for efficient communication between the vendor and the contractor.

Assessing the development schedule. WebApp development schedules should have a high level of granularity since they usually cover a short amount of time—less than one or two months. In other words, job assignments and little goals ought to be planned out on a daily basis. The vendor and the contractor can both identify schedule slippage thanks to this precise granularity before it jeopardises the ultimate completion date.

Managing scope. Given the likelihood of scope changes during a WebApp project, an incremental WebE process model is recommended. In order to achieve an operational WebApp release, this enables the development team to "freeze" the scope for a single increment. Once the second increment starts, scope is again momentarily frozen, albeit it may address scope adjustments recommended by a review of the previous increment.

Notes

By taking this approach, the WebApp team may continue to work without having to adjust to a constant stream of changes, but also acknowledging that most WebApps are always evolving.

These recommendations are not meant to be an infallible recipe book for creating timely, low-cost WebApps. Nonetheless, they will assist the vendor and the contractor in getting things started quickly and with the fewest possible misunderstandings.

SCM Issues for WebE

WebApps have developed over the last ten years from unrefined information-dissemination tools to complex e-commerce websites. Configuration control is becoming more and more necessary as WebApps become more vital to the survival and expansion of businesses. Why? Because improper modifications to a WebApp, in the absence of effective controls, can result in (1) the unapproved publication of new product information, (2) incorrect or ill-tested functionality that annoys website visitors, (3) security flaws that compromise internal company systems and other financially unpleasant or even disastrous outcomes. Keep in mind that immediacy and continuous evolution are the dominant attributes of many WebApps.

When creating strategies for WebApp configuration management, four things need to be taken into account: people, scalability, politics and content.

Content. Text, images, applets, scripts, audio and video files, forms, active page elements, tables, streaming data and many more types of content can be found in a standard WebApp. The difficulty lies in sorting through this deluge of information to create a sensible set of configuration objects and then creating suitable configuration control methods for each of these objects. One method is to use traditional data modelling approaches to model the WebApp content, giving each object a set of specialised features. Examples of characteristics needed to create an efficient SCM strategy are each object's static or dynamic character and its anticipated lifespan (e.g., transient, fixed existence, or permanent object). A content item with hourly changes, for instance, has a limited lifespan. Compared to a forms component that is a permanent object, this item would have different (less formal) control mechanisms.

People. Anybody working in the WebApp can (and often does) contribute content because a large portion of WebApp development is still done on an as-needed basis. A large number of content creators lack any experience with software engineering and are not aware of the necessity of configuration management at all. The application develops and morphs in an erratic way.

Scalability. The methods and constraints used for tiny WebApps don't scale well for larger applications. When connections are made to databases, data warehouses, portal gateways and other current information systems, it is usual for a basic WebApp to expand dramatically. Small adjustments can have far-reaching, unforeseen repercussions that might be troublesome when scale and complexity increase. As a result, the scale of the application should immediately correlate with the strictness of configuration control measures.

Politics. A WebApp is owned by whom? Both large and small businesses debate this issue and the resolution greatly affects the WebE management and control processes. Sometimes Web developers are located outside of the IT department, which could lead to communication issues. To better grasp the politics around WebE, Dart proposes the following inquiries: Who is in charge of ensuring that the data on the website is accurate? Who verifies that procedures for quality control have been followed before material is

posted on the website? Who is in charge of implementing changes? Who bears the expense of change? The persons in an organisation who need to implement a WebApps configuration management procedure can be identified with the help of the answers to these questions.

WebE configuration management is still in its early stages. It could be too laborious to use a traditional SCM procedure. Most SCM tools do not have the necessary functionality to be readily converted to WebE. Some of the problems that still need to be resolved are

- How can we design a configuration management procedure that can adapt quickly enough to WebApps' constant evolution and immediacy?
- How can we best convey concepts and techniques related to configuration management to developers who have no prior experience with the technology?
- How can we help teams of WebApp developers who are spread out?
- In a quasi-publishing setting where content is updated almost constantly, how can we maintain control?
- How do we get the level of detail needed to manage a wide range of configuration objects?
- How can the WebE tools that are already in use be enhanced with configuration management functionality?
- How are modifications to items that have links to other things to be managed?

Before WebE has access to efficient configuration management, these and numerous other problems need to be resolved.

4.3 Service Oriented Software Engineering

An approach to software development known as "service-oriented software engineering" (SOSE) centres on the idea of services, which are encapsulations of discrete functionalities that promote modular, interoperable and scalable systems. The design, development and implementation of software components as services—each fulfilling a distinct business purpose and interacting with other components via clearly specified interfaces—define SOSE. This paradigm encourages adaptability, reuse and the development of distributed systems that can change to meet changing needs.

A step in the development of application development and/or integration is called service-oriented architecture, or SOA. It outlines how to use the interfaces to make software components reusable.

Formally, service-oriented architecture (SOA) is an architectural methodology that allows programs to leverage network services. Services are made available to applications in this architecture via an internet-based network call. Common communication standards are used to expedite and simplify application service interactions. In SOA, every service is a stand-alone full-featured business function. The way the services are released makes it simple for developers to use them to put together their apps. Keep in mind that microservice architecture and SOA are not the same.

- Applications can be created by combining numerous features from pre-existing services thanks to SOA.
- The term "system of architecture" (SOA) refers to a collection of design guidelines that organise the creation of systems and offer ways to combine various system components into a single, decentralised unit.

Notes

- Functionalities are bundled into a collection of interoperable services using SOA-based computing, which may then be incorporated into various software systems that belong to distinct business domains.

4.3.1 Introduction to Service Oriented Software Engineering

Service-oriented architectures as a tool for enabling computing across organisations. Service-oriented architectures, or SOAs, are essentially a method of creating distributed systems where the individual services make up the components of the system. These services could run on computers that are dispersed geographically. To facilitate information sharing and service communication, standard protocols have been developed. Services are therefore platform- and implementation-language-neutral. Software systems can be built with services from several sources interacting with one another without any problems.

A figure that shows the use of web services is shown below. The WSDL language is used by service providers to develop, implement and specify their services. They also use the UDDI publication standard to publish details about these services in a register that is open to the public. When seeking to utilise a service, those seeking services (also known as service clients) look up the service provider and its specifications in the UDDI registry. After that, they can link their application to that particular service and interact with it, usually through the use of the SOAP protocol.

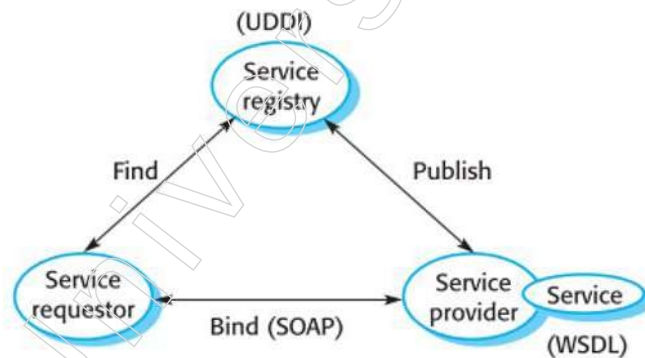


Figure: Service-oriented architecture

Image Source: Software Engineering by roger s. pressman Eighth Edition

Nowadays, most people acknowledge that service-oriented architecture has made considerable advancements, especially for commercial application systems. Because services can be rendered locally or by outside sources, it offers flexibility. It is possible to implement services in any programming language. Companies can protect their investment in valuable software and make it available to a wider range of applications by packaging legacy systems as services. SOA enables interoperability between various platforms and implementation technologies that may be utilised in various departments within an organisation. Perhaps most crucially, developing apps based on services enables businesses and other organisations to collaborate and utilise one another's business operations. As a result, systems that require a lot of information sharing between businesses, like supply chain systems where one company orders things from another, are simple to automate.

The fact that an active standardisation process has been working alongside technical advancements from the beginning may be the primary factor contributing to the success of service-oriented architectures. These standards are the commitment of all the major hardware and software vendors. Because of this, incompatibilities that typically occur with technological innovations—where many providers maintain their proprietary version of the

technology—have not affected service-oriented architectures. Thus, issues arise, like the several incompatible component models in CBSE.

The stack of important standards that have been developed to support web services is depicted in the figure below. Although web services predominate in practice, a service-oriented approach may be employed in scenarios where alternative protocols are in use. Despite the widespread use of HTTP and HTTPS protocols in practice, web services are independent of any specific transport protocol for information exchange.

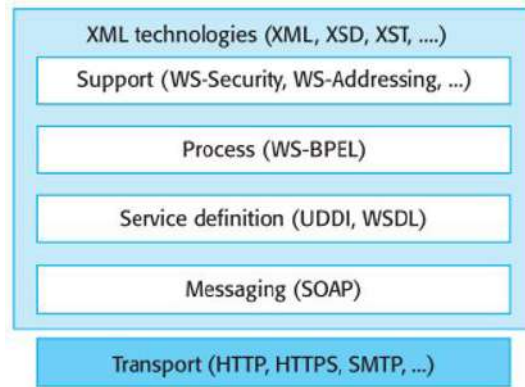


Figure: Web service standards

Image Source: Software Engineering by roger s. pressman Eighth Edition

All facets of service-oriented architectures are covered by web service protocols, ranging from programming language standards (WS-BPEL) to the fundamental methods for service information sharing (SOAP). All these standards rest on XML, a machine- and human-readable language that defines structured data containing text annotated with meaningful identifiers. To extend and modify XML descriptions, a variety of supporting technologies are available, such as XSD for schema definition. Erl does a fantastic job at summarising XML technologies and how they work with web services.

In a nutshell, the fundamental guidelines for web service-oriented architectures are:

1. **SOS** This standard for message transfer facilitates communication between services. The basic and optional components of messages transmitted between services are defined.
2. **WSDL** The Web Service Definition Language (WSDL) standard specifies how service providers are expected to specify these services' interfaces. In essence, it makes it possible to specify a service's interface—that is, the operations, types and parameters of the service—as well as its bindings in a uniform manner.
3. **UDDI** The components of a service specification that can be used to determine whether a service exists are defined by the UDDI (Universal Description, Discovery and Integration) standard. These consist of details regarding the service provider, the services rendered, the location of the service description (often stated in WSDL) and business relationship information. UDDI registries make it possible for prospective customers to learn about the services that are offered.
4. **WS-BPEL** The workflow language defined by this standard is used to create process programs involving many services.

Numerous supplementary standards that concentrate on more specialist areas of SOA complement these core standards. Because they are meant to facilitate SOA in many application kinds, there are a tonne of supporting standards. Several instances of these criteria are as follows:

Notes

Notes

1. A standard for message exchange called WS-Reliable Messaging guarantees that messages will only be transmitted once.
2. A collection of standards known as WS-Security is designed to enable web service security. These standards include guidelines for defining security rules and guidelines for using digital signatures.
3. The SOAP message format for address information is specified by WS-Addressing.
4. The coordination of transactions between dispersed services is defined by WS-Transactions.

These days, “hot topics” include service-oriented software engineering and service-oriented architectures. Businesses are very interested in using a service-oriented approach to software development; but, as of this writing, there isn’t much real-world experience with these kinds of systems. Popular subjects can inspire lofty expectations and make more promises than they can ever keep. For instance, Newcomer and Lomow write the following in their book on SOA:

The service-oriented enterprise, which is propelled by the convergence of important technologies and the widespread adoption of Web services, promises to dramatically increase company agility, accelerate the time to market for new goods and services, lower IT costs and boost operational effectiveness.

Just as significant as the evolution of object-oriented software engineering is that of service-oriented software engineering. But in actuality, it will take a number of years to actualize these advantages and make the SOA vision a reality. We do not yet have well-established software engineering methods for this kind of system because service-oriented software development is so new.

4.3.2 Services as Reusable Components

Software systems are built using component-based software engineering (CBSE), in which standard component models serve as the foundation for the composition of software components. Services are the logical evolution of software components, where the component model is essentially the collection of web service standards. Thus, a loosely connected, reusable software component that encapsulates discrete functionality that may be distributed and accessed programmatically is what is meant to be understood as a service. A web service is one that can be accessible through XML-based protocols and the regular Internet.

According to CBSE, a crucial difference between a software component and a service is that the former must be loosely coupled and independent of the latter. That is, regardless of the execution environment, they should always function in the same manner. They offer access to the service capabilities through their “provides” interface. The goal of services is to be flexible and autonomous in many settings. As a result, they lack the “requires” interface, which is necessary for CBSE to specify which other system components are required.

The Internet can also be used to disseminate services. They exchange messages, which are represented in XML and these messages are sent over common Internet transport protocols like TCP/IP and HTTP. A service sends a message to another service outlining what it requires from it and that service receives it. After parsing the message and performing the computation, the receiving service sends a message to the asking service. After that, this service parses the response to get the necessary data. Services don’t “call” methods connected to other services, in contrast to software components.

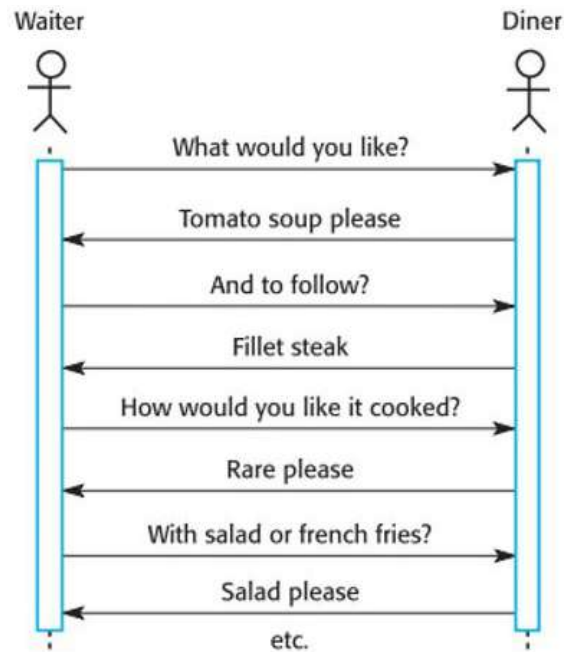


Figure: Synchronous interaction when ordering a meal

Image Source: Software Engineering by roger s. pressman Eighth Edition

Take into consideration an instance where you are placing an order at a restaurant to demonstrate the distinction between communication via method calls and communication via message passing. Your order is defined by a sequence of synchronous exchanges that take place during a chat with the waiter. This is analogous to how components of a software system communicate with one another by calling methods on other components. Your order and the orders of the other patrons are noted by the waiter, who then delivers this message—which contains specifics on everything that has been ordered—to the kitchen so that the meal may be prepared. In essence, the kitchen service receives a message from the waiter service specifying the food that needs to be cooked. Shown this in the Figure above, which depicts the synchronous ordering process and in the Figure below, which depicts a hypothetical XML message that represents an order placed by the three-person table. The distinction is obvious: the waiter views the order as a sequence of exchanges, where each exchange defines a certain aspect of the order. But in the waiter's one exchange with the kitchen, the message conveyed outlines the entirety of the order.

```

<starter>
  <dish name = "soup" type = "tomato" />
  <dish name = "soup" type = "fish" />
  <dish name = "pigeon salad" />
</starter>
<main course>
  <dish name = "steak" type = "sirloin" cooking = "medium" />
  <dish name = "steak" type = "fillet" cooking = "rare" />
  <dish name = "sea bass">
</main>
<accompaniment>
  <dish name = "french fries" portions = "2" />
  <dish name = "salad" portions = "1" />
</accompaniment>
  
```

Figure: A restaurant order expressed as an XML message

Image Source: Software Engineering by roger s. pressman Eighth Edition

Notes

When utilising a web service, you must be aware of its location (URI) and the specifics of its user interface. These are explained via a service description written in WSDL (Web Service Description Language), an XML-based language. Three characteristics of a Web service are defined by the WSDL specification. It outlines the functions, communication methods and location of the service:

1. The “what” portion of a WSDL document, known as an interface, describes the operations the service supports and the message format that the service sends and receives.
2. A binding, which is the “how” portion of a WSDL document, translates an abstract interface into a specific collection of protocols. The technical aspects of communicating with a Web service are specified in the binding.
3. The location of a particular Web service implementation is specified in the “where” section of a WSDL document, which is confusingly referred to as a service.

Every component of a service description is displayed in the WSDL conceptual model (see Figure below). All of these can be supplied as distinct files and are expressed in XML. These components are:

1. An introduction that often includes a definition of the XML namespaces being utilised as well as a documentation section with more details about the service.
2. An optional explanation of the message types that the service uses to exchange messages.
3. An explanation of the functions that the service interface offers.
4. an explanation of the messages that the service processes as input and output.
5. a description of the messaging protocol—that is, the binding that the service will use—for sending and receiving messages. Other bindings can be used, although SOAP is the default. The binding defines the communication protocols to be utilised as well as how the input and output messages related to the service should be combined into a single message. The binding may also dictate how supplementary data—like transaction identifiers or security credentials—is provided.
6. An endpoint definition, or the address of a resource that can be accessed over the Internet, specifies the physical location of the service and is expressed as a Uniform Resource Identifier (URI).

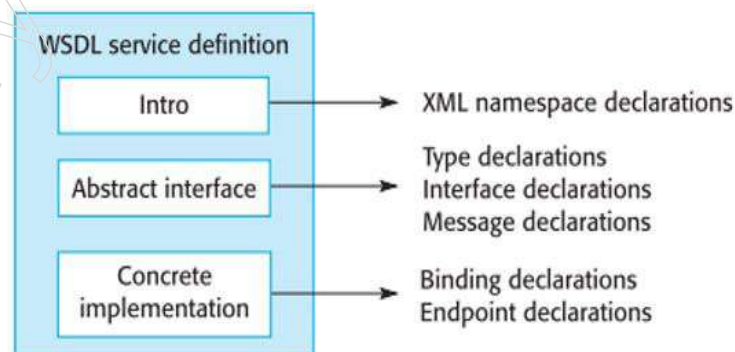


Figure: Organisation of a WSDL specification

Image Source: Software Engineering by roger s. pressman Eighth Edition

XML-formatted complete service descriptions are lengthy, intricate and difficult to interpret. XML namespace definitions, which are qualifiers for names, are typically included in them. Any identifier used in the XML description may come before a namespace identifier. It implies that identifiers with the same name that have been declared in various

sections of an XML description can be distinguished from one another. We won't delve into namespace specifics at this time. All you need to know to grasp this topic is that names must be unique and that namespace:name pairs can be prefixed with namespace identifiers.

This topic of the WSDL corresponds to a software component's "provides" interface. The interface for a basic service that provides the highest and lowest temperatures ever recorded in a location on a certain day, given the location (town and nation), is depicted in detail in the above figure. Depending on where the temperature was measured, these readings could be returned in either degrees Celsius or degrees Fahrenheit.

A portion of the element and type definition used in the service specification is displayed in the first section of the description in the figure below. The elements PlaceAndDate, MaxMinTemp and InDataFault are defined in this way. We've simply given the PlaceAndDate specification, which is essentially a record with three fields: town, nation and date. MaxMinTemp and InDataFault would be defined in a manner akin to this.

The definition of the service interface is presented in the second section of the description. Although the number of operations that can be defined is unlimited, the weather Info service in this example only has one operation. There is an in-out pattern connected with the weather Info operation, which means that it receives one input message and outputs one message. Many message exchange patterns, including in-only, in-out, out-only, in-optional-out, out-in, etc., are supported by the WSDL 2.0 specification. The next step is to specify the input and output messages, which make reference to the definitions already made in the types section.

The main issue with WSDL is that no information regarding the semantics of the service or its non-functional attributes, including dependability and performance, is included in the design of the service interface. It is merely an explanation of the service signature; the user must infer the real function of the service and the meaning of the various fields in the input and output messages. Although descriptive labels and service manuals are helpful in this regard, there is still potential for misinterpretation and misuse of the service.

Define some of the types used. Assume that the namespace prefixes 'ws' refers to the namespace URI for XML schemas and the namespace prefix associated with this definition is weathns.

```
<types>
  <xs:schema targetNamespace = "http://.../weathns"
    xmlns:weathns = "http://.../weathns" >
    <xs:element name = "PlaceAndDate" type = "pdrec" />
    <xs:element name = "MaxMinTemp" type = "mmtrec" />
    <xs:element name = "InDataFault" type = "errmess" />

    <xs:complexType name = "pdrec"
      <xs:sequence>
        <xs:element name = "town" type = "xs:string" />
        <xs:element name = "country" type = "xs:string" />
        <xs:element name = "day" type = "xs:date" />
      </xs:sequence>
    </xs:complexType>
```

Definitions of MaxMinType and InDataFault here

```
</schema>
</types>
```

Now define the interface and its operations. In this case, there is only a single operation to return maximum and minimum temperatures

Notes

```
<interface name = "weatherInfo">
  <operation name = "getMaxMinTemps" pattern = "wsdl:in-out">
    <input messageLabel = "In" element = "weathns:PlaceAndDate" />
    <output messageLabel = "Out" element = "weathns:MaxMinTemp" />
    <outfault messageLabel = "Out" element = "weathns:InDataFault" />
  </operation>
</interface>
```

Figure: Part of a WSDL description for a web service

Image Source: Software Engineering by roger s. pressman Eighth Edition

4.3.3 Service Engineering

The practice of creating services for reuse in applications that are service-oriented is known as service engineering. It is similar to component engineering in many ways. It is the responsibility of service engineers to guarantee that the service is a reusable abstraction that can be applied to many systems. In order for the service to perform consistently across a variety of applications, they must design and implement generally useful functionality related to that abstraction. They also need to make sure the service is resilient and dependable. In order for prospective consumers to find and comprehend the service, they must document it.

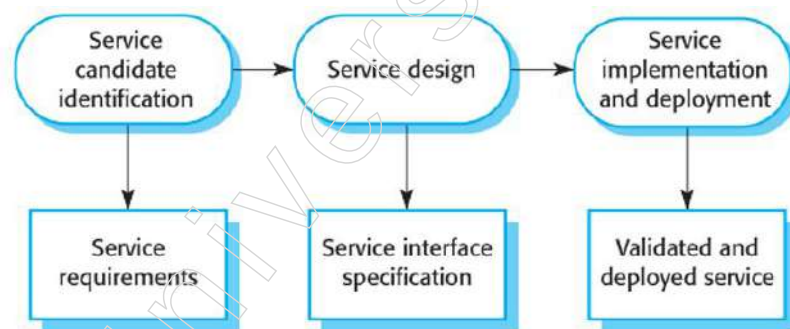


Figure: The service engineering process

The service engineering process consists of three logical steps (see Figure above). These are the following:

1. The process of identifying potential service implementations and defining service needs is known as service candidate identification.
2. The logical and WSDL service interfaces are designed during the service design process.
3. The process of implementing, testing and deploying a service involves making it usable.

Service Candidate Identification

The fundamental idea behind service-oriented computing is that business processes should be supported by services. Every business has a vast array of processes, thus there are a plethora of potential services that may be put into place. In order to determine which reusable services are necessary to support the organisation's business operations, service candidate identification entails comprehending and assessing these processes.

Three basic service categories are described by Erl as potentially identifiable:

1. Services provided by utilities These services carry out some general functionality that many business processes could find useful. A currency conversion service, which may be used to calculate the conversion of one currency (such as dollars) to another (such as euros), is an example of a utility service.

2. **Commercial services** These are services connected to a particular commercial purpose. The process of enrolling students in classes is an illustration of a commercial function in a university.
3. **Organising or handling services** These services assist a broader business process, which often entails a variety of participants and actions. An ordering service that enables orders to be placed with suppliers, goods to be received and payments to be made is an example of a coordination service in a business.

	Utility	Business	Coordination
Task	Currency convertor Employee locator	Validate claim form Check credit rating	Process expense claim Pay external supplier
Entity	Document style checker Web form to XML converter	Expenses form Student application form	

Additionally, Erl proposes that services may be categorised as entity- or task-oriented. Task-oriented services are linked to specific tasks, while entity-oriented services are similar to objects in that they are connected to specific business entities, like a job application form. A few task- or entity-oriented service examples are shown in the above figure. Coordination services are always task-oriented, even if they might also be business or utility services.

Finding logically consistent, autonomous and reusable services should be your aim when searching for potential candidates. In this regard, Erl's classification is useful since it offers advice on how to look at business organisations and business activities in order to find reusable services. However, the identification of service candidates is just as challenging as the identification of objects and components. In order to determine if a potential candidate will be beneficial, you must first identify potential prospects and then ask them a series of questions. You can use the following queries to determine reusable services:

1. Is a service that is entity-oriented linked to a single logical entity that is employed in several business processes? Which actions on that entity are typically taken that need to be supported?
2. Is the task for a task-oriented service one that is completed by several employees inside the company? When a single support service is offered, standardisation inevitably takes place. Will they be able to accept this?
3. To what degree does the service depend on the availability of other services and is it independent?
4. Does the service need to keep state in order to function? If so, will state maintenance make use of a database? Systems that depend on internal state are typically less reusable than those that have external state maintenance capabilities.
5. Could customers from outside the company use the service? For instance, is it possible to access an entity-oriented service connected to a catalogue from the inside as well as the outside?
6. Are there likely to be differences in the non-functional requirements of different service users? If they do, this implies that it could be a good idea to deploy multiple versions of a service.

You can choose and improve the abstractions that can be used as services with the aid of the answers to these questions. Nonetheless, the process of identifying the finest services is skill- and experience-based because there is no set formula for doing so.

Notes

A list of recognised services along with the corresponding requirements for each service is the result of the candidate selection process. What the service is expected to do should be specified in the functional service requirements. The security, performance and availability requirements for the service should be specified in the non-functional requirements.

Let's say a big computer equipment retailer has set aside special rates for certain clients' authorised setups. The company wants to provide a catalogue service where clients can choose the equipment they require, making automated ordering easier. In contrast to a consumer catalogue, orders are placed via each company's web-based procurement system rather than directly through a catalogue interface. The majority of businesses have their own budgeting and order approval processes and when an order is placed, it must go via their own ordering procedure.

One entity-oriented service that helps with business operations is the catalogue service. The following are necessary for a functional catalogue service:

1. Every user company must have a customised copy of the catalogue. This will cover the equipment and configurations that staff members of the client company may order, as well as the agreed-upon costs for catalogue products.
2. A customer staff must be able to download a copy of the catalogue for offline viewing.
3. Users of the catalogue will be able to compare the features and costs of up to six catalogue items.
4. Users will have the ability to browse and search the catalogue.
5. Users of the catalogue will be able to find out when a specified quantity of catalogue goods are expected to be delivered.
6. Customers will be able to place "virtual orders" using the catalogue, whereby the necessary items will be held for them for a period of 48 hours. A genuine order submitted by a procurement system needs to validate virtual orders. The virtual order must be received within 48 hours of this.

The catalogue also has a number of nonfunctional requirements in addition to these functional needs:

1. The catalogue service will only be available to staff members of approved organisations.
2. The costs and options presented to a single client will be kept private and inaccessible to staff members of other clients.
3. The catalogue will be accessible without any interruptions between 0700 GMT and 1100 GMT.
4. During peak load, the catalogue service must be able to handle up to 10 requests per second.

Observe that there isn't a non-functional requirement about the catalogue service's response time. This is contingent upon the magnitude of the database and the anticipated count of concurrent users. It is not necessary to specify it at this time because the service is not time-sensitive.

Service Interface Design

Designing the service interfaces is the next step in the service engineering process once you have chosen candidate services. This entails specifying the parameters and

operations connected to the service. It is imperative to carefully consider how the service's activities and messages might be structured to limit the number of message exchanges required to fulfil the service request. Rather than requiring synchronous service interactions, you must make sure that as much information as feasible is sent to the service in a message.

Additionally, keep in mind that services are stateless, meaning that service users—rather than the service itself—are in charge of maintaining an application state specific to a particular service. As a result, you might need to send input and output messages to and from services carrying this state information.

The design of a service interface has three stages:

1. The processes connected to the service, their inputs and outputs and any exceptions related to these operations are all identified in a logical interface design.
2. Message design is the process of creating the format for messages that the service sends and receives.
3. WSDL development is the process of translating your message and logical designs into an abstract WSDL interface definition.

The service requirements serve as the foundation for the first stage of logical interface design, which also determines the operation names and parameters related to the service. You should now specify any potential exceptions that could occur from invoking a service operation.

The processes that carry out the requirements as well as the inputs, outputs and exceptions for every catalogue operation are depicted in the two figures below. These don't need to be detailed in great depth at this time; it comes later in the design process.

Operation	Description
MakeCatalogue	Creates a version of the catalogue tailored for a specific customer. Includes an optional parameter to create a downloadable PDF version of the catalogue.
Compare	Provides a comparison of up to six characteristics (e.g. price, dimensions, processor speed, etc.) of up to four catalogue items for comparison.
Lookup	Displays all of the data associated with a specified catalogue item.
Search	This operation takes a logical expression and searches the catalogue according to that expression. It displays a list of all items that match the search expression.
CheckDelivery	Returns the predicted delivery date for an item if it is ordered today.
MakeVirtualOrder	Reserves the number of items to be ordered by a customer and provides item information for the customer's own procurement system.

Figure: Functional descriptions of catalogue service operation

Image Source: Software Engineering by roger s. pressman Eighth Edition

Notes

Operation	Inputs	Outputs	Exceptions
MakeCatalogue	<i>mcln</i> Company id PDF-flag	<i>mcOut</i> URL of the catalogue for that company	<i>mcFault</i> Invalid company id
Compare	<i>compln</i> Company id Entry attribute (up to 6) Catalogue number (up to 4)	<i>compOut</i> URL of page showing comparison table	<i>compFault</i> Invalid company id Invalid catalogue number Unknown attribute
Lookup	<i>lookIn</i> Company id Catalogue number	<i>lookOut</i> URL of page with the item information	<i>lookFault</i> invalid company id Invalid catalogue number
Search	<i>searchIn</i> Company id Search string	<i>searchOut</i> URL of web page with search results	<i>searchFault</i> invalid company id Badly-formed search string
CheckDelivery	<i>gdIn</i> Company id Catalogue number Number of items required	<i>gdOut</i> Catalogue number Expected delivery date	<i>gdFault</i> Invalid company id Invalid catalogue number No availability Zero items requested
PlaceOrder	<i>poln</i> Company id Number of items required Catalogue number	<i>poOut</i> Catalogue number Number of items required Predicted delivery date Unit price estimate Total price estimate	<i>poFault</i> Invalid company id Invalid catalogue number Zero items requested

Figure: Catalogue interface design

Image Source: Software Engineering by roger s. pressman Eighth Edition

It is especially crucial to define exceptions and provide service users with information about them. It is usually foolish to assume that service users have fully comprehended the service specification because service engineers have no idea how their services will be used. It is possible for input messages to be inaccurate, thus you should provide exceptions that alert the service client to improper inputs. In general, it is best practice in the construction of reusable components to delegate all exception handling to the component's user; the service developer shouldn't impose their preferences on this matter.

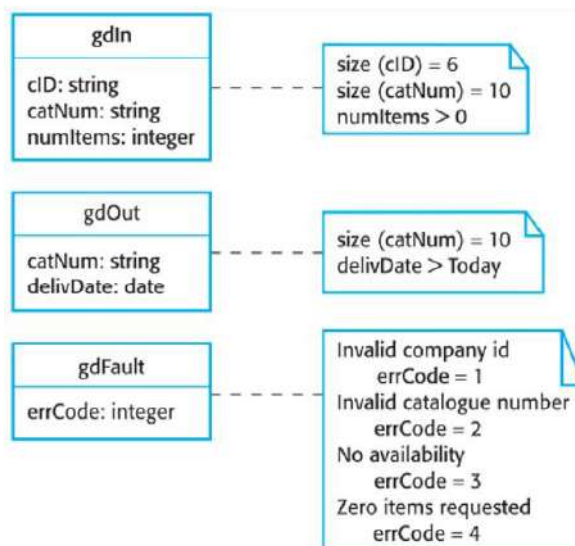


Figure: UML definition of input and output messages

Image Source: Software Engineering by roger s. pressman Eighth Edition

The next step is to specify the input and output message structures and types once you have produced an informal logical description of what the service should perform. Right now, XML is a cumbersome notation to use. We believe it is preferable to specify the messages as objects using a programming language like Java or by utilizing the UML. After that, they can be transformed to XML either automatically or manually. The structure of the input and output messages for the getDelivery action in the catalogue service is depicted in the UML diagram above.

Take note of how we have annotated the UML diagram with restrictions to provide further detail to the explanation. These stipulate that the number of goods must be greater than zero and that delivery must take place after the current date. They also determine the length of the strings that indicate the firm and the catalogue item. The error codes linked to each potential defect are also displayed in the annotations.

Converting the service interface design into WSDL is the last step in the service design process. Because a WSDL representation is lengthy and intricate, mistakes are common at this point. The majority of programming environments (like the ECLIPSE environment) that facilitate service-oriented development are equipped with tools that can convert a logical interface definition into the matching WSDL representation.

Service Implementation and Deployment

Service implementation is the last step in the service engineering process, which begins after you have chosen potential services and created their interfaces. Programming the services in a standard programming language like Java or C# may be part of this implementation. There are currently libraries for both of these languages that provide strong support for service creation.

As an alternative, services could be created using pre-existing parts, legacy systems. This implies that software assets that have shown to be beneficial can now be made more broadly accessible. It might imply that new apps can access the system's capabilities in the case of legacy systems. It is also possible to create new services by declaring how existing services should be composed.

A service must first undergo testing after implementation before being made available to users. In order to do this, the service inputs must be examined and divided, input messages reflecting these input combinations must be created and the expected outputs must then be verified. To ensure that the service can handle erroneous inputs, you should always attempt to generate exceptions when running the test. There are currently many testing tools available that build tests from a WSDL specification and enable services to be inspected and tested. These, however, are limited to evaluating the service interface's adherence to the WSDL. They are unable to verify that the functioning performance of the service is as promised.

Making the service usable on a web server is the last step in the process, known as service deployment. This is made very easy by most server software. The executable service file only has to be installed in one particular directory. It then turns on automatically and is usable. You must create a UDDI description if the service is meant to be accessible to the general public in order for potential users to learn about it. He provides a helpful synopsis of UDDI in his book.

Businesses may also keep their own private UDDI registries and there are now several public registers for UDDI descriptions. A UDDI description is made up of several different kinds of data:

1. Information about the company offering the service. This is crucial for reasons of trust.

Notes

A service must provide users with the assurance that it won't act maliciously. Users can verify a service provider's credentials by viewing information about them.

2. a casual explanation of the features the service offers. This aids prospective customers in determining whether the service is what they desire. It is not an obvious semantic explanation of the service's functions, nevertheless, because the functional description is written in normal language.
3. information about the location of the service's WSDL standard.
4. subscription details that let consumers sign up to receive alerts regarding service updates.

The fact that the functional behaviour of the service is described informally using normal language in UDDI specifications could provide a challenge. A vibrant research community is looking into ways to specify the semantics of services in order to overcome this issue. Ontology-based description, in which the precise meaning of terms in a description is described in an ontology, is the most promising method of semantic definition. Web service ontologies can be described using a language called OWL-S. These methods for defining semantic services are still in their infancy as of this writing, but in the coming years, they should gain traction.

Legacy System Services

Essentially, it would be possible to reuse the functionality of the legacy systems. The component's implementation merely focused on offering a broad interface to that system. The implementation of these "wrappers" for legacy systems is one of the most significant uses of services. After that, these systems can be accessed online and combined with other apps.

Consider a major corporation that keeps track of all of its equipment and the related maintenance database in order to demonstrate this. This maintains track of the maintenance requests submitted for various equipment parts, the scheduled routine maintenance, the day and time of the repair, the amount of time spent on the maintenance, etc. Originally intended to provide daily task lists for maintenance personnel, this historical system has since been expanded to include additional features.

These include information to help determine the cost of maintenance work to be performed by outside contractors as well as data regarding the amount of money spent on maintenance for each piece of equipment. Specialised client software installed on a PC powers the system, which functions as a client-server architecture.

The organisation now wants to give repair workers portable terminal access to this system in real time. In order to locate their next maintenance task, they will query the system and immediately update it with the time and resources spent on maintenance. In order to record repair requests and track their progress, call centre employees also need access to the system.

Since it would be nearly impossible to improve the system to meet these demands, the business chooses to give maintenance and call centre employees new applications. These applications are dependent on the legacy system, which will serve as the foundation for the implementation of several services.

This is demonstrated in the figure below, where We've indicated a service using a UML stereotype. To access the functionality of the legacy system, new applications just need to exchange messages with these services.

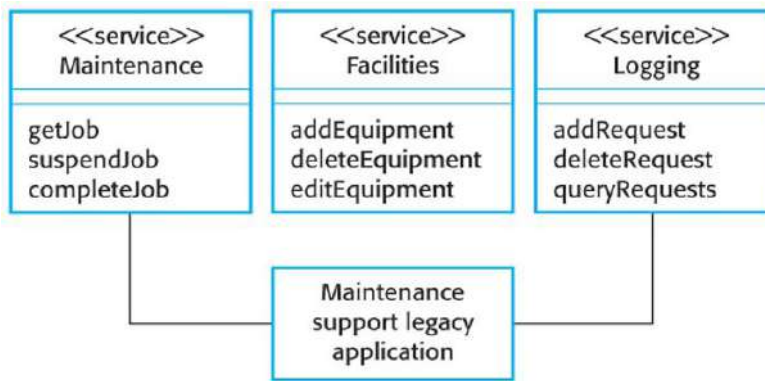


Figure: Services providing access to a legacy system

Image Source: Software Engineering by roger s. pressman Eighth Edition

Among the services offered are:

1. A repair and upkeep service This comprises uploading information about completed maintenance to the maintenance database and retrieving a maintenance job based on its work number, priority and geographic location. It also facilitates an action that enables the suspension of initiated but unfinished maintenance.
2. A service for facilities This covers actions to add and remove new equipment as well as changes to equipment-related database entries.
3. A recording service This covers the processes to create new service requests, remove maintenance requests and find out how many requests are still pending.

There is more to the current legacy system than just one service. Instead, the services that are created are logical and cater to a specific functional domain. As a result, they become less complex and are simpler to comprehend and repurpose in new contexts.

4.3.4 Software Development with Services

The foundation of service-based software development is the notion that services can be combined and configured to build new, composite services. These might be utilised as parts of another service composition, or they could be combined with an online user interface to build a web application. The services included in the composition might have been created especially for the application, might have been business services created internally at a corporation, or might have come from an outside source.

The transformation of enterprise programs into service-oriented systems is currently a major problem for many businesses. This creates the opportunity for increased internal corporate reuse. The creation of inter-organisational applications between reliable providers will be the following phase. The establishment of a "services market" is necessary for the long-term ambition of service-oriented architectures to be fully realised. It seems improbable that this will become apparent in the course of this book. As of this writing, the majority of business services that could be integrated in business applications are not readily accessible to the general public.

Separate business processes can be combined using service composition to create an integrated process with more capability. Let's say a travel agency wants to give vacation packages to customers. In addition to scheduling their flights, passengers can reserve hotels in the locations of their choice, rent a car or hail a taxi from the airport, peruse a travel guide and make plans to see nearby sites. In order to build this application, the airline combines its own booking services with those provided by local attraction providers, car

Notes

rental and taxi firms and hotel booking agencies. As a result, these disparate services from several sources are combined into a single offering.

This procedure can be viewed as a series of discrete processes, as illustrated in the figure below, where information is transferred from one stage to the next (e.g., the flight schedule time is communicated to the automobile rental business). A workflow is a series of actions arranged in a timely manner, each of which completes a portion of the task at hand. A workflow can be thought of as a model for a business process, which consists of the stages necessary to accomplish a crucial objective for a company. The airline's vacation booking service is the business process in this instance.

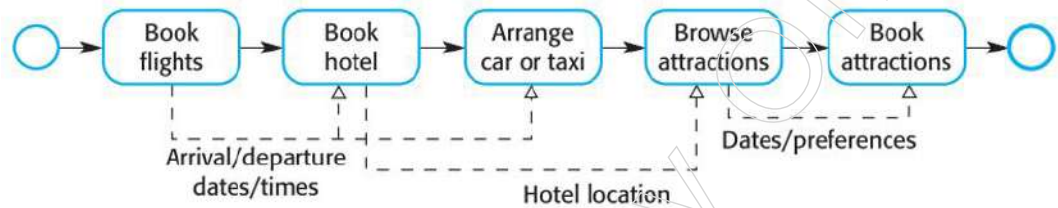


Figure: Vacation package workflow

Image Source: Software Engineering by roger s. pressman Eighth Edition

The concept of workflow is basic and scheduling a vacation in the example above appears like an easy task. Service composition is actually far more complicated than this straightforward model suggests. For instance, you need to build in procedures to deal with service outages and account for the potential for them. You also need to consider the extraordinary requests that users of the program may make. Let's take an example where a passenger needed a wheelchair to be rented and transported to the airport due to their disability.

When one service's usual execution causes an incompatibility with another service's execution, you must be prepared to handle scenarios where the workflow needs to be modified. Let's take an example where a flight is scheduled to depart on June 1 and return on June 7. The process then moves on to the stage of booking hotels. Nevertheless, there are no hotel rooms available because the resort is hosting a significant meeting till June 2. This lack of availability is reported by the hotel booking agency. This is not a failure; being unavailable is a frequent occurrence. After that, you must "undo" the flight reservation and inform the user that the flight is not available. After then, he or she must choose whether to adjust the resort or the dates. A "compensating action" is what's referred to in workflow terminology when an action that has already been completed is undone.

In essence, creating new services by assembling pre-existing ones is a software design method that incorporates reuse (see Figure below). Reusing design always necessitates compromising needs. The "ideal" system requirements must be adjusted to take into account the services that are really offered, whose prices are reasonable and whose level of service is acceptable.

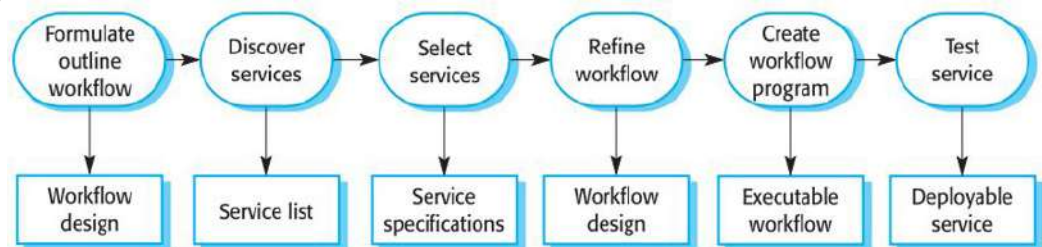


Figure: Service construction by composition

Image Source: Software Engineering by roger s. pressman Eighth Edition

We've highlighted six crucial steps in the process of building services via composition in the above figure:

1. Create a workflow outline. During the first phase of service design, you create a "ideal" service design based on the criteria for the composite service. At this point, you should design something pretty abstract, intending to add features later on when you have more information about the services that are available.
2. Find out about services In this phase of the procedure, you look for service registries to find out what services are available, who offers them and how they are provided.
3. Choose potential services. You next choose potential services that can carry out process tasks from the list of potential service candidates that you have found. Of course, one of your choosing criteria will be how well the services work. They might also cover the price of the services and their level of quality (availability, responsiveness, etc.). Depending on the specifics of pricing and service quality, you may select a variety of functionally identical services, some of which may be tied to a workflow activity.
4. Streamline the process You then fine-tune the procedure based on details about the services you have chosen. This entails maybe adding or deleting workflow tasks as well as providing more information to the abstract description. The steps of service discovery and selection can then be repeated. You proceed to the next step of the process once a stable set of services has been selected and the final workflow architecture has been defined.
5. Make a workflow application. In this phase, the service interface is defined and the abstract workflow design is converted into an executable program. For service implementation, you can use a standard programming language like Java or C# or a more specialist workflow language like WS-BPEL. It is recommended to write the service interface specification in WSDL. The development of web-based user interfaces to enable browser access to the new service may also be part of this phase.
6. Check the finished product or program. When external services are involved, testing the full composite service is a more complicated procedure than testing individual components.

Workflow Design and Implementation

In order to describe the process being developed in a workflow design notation, workflow design entails first assessing current or projected business processes to comprehend the various steps of these processes.

This illustrates the steps needed to carry out the procedure as well as the data that is sent between the various steps. There may not be a "normal" manner of functioning; instead, current procedures may be informal and reliant on the knowledge and abilities of the individuals involved. In these situations, creating a workflow that meets the objectives of the current business processes requires the use of process knowledge.

Business process models are represented by workflows, which are typically shown using graphical notation like YAWL or BPMN. As of this writing, BPMN appears to be the process modelling language most likely to become a standard. This is a really simple to grasp graphical language.

To translate the language to XML-based, lower-level descriptions in WS-BPEL, mappings have been established. As a result, BPMN complies with the web service standards stack that we displayed in the figure below. Here, We demonstrate the idea of business process programming using BPMN.

Notes

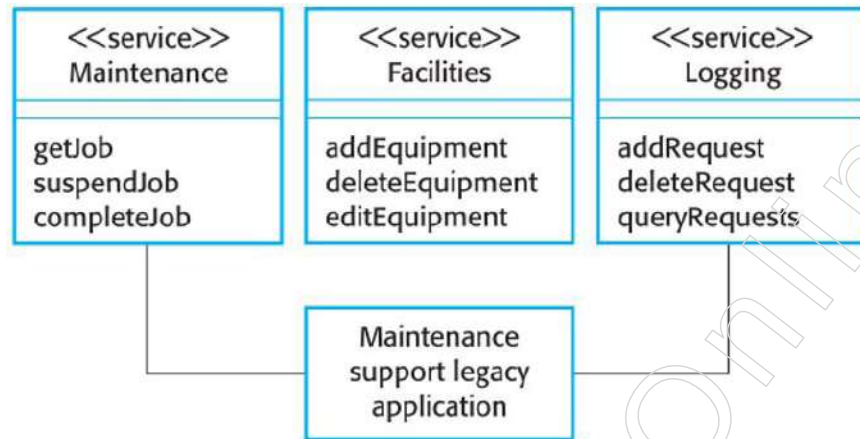


Figure: Services providing access to a legacy system

Image Source: Software Engineering by roger s. pressman Eighth Edition

A basic BPMN model of a portion of the vacation package scenario mentioned above is shown in the figure below. The model assumes the presence of a Hotels service with related actions named GetRequirements, CheckAvailability, ReserveRooms, NoAvailability, ConfirmReservation and CancelReservation. It also depicts a simplified workflow for reserving a hotel. Obtaining the customer's requirements, determining whether rooms are available and then making a reservation for the necessary dates comprise the procedure.

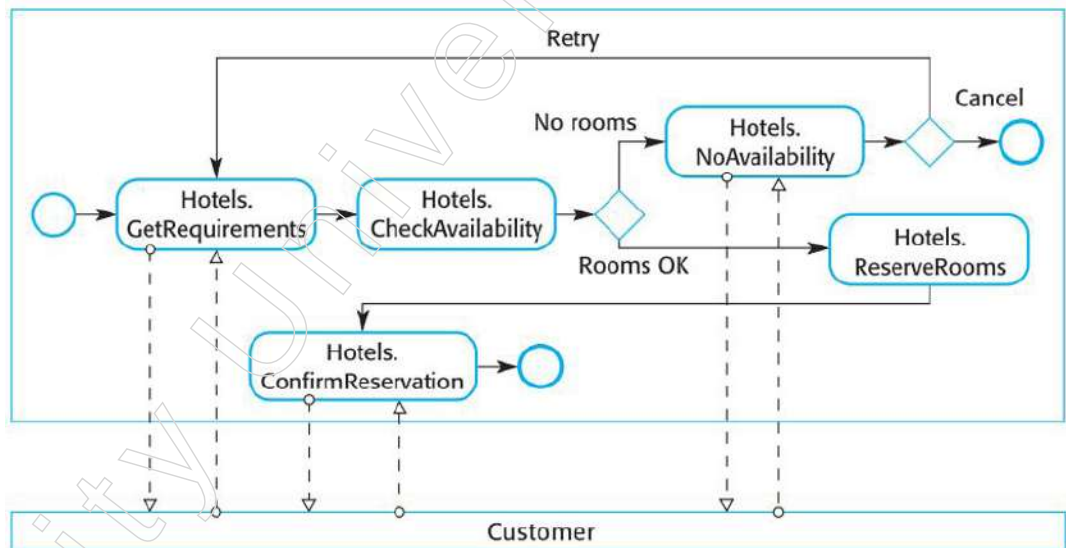


Figure: Hotel booking workflow

Image Source: Software Engineering by roger s. pressman Eighth Edition

Some of the fundamental BPMN ideas that are utilised to build process models are introduced in this model:

1. A rectangular shape with rounded corners is used to symbolise activities. An automated service or a human can carry out an activity.
2. Circles are used to symbolise events. Anything that occurs throughout a business process is called an event. A beginning event is represented by a simple circle, while an ending event is represented by a darker circle. A middle event is represented by a double circle (not visible). Events can be clock events, enabling the periodic execution or time-out of workflows.
3. A gateway is symbolised by a diamond. A gateway is a decision-making point in the

Notes

process. For instance, a decision is made in the above Figure based on the availability of rooms.

4. A dashed line indicates the flow of messages between activities; in the above figure, these messages are transmitted between the customer and the hotel booking service. A solid arrow indicates the order of activities.

The majority of workflows may be summarised by these essential components. These enhance a business process description with details that enable an executable form to be automatically generated. Consequently, a business process model can be used to construct web services based on service compositions that are defined in BPMN.

The procedure used by one organisation—a booking service provider—is depicted in the above figure. But a service-oriented approach's main advantage is that it facilitates inter-organisational computing. This indicates that services from several businesses are included in the total computation. Creating unique workflows with interactions for each of the participating organisations is how BPMN represents this.

We'll use a different example from grid computing to demonstrate this. It has been suggested to use a service-oriented approach to enable the sharing of resources like powerful computers. Assume for the purposes of this example that a research lab is providing a vector processing computer (a device that can do parallel computations on arrays of values) as a service (VectorProcService). SetupComputation is a different service that allows access to this. The figure below illustrates these services and how they interact.

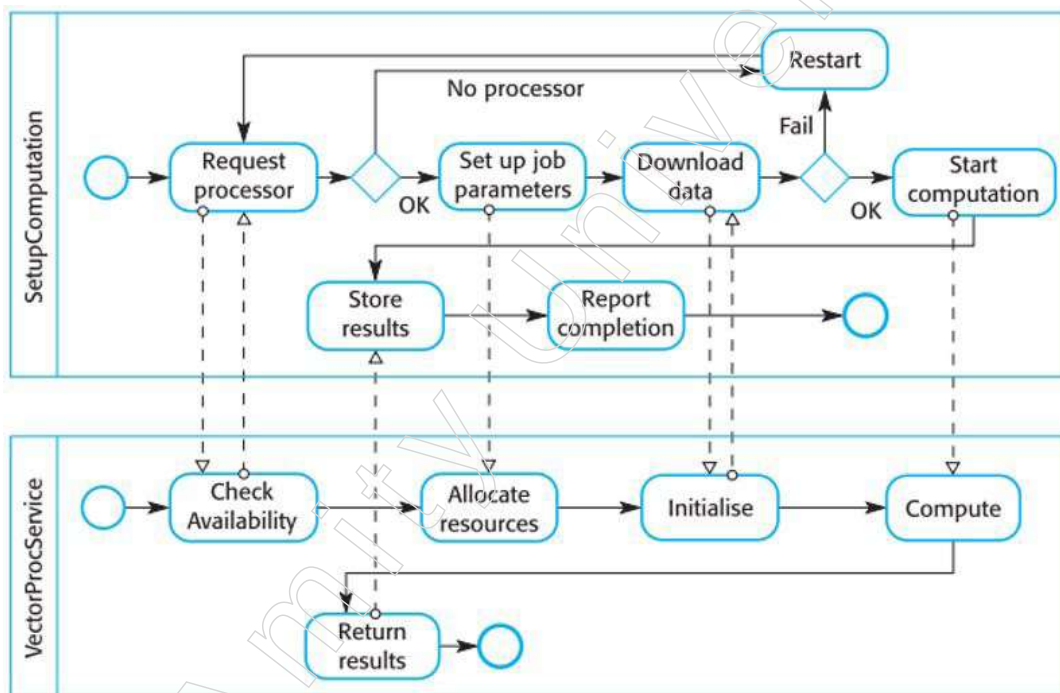


Figure: Interacting workflows

Image Source: Software Engineering by roger s. pressman Eighth Edition

The workflow for the SetupComputation service in this example determines the necessary computation and downloads data to the processing service after requesting access to a vector processor, if one is available. The output is saved locally on the computer after the computation is finished. The VectorProcService workflow first determines whether a processor is available, then it sets up the system, starts the computation, completes the computation and sends the results back to the client service.

Notes

Each organisation's workflow is represented in a different pool in BPMN. The process is represented visually by enclosing each participant's workflow in a rectangle and writing their name vertically on the left border. Each pool's stated workflows are coordinated through message exchanges; sequence flow between activities inside different pools is prohibited. When many departments within an organisation are participating in a workflow, this can be demonstrated by dividing pools into designated "lanes." Every lane displays the operations within that division of the company.

Upon designing a business process model, it must be further developed in light of the services identified. The model could go through several iterations until a design is produced that maximises the amount of services that can be reused. The next step is to turn this concept into an executable program once it is available. This can be built in any language because services are implementation-language agnostic and web service composition is supported in both the Java and C# development environments.

Numerous web service standards have been developed in order to directly enable the implementation of web service compositions. The most well-known of these is called WS-BPEL, or Business Process Execution Language. It is a "programming language" based on XML that regulates how services communicate with one another. Further standards like WS-Coordination, which describes how services are coordinated and WS-CDL (Choreography Description Language), which describes the messages sent between participants, complement this.

Since these are all XML standards, the lengthy and challenging to comprehend descriptions that arise. Direct program writing in XML-based notations is laborious and prone to mistakes. Because they are not necessary to comprehend the concepts of workflow and service-composition. These XML descriptions will be generated automatically as support for service-oriented computing gains traction. A graphical workflow description will be parsed by tools, which will then provide executable service compositions.

Service Testing

In order to show that a system satisfies its functional and non-functional criteria and to find errors that have been introduced throughout the development process, testing is crucial to all system development processes. To aid in the testing process, a variety of methods for system validation and testing have been created. A lot of these methods, including coverage testing and program inspections, depend on examining the source code of the product. Nevertheless, the source code for the service implementation is unavailable when services are provided by an outside source. Thus, tried-and-true source code-based methodologies cannot be used to service-based system testing.

When testing services and service compositions, testers may encounter other challenges in addition to comprehension issues with the service's operation:

1. Instead of the service user, the service provider is in charge of external services. Any past testing experience is nullified by the service provider's right to modify or cancel these services at any moment. Different versions of software components are maintained to address these issues. To cope with service versions, however, no standards have yet been established.
2. The long-term goal of service-oriented architectures is to dynamically bind services to applications that are service-oriented. This implies that an application might not always run on the same service every time. Because of this, tests that bind an application to a specific service may succeed, but there is no guarantee that the system will really use that service when it is being executed.

Notes

3. Because services are typically made available to a variety of users, the nonfunctional behaviour of a service cannot always be attributed to the way the application under test uses it. When undergoing testing, a service might function properly since it isn't experiencing a high load. Due to requests from other users, the reported service behaviour may differ in practice.
4. Service testing could be quite costly due to the services' payment model. There are various alternative payment methods: certain services can be offered for free, while others might require a subscription or be paid for on an as-needed basis. When a service is free, the provider will not want applications that are testing it to load it; when a subscription is needed, a service user might be hesitant to sign up before testing the service; and when usage is fee-based, users might find the cost of testing to be extremely high.
5. We've talked about compensation proceedings that are brought about when something unusual happens and prior agreements (like a plane ticket) need to be cancelled. Testing such actions presents a challenge because they can be dependent on other services failing. It may be rather challenging to guarantee that these services genuinely malfunction within the testing phase.

When outside services are used, these issues become much more severe. When services are used within the same organisation or when cooperating enterprises have faith in the services provided by their partners, they are less serious. In these situations, paying for services is probably not going to be an issue and source code might be accessible to help with the testing process. Currently, there is a significant research focus on finding solutions to these testing issues and developing standards, instruments and methods for testing service-oriented systems.

Summary

- Software engineering for client-server systems involves applying systematic principles to design, develop and maintain applications in a distributed computing environment. It encompasses requirements analysis, architecture design, development of client and server components, implementation of communication protocols, data management, security measures, rigorous testing, deployment and ongoing maintenance. Key considerations include scalability, load balancing, security and documentation to ensure the creation of reliable, efficient and secure client-server systems.
- Peer-to-Peer architecture provides a distributed and scalable model for resource sharing, offering advantages such as decentralisation and redundancy. However, it also presents challenges related to security, resource management and dynamic nature that need to be addressed in design and implementation.
- TCP/IP stands for Transmission Control Protocol/Internet Protocol. It is a suite of communication protocols used to interconnect network devices on the internet and other computer networks. TCP/IP provides a reliable, end-to-end communication service, defining how data should be formatted, addressed, transmitted, routed and received. It includes protocols such as TCP (Transmission Control Protocol) and IP (Internet Protocol), among others, which work together to facilitate communication between devices in a network.
- The Web Engineering Process is iterative and activities in different phases may overlap. Adopting agile methodologies can enhance flexibility, allowing teams to respond to changing requirements efficiently. Additionally, continuous monitoring and improvement are key aspects of successful web engineering.

Notes

- Service-Oriented Software Engineering provides a framework for creating flexible, interoperable and scalable software systems by organising functionality into modular and reusable services. It is particularly valuable in complex and dynamic business environments.
- Software development with services promotes modularity, reusability and scalability, enabling organisations to build flexible and interoperable applications that can adapt to changing business needs. This approach is commonly associated with Service-Oriented Architecture (SOA) and microservices architecture.

Glossary

- HTTP: Hypertext Transfer Protocol
- REST: Representational State Transfer
- API: Application programming interface
- TCP/IP: Transmission Control Protocol/Internet Protocol
- LAN: Local Area Network
- GUI: Graphical User Interface
- COTS: Commercial Off-The-Shelf
- OODBMS: Object-Oriented Database Management System
- RDMS: Relational Database Management System
- P2P: Peer to Peer
- CPU: Central Processing Unit
- BPEL: Business Process Execution Language
- SOA: Service-Oriented Architectures
- CI/CD: Continuous Integration/Continuous Deployment
- IaC: Infrastructure as Code

Check Your Understanding

1. What is a characteristic feature of Peer-to-Peer (P2P) Architecture?
 - a. Centralised control
 - b. Hierarchical structure
 - c. Client-server model
 - d. Distributed decision-making
2. Which of the following is a common communication protocol used in Peer-to-Peer networks?
 - a. FTP (File Transfer Protocol)
 - b. SMTP (Simple Mail Transfer Protocol)
 - c. SOAP (Simple Object Access Protocol)
 - d. P2P (Peer-to-Peer) protocol
3. In Peer-to-Peer systems, what is the primary purpose of service discovery?
 - a. Load balancing
 - b. Security enforcement
 - c. Resource sharing

Notes

- d. Dynamic peer identification
- 4. What is the role of an Enterprise Service Bus (ESB) in a Peer-to-Peer architecture?
 - a. Centralised control of services
 - b. Decentralised routing of messages
 - c. Peer authentication
 - d. Service discovery mechanism
- 5. Which type of Peer-to-Peer architecture involves collaboration between peers without a central controller?
 - a. Unstructured Peer-to-Peer
 - b. Structured Peer-to-Peer
 - c. Hybrid Peer-to-Peer
 - d. Decentralised Peer-to-Peer

Exercise

- 1. What do you mean by software engineering for client server systems?
- 2. Define:
 - a. Peer to Peer Architecture
 - b. Service Oriented Software Engineering
 - c. Service Engineering
 - d. Software Development with Services
- 3. Define various software testing issues
- 4. Define WebE Process.
- 5. Explain the framework for WebE

Learning Activities

- 1. Imagine you are developing a web application that requires integration with multiple external services. These services include a payment gateway, a user authentication service and a notification service. Discuss the steps and considerations you would take to integrate these services into your application, ensuring they function as reusable components.
- 2. Imagine you are tasked with designing and implementing a P2P file-sharing network similar to BitTorrent. Outline the key components, protocols and steps involved in building this P2P network. Consider aspects such as peer discovery, file distribution and handling of data integrity. Discuss the challenges you might encounter and propose solutions.

Check Your Understanding- Answers

- 1. d) 2. d) 3. d) 4. a)
- 5. a)

Module - V: Re-engineering and CASE

Learning Objectives

At the end of this module, you will be able to:

- Define business process re-engineering and software re-engineering
- Understand the building block of case
- Analyse forward re-engineering and economics of re-engineering
- Explain reverse engineering and restructuring engineering
- Define taxonomy of case tools
- Understand integration architecture and case repository

Introduction

Applications Software can be updated by reengineering without compromising its usefulness. It is a software development approach used to increase a software system's maintainability. Re-engineering is the process of dissecting and modifying a system to reconstruct it in a different way. In order to improve and streamline the software experience, this process may involve adding features and functionalities that are either necessary or optional. It has a favourable impact on software cost, quality, customer service and delivery speed.

Need of software Re-engineering

- Processes in continuity: While software is being tested or developed, earlier software products can still be utilised with their functionality.
- Boost up productivity: Through speedier processing of the code and database, software reengineering increases productivity.
- Reduction in risks: Here, developers improve the software product from its current state to improve certain elements that stakeholders or consumers have expressed concern about, rather than starting from scratch or the beginning.
- Drastic change in technology: In the field of IT, it is not uncommon for once-promising technologies to give way to more sophisticated and effective competitors. The market is always changing and reengineering becomes necessary if the organisation wants to stay up to date with technology.
- Saves time: As previously mentioned, software engineering takes less time because the product is created from the existing stage rather than the beginning.
- Optimisation: Through constant optimisation to the greatest extent feasible, this approach improves the system's features and functionalities while lowering the product's complexity.

In reverse engineering, the current system is examined to recapture the requirements (together with all connections and dependencies), data structure, data design and software component interfaces. Business analysts conduct interviews with programmers and stakeholders to gather missing information on the application's status. They also review the application's current documentation, analyse its lexical and syntactic code, look into control and data flows and consider use cases and test cases.

CASE (Computer-Aided Software Engineering) software systems designed to assist software process tasks automatically.

For method support, CASE systems are frequently employed:

Upper- CASE: Tools to assist with requirements and design-related early process activities.

Lower- CASE: instruments to assist with subsequent tasks like testing, debugging and programming. Model descriptions: These are summaries of graphical models that need to be created.

The use of computer-assisted tools and techniques in software development is known as computer-aided software engineering, or CASE. To guarantee flawless and high-quality software, CASE is utilised. Designers, developers, testers, managers and others can view the project milestones during development with the use of CASE, which guarantees a check pointed and disciplined approach.

Additionally, CASE can serve as a repository for project-related materials such as requirements, design specifications and business strategies. Delivering the finished product, which is more likely to satisfy real-world needs since it guarantees that consumers stay involved in the process, is one of the main benefits of adopting CASE.

A vast array of labor-saving tools used in software development are illustrated by CASE. It helps to increase efficiency by establishing a framework for project organisation. Years ago, there was greater interest in the concept of CASE tools; today, however, that enthusiasm has diminished because the tools have evolved into new roles, frequently in response to the needs of software developers. When it was announced, the CASE concept was likewise met with a great deal of criticism.

5.1 Re-engineering

Application Software Re-engineering is a software development method used to increase a software system's maintainability. Re-engineering is the process of dissecting and modifying a system to reconstruct it in a different way. This process is made up of several smaller operations, such as rebuilding, forward engineering and reverse engineering.

Re-engineering is the process of examining, creating and altering current software systems in order to enhance their quality, functionality and maintainability. It is sometimes referred to as reverse engineering or software re-engineering. This can involve adding new features, upgrading the software's basic architecture and design, or updating the program to support new hardware or software platforms.

Software re-engineering is the process of enhancing or upgrading current software systems to raise their level of quality, maintainability, or functionality. It is also referred to as software restructuring or software renovation. It entails altering already-existing software artifacts—such as code, designs and documentation—to satisfy newly created or modified requirements while reusing them.

Enhancing the software system's quality and maintainability while lowering the risks and expenses related to starting from scratch is the major objective of software re-engineering. Software re-engineering might start for a number of reasons, including:

- Improving software quality: Re-engineering can assist raise software quality by removing flaws, boosting dependability and maintainability and optimising performance.
- Updating technology: By upgrading the technology used in the system's development, testing and deployment, re-engineering can aid in modernising the software system.

Notes

- Enhancing functionality: Re-engineering can help increase the software system's functionality by introducing new features or enhancing current ones.
- Resolving issues: Re-engineering can assist in resolving problems with security, scalability and interoperability.

The following steps are involved in the software re-engineering process:

- Planning: The re-engineering process must first be planned, which entails determining the process's goals and objectives, defining its scope and determining the reasons behind the re-engineering.
- Analysis: Analysing the current system, including the code, documentation and other artifacts, is the next stage. This includes determining the system's advantages and disadvantages as well as any problems that require fixing.
- Design: The next stage is to develop the new or updated software system based on the analysis. This entails determining the adjustments that must be made and creating a strategy to put them into action.
- Implementation: The following stage involves putting the changes into practice by updating the documentation and other artifacts, adding new features and changing the current code.
- Testing: The software system must be tested once the modifications have been made to make sure it satisfies the updated requirements and standards.
- Deployment: Deploying the redesigned software system and making it accessible to end users is the last stage.

5.1.1 Business Process Re-engineering and Software Re-engineering

Software engineering and information technology are only small parts of business process reengineering (BPR). BPR has been defined in a variety of ways, most of them somewhat abstract. Fortune magazine defined it as "the search for and the implementation of, radical change in business process to achieve breakthrough results."

Business Processes

"A set of logically related tasks performed to achieve a defined business outcome" is the definition of a business process. People, machinery, material resources, business procedures and equipment are all joined inside the business process to achieve a specific outcome. Purchasing services and materials, hiring new staff, paying suppliers and creating new products are a few examples of business processes. Each requires a certain set of work and makes use of various resources available to the company.

A specific individual or group that receives the output (such as an idea, report, design, or product) is known as the customer for each business process. Business processes also transcend organisational boundaries. They stipulate that the "logically related tasks" that define the process must be completed by members of various organisational groups.

As we observed, all systems are really composed of a hierarchy of subsystems. A company is not an exception. The following is how the entire company is divided into segments:

- ❖ The business
- ❖ business systems
- ❖ business process
- ❖ business subprocesses

One or more business processes make up each business system (also known as a business function) and each business process is defined by a group of related procedures.

Although BPR can be implemented at any level of the hierarchy, its hazards increase significantly as its application becomes more widespread and as we advance up the hierarchy. Because of this, the majority of BPR initiatives concentrate on specific processes or subprocesses.

Principles of Business Process Reengineering

Business Process Reengineering (BPR) and business process management (BPE) share many similarities. BPR should ideally take place in a top-down fashion, starting with the identification of the main aims and objectives of the organisation and ending with a much more thorough description of the tasks that constitute a particular business process.

Hammer offers several guidelines for BPR initiatives that start at the highest (company) level, including the following:

Organize around outcomes, not tasks. Many businesses have divided up their operations such that no one individual or group is in charge of or in charge of a particular business outcome. In these situations, it can be challenging to ascertain the state of the job and, in the event that process issues arise, even more so to troubleshoot. BPR needs to create procedures that steer clear of this issue.

Have those who use the output of the process perform the process. This advice aims to give those who require business output complete control over all the factors that affect their ability to receive the output on time. The path to a quick conclusion is smoother when there are fewer distinct stakeholders participating in the process.

Incorporate information processing work into the real work that produces the raw information. Most information processing can be found inside the company that generates the raw data as IT becomes increasingly dispersed. This places processing power in the hands of those who have a stake in the information generated, localises control and shortens transmission times.

Treat geographically dispersed resources as though they were centralised. With the advancement of computer-based communications, organisations from different geographical locations can work together in a single “virtual office.” For instance, a multinational corporation can operate one engineering shift in Europe, one in North America and one in Asia rather than three shifts at one site. Engineers will collaborate via high-bandwidth networks and work during the day in each scenario.

Link parallel activities instead of integrating their results. When various stakeholders carry out tasks concurrently, it's critical to create a process that necessitates ongoing collaboration and communication. Integration issues will undoubtedly arise otherwise.

Put the decision point where the work is performed and build control into the process. This principle proposes a flatter organisational architecture with less factoring, using technical terms from software design.

Capture data once, at its source. Online data storage makes it unnecessary to ever reenter data once it has been gathered.

A “big picture” perspective of BPR is represented by each of these tenets. Process designers and business planners need to start rethinking processes based on these guidelines. We look more closely at the BPR process in the following section.

Notes

A BPR Model

Business process reengineering is an iterative process, just like other engineering endeavours. A dynamic business environment necessitates the adaptation of corporate objectives and the procedures that achieve them. BPR is an evolving process, thus it lacks a beginning and an end. The figure below shows a business process reengineering model. Six actions are defined by the model:

Business definition. Four primary drivers are considered when identifying business goals: time and cost savings, quality enhancement, employee development and empowerment. Objectives might be specified at the corporate level or for a particular division of the company.

Process identification. Procedures that are necessary to accomplish the objectives listed in the business definition are noted. Next, they can be ordered according to significance, the necessity for modification, or any other criterion that makes sense for the reengineering project.

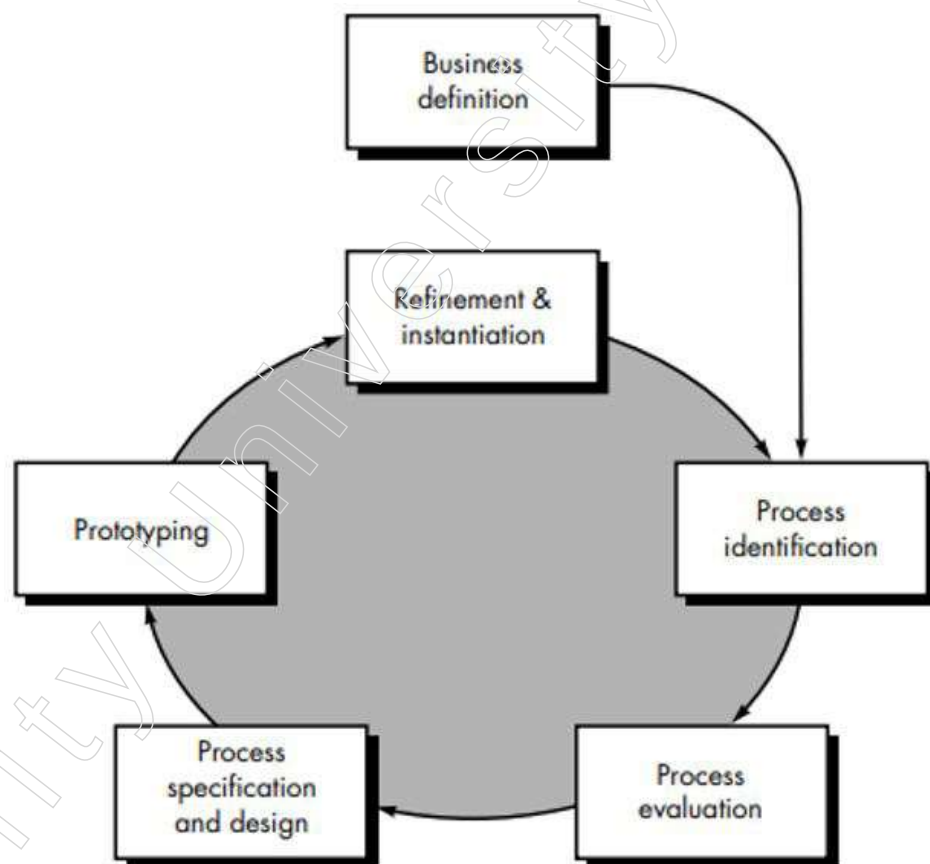


Figure: A BPR model

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Process evaluation. The current procedure is measured and carefully examined. Process tasks are identified, quality/performance issues are separated and the expenses and time incurred by process tasks are recorded.

Process specification and design. Use-cases are created for every process that has to be changed using data gathered from the first three BPR activities. Use-cases in the framework of BPR pinpoint a situation that provides a client with a certain result. A new collection of tasks with the use-case serving as the process definition.

Prototyping. Before a modified business process is completely incorporated into the company, it needs to be prototyped. In order to make improvements, this task “tests” the procedure.

Refinement and instantiation. After the prototype is developed, the business process is implemented within a business system based on user feedback.

Workflow analysis tools are occasionally utilised in conjunction with these BPR tasks. By creating a model of the current workflow, these technologies aim to improve the analysis of current operations. The first four activities in the process model can also be implemented using modelling techniques that are frequently linked with business process engineering activities, such as business area analysis and information strategy planning.

Words of Warning

It happens frequently that a novel business strategy—BPR in this case—is originally heralded as a cure-all before facing such harsh criticism that it is written off as a pariah. The effectiveness of BPR has been hotly debated throughout the years. Weisz provides a thorough analysis of the arguments for and against BPR, which he summarises as follows:

It is easy to criticise BPR as just another gimmick. You would have to anticipate high failure rates for the concept from a number of perspectives—systems thinking, people ware, simple history—rates that appear to be supported by real data. It appears like the silver bullet has missed a lot of companies. However, it appears that the reengineering work has paid off for others.

If motivated, skilled individuals who understand that process reengineering is an ongoing endeavour implement BPR, it can be successful. Information systems are better integrated into business processes when business process reengineering (BPR) is done correctly. It is possible to consider reengineering older programs within the framework of a comprehensive business plan and to wisely determine software reengineering priorities.

Software reengineering, however, is a necessary tactic even if business reengineering is rejected by an organisation. Tens of thousands of legacy systems—applications essential to both small and large enterprises’ success—need to be completely rebuilt or renovated.

Software Reengineering

The situation is all too typical: An application has satiated a company’s business requirements for ten or fifteen years. It has undergone numerous revisions, modifications and enhancements during that time. Despite the best of intentions, good software engineering techniques were consistently ignored in favour of other pressing issues when working on this project. The application is unstable right now. It still functions, but if any changes are made, unanticipated and detrimental side consequences happen. However, the application needs to keep changing. How should one proceed? The issue of unmaintainable software is not new. In actuality, a software maintenance “iceberg” that has been accumulating for more than three decades is what gave rise to the growing emphasis on software reengineering.

Software Maintenance

A term used to describe software maintenance thirty years ago was “iceberg.” While we hope that the initial appearance is all that exists, there is a vast amount of possible issues and expenses hidden beneath the surface. The maintenance iceberg was large enough to sink an aircraft carrier in the early 1970s. It could easily drown the whole navy today!

Notes

Over sixty percent of a development organisation's labour can go towards maintaining software that already exists and that number only goes up as more software is developed. Readers who are not familiar with the subject may wonder why there is such a high need for maintenance and effort. Osborne and Chikofsky offer a succinct response:

The majority of the software we use every day is, on average, between 10 and 15 years old. Program size and storage capacity were major considerations when these programs were designed, even if most of them did not use the finest design and coding approaches available at the time. Without giving overall architecture adequate thought, they were then moved to new platforms, modified to account for advancements in hardware and operating systems and improved to satisfy evolving user requirements.

As a result, the software systems that we are now expected to maintain have badly written documentation, badly written coding, badly thought out structures and poor reasoning.

The foundation of any software development is the ubiquitous nature of change. When computer-based systems are designed, change is unavoidable; consequently, we need to create systems for assessment, management and adjustment.

Some readers could object, saying, "But We don't spend 60 percent of my time fixing mistakes in the programs we develop." after reading the previous lines. Of course, software maintenance entails much more than simply "fixing mistakes." Four tasks that are carried out following the release of a program for usage can be used to define maintenance. Corrective maintenance, adaptive maintenance, perfective maintenance or improvement and preventive maintenance or reengineering are the four distinct maintenance tasks that we have identified. Just 20% of all maintenance tasks are devoted to "fixing mistakes." The remaining 80% is devoted to reengineering a program for future usage, implementing user-requested additions and adjusting current systems to changes in their external environment. When all of these tasks are included under maintenance, it becomes rather clear why it requires so much work.

A Software Reengineering Process Model

Reengineering consumes resources that may be used for more pressing issues, takes a long time and costs a lot of money. Reengineering takes time—months or even years—to complete for all of these reasons. For many years to come, information technology resources will be devoted to the reengineering of information systems. For this reason, every company requires a realistic software reengineering plan.

An effective approach is included in a reengineering process model. Reengineering is a rebuilding process and by comparing it to the reconstruction of a house, we can have a better understanding of the reengineering of information systems. Think about the following circumstance.

You bought a home in a different state. Although you've never really seen the house, you bought it at an incredible discount with the understanding that it might need to be fully renovated. How would you go about it?

- It would seem sense to inspect the house before you begin to rebuild. You (or a qualified inspector) would make a list of requirements so that your inspection would be methodical and ascertain whether it needs to be rebuilt.
- Verify that the house's structure is weak before demolishing and starting over. It might be feasible to "remodel" rather than rebuild if the house is structurally sound (at considerably lower cost and in much less time).

- Make sure you comprehend how the original was constructed before you begin to rebuild. Peer around the corners. Recognise the plumbing, electrical and structural internals. When building begins, the knowledge you will acquire will be useful, even if you destroy them all.
- If you decide to reconstruct, make sure to utilise only the most durable, contemporary materials. Although there may be a slight up-front expense, doing this now can save costly and time-consuming maintenance down the road.
- Rebuilding requires discipline, so approach it that way. Adopt procedures that will yield excellent work both now and in the future.

These ideas are centred around house reconstruction, but they also hold true when reengineering computer-based programs and systems.

We use a software reengineering process model, which is depicted in the figure below, which outlines six tasks, to put these concepts into practice. These tasks don't usually happen in a straight line; sometimes they do. For instance, it could be necessary to perform reverse engineering—that is, to comprehend the inner workings of a program—before beginning document reorganisation.

The reengineering paradigm that is depicted in the figure follows a cycle. This implies that any task listed inside the paradigm could be reviewed. Any one of these actions can cause the process to end for that particular cycle.

Inventory analysis. An inventory of all programs ought to be kept by any software organisation. All that is required for the inventory to function is a spreadsheet model with all the data needed to provide each current application a thorough description (such as its size, age and business criticality).

Candidates for reengineering emerge when this data is sorted based on factors like as lifespan, business criticality, current maintainability and other locally significant criteria. Following that, resources for potential reengineering work applications might be assigned.

It is crucial to remember that the inventory needs to be reviewed on a regular basis. Reengineering priorities will fluctuate in response to changes in the status of applications (e.g., business criticality).

Document restructuring. Poor documentation is a common characteristic of many older systems. However, how should we address it? What choices do we have?

1. The process of creating documentation takes far too long. We will make do with what we have if the system functions. Sometimes, this is the best course of action. Rewriting the documentation for hundreds of computer programs is not feasible. Let a program go if it is largely stagnant, nearing the end of its useful life and not likely to alter significantly!
2. We need to update the documentation, but our resources are limited. Our strategy will be to “document when touched.” It might not be required to redo an application in its entirety. Instead, the extensively documented parts of the system are the ones that are currently changing. A body of pertinent and helpful documentation will build up over time.
3. Given the system's importance to the business, all documentation has to be updated. Reducing documentation to bare minimum is a wise move even in this situation.

All these solutions are feasible. A software company needs to select the best option for every situation.

Notes

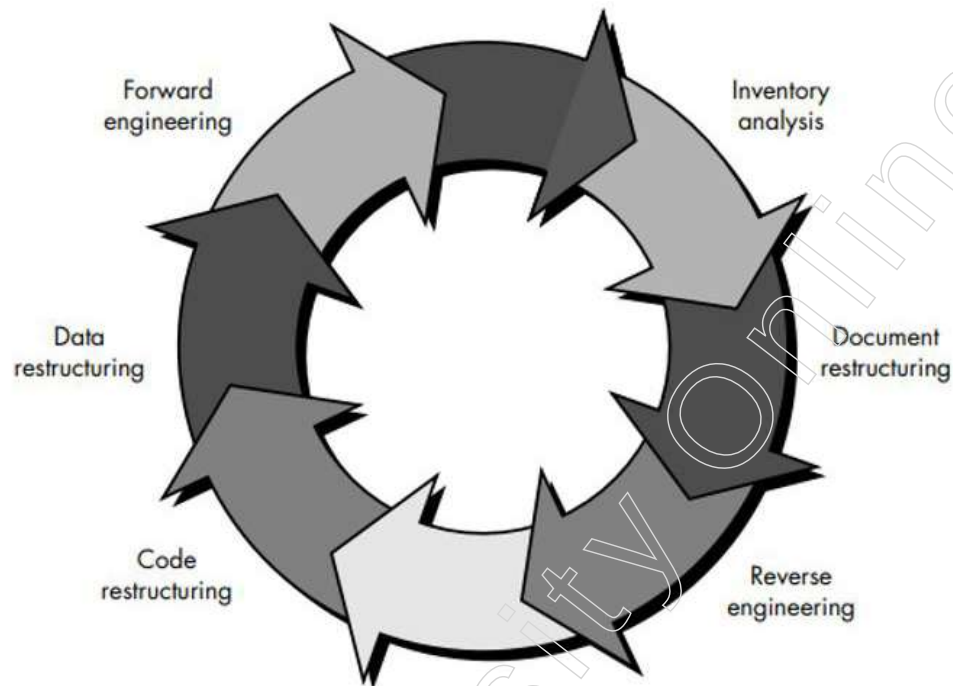


Figure: A software reengineering process model

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Reverse engineering. The hardware industry is where the term “reverse engineering” first appeared. A business disassembles a rival hardware product to try and figure out the “secrets” of its production and design. If one were to access the design and manufacturing specifications of the opponent, these secrets would be clearly understood. However, the company conducting the reverse engineering cannot access these records since they are proprietary. Reverse engineering that works basically looks at real product specimens to extract one or more design and manufacturing requirements for a product.

Software reverse engineering is very similar. However, the program that needs to be reverse-engineered is typically not one that belongs to a rival. Instead, it is the company's own work that was frequently completed years previously. The lack of a specification means that the “secrets” to be grasped are unclear. Thus, the process of examining a program to produce a representation of the program at a higher level of abstraction than source code is known as reverse engineering for software. Design recovery is the process of reverse engineering. Tools for reverse engineering take data out of an existing program, including procedural and architectural design information.

Code restructuring. Code restructuring is the most popular kind of reengineering—in fact, it's debatable whether the name “reengineering” is appropriate in this context. While the program design of certain older systems is rather sound, the coding style of some of the individual modules makes them challenging to debug, test and maintain. The code in the dubious modules may be reorganised in such circumstances.

A restructuring tool is used to analyse the source code in order to do this task. When structured programming constructs are broken, the code is automatically reorganised and reported. To make sure that no abnormalities have been introduced, the restructured code that results is examined and evaluated. The documentation for internal codes has been revised.

Data restructuring. Improving and making adjustments to a program with a poor data architecture will be challenging. The long-term survival of a program is often determined more by data architecture than by the source code itself, in many cases.

Data structuring is a comprehensive reengineering process, in contrast to code restructuring, which takes place at a lower level of abstraction. Most often, reverse engineering is the first step in the data rearrangement process. The existing data architecture is broken down and the required data models are established. Existing data structures are examined for quality and data objects and attributes are discovered.

The data are reengineered when the data structure is poor (for example, when flat files are used now, even if a relational approach would significantly ease processing). Changes to the data will inevitably lead to either architectural or code-level changes because data architecture strongly influences program architecture and the algorithms that populate it.

Forward engineering. An automated “reengineering engine” would be used in a perfect world to rebuild applications. The outdated program would be loaded into the engine, which would then analyse, reorganise and build a new version that showcased the greatest features of high-quality software. While it is unlikely that such a “engine” would materialise in the near future, CASE manufacturers have launched tools that offer a constrained subset of these capabilities and cater to particular application domains (e.g., applications that are implemented using a specific database system). What’s more, the sophistication of these reengineering technologies is rising.

In an effort to enhance the overall quality of the current system, forward engineering, also known as renovation or reclamation, not only retrieves design information from the software that already exists but also modifies or reconstitutes it. Reengineered software typically adds new features, enhances overall performance and reimplements the functionality of the original system.

5.1.2 Introduction to Building Blocks of CASE

Computer-aided software engineering (CASE) tools assist software engineering managers and practitioners in every activity associated with the software process. They oversee all work products generated during the process, automate project management tasks, and support engineers with their analysis, design, coding, and testing needs.

It is possible to incorporate CASE tools into an advanced environment. Software engineers can increase their engineering insight and automate tedious tasks with CASE.

When it comes to reengineering, CASE (Computer-Aided Software Engineering) describes how software tools are used to help with software system creation and maintenance. The requirements analysis, design, coding, testing, and maintenance phases of the software development lifecycle are all supported by CASE tools.

The fundamental elements of CASE can be divided into multiple major categories:

1. **Diagramming Tools:** Using flowcharts, data flow diagrams, entity-relationship diagrams, and UML diagrams, these tools assist in visualizing the software requirements, design, and architecture.
2. **Tools for Analysis and Design:** These resources aid in the requirements analysis and software system design. They frequently have functionality for modelling, simulation, and requirements management.
3. **Code Generation Tools:** By automating the process of creating code from design models, these tools lessen the amount of human labour needed to code.
4. **Testing Tools:** These tools support the development and implementation of test cases to guarantee that the program satisfies specifications and operates as intended.

Notes

5. Configuration management Tools: Solutions, such as version control, configuration control, and change management, aid in the administration of software modifications.
6. Tools for Documentation: These resources support the production and upkeep of user manuals, technical documentation, and design documents for the software.
7. Tools for project management: These resources support the organizing, arranging, and monitoring of software development projects.
8. Collaboration Tools: These tools make it easier for team members to work together on the software project by facilitating task management, file sharing, and communication.

Organizations may produce high-quality software products, increase productivity, and streamline the software development process by making efficient use of these building pieces.

5.1.3 Forward Re-engineering and Economics of Re-engineering

290,000 source statements contain few significant comment lines, “modules” with a control flow the visual equivalent of a bowl of spaghetti and no further documentation that has to be changed to account for evolving user needs. The following choices are available to us:

1. To make the required adjustments, we can combat the implicit design and source code modification after modification.
2. To make changes more successfully, we can try to comprehend the program’s more intricate inner workings.
3. We can take a software engineering approach to all altered segments of the program, redesigning, recoding and testing the parts that need to be changed.
4. We can use CASE (reengineering) tools to help us comprehend the current design and then entirely rebuild, recode and test the program.

There isn’t just one “correct” answer. Even when the other options are preferable, the first one may be required by the circumstances.

Using the results of inventory analysis, the development or support organisation chooses a program that will (1) last for a predetermined number of years, (2) is being used successfully right now and (3) is probably going to undergo significant modification or enhancement soon, rather than waiting to hear about a maintenance request. Option 2, 3, or 4 is then used.

Miller invented this method of preventive maintenance, which they called structured retrofit. According to its definition, this idea is “the application of today’s methodologies to yesterday’s systems to support tomorrow’s requirements.”

The idea of completely rewriting a major program when there is already a functional version may sound very bold at first. Prior to making a decision, take into account the following:

1. One source code line’s maintenance expenses can range from 20 to 40 times the line’s original development costs.
2. Future maintenance can be made much easier by redesigning the software architecture (program and/or data structure) using contemporary design principles.
3. The software already has a prototype, thus development productivity should be significantly greater than typical.

4. The user is now familiar with the program. As a result, it is easier to determine the direction of change and new requirements.
5. Some of the work will be automated by CASE reengineering tools.
6. After preventive maintenance is finished, a complete software configuration (documents, programs and data) will be present.

“New releases” of a program are what constitute preventive maintenance when a software development company offers software as a product. A sizable in-house software development team (such as the business systems software development department of a major consumer products corporation) might be in charge of 500–2000 production programs. Prior to being evaluated as potential candidates for preventative maintenance, these programs can be prioritised.

The forward engineering technique recreates an existing application using the ideas, concepts and practices of software engineering. Forward engineering typically involves more than just producing an updated version of an older program. Rather, the reengineering process incorporates new user and technological requirements. The enhanced software expands upon the features of the previous version.

Forward Engineering for Client/Server Architectures

Many mainframe applications have been redesigned in the last ten years to support client/server architectures. Essentially, numerous client platforms share centralised computing resources, including software. While other distributed environments can be created, a mainframe program redesigned with a client/server architecture typically possesses the following characteristics:

- Application functionality migrates to each client computer.
- New GUI interfaces are implemented at the client sites.
- Database functions are allocated to the server.
- Specialised functionality (e.g., compute-intensive analysis) may remain at the server site.
- New communications, security, archiving and control requirements must be established at both the client and server sites.

It is crucial to remember that business and software reengineering are necessary for the transition from mainframe to c/s computing. Furthermore, the establishment of a “enterprise network infrastructure” is necessary.

The first step in reengineering for c/s applications is a detailed examination of the business environment that includes the current mainframe. There are three distinct layers of abstraction. A client/server architecture is built on a database, which also handles queries and transactions from server apps. However, these queries and transactions need to be managed within the framework of a set of business rules, which are established by a redesigned or current business process. The user community can access specific functionality using client applications.

Before the database foundation layer is redesigned, the functionalities of the current database management system and the data architecture of the current database must be reverse engineered. Sometimes a brand-new data model is developed. Every time, the c/s database is redesigned to guarantee consistent transaction execution, that only authorised users can make updates, that core business rules are upheld (for example, the server makes sure that no related contracts, accounts payable, or communications for a vendor

Notes

exist before deleting a record), that queries can be handled quickly and that full archiving capability has been established.

Software that is installed on both the client and the server is represented by the business rules layer. In order to guarantee that queries and transactions between the client application and the database follow the defined business process, this software handles control and coordination duties. The business functions needed by particular end-user groups are implemented via the client applications layer. A mainframe application is often divided into several smaller, redesigned desktop applications. The business rules layer regulates, when needed, communication between the desktop apps. It is advisable to leave a thorough discussion of client/server software design and reengineering to books that specifically address the topic.

Forward Engineering for Object-Oriented Architectures

For many software companies, object-oriented software engineering has emerged as the preferred development paradigm. However, what about applications that are already in use that were created with traditional techniques? Leaving such apps “as is” is sometimes the best course of action. In other cases, it is necessary to rework older applications in order to make them easily integrable into sizable object-oriented systems. Rewriting traditional program to be implemented in an object-oriented manner. Reverse engineering the current program is the first step in creating the necessary functional, behavioural and data models. Use-cases are developed if the reengineered system expands upon the functionality or behaviour of the original application. The foundation for class definition is then established by combining CRC modelling with the data models generated during reverse engineering. Object-oriented design starts with the definition of class hierarchies, object-relationship models, object-behaviour models and subsystems.

A CBSE process model can be used to guide object-oriented forward engineering as it moves from analysis to design. It's possible that a strong component library exists and may be utilised during forward engineering if the current application runs in a domain that is already home to a large number of object-oriented apps. Algorithms and data structures from the current conventional application may be reusable for those classes that need to be redesigned from start. But in order to comply with the object-oriented architecture, these need to be modified.

Forward Engineering User Interfaces

As desktop applications replace mainframe programs, consumers are unable to put up with obscure, character-based user interfaces. In reality, reengineering client application user interfaces can account for a large percentage of the total effort invested in the shift from mainframe to client/server computing.

The model that Merlo and his associates recommend for reengineering user interfaces is as follows:

1. Recognise the data that moves between the original interface and the rest of the application. The goal is to comprehend the interactions between existing code that implements the interface and other program elements. The data that flow between the new GUI and the surviving program must match the data that flow between the program and the character-based interface at the moment if a new GUI is to be constructed.
2. Transform the behaviour suggested by the current interface into a set of meaningful abstractions within the GUI context. When viewed in the context of a usage scenario, the business behaviour displayed by users of the old and new interfaces must not change, despite the possibility of a drastically changed manner of interaction. The necessary

business behaviour must still be exhibited by a user on a revised interface. For instance, the previous interface might have required a lengthy string of text-based commands to specify the query when a database query was to be made. The purpose and content of the inquiry are still the same, even though the redesigned GUI may condense it to a short series of mouse clicks.

3. Introduce upgrades that increase the effectiveness of the interaction mode. The new GUI's design addresses and improves upon the ergonomic shortcomings of the current one.
4. Construct the new GUI and incorporate it. The amount of work needed to create the GUI can be greatly decreased by the availability of class libraries and fourth generation tools. Integration with already-existing application software, however, may take longer. It is important to take precautions to make sure that the GUI doesn't spread unfavourable side effects throughout the rest of the application.

Reverse engineering is the process of analyzing a product, system, or component to understand its design, functionality, and behavior, often with the goal of reproducing or modifying it. It involves examining the structure, behavior, and inner workings of a system through various techniques, such as examination, disassembly, and analysis of its components, code, or documentation. Reverse engineering can be applied to a wide range of artifacts, including software, hardware, mechanical devices, and even biological systems. Here are key concepts associated with reverse engineering:

- **Understanding Existing Systems:** Reverse engineering is used to gain insight into how existing systems work, especially when documentation or source code is not available. This allows engineers to understand legacy systems, proprietary formats, or third-party components.
- **Recovery of Design Information:** Reverse engineering helps extract design information from artifacts, such as software binaries, hardware components, or physical models. This includes identifying interfaces, protocols, algorithms, data formats, and other design elements.
- **Interoperability and Compatibility:** Reverse engineering enables the creation of interoperable or compatible systems that can interact with existing systems or data formats. By reverse engineering proprietary protocols or file formats, interoperability between different systems can be achieved.
- **Bug Fixes and Optimization:** Reverse engineering is used to identify and fix bugs, performance bottlenecks, or security vulnerabilities in existing systems. By analyzing the code or behavior of a system, engineers can identify and address issues that may not be apparent from the documentation alone.
- **Product Analysis and Competitive Intelligence:** Reverse engineering is employed to analyze competitor products or commercial off-the-shelf (COTS) components to understand their features, strengths, and weaknesses. This information can inform product development strategies and competitive positioning.
- **Legacy System Maintenance:** Reverse engineering helps maintain and extend legacy systems by providing insights into their architecture, dependencies, and constraints. This allows organizations to continue supporting and enhancing legacy systems that are critical to their operations.
- **Intellectual Property Protection:** Reverse engineering can be used defensively to protect intellectual property rights by identifying unauthorized use or infringement of proprietary technology. It helps detect reverse-engineered copies or clones of proprietary products, allowing legal action to be taken if necessary.

Notes

- **Ethical Considerations:** While reverse engineering can offer significant benefits, it also raises ethical considerations, particularly when it involves analyzing or modifying proprietary or copyrighted material without authorization. Engineers and organizations must adhere to legal and ethical guidelines when engaging in reverse engineering activities.

Economics of Re-engineering

In an ideal world, all unmaintainable programs would be decommissioned right away and replaced by superior, redesigned applications created with contemporary software engineering techniques. However, the resources in our world are scarce. Resources that may be applied to other company objectives are depleted by reengineering. Therefore, a cost/benefit analysis should be carried out by an organisation before attempting to reengineer an existing program. Sneed has suggested a cost/benefit analysis methodology for reengineering. There are nine defined parameters:

- P1 = current annual maintenance cost for an application.
- P2 = current annual operation cost for an application.
- P3 = current annual business value of an application.
- P4 = predicted annual maintenance cost after reengineering.
- P5 = predicted annual operations cost after reengineering.
- P6 = predicted annual business value after reengineering.
- P7 = estimated reengineering costs.
- P8 = estimated reengineering calendar time.
- P9 = reengineering risk factor (P9 = 1.0 is nominal).
- L = expected life of the system.

If reengineering is not done, the cost of maintaining a candidate application might be described as follows:

$$C_{\text{maint}} = [P_3 - (P_1 + P_2)] L$$

The following relationship is used to define the reengineering costs:

$$C_{\text{reeng}} = [P_6 - (P_4 + P_5) \times (L - P_8) - (P_7 - P_9)]$$

The total benefit of reengineering can be calculated using the expenses shown in the calculations above and

$$\text{cost benefit} = C_{\text{reeng}} - C_{\text{maint}}$$

All high-priority applications found during inventory analysis can undergo the cost/benefit analysis shown in the equations. Reengineering efforts can be directed towards the applications with the highest cost/benefit ratios, with work on the others being put off until resources are available.

5.1.4 Reverse Engineering and Restructuring Engineering

It is common to associate reverse engineering with the “magic slot.” We insert an unorganised, undocumented source listing into the slot and the computer program’s complete documentation emerges on the other end. Regretfully, there isn’t a magic slot. Design information can be extracted from source code by reverse engineering, but there are a lot of variables in the process, including the degree of abstraction, the documentation’s completeness, the degree to which tools and human analysts collaborate and the process’s directionality.

The sophistication of the design knowledge that can be gleaned from source code is referred to as the abstraction level of a reverse engineering process and the tools required to carry it out. The abstraction level ought to be as high as it gets. In other words, procedural design representations—a low level of abstraction—program and data structure information—a somewhat higher level of abstraction—data and control flow models—a relatively high level of abstraction—and entity relationship models—a high level of abstraction—should all be derived through the reverse engineering process. The software engineer is given knowledge that will make it easier for them to understand the program as the abstraction level rises.

The degree of detail offered at an abstraction level is referred to as the completeness of a reverse engineering process. Generally speaking, completeness diminishes with increasing abstraction level. For instance, creating a comprehensive procedural design representation is not too difficult when provided with a source code listing. While it is possible to extract simple data flow representations, creating an entire set of entity-relationship models or data flow diagrams is significantly more challenging.

The degree of analysis carried out by the reverse engineer directly relates to how complete the work is. The degree to which a human is “integrated” with automated technologies to produce a successful reverse engineering process is known as interaction. Generally speaking, completeness suffers when abstraction level rises and interactivity must rise instead.

All information retrieved from the source code is given to the software engineer if the reverse engineering process is one-way and they can utilise it for any maintenance task after that. In the event that directionality is bidirectional, the data is supplied to a reengineering tool, which endeavours to reconstruct or revitalise the previous program.

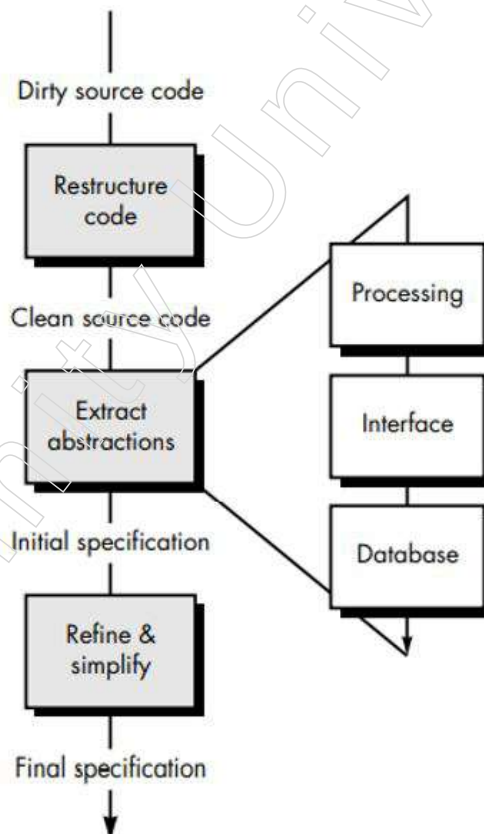


Figure: The reverse engineering process

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Notes

The figure above illustrates the reverse engineering procedure. Unstructured (or “dirty”) source code is reorganised to contain only structured programming structures before reverse engineering work may begin.² This facilitates reading the source code and serves as the foundation for all ensuing reverse engineering tasks.

The process of extracting abstractions is the foundation of reverse engineering. The engineer is required to assess the outdated program and derive a reasonable specification of the processing carried out, the user interface utilised and the program’s data structures or database from the (sometimes undocumented) source code.

Reverse Engineering to Understand Processing

Trying to comprehend and then extract procedural abstractions from the source code is the first real step in reverse engineering. The code is examined at different levels of abstraction, including system, program, component, pattern and statement, in order to comprehend procedural abstractions.

It is necessary to comprehend the application system’s overall functionality before delving deeper into the reverse engineering process. This creates a framework for additional investigation and sheds light on problems with application interoperability within the system. At a high degree of detail, every program that comprises the application system represents a functional abstraction. The relationship between these functional abstractions is shown by a block diagram that is made. Every element carries out a certain subtask and symbolises a predetermined procedural abstraction. Every component has a processing story written for it. System, program and component specifications are already in place in certain cases. In this situation, the specs are examined to make sure they comply with the current code.

When the code inside a component is taken into account, things get more complicated. The coder searches for chunks of code that illustrate common procedural patterns. Almost all components have three separate pieces of code: one prepares the data for processing (inside the module), another does the processing and a third portion prepares the processed data for export from the component. Smaller patterns can be found within each of these parts; for instance, bounds checking and data validation frequently take place within the code that gets the data ready for processing.

Reverse engineering for large systems is typically carried out using a semiautomated method. Code that already exists is “parsed” semantically using CASE tools. To finish the reengineering process, the output of this procedure is subsequently sent to restructure and forward engineering tools.

Reverse Engineering to Understand Data

Data reverse engineering takes place at several abstraction levels. Internal program data structures frequently require reverse engineering at the program level as a component of a larger reengineering endeavour. Global data structures, such as files and databases, are frequently redesigned at the system level to support new database management paradigms (such as the transition from flat file to relational or object-oriented database systems). Setting up a new system wide database requires reverse engineering of the existing global data structures. Internal data structures. The definition of classes of objects is the main focus of reverse engineering approaches for internal program data. Examining the program code with the goal of organising similar program variables is how this is done. Abstract data types are often identified by the way the data is organised within the code. Files, lists, record structures and other data structures, for instance, frequently include a preliminary class indicator.

Breuer and Lano recommend the subsequent method for class reverse engineering:

1. Within the program, locate flags and local data structures that store crucial details about global data structures (such as a file or database).
2. Describe the connection between the global and local data structures and flags. A local data structure might be used as a buffer to hold the most recent 100 records that were obtained from a central database, or a flag could be set in the event that a file is empty.
3. List all the variables that are logically related to each variable (inside the program) that represents an array or file.

A software engineer can find program classes that communicate with the global data structures by following these procedures.

Database structure. A database permits the definition of data objects and enables a mechanism for establishing relationships between the objects, regardless of the logical arrangement and physical structure of the database. As a result, comprehending current items and their relationships is necessary for reengineering one database schema into another.

Before reengineering a new database model, the current data model can be defined using the procedures listed below:

1. Build an initial object model. Examining data from tables in a relational schema or entries in a flat file database can yield the classes established as part of the model. Records and tables hold items that are assigned to a class as attributes.
2. Determine candidate keys. If the characteristics are used to point to another record or table, this is determined by looking at them. Those that act as identifiers become potential keys.
3. Refine the tentative classes. Check to see if classes that are close enough can be consolidated into one.
4. Define generalisations. To decide whether to create a class hierarchy with a generalisation class at the top, look at classes that share a lot of characteristics.
5. Discover associations. Make use of methods similar to the CRC approach to create relationships between classes.

Once the data defined in the previous steps is known, the old database structure can be mapped into a new database structure using a number of transformations.

Reverse Engineering User Interfaces

Complex graphical user interfaces (GUIs) are now standard for all computer-based systems and products. As a result, one of the most popular kinds of reengineering work nowadays is the redesign of user interfaces. However, reverse engineering needs to be done before a user interface is redesigned.

The structure and behaviour of a current user interface (UI) must be described in order to properly comprehend it. Three fundamental problems are posed by Merlo and colleagues that need to be addressed before reverse engineering of the user interface can begin:

- What fundamental inputs, like keystrokes and mouse clicks, does the interface need to process?
- What succinct explanation would you give of the system's behavioural reaction to these actions?

Notes

- What does “replacement” entail, or more specifically, which idea of interface equivalency matters in this context?

Answers to the first two questions can be developed using behavioural modelling notation. Seeing how the current interface appears on the outside provides a lot of the data needed to build a behavioural model. However, more data must be collected from the code in order to build the behavioural model. It is crucial to remember that a replacement graphical user interface (GUI) might not exactly look like the original one—in fact, it might look very different. It is frequently beneficial to create new interaction metaphors. For instance, an outdated user interface asks the user to select a scale factor (1–10) in order to enlarge or reduce a graphical image. A redesigned graphical user interface could combine a mouse and slide bar to perform the same task.

Restructuring

In order to make software more adaptable to future changes, restructuring adjusts the source code and/or data. Restructuring typically has little effect on the program’s overall architecture. It frequently concentrates on the specifics of each module’s design as well as the local data structures established within modules. Restructuring transforms into forward engineering if it crosses module boundaries and incorporates the software architecture.

Arnold lists several advantages that come from reorganising software, including:

- Programs are of a higher calibre because they follow contemporary software engineering standards and practices, have greater documentation and are simpler.
- Software engineers that have to work on the program experience less frustration, which boosts output and facilitates learning.
- The amount of work needed to complete maintenance tasks is decreased.
- Software is simpler to test and troubleshoot.

Restructuring happens when an application’s technical internals require improvement but its overall architecture is sound. It starts when the majority of the software is still functional and just a small portion of the modules and data require significant alteration.

Code Restructuring

Restructuring the code results in a design that outperforms the original program while still producing the same function. Restructuring code generally involves modelling program logic using Boolean algebra and then applying a set of transformation rules to produce restructured logic (e.g., Warnier’s logical simplification techniques). The goal is to take “spaghetti-bowl” code and use the structured programming philosophy to generate a procedural design.

Use of additional restructuring strategies in conjunction with reengineering tools has also been suggested. Each program module and the resources (variables, processes and data kinds) that are shared between it and other modules are mapped in a resource exchange diagram. It is possible to reorganise the program architecture to ensure minimal connection between modules by generating representations of resource flow.

Data Restructuring

Analysis of source code, a type of reverse engineering, needs to be done before data reorganisation can start. Statements in programming languages containing I/O, interface, file and data descriptions are all assessed. The goal is to extract data items and objects, gather data flow information and comprehend the implemented data structures that are currently in place. This process is occasionally referred to as data analysis.

Data redesign starts when data analysis is finished. When a data structure or file format already exists, a data record standardisation phase, in its most basic form, defines data definitions to ensure consistency in data item names or physical record forms. Data name rationalisation, a different type of redesign, makes ensuring that aliases are removed as data flows through the system and that all data naming conventions adhere to local standards. Physical alterations to pre-existing data structures are undertaken when restructuring goes beyond standardisation and rationalisation in order to improve the effectiveness of the data design. This could entail translating between different file formats or, perhaps, between different kinds of databases.

5.2 Computer-Aided Software Engineering

Computer Aided Software Engineering is referred to as CASE. It refers to the creation and upkeep of software projects with the aid of several automated software tools.

CASE Tools

A collection of software applications known as CASE tools are used to automate SDLC tasks. Software project managers, analysts and engineers utilise CASE tools in the development of software systems. The fundamental tenet of CASE tools is that pre-written programs may aid in the analysis of creating systems to improve quality and yield superior results. Throughout the 1990s, CASE tools entered the software vernacular and were used by major software development firms such as IBM.

To mention a few, Analysis Tools, Design Tools, Project Management Tools, Database Management Tools and Documentation Tools are just a few of the CASE tools available to streamline different stages of the Software Development Life Cycle.

By using CASE tools, projects can be completed more quickly and effectively while also identifying problems before going on to the next phase of software development.

Components of CASE Tools

Based on their application at a specific level of the SDLC, CASE tools can be roughly classified into the following categories:

- **Central Repository** - A central repository that can provide a source of shared, integrated and consistent data is necessary for CASE tools. A central repository is a location where important management information, such as product specifications, requirement documents, associated reports and diagrams, are kept. The central repository functions as a data dictionary as well.
- **Upper Case Tools** - During the SDLC's planning, analysis and design phases, upper CASE tools are employed.
- **Lower Case Tools** - For maintenance, testing and implementation, lower CASE tools are employed.
- **Integrated Case Tools** - All phases of the SDLC, from requirement collecting to testing and documentation, benefit from the use of integrated CASE tools.

If CASE tools share the same functionality, process activities and integration potential with other tools, they might be grouped together.

Advantages of the CASE approach:

- The servicing cost of a product over its anticipated lifetime is significantly decreased since extra attention is given to testing and redesign.

Notes

- An organised strategy is used during the development process, which improves the product's overall quality.
- Using computer-aided software engineering increases the likelihood and facilitates the process of meeting real-world requirements.
- A company can get an indirect competitive advantage through CASE by assisting in the production of high-quality products.
- It offers superior documentation.
- Accuracy is enhanced.
- It offers benefits that are intangible.
- It lowers upkeep over time.
- It's a chance for people who don't program.
- It affects the way the business operates.
- It lessens the tedious job that software engineers do.
- It speeds up the processing rate.
- Software programming is simple.

Disadvantages of the CASE Approach:

- **Cost:** It is highly expensive to use a case tool. Because they believe that CASE is only beneficial when developing large systems, the majority of small software development companies do not invest in CASE tools.
- **Learning Curve:** Programmers' productivity typically declines during the first stages of deployment because users require time to become familiar with the new technology. Numerous consultants provide on-site services and training, which can be crucial for quickening the learning curve and advancing the creation and application of CASE tools.
- **Tool Mix:** Creating the right combination of selecting tools is crucial to maximising cost benefit. Data integration and CASE integration are crucial for all platforms.

5.2.1 Introduction and Building Blocks of CASE

For any craftsperson, whether they be software engineers, mechanics, or carpenters, a good workshop should have three main features: (1) a selection of practical tools that will aid in each stage of creating a product; (2) a well-organised layout that makes it easy to locate and use tools; and (3) a knowledgeable artisan who knows how to use the tools effectively. Software developers now understand that in addition to a more extensive and diversified toolkit, they also want a well-organised and functional workshop. The tools used in the software engineering workshop are collectively referred to as computer-aided software engineering and the workshop itself has been referred to as an integrated project support environment. Software engineers can increase their engineering insight and automate tedious tasks with CASE. CASE tools aid in ensuring that quality is built in before the product is constructed, much like computer-aided engineering and design tools used by engineers in other fields.

Computer-assisted software engineering can be as straightforward as a single tool supporting a particular software engineering task or as intricate as an entire "environment" consisting of hardware, software, people, databases, networks, operating systems, standards and a host of other elements. The figure below shows the components that make up CASE. Every construction step lays the groundwork for the subsequent one, with tools at the top of the hierarchy. It is interesting to note that software engineering tools themselves have a

relatively little role in the foundation of efficient CASE environments. Instead, an environment architecture that includes the right hardware and systems software is the foundation of productive settings for software engineering. The environment design also needs to take into account the work patterns used by humans in the software engineering process.

Notes

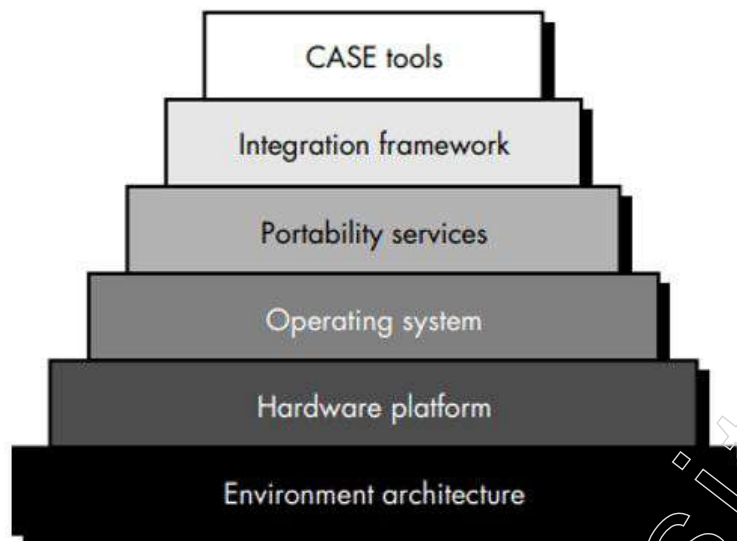


Figure: CASE building blocks

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

The foundation for CASE is laid by the environment architecture, which is made up of the object management services, database management, networking software and hardware platform. However, the CASE environment necessitates additional components. A bridge between CASE tools, their integration framework and the environment architecture is provided by a set of portability services. The integration framework is a group of specialized programmes that allow different CASE tools to interact with one other, build a project database and present a consistent user interface to the software engineer. Without requiring extensive adaptive maintenance, portability services enable CASE tools and their integration framework to move across many hardware platforms and operating systems.

The components shown in the above figure provide a thorough basis for integrating CASE tools. But not every one of these building components has been used to create the majority of CASE tools that are used today. Certain CASE tools are actually still “point solutions.” In other words, a tool is utilised to help with a specific software engineering task (analytical modelling, for example), but it is not incorporated into a project database, does not interface with other tools directly and is not a component of an integrated CASE environment (ICASE). Even though this is a point solution, a CASE tool can be used rather well, despite the unfavourable circumstances.

The figure below displays the respective levels of CASE integration. The individual (point solution) tool is at the bottom of the integration spectrum. A small improvement in integration occurs when individual tools offer data exchange capabilities, which is the case for the majority of them. These programs generate output in a common format that ought to work with other tools that can read it. A bridge between complementing CASE tools can occasionally be created by their creators working together (e.g., an analysis and design tool that is paired with a code generator). By employing this strategy, the combined effect of both technologies can yield results that would be challenging to achieve with just one tool alone. When a single provider of CASE tools combines several different tools and offers them as a package, this is known as single-source integration. While this method works

Notes

very well, most single-source systems are locked architectures that make it difficult to incorporate tools from other suppliers.

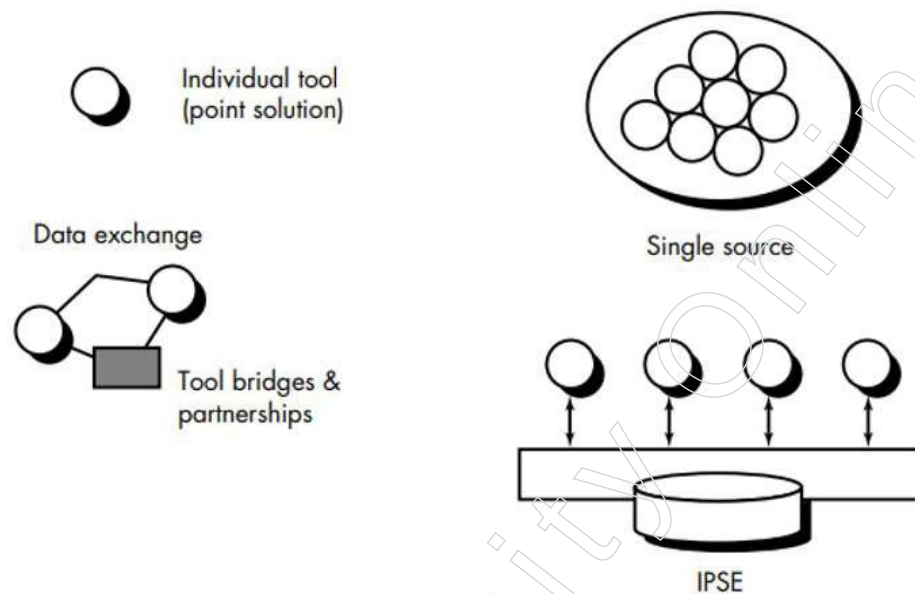


Figure: Integration options

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

The integrated project support environment is at the top of the integration spectrum (IPSE). There are now established standards for every one of the previously discussed building blocks. IPSE standards are used by CASE tool providers to create products that work with the IPSE and, consequently, with each other.

5.2.2 Taxonomy of Case Tools

Any time we group CASE tools, there are risks. Contrary to popular belief, one does not need to use all tool categories to create a successful CASE environment. Putting a tool in one category when others think it belongs in another may provoke confusion (or anger). Some readers may think a category was left out, which would remove an entire set of tools from the CASE environment. Flat classification does not show hierarchical linkages or tool interactions. Despite these risks, a CASE tool taxonomy is needed to understand the area and how software engineering processes might use these tools.

❖ CASE tools can be categorised by purpose, use as tools for technical personnel or management, how they are used in different stages of the software engineering process, hardware and software environment architecture, price, or place of manufacture. This taxonomy's fundamental criterion is function.

Process engineering tools. Business process engineering technologies create a “meta-model” of an organization's strategic information needs from which information systems are produced. Business information travels across organisational entities and is represented as it does so, not for a specific application. This category's technologies focus on representing business data objects, their relationships, and their migration between business divisions.

Tools for process modelling and management. Before improving a business (or software) process, an organisation must understand it. Process modelling, or process technology, can express a process's core components to simplify it. These tools can also link to process explanations to help participants comprehend the activities. Process management tools link to technology that aid process activity.

Design and analysis tools. Analysis and design tools help software engineers model the system to be built. The models characterise data, architectural, component-level, and interface design, as well as data, function, and behaviour (analysis level). analytical and design tools enable software engineers understand the analytical representation and identify problems before they spread to the design or implementation by validating model consistency and validity.

PRO/SIM tools. PRO/SIM (prototyping and simulation) technologies allow software engineers to predict real-time system behaviour before development. These tools also allow the software engineer to construct workable mock-ups of the real-time system so the customer can test it before implementation.

Tools for UI development. Interface design and development tools include menus, buttons, window structures, icons, scrolling algorithms, device drivers, and more. Interface prototype tools allow speedy onscreen design of complicated user interfaces that follow the software's interfacing standard, replacing traditional tool kits.

Prototyping tools. One can use many prototyping tools. Software engineers can easily define interactive app screen layouts with screen painters. Advanced CASE prototype tools may develop screen, report, and data designs. Many analytical and design programmes have prototyping extensions. PRO/SIM generates Ada and C skeleton source code for real-time engineering applications. Finally, several fourth-generation tools offer prototyping.

Programming tools. Programming tools include compilers, editors, and debuggers for most prominent languages. Database query languages, application generators, graphical programming environments, OOP environments, and fourth-generation languages are also included.

Web-development tools: Many WebApp development tools support Web engineering. These programmes create forms, scripts, graphics, text, applets, and other website elements.

Integration and testing tools: Software Quality Engineering identifies the following types of software testing tools in their directory:

- Tools for acquiring data to be used in testing are known as data acquisition tools.
- Tools that analyse source code without running test cases are known as static measurement tools.
- Dynamic measurement: programs that examine source code as it's being executed.
- Tools for simulating hardware or other external functions are called simulations.
- Tools for test planning, execution and control are known as test management tools.
- Tools that transcend the boundaries of the previous categories are called cross-functional tools.

It is important to highlight that a lot of testing tools incorporate functionality from two or more categories.

Static analysis tools. Software engineers can generate test cases with the help of static testing technologies. The industry uses three basic kinds of static testing tools: requirements-based testing tools, code-based testing tools and specialised testing languages. Source code (or PDL) is accepted as input by code-based testing tools, which then run through a series of evaluations to produce test cases. Software engineers can create comprehensive test specifications that outline each test case and the procedures for carrying it out using specialised testing languages like ATLAS. Tools for requirements-based

Notes

testing identify particular user requirements and provide test cases, or test classes, that will exercise those requirements.

Dynamic analysis tools. Dynamic testing tools work in tandem with a running program to instrument the program's execution flow, verify path coverage and test assertions regarding the value of certain variables. There are two types of dynamic tools: invasive and noninvasive. An invasive tool modifies the software under test by adding probes, or additional instructions, that carry out the previously listed tasks. Nonintrusive testing tools operate in parallel with the processor housing the program under test on a different piece of hardware.

Test management tools. Software testing is coordinated and controlled using test management solutions for all of the main testing phases. Regression testing management and coordination tools, comparison tools for determining discrepancies between output and expectations and batch testing tools for interactive human-computer interaction programs are all included in this area. Many test management technologies also work as generic test drivers in addition to the above mentioned uses. After reading one or more test cases from a testing file, a test driver formats the test data to meet the requirements of the software that is being tested, then launches the software for testing.

Client/server testing tools. Specialised testing tools that simulate the graphical user interface and the network communications requirements for both the client and server are necessary in the c/s environment.

Reengineering tools. Tools for legacy software provide a range of maintenance tasks that account for a sizable portion of software-related work at the moment. The following roles can be found within the category of reengineering tools:

- Using source code as input, reverse engineering to specification tools produce graphical structured analysis and design models, where-used lists and other design data.
- Program syntax is examined by code reformation and analysis tools, which also produce a control flow diagram and an automatically structured program.
- Online database systems can be changed (e.g., by converting IDMS or DB2 files into entity-relationship format) using online system reengineering tools.

These tools need some interaction from the software engineer and are restricted to certain programming languages (though they cover most major languages).

5.2.3 TCS Robot: Case Tools

Smart logistics: Where machines do the heavy lifting

TCS's IP-backed autonomous mobile robot increases output in a collaborative human-machine setting while maintaining safety.

TCS has developed an industry-first portfolio of disruptive robotics solutions to automate various logistics industry processes.

Supported by TCS intellectual property, the suite includes robotic orchestration platforms, pickers and packers and AMRs. This is the result of the TCS Research and TCS Pace™ teams' rigorous work over the course of around ten years.

TCS reimagines and provides as-a-service the next generation of fully automated material handling, sorting and distribution systems with robotics. These systems can be used for a variety of industries, sizes and commodities.

Because of the nature of work in semi-structured environments, CEP firms, e-tailers, fast-moving consumer goods companies, manufacturers, professional manpower supply companies and third-party logistics providers have been facing difficult-to-automate issues for years. Some of the issues facing the sector in the past ten years include a lack of labour, pressure to control costs, the size and increase of volumes and health and safety regulations.

In addition to optimising sorting, distribution and fulfilment centre performance through task automation, TCS's solution helps businesses achieve their growth strategy through integration, orchestration, capacity and throughput enhancement, knowledge management and the robotics and all-pervasive intelligence that enable wall-to-wall processes.

What is TCS AMR?

TCS AMR is a multipurpose, industrial-grade forklift mobile robot that can handle various payloads for intralogistics tasks. The TCS AMR's small design facilitates safe, autonomous navigation along constrained pathways. For efficient fleet management and control, a vendor-neutral fleet management system (FMS) is included with it. For material handling orders, TCS FMS offers smooth connection with corporate systems like a warehouse management system.

Technology Driving AMR

The robots are outfitted with cargo cage detecting modules that are exclusive to TCS, adaptive obstacle avoidance, zone definition for navigation and unique navigation behaviour models for charging and queuing. The robots are also capable of communicating with people and other robots in a fleet. Intelligent handling of dynamic barriers and the planning of collision-free multi-robot pathways are achieved through the use of robot sensor data and digital twin technology.

Material handling robots hold the promise of a massive industry transformation in logistics, along with other robotic interventions like autonomous truck loose-loading and unloading systems, 3D bin packers, sorting robots, multi-AMR collaborative transport systems for uglies (non-machinable goods) and singulators. To fully utilise robots and automation, the wall-to-wall operations of a lights-out sorting or last-mile centre typically call for integration, orchestration and next-generation optimisation and control algorithms.

"Systems with AMRs could ensure the ability to do time-definite deliveries in various segments with right prioritisation across the production chain and flexible sorting to accommodate varying peak-to-average ratios of demand."

Industry Transformation Through AMRS

Logistics organisations may find that their cost and business models are drastically altered by touchless, energy-efficient, always-on warehouses or sorting centres that are also cost and energy-optimized. These centres can range from temporary and shipper-adjacent to enhanced current centres and those with greenfield operations.

Without hiring more people, systems using AMRs might guarantee new or incremental capacity expansions to logistical networks. In order to handle different peak-to-average ratios of demand, it can support contactless operations, execute time-definite deliveries across the production chain and perform flexible sorting. Furthermore, automatic data collecting about robot movements aids in the evaluation of energy and cost expenditures in real time as well as the assurance of proper product and service pricing.

The main way that AMRs improve physical labour management is by assigning workers to jobs requiring human judgement and delegating heavy lifting to a robot.

5.2.4 Integration Architecture and Case Repository

Notes

The Integration Architecture

A software engineering team builds a repository of software engineering knowledge using matching techniques, CASE tools and a process framework. Information entering and leaving the pool is made easier via the integration architecture. The following architectural elements must be present in order to achieve this: To store the information, a database must be made; to manage changes to the information, an object management system must be developed; to coordinate the use of CASE tools, a tools control mechanism must be built; and finally, a user interface must offer a consistent path between user actions and the tools present in the environment. The majority of integration framework models depict these elements as layers. The figure below shows a basic model of the framework that simply shows the previously mentioned components.

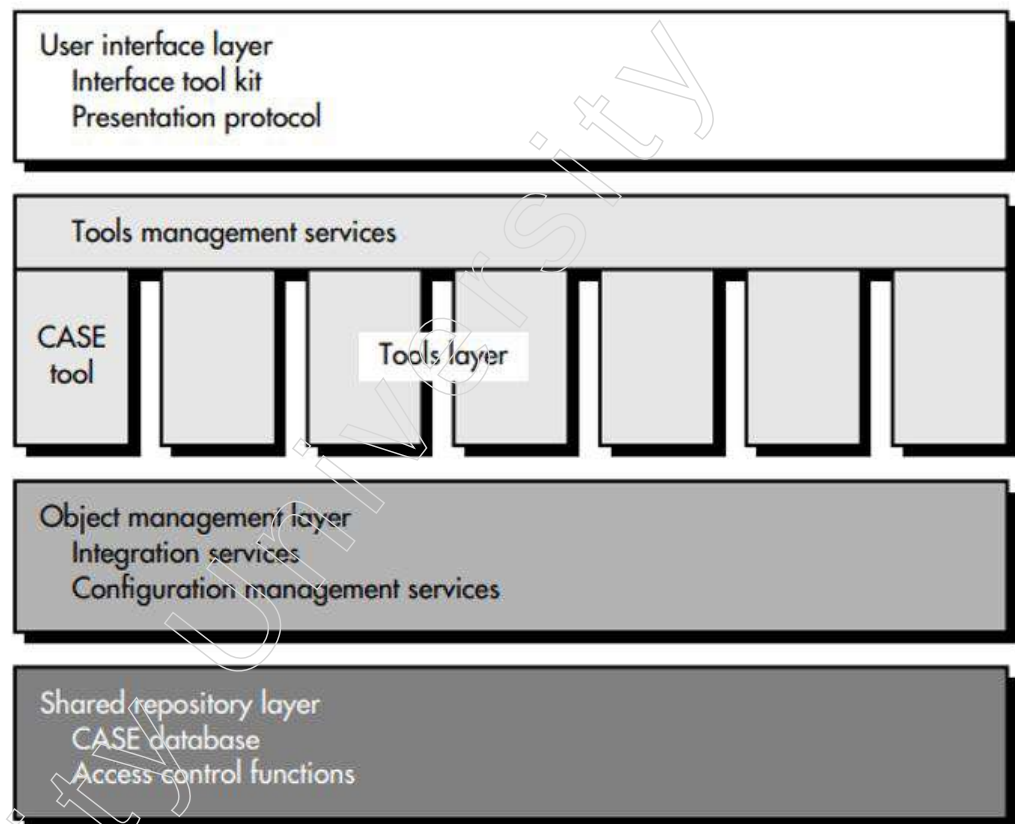


Figure: Architectural model for the integration framework

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

A consistent presentation protocol and standardised interface toolkit are integrated into the user interface layer. A library of display items and software for managing human-computer interfaces are included in the interface tool kit. Both offer a standardised means of communication between the CASE tool and the UI. The collection of rules known as the presentation protocol is what unites the appearance and feel of all CASE tools. The presentation protocol defines the norms for screen layout, the names and organisation of menus, icons, object names, keyboard and mouse usage and the method for accessing tools.

Together with the CASE tools itself, the tools layer includes a collection of tools management services. Tools behave in the environment under the control of tools management services (TMS). When one or more tools are executed while multitasking is enabled, TMS handles multitask synchronisation and communication, organises data

transfer from the object management system and repository to the tools, carries out security and auditing tasks and gathers tool usage metrics.

This layer of the framework architecture's software essentially acts as the mechanism for integrating tools. The object management layer is "plugged into" by all CASE tools. The OML offers integration services, which are a collection of common modules that couple tools with the repository, in cooperation with the CASE repository. Furthermore, by facilitating the identification of all configuration items, managing versions and offering assistance with change control, audits and status accounting, the OML offers configuration management services. The CASE database and the access control features that let the object management layer communicate with the database are part of the shared repository layer. The common repository and object management layers enable data integration.

The Case Repository

The definition of a repository according to Webster's Dictionary is "anything or person thought of as a centre of accumulation or storage." In the early days of software development, the repository was actually a person: the programmer, who had to recollect where all pertinent information was stored for a project, recreate missing information and remember information that was never recorded. Unfortunately, even if it fits Webster's definition, utilising a person as "the centre for accumulation and storage" does not function very well. These days, the repository is a "thing"—a database that serves as the hub for the gathering and archiving of knowledge related to software engineering. Utilising CASE tools that are integrated with the repository, the software engineer's job is to communicate with it.

The location where software engineering data is stored has been referred to by a variety of names, including requirements dictionary (a constrained database), CASE database, project database, integrated project support environment (IPSE) database and repository. All of these words allude to the centre for accumulation and storage, notwithstanding some minor variations.

The Role of the Repository in I-CASE

The collection of data structures and methods that enable data/tool and data/data integration is the repository for an I-CASE environment. Along with the apparent duties of a database management system, the repository also executes or initiates the following tasks:

- Validating entries into the repository, guaranteeing consistency between linked objects and automatically carrying out "cascading" modifications—modifications to one object that require changes to other connected objects—are all examples of data integrity functions.
- Information sharing offers a way to manage and govern multiuser access to data, exchange information between developers and tools and lock or unlock objects to prevent changes from accidentally being overwritten on top of each other.
- Data/tool integration creates a data model that is accessible to all tools inside the I-CASE environment, manages data access and carries out relevant configuration management tasks.
- The database management system that links data items to enable other functions is known as data/data integration.
- The connections and objects describe, at the very least, a set of procedures that must be taken in order to construct the contents of the repository. Methodology enforcement defines an entity-relationship model stored in the repository that indicates a specific paradigm for software engineering.

Notes

Notes

- The defining of items in the database that directly results in a standardised procedure for the development of software engineering documents is known as document standardisation.

The repository is specified in terms of a meta-model in order to accomplish these goals. The repository's information storage, tool access and software engineer viewing capabilities, data security and integrity maintenance and the ease with which the current model can be expanded to meet new requirements are all determined by the meta-model. The template that software engineering knowledge is inserted into is called the meta-model.

Features and Content

The best way to understand the repository's features and content is to consider it from two angles: what will be stored there and what particular services it offers. Generally speaking, the kinds of items that should be kept in the repository include

- the issue that needs to be resolved.
- details regarding the issue domain.
- The emergent system solution.
- regulations and guidelines relevant to the software process (methodology) being used.
- The history, resources and project plan.
- details regarding the organisational setting.

The table below provides a thorough inventory of the various kinds of deliverables, documents and representations that are kept in the CASE repository.

Table: Case Repository Contents

Enterprise information Organizational structure Business area analyses Business functions Business rules Process models (scenarios) Information architecture	Construction Source code; Object code System build instructions Binary images Configuration dependencies Change information
Application design Methodology rules Graphical representations System diagrams Naming standards Referential integrity rules Data structures Process definitions Class definitions Menu trees Performance criteria Timing constraints Screen definitions Report definitions Logic definitions Behavioral logic Algorithms Transformation rules	Validation and verification Test plan; Test data cases Regression test scripts Test results Statistical analyses Software quality metrics
	Project management information Project plans Work breakdown structure Estimates; Schedules Resource loading; Problem reports Change requests; Status reports Audit information
	System documentation Requirements documents External/internal designs User manuals

Image Source: Software Engineering A Practitioner's approach by roger s. pressman

Two distinct categories of services are offered by a strong CASE repository: (1) services that are typical of any advanced database management system and (2) services that are unique to the CASE environment.

Numerous criteria for repositories are similar to those of standard programs developed on commercial database management systems (DBMS). Actually, a database management system (DBMS; often relational or object oriented) serves as the foundation for the majority of modern CASE repositories. The information management of software development is supported by the following DBMS functionalities.

- non-redundant storage of data. All CASE tools that require access to an object can access it even though it is only saved once.
- elevated access. To avoid duplicating data handling facilities across all CASE tools, a standard data access mechanism is established.
- independence of data. Changes to the hardware configuration have no effect on CASE tools or the target applications since they are isolated from physical storage.
- control over transactions. When there are several users interacting with the repository at the same time, record locking, two-stage commits, transaction logging and recovery protocols are put in place to ensure data integrity.
- safety. The repository offers controls over who can access and alter the data it contains.
- ad hoc reports and data inquiries. Through a handy user interface like SQL or a forms-oriented “browser,” the repository facilitates direct access to its contents, allowing for user-defined analysis beyond the basic reports that come with the CASE tool set.
- Transparency. Generally speaking, repositories offer a straightforward import/export feature to facilitate bulk loading or transfer.
- support for many users. Multiple developers must be able to work on an application at once in a robust repository. It must control several tools’ and users’ simultaneous access to the database while providing access arbitration and locking at the file or record level. Multiuser support for networking contexts also means that the repository can communicate with common networking protocols (object request brokers) and infrastructure.

In addition, the repository must meet additional requirements from the CASE environment that go beyond those of a commercial DBMS. CASE repositories have unique characteristics that include

- storage of complex data structures. Both simple and complicated data elements—such as files, documents and diagrams—must be supported by the repository. An information model, also known as a metamodel, is a component of a repository that describes the relationships, structure and semantics of the data kept within. In order to support additional representations and distinct organisational information, the meta-model needs to be extendable. The repository contains not only descriptions and models of systems in development, but also related meta-data (i.e., extra details describing the software engineering data itself, like the creation date, status and dependencies of a specific design component).
- enforcement of integrity. In addition, the repository information model includes policies, or rules, that outline appropriate business rules as well as additional limitations and specifications on data that must be entered into the repository (either directly or through the use of a CASE tool). Design models can be validated in real time by using a capability called a trigger, which can be used to activate the rules associated with an item anytime it is modified.

Notes

- Tool interface rich in semantics. Semantics in the repository information model (meta-model) allow several tools to understand the meaning of the data kept in the repository. A CASE tool's generated data flow diagram, for instance, is kept in the repository in a format determined by the information model and unaffected by whatever internal representations the tool may employ. The information in the repository can then be interpreted by another CASE tool, which can utilise it for whatever purpose it sees fit. So instead of specialised tool-to-tool conversions or "bridges," the semantics maintained in the repository allow data interchange across several tools.
- Project/process management. In addition to details on the software program itself, a repository also includes information about the specifics of each project as well as the organisation's overall software development process, including tasks, deliverables and phases. This creates opportunities for automatic coordination between project management and technical development activities. One possible outcome of utilising the CASE tools could be the automatic updating of project tasks' statuses. Without requiring developers to step outside of their typical development environment, status updating can be made very simple for them to accomplish. Email can be used for task assignment and question answering as well. Through tools that access the repository, problem reports, maintenance tasks, change authorisation and repair status may all be managed and tracked.

Software configuration management includes all of the functionality listed below for repositories. Here, they are reexamined to highlight how they are related to I-CASE environments:

Versioning. Numerous iterations of distinct work products will be produced as a project moves forward. To facilitate efficient administration of product releases and to allow developers to roll back to earlier versions for testing and debugging, the repository has to be able to store all of these versions.

Text, images, bit maps, complicated documents and unique objects such as object files, test data and results, as well as screen and report definitions, must all be controlled by the CASE repository. A developed repository can keep object versions at any granularity; for instance, it can track a single data definition or a group of modules.

The version control system should allow for several derivatives (variants) from a single ancestor in order to facilitate simultaneous work. As a result, a developer may be working simultaneously on two different approaches to a design problem that originated from the same source.

Dependency tracking and change management. Many different types of relationships between the data pieces stored in the repository are managed by it. These comprise the connections between application design components, enterprise information architecture, enterprise entities and processes, deliverables and design elements and so forth. While some of these interactions are merely affiliations, others are required ties or dependencies. Link management is the process of keeping these ties between development items intact.

One of the most significant ways that the repository concept improves the software development process is through its ability to track all of these relationships, which are essential to the integrity of the information stored in the repository and to the creation of deliverables based on it. Link management supports a multitude of functions, one of which is the ability to recognise and evaluate the consequences of change. The ability to identify every object that could be impacted allows for more precise estimation of cost, downtime and degree of difficulty when designs change to accommodate new requirements.

Additionally, it aids in avoiding unanticipated side effects that would otherwise result in flaws and system malfunctions.

By maintaining synchronisation between the different parts of a design, link management assists the repository mechanism in guaranteeing the accuracy of design information. When a data flow diagram is changed, for instance, the repository can identify if associated data dictionaries, screen definitions and code modules also need to be changed. It may then notify the developer of any affected components.

Requirements tracing. This unique feature, which is reliant on link management, allows for the forward tracking of all design elements and deliverables that originate from a given requirement definition. Furthermore, it offers the capability to determine which need led to the creation of a specific delivery (backward tracking).

Configuration management. To maintain track of a number of configurations that correspond to particular project milestones or production releases, a configuration management facility collaborates closely with the link management and versioning facilities. Link management monitors interdependencies and version management supplies the necessary versions.

Audit trails. Additional details regarding the who, what and when of changes are established by an audit trail. Changes' origin can be specified as a property for particular items in the repository. Every time a design element is changed, a repository trigger mechanism can help remind the developer or the tool being used to start entering audit information (such the rationale for a change).

Summary

- Business Process Re-engineering (BPR) and Software Re-engineering are methodologies aimed at transforming and improving existing processes or software systems to meet evolving needs. Business Process Re-engineering focuses on optimising business processes, while Software Re-engineering targets enhancing existing software systems. Both methodologies aim for improvement and adaptation to meet evolving business and technological demands. Successful implementation requires careful planning, stakeholder involvement and a thorough understanding of existing systems or processes.
- Building blocks in a case refer to the fundamental components or elements that collectively form the structure of a case study or analysis. They serve as the essential parts that, when combined, provide a comprehensive understanding of the subject under examination. The building blocks serve as the structural foundation of a case study or analysis, ensuring a systematic and comprehensive exploration of the subject matter. They guide the development of a coherent narrative, provide a framework for analysis and facilitate clear communication of findings and recommendations. Each component contributes uniquely to creating a holistic understanding of the case and its potential impact or implications.
- Forward re-engineering is a process that involves the restructuring or redevelopment of an existing software system or application to adapt it to newer technologies, platforms, or architectures. It focuses on upgrading and modernising the system's structure and functionality without changing its external behaviour or objectives. The economics of re-engineering refers to the cost-benefit analysis or financial considerations involved in deciding whether to undertake a re-engineering effort, such as software or business process re-engineering. It involves evaluating the economic feasibility, potential returns and risks associated with re-engineering projects. Forward re-engineering involves modernising existing systems, while the economics

Notes

Notes

of re-engineering encompass evaluating the financial viability and potential returns associated with such efforts. Both concepts focus on ensuring the sustainability, efficiency and competitiveness of systems or processes within an organisation. Successful implementation requires careful planning, cost-benefit analysis and risk assessment.

- Reverse engineering is a process of analysing an existing product, system, or software to understand its design, architecture, functionalities and components. It involves dissecting or deconstructing the object of study to comprehend its internal workings or source code. Restructuring engineering refers to the process of modifying, reorganising, or improving the structure, design, or architecture of an existing system or software. It involves making strategic changes to enhance the system's performance, scalability, or maintainability while preserving its core functionalities. Reverse engineering involves understanding existing systems by analysing their internal structure or code, while restructuring engineering focuses on modifying and optimising system structures for improved performance or maintainability. Both processes play vital roles in adapting and evolving systems to meet changing needs but require careful planning, expertise and consideration of potential challenges.
- CASE (Computer-Aided Software Engineering) refers to the use of computer-based tools and methodologies to aid in the development, maintenance and management of software systems. It encompasses a range of software tools and techniques that assist in various phases of the software development life cycle. CASE tools emerged in response to the complexities of software development, aiming to streamline and automate various tasks involved in creating software. These tools provide support across different phases of the software development process, including planning, analysis, design, implementation, testing and maintenance. CASE tools' building blocks collectively provide a suite of software to assist and automate various aspects of software development, from planning and analysis to design, implementation, testing and maintenance. They play a crucial role in improving productivity, quality and collaboration in software engineering processes.
- CASE tools encompass a broad range of software applications used to support various phases of the software development life cycle. They can be classified into different categories based on their functionalities and the specific phases of software development they cater to. TCS (Tata Consultancy Services) Robot is an example of a CASE tool used in software testing and quality assurance. It falls under the category of LCASE tools and is specifically designed for automation testing purposes. TCS Robot, being a specialised tool for automated testing, falls within the LCASE category, focusing on the testing and validation phase of software development, enhancing efficiency, accuracy and reliability in software testing processes.
- Integration Architecture refers to the design, structure and methodologies used to facilitate the seamless integration of different software components, systems, or applications within an organisation. It ensures that diverse systems can communicate, share data and operate cohesively. A Case Repository in CASE tools refers to a centralised storage or database that houses artifacts, documents, models and other elements related to software development projects. It serves as a comprehensive repository for managing and organising project-related information. Both Integration Architecture and Case Repository play crucial roles in ensuring efficient software development, enabling seamless connectivity between systems and providing a structured, centralised repository for managing project-related artifacts and information.

Glossary

- LRSLC: Legacy and Reuse Software Life Cycle
- BPR: Business Process Reengineering
- BPE: Business Process Engine
- GUI: Graphical User Interface
- CASE: Computer-Aided Software Engineering
- CBSE: Component-Based Software Engineering
- ICASE: Integrated CASE Environment
- TMS: Tools Management Services
- IPSE: Integrated Project Support Environment
- DBMS: Database Management Systems

Check Your Understanding

1. What is the primary goal of Business Process Re-engineering (BPR)?
 - a) Radical redesign of processes for significant improvements
 - b) Incremental improvements in existing processes
 - c) Maintaining the status quo of business operations
 - d) Increasing paperwork in business workflows
2. Which of the following is a challenge typically associated with Business Process Re-engineering?
 - a) Embracing the existing workflow without modifications
 - b) Resistance to change from employees
 - c) Minimising customer-centric approaches
 - d) Avoiding technology integration
3. Business Process Re-engineering emphasises:
 - a) Slow and cautious changes
 - b) Technology-agnostic solutions
 - c) Incremental modifications to existing processes
 - d) Radical rethinking and redesign of processes
4. What is the primary objective of Software Re-engineering?
 - a) Development of new software from scratch
 - b) Understanding and updating existing software systems
 - c) Eradicating software bugs
 - d) Maintaining outdated software as is
5. Software Re-engineering involves:
 - a) Rewriting code from scratch with no reference to the existing system
 - b) Replacing the software entirely with a new system
 - c) Making changes to the software's structure while preserving its external behaviour
 - d) Ignoring the original system's architecture

Notes

Notes

Exercise

1. What is business process re-engineering and software re-engineering? Explain briefly.
2. Give introduction to building block of case.
3. Explain with examples forward re-engineering and economics of re-engineering, reverse engineering and restructuring engineering.
4. Give introduction and building blocks of case.
5. What do you understand by taxonomy of case tools?

Learning Activities

1. Imagine you are a consultant hired by a manufacturing company that wants to undergo Business Process Re-engineering to enhance its production efficiency. Describe the steps you would take to analyse and re-engineer their current manufacturing processes.
2. You are a software development team lead responsible for modernising a legacy system that is critical for your organisation's operations. Outline the steps you would take in the software re-engineering process. Discuss how you would assess the existing system, decide on the re-engineering approach (e.g., migration, re-platforming) and manage potential risks and challenges. Consider factors such as maintaining data integrity, minimising downtime and ensuring user acceptance.

Check Your Understanding- Answers

1. a)
2. b)
3. d)
4. b)
5. c)